KAYA VISION

# Vision Point II API

# Data Book

October 2026

Rev 2026.1.3

20283 State Road 7, Suite 350
Boca Raton, FL 33498
USA
+1 (561) 689-2899

info@kaya.vision
support@kaya.vision

www.kaya.vision

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# Revision History

| Version | Date | Notes |
|---|---|---|
| 2024.1.0 | 09.2024 | Initial release |
| 2024.1.1 | 09.2024 | The document remains unchanged |
| 2024.1.2 | 10.2024 | Vision Point API release 2024.1 (service pack 2)<br>- KYParametersHandler_GetParameterAttributeValue() function was added to the KYVP ParametersHandler library.<br>- KYFG_LoadFirmware(), KYFG_GetFirmareUpdateFileInfo() , KY_GetCameraPropertyParameterValue() and KY_GetGrabberPropertyParameterValue() functions were added to the Vision Point API Adapter |
| 2024.1.3 | 10.2024 | The document remains unchanged |
| 2025.1.0 | 12.2024 | Vision Point API release 2025.1<br>- KYVPLibExtension library was added<br>- API Examples for Linux section was added<br>- Legacy Vision Point API Adapter section has been moved to the separate document titled “Vision Point Migration guide” |
| 2025.1.1-1.3 | 01-04.2025 | The document remains unchanged |
| 2025.2.0 - 2025.2.3 | 06-09.2025 | The document remains unchanged |
| 2025.2.4 | 11.2025 | Added KYFoundation library section<br>Added KY_RESULT section |
| 2026.1.0 | 01.2026 | API Examples section updated |
| 2026.1.1 | 04.2026 | Data Stream section updated |
| 2026.1.2 | 05.2026 | The document remains unchanged |
| 2026.1.3 | 10.2026 | Added explanations of API functions and data structures for all libraries |

Table 1 – Revision History

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# Table of Contents

**Revision History** ..... **1**
**1 Figures and Tables** ..... **4**
1.1 List of Figures ..... 4
1.2 List of Tables ..... 4
**2 Introduction** ..... **5**
2.1 Safety precautions ..... 5
2.2 Disclaimer ..... 6
**3 Overview** ..... **7**
3.1 Document Structure ..... 7
**4 API Notes and Limitations** ..... **8**
4.1 API Usage in Multi-Threaded Applications ..... 8
4.2 Important notes ..... 8
**5 Common include headers** ..... **9**
5.1 Common SDK definitions ..... 9
5.2 Foundation types ..... 19
5.3 Common result codes ..... 31
5.4 Hardware result codes ..... 35
5.5 Compiler warning controls ..... 35
5.6 Experimental API annotations ..... 38
**6 KYVPLibTL library** ..... **40**
6.1 Function Call Sequence ..... 40
6.2 System Modules ..... 43
6.2.1 System Module ..... 43
6.2.2 PCI Interface Module ..... 43
6.2.3 Local Device Module ..... 43
6.2.4 Remote Device Module ..... 44
6.2.5 Data Stream Module ..... 44
6.2.6 Buffer Module ..... 44
6.3 Module Enumeration and Instantiation ..... 45
6.3.1 System ..... 45
6.3.2 Interface ..... 45
6.3.3 Local and Remote Devices ..... 46
6.3.4 Data Stream ..... 46
6.3.5 Buffer ..... 47
6.4 KYVPLibTL API functions ..... 48
**7 KYVPLibExtension library** ..... **126**
7.1 KYVPLibExtension API functions ..... 126
**8 KYVPParametersHandler library** ..... **162**
8.1 Function Call Sequence ..... 162
8.2 KYVPParametersHandler API functions ..... 163
**9 KYVPImageProcessing library** ..... **199**

9.1 KYVPImageProcessing API functions ..... 199
**10 KYFoundation library ..... 222**
10.1 KYFoundation API functions ..... 222
**11 KY_RESULT ..... 224**
**12 Library Exiting ..... 225**
12.1 Memory buffers ..... 225
12.2 Background monitoring thread ..... 225
**13 API Examples ..... 226**
13.1 Building API example for Windows ..... 234
13.2 Building API example for Linux ..... 234
**14 Collect Diagnostic Info ..... 236**
14.1 Windows Operating System ..... 236
14.2 Linux Operating System ..... 238
**REFERENCES ..... 239**

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# 1 Figures and Tables

## 1.1 List of Figures

Figure 1 – KYVPLibTL function call sequence ..... 41
Figure 2 – Module hierarchy ..... 43
Figure 3 – Enumeration hierarchy ..... 45
Figure 4 – KYVPParameterHandler function call sequence ..... 162
Figure 5 – Collect diagnostic info from the KAYA Service Controller II tray menu ..... 236
Figure 6 – Collect diagnostic info from the KAYA Vision Studio Help menu ..... 236
Figure 7 – Collect diagnostic info from the Windows Start menu ..... 237
Figure 8 – Collect diagnostic info from the KAYA Service Controller II tray menu on Linux ..... 238
Figure 9 – Collect diagnostic info from the KAYA Vision Studio Help menu ..... 238

## 1.2 List of Tables

Table 1 – Revision History ..... 1

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# 2 Introduction

## 2.1 Safety precautions

Please take the time to read through the precautions listed below to prevent preventable and unnecessary injuries and damage to you, other personnel, or property. Read these safety instructions carefully before your first use of the product, as these precautions contain safety instructions that must be observed. Be sure to follow this manual to prevent misuse of the product.

| ⚠ Caution! Read Carefully and do not disregard these instructions. |
|---|
| **In the event of a failure, disconnect the power supply.**<br>Disconnect the power supply immediately and contact our sales personnel for repair. Continuing to use the product in this state may result in a fire or electric shock. |
| **If an unpleasant smell or smoking occurs, disconnect the power supply.**<br>Disconnect the power supply immediately! Continuing to use the product in this state may result in a fire or electric shock. After verifying that no smoking is observed, contact our sales personnel for repair. |
| **Do not disassemble, repair or modify the product.**<br>This may result in a fire or electric shock due to a circuit shortage or heat generation. Contact our sales personnel before inspection, modification or repair. |
| **Do not place the product on unstable surfaces.**<br>Otherwise, it may drop or fall, resulting in injury to persons or the camera. |
| **Do not use the product if dropped or damaged.**<br>Otherwise, a fire or electric shock may occur. |
| **Do not touch the product with metallic objects.**<br>Otherwise, a fire or electric shock may occur. |
| **Do not place the product in dusty or humid environments, nor where water may splash.**<br>Otherwise, a fire or electric shock may occur. |
| **Do not wet the product or touch it with wet hands.**<br>Otherwise, the product may fail or it may cause a fire, smoking or electric shock. |
| **Do not touch the gold-plated sections of the connectors on the product.**<br>Otherwise, the surface of the connector may be contaminated by sweat or skin-oil, resulting in contact failure of a connector, malfunction, fire or electric shock due to static electricity discharge. |
| **Do not use or place the product in the following locations.**<br>▪ Unventilated areas such as closets or bookshelves.<br>▪ Near oils, smoke or steam.<br>▪ Next to heat sources.<br>▪ A closed (and not running) car where the temperature becomes high.<br>▪ Static electricity replete locations<br>▪ Near water or chemicals.<br>Otherwise, a fire, electric shock, accident or deformation may occur due to a short circuit or heat generation. |
| **Do not place heavy objects on the product.**<br>Otherwise, the product may be damaged. |
| **Be sure to discharge static electricity from the body before touching any sensitive electronic components.**<br>The electronic circuits in the computer and the circuits on the board are sensitive to static electricity and surges. Improper handling may seriously damage the circuits. In addition, do not let your clothing come in contact with the circuit boards or components. Otherwise, the product may be damaged. |

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

## 2.2 Disclaimer

KAYA Instruments will assume no responsibility for any damage that may ensue by the use of this product for any purpose other than intended, as previously stated. Without detracting from what was previously written, please be advised that the company will take no responsibility for any damages caused by:

- Earthquake, thunderstrike, natural disasters, fire caused by use beyond our control, wilful and/or accidental misuse and/or use under other abnormal and/or unreasonable conditions.
- Secondary damages caused by the use of this product or its unusable state (business interruption or others).
- Use of this product in any manner that contradicts this manual or malfunctions that may occur due to connection to other devices. Damage to this product that is out of our control or failure due to modification
- Accidents and/or third parties that may be involved.

Additionally, KAYA Instruments assumes no responsibility or liability for:

- Erasure or corruption of data caused by the use of this product.
- Any consequences or other abnormalities following the use of this product

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# 3 Overview

The purpose of this document is to list and demonstrate the provided functionality of KAYA Frame Grabbers' API.

This API is to be used with KAYA's Frame Grabbers hardware provided by KAYA Vision. This is a high-level API for connecting, configuring and capturing data streaming over 1, 2, 4 or 8 channels. KAYA's Frame Grabbers are capable of connecting to various cameras at various speeds and topologies.

## 3.1 Document Structure

This API guide is divided into few major topics each related to different functionalities:

- Common include headers defines shared SDK types, result codes, and compiler annotations.
- KYVPLibTL library implements the transport layer function.
- KYVPParametersHandler library provides access to devices' Gen<i>Cam parameters.
- KYVPLibExtension library enhances the functionality of the KYVPLibTL library.
- KYVPImage Processing library to perform various image processing tasks with acquired stream images.
- KYFoundation library to handling KY_RESULT.
- API examples.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# 4 API Notes and Limitations

## 4.1 API Usage in Multi-Threaded Applications

Vision Point II API is NOT thread-safe. This means that if a calling application accesses the resources listed below from multiple threads, the serialization of such accesses should be implemented by that application. Resources that require serialized access are:

- KYVPLibTL Library accessed via an instance of KYVP_TL_HANDLE
- PCI Interface accessed via an instance KYVP_PCI_INTERFACE_HANDLE
- Local Device accessed via an instance of KYVP_DEVICE_HANDLE
- Stream accessed via an instance of KYVP_STREAM_HANDLE
- Event accessed via an instance KYVP_EVENT_HANDLE
- Remote Device accessed via an instance of KYVP_REMOTE_DEVICE_HANDLE
- ParametersHandler library accessed via an instance KYVP_COLLECTION_HANDLE
- A frame buffer accessed via an instance of KYVP_BUFFER_HANDLE

## 4.2 Important notes

1. DllMain function:
   KAYA's API should **NOT** be used from DllMain function on Windows OS.
   There are significant limits on what you can safely do at a DLL entry point. See General Best Practices for specific Windows APIs that are unsafe to call in DllMain. If more than the simplest initialization is required, it is recommended to perform it in an initialization function for the DLL. You can require applications to call the initialization function after DllMain has run and before they call any other functions in the DLL.

2. KYVP ParametersHandler performance:
   Functions of KYVP ParametersHandler are inherently relatively slow because they utilize the Gen<I>Cam reference implementation. Therefore, we do not suggest using them in performance-critical parts of the code, such as the stream callback function, etc. Instead, we highly recommend using KYVPLibTL_DSGetBufferInfo() with a relevant command to retrieve the required information. Those functions can still be used in non-performance-critical parts for example at the system initialization, before or after acquisition sessions etc.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# 5 Common include headers

The Vision Point II SDK libraries share public include headers that define common types, result codes, platform macros, and data structures. The references in this section describe these shared declarations. Library-specific function references and their supporting definitions follow in the corresponding library chapters.

## 5.1 Common SDK definitions

KYVPDefines.h defines shared platform and calling-convention macros, event identifiers, CoaXPress link definitions, and structures for auxiliary data, interfaces, devices, and buffers. Preserve the documented structure versions and layouts when passing data to the SDK.

### 5.1.1 KYVPDefines.h API reference

Provides platform macros, event identifiers, CoaXPress link definitions, and structures for auxiliary data, PCI interfaces, devices, and buffers.

Structure packing, member order, enum values, and version constants form part of the SDK ABI. See KYVPDefines Structs for the structure field reference.

#### 5.1.1.1 Macros

##### 5.1.1.1.1 KYVP_CALLCONV

SDK calling convention; __cdecl on Microsoft compilers.

##### 5.1.1.1.2 KYVP_MEMBER_DEFAULT

Expands to a member initializer in C++ and to nothing in C.

##### 5.1.1.1.3 KYVP_EXTERNAL_C

C linkage in C++; empty in C.

##### 5.1.1.1.4 KYVP_EXTERNAL

```
extern
```

External linkage declaration keyword.

##### 5.1.1.1.5 KYVP_EXTERNAL_DLL

SDK DLL export or import attribute on Microsoft compilers.

##### 5.1.1.1.6 KYVP_API

```
KYVP_EXTERNAL_C KYVP_EXTERNAL_DLL
```

Public SDK function linkage and visibility attributes.

##### 5.1.1.1.7 KYVP_IO_CONTROLLER_MASKED_IO

```
((masked_data) & 0xFFFFFFFFFF)
```

Extracts I/O signal bits 0-39 from an auxiliary event mask.

##### 5.1.1.1.8 KYVP_IO_CONTROLLER_MASKED_ENCODERS

```
(((masked_data) >> 40) & 0xF)
```

Extracts encoder bits 40-43 as bits 0-3.

##### 5.1.1.1.9 KYVP_IO_CONTROLLER_MASKED_TIMERS

```
(((masked_data) >> 44) & 0xFF)
```

Extracts timer bits 44-51 as bits 0-7.

##### 5.1.1.1.10 KYVP_IO_CONTROLLER_MASKED_CAMERA_TRIGGERS

```
(((masked_data) >> 52) & 0xFF)
```

Extracts camera-trigger bits 52-59 as bits 0-7.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

#### 5.1.1.1.11 KYVP_IO_CONTROLLER_MASKED_TRIGGERS

```
(((masked_data) >> 60) & 0xF)
```

Extracts acquisition-trigger bits 60-63 as bits 0-3.

#### 5.1.1.1.12 KYVP_AUX_MESSAGE_ID_IO_CONTROLLER

```
11
```

Auxiliary message ID selecting the I/O controller payload.

#### 5.1.1.1.13 KYVP_CXP2_HEARTBEAT_VERSION

```
1
```

Current version of KYVP_CXP2_HEARTBEAT.

#### 5.1.1.1.14 KY_CXP_EVENT_MAX_DATA_SIZE

```
1024
```

Maximum CoaXPress 2.0 event data capacity, in bytes.

#### 5.1.1.1.15 KYVP_CXP2_EVENT_VERSION

```
1
```

Current version of KYVP_CXP2_EVENT.

#### 5.1.1.1.16 KYVP_PCI_INTERFACE_STREAM_UNKNOWN

```
0x0
```

No stream direction capability flags set.

#### 5.1.1.1.17 KYVP_PCI_INTERFACE_STREAM_GRABBER

```
0x1
```

Bit flag for acquisition capability (input streams).

#### 5.1.1.1.18 KYVP_PCI_INTERFACE_STREAM_GENERATOR

```
0x2
```

Bit flag for generation capability (output streams).

#### 5.1.1.1.19 KYVP_PCI_CONFIG_VERSION

```
1
```

Version identifier associated with KYVP_PCI_CONFIG, which has no version field.

#### 5.1.1.1.20 KYVP_MAX_BOARD_INFO_VERSION

```
1
```

Maximum declared version of KYVP_BOARD_INFO.

#### 5.1.1.1.21 MAX_PCI_INTERFACE_LINKS

```
8
```

Maximum number of interface links represented in device topology arrays.

#### 5.1.1.1.22 KYVP_PCI_INTERFACE_INFO_VERSION

```
1
```

Current version of KYVP_PCI_INTERFACE_INFO.

#### 5.1.1.1.23 KY_MAX_PCI_INTERFACE_INFO_STRING_SIZE

```
256
```

Maximum interface information string length, excluding the terminator.

#### 5.1.1.1.24 KY_MAX_DEVICE_INFO_STRING_SIZE

```
64
```

Maximum device information string length, excluding the terminator.

#### 5.1.1.1.25 KYVP_DEVICE_INFO_VERSION

```
1
```

Declared version of KYVP_DEVICE_INFO.

#### 5.1.1.1.26 KYVP_DEVICE_IMAGE_DETAILS_VERSION

1

Current version of KYVP_DEVICE_IMAGE_DETAILS.

#### 5.1.1.1.27 KYVP_BUFFER_INFO_VERSION

1

Current version of KYVP_BUFFER_INFO.

### 5.1.1.2 Enumerations

#### 5.1.1.2.1 _KYVP_PCIINTERFACE_EVENT_ID

PCI interface event identifiers.

##### *5.1.1.2.1.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_PCIINTERFACE_EVENT_PROBE_ID | -1 | Reserved for internal event delivery tests. |
| KYVP_PCIINTERFACE_EVENT_TEMPERATURE_ID | 0x02 | The PCI interface temperature threshold state changed. |
| KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_ID | 0x03 | A CoaXPress 2.0 heartbeat packet was received from a connected device. |
| KYVP_PCIINTERFACE_EVENT_CXP2_EVENT_ID | 0x04 | A CoaXPress 2.0 event packet was received from a connected device. |
| KYVP_PCIINTERFACE_EVENT_INVALID | 0xFF | Invalid or unsupported PCI interface event identifier. |

#### 5.1.1.2.2 _KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID

PCI interface temperature threshold states.

##### *5.1.1.2.2.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_NORMAL | 0x00 | Normal temperature state, outside the warning and critical states. |
| KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_WARNING | 0x01 | Warning temperature state, as determined by the configured thresholds. |
| KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_CRITICAL | 0x02 | Critical temperature state, as determined by the configured thresholds. |
| KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_INVALID | 0xFF | Invalid or undefined temperature threshold state. |

#### 5.1.1.2.3 _KYVP_DEVICE_EVENT_ID

Device event identifiers.

##### *5.1.1.2.3.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_DEVICE_EVENT_PROBE_ID | -1 | Reserved for internal event delivery tests. |
| KYVP_DEVICE_EVENT_CONNECTION_LOST_ID | 0x02 | The connection to the device was lost. |
| KYVP_DEVICE_EVENT_INVALID | 0xFF | Invalid or unsupported device event identifier. |

#### 5.1.1.2.4 _KYVP_TAP_GEOMETRY

Supported image sensor tap geometries.

#### *5.1.1.2.4.1 Values*

| Name | Value | Description |
|---|---|---|
| TAP_GEOMETRY_1X_1Y | 0x00 | One horizontal region and one vertical region, read from top to bottom. |
| TAP_GEOMETRY_1X_2YE | 0x41 | One horizontal region and two vertical regions, read from the top and bottom toward the center. |
| TAP_GEOMETRY_INVALID | 0xFF | Invalid or unsupported tap geometry. |

### 5.1.1.2.5 _KYVP_CXP_LINK_SPEED

CoaXPress link speed encodings; rates are per connection.

#### *5.1.1.2.5.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_CXP_LINK_SPEED_CXP0 | 0 | No link speed selected or known; SDK sentinel, not a CoaXPress transmission rate. |
| KYVP_CXP_LINK_SPEED_CXP1 | 0x28 | CXP-1: 1.25 Gbit/s per connection. |
| KYVP_CXP_LINK_SPEED_CXP2 | 0x30 | CXP-2: 2.5 Gbit/s per connection. |
| KYVP_CXP_LINK_SPEED_CXP3 | 0x38 | CXP-3: 3.125 Gbit/s per connection. |
| KYVP_CXP_LINK_SPEED_CXP5 | 0x40 | CXP-5: 5 Gbit/s per connection. |
| KYVP_CXP_LINK_SPEED_CXP6 | 0x48 | CXP-6: 6.25 Gbit/s per connection. |
| KYVP_CXP_LINK_SPEED_CXP10 | 0x50 | CXP-10: 10 Gbit/s per connection. |
| KYVP_CXP_LINK_SPEED_CXP12 | 0x58 | CXP-12: 12.5 Gbit/s per connection. |

### 5.1.1.2.6 _KYVP_PCI_INTERFACE_PROTOCOL

Transport protocol reported by a PCI interface.

#### *5.1.1.2.6.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_PCI_INTERFACE_PROTOCOL_CoaXPress | 0x0 | CoaXPress transport. |
| KYVP_PCI_INTERFACE_PROTOCOL_CLHS | 0x1 | Camera Link HS transport. |
| KYVP_PCI_INTERFACE_PROTOCOL_PCI | 0x2 | PCI transport. |
| KYVP_PCI_INTERFACE_PROTOCOL_Mixed | 0xFF | Mixed transport protocol designation. |

## 5.1.1.3 Type definitions

### 5.1.1.3.1 KYVP_PCIINTERFACE_EVENT_ID

```
typedef enum _KYVP_PCIINTERFACE_EVENT_ID KYVP_PCIINTERFACE_EVENT_ID
```

PCI interface event identifiers.

### 5.1.1.3.2 KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID

```
typedef enum _KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID
```

PCI interface temperature threshold states.

### 5.1.1.3.3 KYVP_DEVICE_EVENT_ID

```
typedef enum _KYVP_DEVICE_EVENT_ID KYVP_DEVICE_EVENT_ID
```

Device event identifiers.

### 5.1.1.3.4 KYVP_TAP_GEOMETRY

```
typedef enum _KYVP_TAP_GEOMETRY KYVP_TAP_GEOMETRY
```

Supported image sensor tap geometries.

### 5.1.1.3.5 KYVP_CXP_LINK_SPEED

```
typedef enum _KYVP_CXP_LINK_SPEED KYVP_CXP_LINK_SPEED
```

CoaXPress link speed encodings; rates are per connection.

### 5.1.1.3.6 KYVP_PCI_INTERFACE_PROTOCOL

```
typedef enum _KYVP_PCI_INTERFACE_PROTOCOL KYVP_PCI_INTERFACE_PROTOCOL
```

Transport protocol reported by a PCI interface.

#### 5.1.1.4 KYVPDefines Structs

Field reference for auxiliary data, event payloads, and SDK information.

Each table follows declaration order and lists the exact public field types, including array bounds, bit widths, and named union members.

C++ member defaults initialize only the fields shown in the declarations. C callers must initialize input structures explicitly and set their version fields as documented. Library-owned information and callback payloads are populated by the SDK; treat returned information as read-only.

##### 5.1.1.4.1 KYVP_IO_AUX_DATA

I/O controller payload carried in KYVP_AUX_DATA. The SDK populates version with 1. Timestamps are in nanoseconds.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Current version: 1. |
| masked_data | uint64_t | I/O controller event-state mask. | See the bit ranges below. |
| timestamp | uint64_t | I/O event timestamp. | Nanoseconds. |

Within each range, the lowest bit corresponds to signal 0.

| Bits | Signals |
|---|---|
| 0-7 | Optocoupled inputs 0-7 |
| 8-11 | LVDS inputs 0-3 |
| 12-19 | TTL lines 0-7 |
| 20-27 | LVTTL lines 0-7 |
| 28-35 | Optocoupled outputs 0-7 |
| 36-39 | LVDS outputs 0-3 |
| 40-43 | Encoders 0-3 |
| 44-51 | Timers 0-7 |
| 52-59 | Camera triggers 0-7 |
| 60-63 | Acquisition triggers 0-3 |

Use KYVP_IO_CONTROLLER_MASKED_IO, KYVP_IO_CONTROLLER_MASKED_ENCODERS, KYVP_IO_CONTROLLER_MASKED_TIMERS, KYVP_IO_CONTROLLER_MASKED_CAMERA_TRIGGERS, and KYVP_IO_CONTROLLER_MASKED_TRIGGERS to extract the corresponding groups.

##### 5.1.1.4.2 KYVP_FRAME_AUX_DATA

Frame-arrival payload reserved for internal use. The fields following version belong to structure version 1.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Version 1 layout. |
| sequence_number | uint32_t | Sequential frame index within the allocated frame buffer. | |
| timestamp | uint64_t | Frame-arrival timestamp. | Nanoseconds. |
| reserved | uint32_t | Reserved for internal use. | |

##### 5.1.1.4.3 KYVP_AUX_DATA

Auxiliary message delivered by the SDK. The SDK populates version with 1. Access payloads through the named u_data union.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint8_t | Structure version. | Current version: 1. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| messageID | uint32_t | Identifies the auxiliary message and its payload. | KYVP_AUX_MESSAGE_ID_IO_CONTROLLER for I/O data. |
| u_data | union | Message payload. | Select the member using messageID . |
| u_data.io_data | KYVP_IO_AUX_DATA | I/O controller payload. | Used for KYVP_AUX_MESSAGE_ID_IO_CONTROLLER. |
| u_data.frame_data | KYVP_FRAME_AUX_DATA | Frame-arrival payload. | Reserved for internal use. |
| m_InterruptTimestampHw | uint64_t | Hardware timestamp captured for the interrupt. | Nanoseconds. |
| m_InterruptTimestampChrono | uint64_t | Host clock timestamp captured for the interrupt. | Nanoseconds; a separate clock from the hardware timestamp. |

#### 5.1.1.4.4 KYVP_CXP2_HEARTBEAT

Payload of a received CoaXPress 2.0 heartbeat packet. The SDK sets version to KYVP_CXP2_HEARTBEAT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | |
| uMasterHostConnectionID | uint32_t | Host connection ID associated with the device master connection. | Value received in the heartbeat packet. |
| uDeviceTime | uint64_t | Time reported by the remote device in the heartbeat packet. | Nanoseconds. |

#### 5.1.1.4.5 KYVP_CXP2_EVENT

Payload of a received CoaXPress 2.0 event packet. The SDK sets version to KYVP_CXP2_EVENT_VERSION. uDataSize counts 32-bit words, while KY_CXP_EVENT_MAX_DATA_SIZE is in bytes. The payload array holds at most 256 words (1024 bytes).

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | |
| uMasterHostConnectionID | uint32_t | Host connection ID associated with the device master connection. | Value received in the event packet. |
| uTag | uint8_t | Packet tag incremented for each new event packet. | 8-bit value. |
| uDataSize | uint16_t | Number of 32-bit event data words reported by the packet. | Validate against the array capacity before reading. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uDataWordsArr | uint32_t [KY_CXP_EVENT_MAX_DATA_SIZE / 4] | Event data words containing one or more event messages. | Read at most uDataSize words, bounded by the array capacity. |

#### 5.1.1.4.6 KYVP_BUFFER_DATA_CHUNK

Describes one memory chunk of a buffer.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pBufferChunkMemory | void* | Address of the memory chunk. | |
| bufferChunkSize | size_t | Size of the memory chunk. | Bytes. |

#### 5.1.1.4.7 KYVP_PCI_INTERFACE_SLOT

PCI bus, slot, and function identifying an interface.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uBus | uint8_t : 8 | PCI bus number. | 8-bit field. |
| m_uSlot | uint8_t : 5 | PCI slot number. | 5-bit field. |
| m_uFunction | uint8_t : 3 | PCI function number. | 3-bit field; 7 denotes a software-emulated interface in this SDK. |

#### 5.1.1.4.8 KYVP_PCI_CONFIG

PCI Express generation and lane configuration. This structure has no version member.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uGeneration | uint8_t | PCI Express generation number. | C++ default: UINT8_MAX. |
| m_uLanes | uint8_t | Number of PCI Express lanes. | C++ default: UINT8_MAX. |

#### 5.1.1.4.9 KYVP_BOARD_INFO

Board identity, PCI configuration, and stream capabilities. The current layout version is KYVP_MAX_BOARD_INFO_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uVersion | uint8_t | Structure version. | |
| m_Slot | KYVP_PCI_INTERFACE_SLOT | PCI bus, slot, and function. | |
| m_uDevicePID | uint32_t | Board product identifier. | |
| m_Protocol | KYVP_PCI_INTERFACE_PROTOCOL | Transport protocol supported by the interface. | |
| m_uDeviceGeneration | uint8_t | Product generation number. | 1, 2, 3 correspond to generations I, II, III. |
| m_uRevision | uint8_t | Board hardware revision number. | |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uPhysicalLinks | uint32_t | Number of physical interface links. | |
| m_ActivePCIConfig | KYVP_PCI_CONFIG | Current PCI Express connection configuration. | |
| m_MinimalPCIConfig | KYVP_PCI_CONFIG | Minimum PCI Express configuration required by the board. | |
| m_uFlags | uint64_t | Supported stream directions as a bit mask. | KYVP_PCI_INTERFACE_STREAM_GRABBER and KYVP_PCI_INTERFACE_STREAM_GENERATOR. |

#### 5.1.1.4.10 KYVP_PCI_INTERFACE_INFO

Library-owned information describing a PCI interface. The current layout version is KYVP_PCI_INTERFACE_INFO_VERSION. Treat information returned by the SDK as read-only.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uVersion | uint8_t | Structure version. | |
| m_szDeviceDisplayName | char [KY_MAX_PCI_INTERFACE_INFO_STRING_SIZE + 1] | Human-readable interface display name. | Capacity includes one character for the terminator. |
| m_uSerialNumber | uint32_t | Interface serial number. | |
| m_KYVP_BoardInfo | KYVP_BOARD_INFO | Board identity, PCI configuration, and stream capabilities. | |
| m_bIsVirtual | KY_BOOL | Whether the interface is software-emulated. | KY_TRUE for a virtual interface. |

#### 5.1.1.4.11 KYVP_DEVICE_INFO

Library-owned information describing a detected camera or device. The SDK stores these records in shared memory to coordinate processes using an interface. Treat returned records as read-only. The declared layout version is KYVP_DEVICE_INFO_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uVersion | uint32_t | Structure version supplied by the SDK. | |
| m_uMasterLink | uint8_t | Host link connected to the device master link. | Zero-based host link index. |
| m_ProcId | KY_PROCID | ID of the process recorded as owning the device. | |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uLinkMask | uint8_t | Mask of host links connected to the device. | Bit n corresponds to host link n. |
| m_nMaxDeviceLinks | int | Maximum number of device links. | |
| m_vecDeviceConnectionTopology | int [MAX_PCI_INTERFACE_LINKS] | Maps each device link index to its connected host link index. | -1 for an unmapped link in SDK-populated records. |
| m_linkSpeed | KYVP_CXP_LINK_SPEED | Detected CoaXPress link speed. | |
| m_uStreamId | uint32_t [256] | Device stream IDs indexed by image stream. | An entry is valid only when its m_uValidStreamId entry is true. |
| m_uValidStreamId | KY_BOOL [256] | Validity flags for the corresponding m_uStreamId entries. | |
| m_szDeviceVersion | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Device version string. | Capacity includes one character for the terminator. |
| m_szDeviceVendorName | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Device vendor name. | Capacity includes one character for the terminator. |
| m_szDeviceManufacturerInfo | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Additional manufacturer information. | Capacity includes one character for the terminator. |
| m_szDeviceModelName | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Device model name. | Capacity includes one character for the terminator. |
| m_szDeviceID | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Device identifier string. | Capacity includes one character for the terminator. |
| m_szDeviceUserID | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | User-defined device identifier. | Capacity includes one character for the terminator. |
| m_szDeviceFirmwareVersion | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Device firmware version string. | Capacity includes one character for the terminator. |
| m_bOutputCamera | KY_BOOL | Whether the device represents an output camera. | KY_TRUE for output-camera mode. |
| m_bVirtualCamera | KY_BOOL | Whether the camera is virtual. | Used by custom firmware implementations. |
| m_uLinkConfig | uint32_t | Saved master-link configuration. | Used internally when restoring simulated device detection. |

#### 5.1.1.4.12 KYVP_DEVICE_IMAGE_DETAILS

Image dimensions, pixel format, and tap geometry of a device. Initialize m_uVersion to KYVP_DEVICE_IMAGE_DETAILS_VERSION for caller-provided instances.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uVersion | uint32_t | Structure version. | |
| m_uWidth | uint32_t | Image width. | Pixels. |
| m_uHeight | uint32_t | Image height. | Pixels. |
| m_uPixelFormat | uint32_t | Numeric pixel format identifier. | |
| m_szPixelFormatName | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Symbolic pixel format name. | Capacity includes one character for the terminator. |
| m_szTapGeometryType | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Symbolic tap geometry name. | Capacity includes one character for the terminator. |

#### 5.1.1.4.13 KYVP_BUFFER_INFO

Library-owned bundle of pointers to buffer data and metadata. The current layout version is KYVP_BUFFER_INFO_VERSION. Metadata pointers refer to SDK-owned values, not copied snapshots. Read them while the buffer is valid and available to the application; acquisition can update the values. Do not free SDK-owned metadata or retain its pointers after the buffer is revoked. m_pBase addresses payload memory; m_pContext is the application context pointer.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_uVersion | uint32_t | Structure version supplied by the SDK. | |
| m_puFrameId | const uint64_t* | Pointer to the acquired frame identifier. | |
| m_puBufferId | const uint64_t* | Pointer to the SDK buffer identifier. | |
| m_puTimestampNS | const uint64_t* | Pointer to the frame timestamp. | Nanoseconds. |
| m_puInstantFPS | const double* | Pointer to the instantaneous acquisition rate. | Frames per second. |
| m_pBase | const void* | Base address of the buffer payload memory. | Direct address, not a pointer to a pointer. |
| m_puSize | const size_t* | Pointer to the allocated or announced buffer size. | Bytes. |
| m_pContext | const void* | Application context associated with the announced buffer. | Returned as supplied by the application. |
| m_puWidth | const size_t* | Pointer to the image width. | Pixels. |
| m_puHeight | const size_t* | Pointer to the image height. | Pixels. |
| m_puPixelFormat | const uint64_t* | Pointer to the numeric pixel format identifier. | |
| m_puTimestampNS_HW | const uint64_t* | Pointer to the hardware interrupt timestamp. | Nanoseconds. |
| m_puTimestampNS_CHRONO | const uint64_t* | Pointer to the host clock interrupt timestamp. | Nanoseconds; a separate clock from the hardware timestamp. |

## 5.2 Foundation types

KYFoundation_Types.h provides KY_BOOL, KY_TRUE, KY_FALSE, platform-dependent handle and identifier aliases, and integer formatting macros used throughout the SDK.

### 5.2.1 KYFoundation_Types.h API reference

The public aliases describe native module, procedure, thread, and process identifiers. Their representation depends on the target platform. KY_BOOL has a fixed eight-bit representation; use KY_TRUE and KY_FALSE for its values. See Foundation Types for type mappings and formatting requirements.

#### 5.2.1.1 Macros

##### 5.2.1.1.1 __STDC_LIMIT_MACROS

Request integer limit macros from compatible C++ standard-library headers.

Defined on non-Windows targets when not already provided.

##### 5.2.1.1.2 KY_MEMBER_INIT

Preserve a member-initializer expression in C++; omit it in C.

| Parameter | Description |
|---|---|
| exp | Complete initializer tokens, including the assignment operator when needed. |

##### 5.2.1.1.3 KY_FUNC

```
__FUNCTION__
```

Compiler-provided name of the current function for diagnostics.

Use within a function body.

##### 5.2.1.1.4 KY_TRUE

```
1
```

SDK Boolean true value, equal to one.

##### 5.2.1.1.5 KY_FALSE

```
0
```

SDK Boolean false value, equal to zero.

##### 5.2.1.1.6 INT64_MAX

```
0x7fffffffffffffffLL /* maximum signed int64 value */
```

Fallback maximum value for a signed 64-bit integer.

Used only when the standard integer-limit macro is unavailable.

##### 5.2.1.1.7 INT64_MIN

```
0x8000000000000000LL /* minimum signed int64 value */
```

Fallback constant representing the minimum signed 64-bit value.

Used only when the standard integer-limit macro is unavailable.

##### 5.2.1.1.8 __PRI64_PREFIX

```
"ll"
```

##### 5.2.1.1.9 __PRIPTR_PREFIX

##### 5.2.1.1.10 PRId8

```
"d"
```

printf signed decimal fragment for int8_t.

##### 5.2.1.1.11 PRId16

```
"d"
```

printf signed decimal fragment for int16_t.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

#### 5.2.1.1.12 PRId32

`"d"`

printf signed decimal fragment for int32_t.

#### 5.2.1.1.13 PRId64

`__PRI64_PREFIX "d"`

printf signed decimal fragment for int64_t.

#### 5.2.1.1.14 PRIdLEAST8

`"d"`

printf signed decimal fragment for int_least8_t.

#### 5.2.1.1.15 PRIdLEAST16

`"d"`

printf signed decimal fragment for int_least16_t.

#### 5.2.1.1.16 PRIdLEAST32

`"d"`

printf signed decimal fragment for int_least32_t.

#### 5.2.1.1.17 PRIdLEAST64

`__PRI64_PREFIX "d"`

printf signed decimal fragment for int_least64_t.

#### 5.2.1.1.18 PRIdFAST8

`"d"`

printf signed decimal fragment for int_fast8_t.

#### 5.2.1.1.19 PRIdFAST16

`__PRIPTR_PREFIX "d"`

printf signed decimal fragment for int_fast16_t.

#### 5.2.1.1.20 PRIdFAST32

`__PRIPTR_PREFIX "d"`

printf signed decimal fragment for int_fast32_t.

#### 5.2.1.1.21 PRIdFAST64

`__PRI64_PREFIX "d"`

printf signed decimal fragment for int_fast64_t.

#### 5.2.1.1.22 PRIi8

`"i"`

printf signed decimal fragment for int8_t.

#### 5.2.1.1.23 PRIi16

`"i"`

printf signed decimal fragment for int16_t.

#### 5.2.1.1.24 PRIi32

`"i"`

printf signed decimal fragment for int32_t.

#### 5.2.1.1.25 PRIi64

`__PRI64_PREFIX "i"`

printf signed decimal fragment for int64_t.

#### 5.2.1.1.26 PRIiLEAST8

`"i"`

printf signed decimal fragment for int_least8_t.

#### 5.2.1.1.27 PRIiLEAST16

`"i"`

printf signed decimal fragment for int_least16_t.

#### 5.2.1.1.28 PRIiLEAST32

`"i"`

printf signed decimal fragment for int_least32_t.

#### 5.2.1.1.29 PRIiLEAST64

`__PRI64_PREFIX "i"`

printf signed decimal fragment for int_least64_t.

#### 5.2.1.1.30 PRIiFAST8

`"i"`

printf signed decimal fragment for int_fast8_t.

#### 5.2.1.1.31 PRIiFAST16

`__PRIPTR_PREFIX "i"`

printf signed decimal fragment for int_fast16_t.

#### 5.2.1.1.32 PRIiFAST32

`__PRIPTR_PREFIX "i"`

printf signed decimal fragment for int_fast32_t.

#### 5.2.1.1.33 PRIiFAST64

`__PRI64_PREFIX "i"`

printf signed decimal fragment for int_fast64_t.

#### 5.2.1.1.34 PRIo8

`"o"`

printf octal fragment for uint8_t.

#### 5.2.1.1.35 PRIo16

`"o"`

printf octal fragment for uint16_t.

#### 5.2.1.1.36 PRIo32

`"o"`

printf octal fragment for uint32_t.

#### 5.2.1.1.37 PRIo64

`__PRI64_PREFIX "o"`

printf octal fragment for uint64_t.

#### 5.2.1.1.38 PRIoLEAST8

`"o"`

printf octal fragment for uint_least8_t.

#### 5.2.1.1.39 PRIoLEAST16

`"o"`

printf octal fragment for uint_least16_t.

#### 5.2.1.1.40 PRIoLEAST32

`"o"`

printf octal fragment for uint_least32_t.

#### 5.2.1.1.41 PRIoLEAST64

`__PRI64_PREFIX "o"`

printf octal fragment for uint_least64_t.

#### 5.2.1.1.42 PRIoFAST8

"o"

printf octal fragment for uint_fast8_t.

#### 5.2.1.1.43 PRIoFAST16

__PRIPTR_PREFIX "o"

printf octal fragment for uint_fast16_t.

#### 5.2.1.1.44 PRIoFAST32

__PRIPTR_PREFIX "o"

printf octal fragment for uint_fast32_t.

#### 5.2.1.1.45 PRIoFAST64

__PRI64_PREFIX "o"

printf octal fragment for uint_fast64_t.

#### 5.2.1.1.46 PRIu8

"u"

printf unsigned decimal fragment for uint8_t.

#### 5.2.1.1.47 PRIu16

"u"

printf unsigned decimal fragment for uint16_t.

#### 5.2.1.1.48 PRIu32

"u"

printf unsigned decimal fragment for uint32_t.

#### 5.2.1.1.49 PRIu64

__PRI64_PREFIX "u"

printf unsigned decimal fragment for uint64_t.

#### 5.2.1.1.50 PRIuLEAST8

"u"

printf unsigned decimal fragment for uint_least8_t.

#### 5.2.1.1.51 PRIuLEAST16

"u"

printf unsigned decimal fragment for uint_least16_t.

#### 5.2.1.1.52 PRIuLEAST32

"u"

printf unsigned decimal fragment for uint_least32_t.

#### 5.2.1.1.53 PRIuLEAST64

__PRI64_PREFIX "u"

printf unsigned decimal fragment for uint_least64_t.

#### 5.2.1.1.54 PRIuFAST8

"u"

printf unsigned decimal fragment for uint_fast8_t.

#### 5.2.1.1.55 PRIuFAST16

__PRIPTR_PREFIX "u"

printf unsigned decimal fragment for uint_fast16_t.

#### 5.2.1.1.56 PRIuFAST32

__PRIPTR_PREFIX "u"

printf unsigned decimal fragment for uint_fast32_t.

#### 5.2.1.1.57 PRIuFAST64

```
__PRI64_PREFIX "u"
```

printf unsigned decimal fragment for uint_fast64_t.

#### 5.2.1.1.58 PRIx8

```
"x"
```

printf lowercase hexadecimal fragment for uint8_t.

#### 5.2.1.1.59 PRIx16

```
"x"
```

printf lowercase hexadecimal fragment for uint16_t.

#### 5.2.1.1.60 PRIx32

```
"x"
```

printf lowercase hexadecimal fragment for uint32_t.

#### 5.2.1.1.61 PRIx64

```
__PRI64_PREFIX "x"
```

printf lowercase hexadecimal fragment for uint64_t.

#### 5.2.1.1.62 PRIxLEAST8

```
"x"
```

printf lowercase hexadecimal fragment for uint_least8_t.

#### 5.2.1.1.63 PRIxLEAST16

```
"x"
```

printf lowercase hexadecimal fragment for uint_least16_t.

#### 5.2.1.1.64 PRIxLEAST32

```
"x"
```

printf lowercase hexadecimal fragment for uint_least32_t.

#### 5.2.1.1.65 PRIxLEAST64

```
__PRI64_PREFIX "x"
```

printf lowercase hexadecimal fragment for uint_least64_t.

#### 5.2.1.1.66 PRIxFAST8

```
"x"
```

printf lowercase hexadecimal fragment for uint_fast8_t.

#### 5.2.1.1.67 PRIxFAST16

```
__PRIPTR_PREFIX "x"
```

printf lowercase hexadecimal fragment for uint_fast16_t.

#### 5.2.1.1.68 PRIxFAST32

```
__PRIPTR_PREFIX "x"
```

printf lowercase hexadecimal fragment for uint_fast32_t.

#### 5.2.1.1.69 PRIxFAST64

```
__PRI64_PREFIX "x"
```

printf lowercase hexadecimal fragment for uint_fast64_t.

#### 5.2.1.1.70 PRIX8

```
"X"
```

printf uppercase hexadecimal fragment for uint8_t.

#### 5.2.1.1.71 PRIX16

```
"X"
```

printf uppercase hexadecimal fragment for uint16_t.

#### 5.2.1.1.72 PRIX32

```
"X"
```

printf uppercase hexadecimal fragment for uint32_t.

#### 5.2.1.1.73 PRIX64

```
__PRI64_PREFIX "X"
```

printf uppercase hexadecimal fragment for uint64_t.

#### 5.2.1.1.74 PRIXLEAST8

```
"X"
```

printf uppercase hexadecimal fragment for uint_least8_t.

#### 5.2.1.1.75 PRIXLEAST16

```
"X"
```

printf uppercase hexadecimal fragment for uint_least16_t.

#### 5.2.1.1.76 PRIXLEAST32

```
"X"
```

printf uppercase hexadecimal fragment for uint_least32_t.

#### 5.2.1.1.77 PRIXLEAST64

```
__PRI64_PREFIX "X"
```

printf uppercase hexadecimal fragment for uint_least64_t.

#### 5.2.1.1.78 PRIXFAST8

```
"X"
```

printf uppercase hexadecimal fragment for uint_fast8_t.

#### 5.2.1.1.79 PRIXFAST16

```
__PRIPTR_PREFIX "X"
```

printf uppercase hexadecimal fragment for uint_fast16_t.

#### 5.2.1.1.80 PRIXFAST32

```
__PRIPTR_PREFIX "X"
```

printf uppercase hexadecimal fragment for uint_fast32_t.

#### 5.2.1.1.81 PRIXFAST64

```
__PRI64_PREFIX "X"
```

printf uppercase hexadecimal fragment for uint_fast64_t.

#### 5.2.1.1.82 PRIdMAX

```
__PRI64_PREFIX "d"
```

printf signed decimal fragment for intmax_t.

#### 5.2.1.1.83 PRIiMAX

```
__PRI64_PREFIX "i"
```

printf signed decimal fragment for intmax_t.

#### 5.2.1.1.84 PRIoMAX

```
__PRI64_PREFIX "o"
```

printf octal fragment for uintmax_t.

#### 5.2.1.1.85 PRIuMAX

```
__PRI64_PREFIX "u"
```

printf unsigned decimal fragment for uintmax_t.

#### 5.2.1.1.86 PRIxMAX

```
__PRI64_PREFIX "x"
```

printf lowercase hexadecimal fragment for uintmax_t.

#### 5.2.1.1.87 PRIXMAX

```
__PRI64_PREFIX "X"
```

printf uppercase hexadecimal fragment for uintmax_t.

#### 5.2.1.1.88 PRIdPTR

```
__PRIPTR_PREFIX "d"
```

printf signed decimal fragment for intptr_t.

#### 5.2.1.1.89 PRIiPTR

```
__PRIPTR_PREFIX "i"
```

printf signed decimal fragment for intptr_t.

#### 5.2.1.1.90 PRIoPTR

```
__PRIPTR_PREFIX "o"
```

printf octal fragment for uintptr_t.

#### 5.2.1.1.91 PRIuPTR

```
__PRIPTR_PREFIX "u"
```

printf unsigned decimal fragment for uintptr_t.

#### 5.2.1.1.92 PRIxPTR

```
__PRIPTR_PREFIX "x"
```

printf lowercase hexadecimal fragment for uintptr_t.

#### 5.2.1.1.93 PRIXPTR

```
__PRIPTR_PREFIX "X"
```

printf uppercase hexadecimal fragment for uintptr_t.

#### 5.2.1.1.94 SCNd8

```
"hhd"
```

scanf signed decimal fragment for a pointer to int8_t.

#### 5.2.1.1.95 SCNd16

```
"hd"
```

scanf signed decimal fragment for a pointer to int16_t.

#### 5.2.1.1.96 SCNd32

```
"d"
```

scanf signed decimal fragment for a pointer to int32_t.

#### 5.2.1.1.97 SCNd64

```
__PRI64_PREFIX "d"
```

scanf signed decimal fragment for a pointer to int64_t.

#### 5.2.1.1.98 SCNdLEAST8

```
"hhd"
```

scanf signed decimal fragment for a pointer to int_least8_t.

#### 5.2.1.1.99 SCNdLEAST16

```
"hd"
```

scanf signed decimal fragment for a pointer to int_least16_t.

#### 5.2.1.1.100SCNdLEAST32

```
"d"
```

scanf signed decimal fragment for a pointer to int_least32_t.

#### 5.2.1.1.101SCNdLEAST64

```
__PRI64_PREFIX "d"
```

scanf signed decimal fragment for a pointer to int_least64_t.

#### 5.2.1.1.102 SCNdFAST8

```
"hhd"
```

scanf signed decimal fragment for a pointer to int_fast8_t.

#### 5.2.1.1.103 SCNdFAST16

```
__PRIPTR_PREFIX "d"
```

scanf signed decimal fragment for a pointer to int_fast16_t.

#### 5.2.1.1.104 SCNdFAST32

```
__PRIPTR_PREFIX "d"
```

scanf signed decimal fragment for a pointer to int_fast32_t.

#### 5.2.1.1.105 SCNdFAST64

```
__PRI64_PREFIX "d"
```

scanf signed decimal fragment for a pointer to int_fast64_t.

#### 5.2.1.1.106 SCNi8

```
"hhi"
```

scanf base-detected signed integer fragment for a pointer to int8_t.

#### 5.2.1.1.107 SCNi16

```
"hi"
```

scanf base-detected signed integer fragment for a pointer to int16_t.

#### 5.2.1.1.108 SCNi32

```
"i"
```

scanf base-detected signed integer fragment for a pointer to int32_t.

#### 5.2.1.1.109 SCNi64

```
__PRI64_PREFIX "i"
```

scanf base-detected signed integer fragment for a pointer to int64_t.

#### 5.2.1.1.110 SCNiLEAST8

```
"hhi"
```

scanf base-detected signed integer fragment for a pointer to int_least8_t.

#### 5.2.1.1.111 SCNiLEAST16

```
"hi"
```

scanf base-detected signed integer fragment for a pointer to int_least16_t.

#### 5.2.1.1.112 SCNiLEAST32

```
"i"
```

scanf base-detected signed integer fragment for a pointer to int_least32_t.

#### 5.2.1.1.113 SCNiLEAST64

```
__PRI64_PREFIX "i"
```

scanf base-detected signed integer fragment for a pointer to int_least64_t.

#### 5.2.1.1.114 SCNiFAST8

```
"hhi"
```

scanf base-detected signed integer fragment for a pointer to int_fast8_t.

#### 5.2.1.1.115 SCNiFAST16

```
__PRIPTR_PREFIX "i"
```

scanf base-detected signed integer fragment for a pointer to int_fast16_t.

#### 5.2.1.1.116 SCNiFAST32

```
__PRIPTR_PREFIX "i"
```

scanf base-detected signed integer fragment for a pointer to int_fast32_t.

#### 5.2.1.1.117 SCNiFAST64

```
__PRI64_PREFIX "i"
```

scanf base-detected signed integer fragment for a pointer to int_fast64_t.

#### 5.2.1.1.118 SCNu8

```
"hhu"
```

scanf unsigned decimal fragment for a pointer to uint8_t.

#### 5.2.1.1.119 SCNu16

```
"hu"
```

scanf unsigned decimal fragment for a pointer to uint16_t.

#### 5.2.1.1.120 SCNu32

```
"u"
```

scanf unsigned decimal fragment for a pointer to uint32_t.

#### 5.2.1.1.121 SCNu64

```
__PRI64_PREFIX "u"
```

scanf unsigned decimal fragment for a pointer to uint64_t.

#### 5.2.1.1.122 SCNuLEAST8

```
"hhu"
```

scanf unsigned decimal fragment for a pointer to uint_least8_t.

#### 5.2.1.1.123 SCNuLEAST16

```
"hu"
```

scanf unsigned decimal fragment for a pointer to uint_least16_t.

#### 5.2.1.1.124 SCNuLEAST32

```
"u"
```

scanf unsigned decimal fragment for a pointer to uint_least32_t.

#### 5.2.1.1.125 SCNuLEAST64

```
__PRI64_PREFIX "u"
```

scanf unsigned decimal fragment for a pointer to uint_least64_t.

#### 5.2.1.1.126 SCNuFAST8

```
"hhu"
```

scanf unsigned decimal fragment for a pointer to uint_fast8_t.

#### 5.2.1.1.127 SCNuFAST16

```
__PRIPTR_PREFIX "u"
```

scanf unsigned decimal fragment for a pointer to uint_fast16_t.

#### 5.2.1.1.128 SCNuFAST32

```
__PRIPTR_PREFIX "u"
```

scanf unsigned decimal fragment for a pointer to uint_fast32_t.

#### 5.2.1.1.129 SCNuFAST64

```
__PRI64_PREFIX "u"
```

scanf unsigned decimal fragment for a pointer to uint_fast64_t.

#### 5.2.1.1.130 SCNo8

```
"hho"
```

scanf octal fragment for a pointer to uint8_t.

#### 5.2.1.1.131 SCNo16

```
"ho"
```

scanf octal fragment for a pointer to uint16_t.

#### 5.2.1.1.132 SCNo32

```
"o"
```

scanf octal fragment for a pointer to uint32_t.

#### 5.2.1.1.133 SCNo64

```
__PRI64_PREFIX "o"
```

scanf octal fragment for a pointer to uint64_t.

#### 5.2.1.1.134 SCNoLEAST8

```
"hho"
```

scanf octal fragment for a pointer to uint_least8_t.

#### 5.2.1.1.135 SCNoLEAST16

```
"ho"
```

scanf octal fragment for a pointer to uint_least16_t.

#### 5.2.1.1.136 SCNoLEAST32

```
"o"
```

scanf octal fragment for a pointer to uint_least32_t.

#### 5.2.1.1.137 SCNoLEAST64

```
__PRI64_PREFIX "o"
```

scanf octal fragment for a pointer to uint_least64_t.

#### 5.2.1.1.138 SCNoFAST8

```
"hho"
```

scanf octal fragment for a pointer to uint_fast8_t.

#### 5.2.1.1.139 SCNoFAST16

```
__PRIPTR_PREFIX "o"
```

scanf octal fragment for a pointer to uint_fast16_t.

#### 5.2.1.1.140 SCNoFAST32

```
__PRIPTR_PREFIX "o"
```

scanf octal fragment for a pointer to uint_fast32_t.

#### 5.2.1.1.141 SCNoFAST64

```
__PRI64_PREFIX "o"
```

scanf octal fragment for a pointer to uint_fast64_t.

#### 5.2.1.1.142 SCNx8

```
"hhx"
```

scanf lowercase hexadecimal fragment for a pointer to uint8_t.

#### 5.2.1.1.143 SCNx16

```
"hx"
```

scanf lowercase hexadecimal fragment for a pointer to uint16_t.

#### 5.2.1.1.144 SCNx32

```
"x"
```

scanf lowercase hexadecimal fragment for a pointer to uint32_t.

#### 5.2.1.1.145 SCNx64

```
__PRI64_PREFIX "x"
```

scanf lowercase hexadecimal fragment for a pointer to uint64_t.

#### 5.2.1.1.146 SCNxLEAST8

```
"hhx"
```

scanf lowercase hexadecimal fragment for a pointer to uint_least8_t.

#### 5.2.1.1.147 SCNxLEAST16

```
"hx"
```

scanf lowercase hexadecimal fragment for a pointer to uint_least16_t.

#### 5.2.1.1.148 SCNxLEAST32

```
"x"
```

scanf lowercase hexadecimal fragment for a pointer to uint_least32_t.

#### 5.2.1.1.149 SCNxLEAST64

```
__PRI64_PREFIX "x"
```

scanf lowercase hexadecimal fragment for a pointer to uint_least64_t.

#### 5.2.1.1.150 SCNxFAST8

```
"hhx"
```

scanf lowercase hexadecimal fragment for a pointer to uint_fast8_t.

#### 5.2.1.1.151 SCNxFAST16

```
__PRIPTR_PREFIX "x"
```

scanf lowercase hexadecimal fragment for a pointer to uint_fast16_t.

#### 5.2.1.1.152 SCNxFAST32

```
__PRIPTR_PREFIX "x"
```

scanf lowercase hexadecimal fragment for a pointer to uint_fast32_t.

#### 5.2.1.1.153 SCNxFAST64

```
__PRI64_PREFIX "x"
```

scanf lowercase hexadecimal fragment for a pointer to uint_fast64_t.

#### 5.2.1.1.154 SCNdMAX

```
__PRI64_PREFIX "d"
```

scanf signed decimal fragment for a pointer to intmax_t.

#### 5.2.1.1.155 SCNiMAX

```
__PRI64_PREFIX "i"
```

scanf base-detected signed integer fragment for a pointer to intmax_t.

#### 5.2.1.1.156 SCNoMAX

```
__PRI64_PREFIX "o"
```

scanf octal fragment for a pointer to uintmax_t.

#### 5.2.1.1.157 SCNuMAX

```
__PRI64_PREFIX "u"
```

scanf unsigned decimal fragment for a pointer to uintmax_t.

#### 5.2.1.1.158 SCNxMAX

```
__PRI64_PREFIX "x"
```

scanf lowercase hexadecimal fragment for a pointer to uintmax_t.

#### 5.2.1.1.159 SCNdPTR

```
__PRIPTR_PREFIX "d"
```

scanf signed decimal fragment for a pointer to intptr_t.

#### 5.2.1.1.160 SCNiPTR

```
__PRIPTR_PREFIX "i"
```

scanf base-detected signed integer fragment for a pointer to intptr_t.

#### 5.2.1.1.161 SCNoPTR

```
__PRIPTR_PREFIX "o"
```

scanf octal fragment for a pointer to uintptr_t.

#### 5.2.1.1.162 SCNuPTR

`__PRIPTR_PREFIX "u"`

scanf unsigned decimal fragment for a pointer to uintptr_t.

#### 5.2.1.1.163 SCNxPTR

`__PRIPTR_PREFIX "x"`

scanf lowercase hexadecimal fragment for a pointer to uintptr_t.

#### 5.2.1.1.164 PRISIZET

`"zu"`

printf decimal format fragment for size_t.

#### 5.2.1.1.165 PRIKY_THREADID_X

`PRIX64`

printf uppercase hexadecimal fragment for a numeric thread identifier.

The argument must match the target's PRIX32 (Windows) or PRIX64 (POSIX) type.

#### 5.2.1.1.166 PRIKY_THREADID_U

`PRIu64`

printf decimal fragment for a numeric thread identifier.

The argument must match the target's PRIu32 (Windows) or PRIu64 (POSIX) type.

#### 5.2.1.1.167 PRIKY_PROCID

`"d"`

printf signed decimal fragment for a process identifier.

The argument must match the int conversion used by this fragment.

### 5.2.1.2 Type definitions

#### 5.2.1.2.1 KY_HMODULE

`typedef void* KY_HMODULE`

Native loaded-module handle; ownership follows the returning API.

#### 5.2.1.2.2 KY_PROCADDR

`typedef void* KY_PROCADDR`

Native procedure address; use the referenced function's declared signature.

#### 5.2.1.2.3 KY_THREADID

`typedef pthread_t KY_THREADID`

Native thread identifier with a platform-dependent representation.

#### 5.2.1.2.4 KY_PROCID

`typedef pid_t KY_PROCID`

Native process identifier with a platform-dependent representation.

#### 5.2.1.2.5 KY_BOOL

`typedef uint8_t KY_BOOL`

Eight-bit SDK Boolean; use KY_TRUE or KY_FALSE.

### 5.2.1.3 Foundation Types

Platform type aliases, Boolean values, and integer formatting.

#### 5.2.1.3.1 Native platform types

| Type | Windows | POSIX | Description |
|---|---|---|---|
| KY_HMODULE | HMODULE | void* | Native loaded-module handle. |

| Type | Windows | POSIX | Description |
|---|---|---|---|
| KY_PROCADDR | FARPROC | void* | Native procedure address. Use the declaration and calling convention of the referenced function. |
| KY_THREADID | DWORD | pthread_t | Native thread identifier; representation is platform-dependent. |
| KY_PROCID | int | pid_t | Native process identifier. |
| KY_BOOL | uint8_t | uint8_t | Eight-bit SDK Boolean value. |

These aliases do not create resources or transfer ownership. Follow the lifetime and release requirements of the API that returns each value.

#### 5.2.1.3.2 KY_BOOL

| Defines | Type | Value | Description |
|---|---|---|---|
| KY_TRUE | KY_BOOL in C++; integer macro in C | 0x1 | Boolean true. |
| KY_FALSE | KY_BOOL in C++; integer macro in C | 0x0 | Boolean false. |

#### 5.2.1.3.3 Integer format strings

Use the PRI* and SCN* format fragments with their matching fixed-width, least-width, fast-width, maximum-width, or pointer-width integer types. Prefix the fragment with a percent sign in a string literal. For scanning, pass a pointer to the exact destination type; printf and scanf fragments are not interchangeable.

| Format family | Purpose | Matching types |
|---|---|---|
| PRId*, PRIi* | Signed decimal output. | intN_t, int_leastN_t, int_fastN_t, intmax_t, intptr_t |
| PRIu*, PRIo*, PRIx*, PRIX* | Unsigned decimal, octal, or hexadecimal output. | uintN_t, uint_leastN_t, uint_fastN_t, uintmax_t, uintptr_t |
| SCNd*, SCNi* | Signed input; SCNi* accepts decimal, octal, or hexadecimal prefixes. | Corresponding signed integer pointer. |
| SCNu*, SCNo*, SCNx* | Unsigned decimal, octal, or hexadecimal input. | Corresponding unsigned integer pointer. |
| PRISIZET | Decimal output for size_t. | size_t |
| PRIKY_THREADID_X, PRIKY_THREADID_U | Hexadecimal or decimal output for numeric thread identifiers on supported targets. | KY_THREADID with the target's matching format argument type. |
| PRIKY_PROCID | Decimal output for a process identifier. | KY_PROCID |

Par: For a uint32_t variable named frame_count, use printf("Frames: %" PRIu32 "\n", frame_count); . Include <stdio.h> when using the formatted I/O functions.

Foundation types and Boolean values

Integer format fragments

Compiler compatibility definitions

## 5.3 Common result codes

KYFoundation_Errors.h defines KY_RESULT, common result codes, and severity values. Include KYFoundation.h to inspect returned results with KY_RESULT_SUCCEEDED(), KY_RESULT_FAILED(), and the result inspection functions described in the KYFoundation library chapter.

### 5.3.1 KYFoundation_Errors.h API reference

Include KYFoundation.h to access the result inspection functions. Numeric codes and severities describe components of a KY_RESULT; they are not complete return values and must not be assigned directly to its uResult field.

#### 5.3.1.1 Macros

##### 5.3.1.1.1 KY_RESULT_CODE_GENERIC_SUCCESS

`0x0`

The operation completed successfully.

##### 5.3.1.1.2 KY_RESULT_CODE_GENERIC_ERROR

`0x1`

The operation failed without a more specific common code.

##### 5.3.1.1.3 KY_RESULT_CODE_INTERFACE_NOT_SET

`0x2`

A required interface has not been configured.

##### 5.3.1.1.4 KY_RESULT_CODE_BAD_CONTEXT

`0x3`

The operation is not valid in the current context.

##### 5.3.1.1.5 KY_RESULT_CODE_MEMORY_ERROR

`0x4`

A memory allocation or memory operation failed.

##### 5.3.1.1.6 KY_RESULT_CODE_INVALID_CONFIG

`0x5`

The supplied configuration is invalid.

##### 5.3.1.1.7 KY_RESULT_CODE_INITIALIZATION_FAILED

`0x6`

Initialization could not be completed.

##### 5.3.1.1.8 KY_RESULT_CODE_INVALID_PARAMETER

`0x7`

An argument has an invalid value or combination of values.

##### 5.3.1.1.9 KY_RESULT_CODE_INVALID_CRC_CALCULATION

`0x8`

A cyclic redundancy check failed.

##### 5.3.1.1.10 KY_RESULT_CODE_CANNOT_LOCK_PROCESS_RESOURCE

`0x9`

A required process resource could not be locked.

##### 5.3.1.1.11 KY_RESULT_CODE_FILE_NOT_FOUND

`0xA`

The requested file could not be found.

##### 5.3.1.1.12 KY_RESULT_CODE_FILE_CANNOT_OPEN

`0xB`

The requested file could not be opened.

##### 5.3.1.1.13 KY_RESULT_CODE_FILE_READ_ERROR

`0xC`

Reading from a file failed.

#### 5.3.1.1.14 KY_RESULT_CODE_FILE_WRITE_ERROR

`0xD`

Writing to a file failed.

#### 5.3.1.1.15 KY_RESULT_CODE_OPERATION_BLOCKED

`0xE`

The operation is blocked by an access restriction or current state.

#### 5.3.1.1.16 KY_RESULT_CODE_BUFFER_TOO_SMALL

`0xF`

The supplied buffer is too small for the requested data.

#### 5.3.1.1.17 KY_RESULT_CODE_FUNCTIONALITY_NOT_IMPLEMENTED

`0x10`

The requested functionality is not implemented.

#### 5.3.1.1.18 KY_RESULT_CODE_TIMEOUT

`0x11`

The operation did not complete within the allowed time.

#### 5.3.1.1.19 KY_RESULT_CODE_INVALID_INDEX

`0x12`

The supplied index is outside the valid range.

#### 5.3.1.1.20 KY_RESULT_CODE_INVALID_ID

`0x13`

The supplied identifier is invalid or unrecognized.

#### 5.3.1.1.21 KY_RESULT_CODE_ABORT

`0x14`

The operation was aborted before completion.

#### 5.3.1.1.22 KY_RESULT_CODE_INVALID_HANDLE

`0x15`

The supplied handle is not valid for the operation.

#### 5.3.1.1.23 KY_RESULT_CODE_RESOURCE_IN_USE

`0x16`

The requested resource is already in use.

#### 5.3.1.1.24 KY_RESULT_CODE_INVALID_JSON

`0x17`

The supplied JSON data is invalid.

#### 5.3.1.1.25 KY_RESULT_CODE_JSON_PROPERTY_NOT_FOUND

`0x18`

A required JSON property is missing.

#### 5.3.1.1.26 KY_RESULT_CODE_TARGET_IS_READONLY

`0x19`

The target does not allow the requested write operation.

#### 5.3.1.1.27 KY_RESULT_CODE_STREAM_NOT_READY_FOR_START

`0x1A`

The stream is not in a state that permits acquisition to start.

#### 5.3.1.1.28 KY_RESULT_CODE_DEVICE_NOT_CONNECTED

`0x1B`

The operation requires a device that is not connected.

#### 5.3.1.1.29 KY_SEVERITY_FATAL

`0x0`

Fatal condition; the affected system or service cannot be used.

#### 5.3.1.1.30 KY_SEVERITY_ALERT

`0x1`

Condition requiring immediate attention.

#### 5.3.1.1.31 KY_SEVERITY_CRITICAL

`0x2`

Critical condition affecting operation.

#### 5.3.1.1.32 KY_SEVERITY_ERROR

`0x3`

Error condition.

#### 5.3.1.1.33 KY_SEVERITY_WARNING

`0x4`

Warning condition requiring attention.

#### 5.3.1.1.34 KY_SEVERITY_NOTICE

`0x5`

Normal but significant condition.

#### 5.3.1.1.35 KY_SEVERITY_INFO

`0x6`

Informational diagnostic.

#### 5.3.1.1.36 KY_SEVERITY_DEBUG

`0x7`

Diagnostic detail intended for troubleshooting.

### 5.3.1.2 Type definitions

#### 5.3.1.2.1 KY_RESULT

```
typedef struct _KY_RESULT KY_RESULT
```

Encoded SDK operation result.

See: KYFoundation_KY_RESULT

### 5.3.1.3 KY_RESULT

Encoded SDK operation result.

See: KYFoundation_KY_RESULT

#### 5.3.1.3.1 uResult

```
uint32_t _KY_RESULT::uResult
```

Encoded result value; inspect through the Foundation result helpers.

### 5.3.1.4 KYFoundation Errors Structs

#### 5.3.1.4.1 KY_RESULT

Result value returned by SDK operations. The declaration uses one-byte packing; preserve the field type and layout when declaring the type in another language.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uResult | uint32_t | Encoded SDK result. | Preserve the complete returned value. Use KY_RESULT_SUCCEEDED(), KY_RESULT_FAILED(), and the KYFoundation accessors to inspect it. |

## 5.4 Hardware result codes

KYVPHWLib_Errors.h defines the hardware result codes for outdated firmware and limited operating mode. Interpret these codes as part of the KY_RESULT returned by an SDK operation.

### 5.4.1 KYVPHWLib_Errors.h API reference

#### 5.4.1.1 Macros

##### 5.4.1.1.1 KY_RESULT_CODE_HWLIB_FW_VERSION_OUTDATED

`0x400`

##### 5.4.1.1.2 KY_RESULT_CODE_HWLIB_LIMITED_MODE

`0x401`

## 5.5 Compiler warning controls

KYFoundation_CompilerWarnings.h provides scoped compiler-warning controls.
Pair KY_DISABLE_WARNING_PUSH with KY_DISABLE_WARNING_POP around the code that needs a suppression; the available mappings depend on the compiler.

### 5.5.1 KYFoundation_CompilerWarnings.h API reference

Pair KY_DISABLE_WARNING_PUSH with KY_DISABLE_WARNING_POP around the smallest region that needs a suppression. The named controls have compiler-specific mappings; an empty mapping has no effect. See Foundation Compiler Warnings for the complete mapping, including names not defined on a given compiler.

#### 5.5.1.1 Foundation Compiler Warnings

Warning controls for MSVC and GNU-compatible compilers.
This header supports MSVC and GCC/Clang diagnostic syntax. Other compiler families receive no definitions. Named suppressions do not correct a code defect and do not change the generated program's required behavior.

##### 5.5.1.1.1 Saving and restoring warning settings

| Macro | Usage |
|---|---|
| KY_DISABLE_WARNING_PUSH | Save the current diagnostic settings before a suppression. |
| KY_DISABLE_WARNING_POP | Restore the settings saved by the matching push. |
| KY_DISABLE_WARNING(value) | Pass an MSVC warning number, or an unquoted GCC/Clang warning option supported by the compiler. |

```
KY_DISABLE_WARNING_PUSH
KY_DISABLE_WARNING_UNREFERENCED_FORMAL_PARAMETER
KY_DISABLE_WARNING_POP
```

Place the declarations or definitions that need the suppression between the named control and KY_DISABLE_WARNING_POP.
Direct controls use KY_DISABLE_WARNING(4100) on MSVC or KY_DISABLE_WARNING(-Wunused-parameter) on GCC/Clang. Prefer the named controls when their mapping suits the application. Diagnostic availability depends on the compiler version and enabled analysis tools.

#### 5.5.1.1.2 Named warning controls

Each name below follows the prefix KY_DISABLE_WARNING_. "None" means the macro expands to nothing; "Not defined" means the name is unavailable in that compiler branch. Existing macro spellings are retained for source compatibility.

| Name suffix | MSVC | GCC/Clang | Diagnostic |
|---|---|---|---|
| UNREFERENCED_FORMAL_PARAMETER | C4100 | -Wunused-parameter | Unused function parameter. |
| UNARY_MINUS_OPERATOR | C4146 | None | Unary minus applied to an unsigned value. |
| CONVERSION_FROM_TYPE2TYPE | C4244 | None | Conversion that may lose data. |
| NEED_DLL_INTERFACE | C4251 | None | Member type without the required DLL interface. |
| CONVERSION_FROM_SIZET2TYPE | C4267 | None | Conversion from size_t that may lose data. |
| CAST_GREATER_SIZE | C4312 | None | Conversion to a pointer of a larger size. |
| STRUCT_WAS_PADDED | C4324 | None | Structure padding required by an alignment specification. |
| HIDES_PREVIOUS_DECLARATION | C4456 | None | Declaration that hides a previous local declaration. |
| HIDES_GLOBAL_DECLARATION | C4459 | None | Declaration that hides a global declaration. |
| RELATIVE_INCLUDE_PATH | C4464 | None | Relative include path containing a parent-directory component. |
| UNREFERENCED_FUNCTION | C4505 | -Wunused-function | Unused function. |
| UNREFERENCED_INLINE_FUNCTION | C4514 | None | Unused inline function. |
| SEH_ARE_NO_LONGER_CAUHT | C4571 | None | MSVC diagnostic about catch-all handling of structured exceptions. |
| NON_MEMBER_NEW_INLINE | C4595 | None | Non-member allocation or deallocation function declared inline. |
| COPY_CTOR_IMPLICITLY_DELETED | C4625 | None | Copy constructor that is implicitly deleted. |
| ASSIGMENT_OPERATOR_IMPLICITLY_DELETED | C4626 | None | Copy assignment operator that is implicitly deleted. |
| FUNCTION_NOT_INLINED | C4710 | None | Function that was not inlined. |
| FMT_STRING_EXPECTED | C4774 | None | Format argument that is not a string literal. |

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

| Name suffix | MSVC | GCC/Clang | Diagnostic |
|---|---|---|---|
| BYTES_PADDING_ADDED | C4820 | None | Padding bytes added to a structure or class. |
| NON_PORTABLEUSE_OF_CLASS | C4840 | Not defined | Class object passed to a variadic function in a non-portable way. |
| DEPRECATED | C4996 | -Wdeprecated-declarations | Use of a deprecated declaration. |
| MOVE_CTOR_IMPLICITLY_DELETED | C5026 | None | Move constructor that is implicitly deleted. |
| PTR_OR_REF_TO_THROWING_FUNC | C5039 | None | Potentially throwing function passed through a C-linkage interface. |
| MOVE_ASSIGMENT_IMPLICITLY_DELETED | C5027 | None | Move assignment operator that is implicitly deleted. |
| UNINITIALIZED_MEMORY | C6001 | None | Use of uninitialized memory. |
| DEREFERENCING_NULL | C6011 | None | Possible null-pointer dereference. |
| INDEX_OUT_OF_RANGE | C6201 | None | Index outside the valid buffer range. |
| HIDES_PREVIOUS_DEFINITION | C6244 | None | Declaration that hides a previous definition during code analysis. |
| SECURITY_DESCRIPTOR_NULL | C6248 | None | Security descriptor with a null discretionary access-control list. |
| INCORRECT_OPERATOR | C6282 | None | Potentially incorrect operator in an expression. |
| ALWAYS_NONZERO_CONSTANT | C6285 | None | Logical OR of nonzero constants that always evaluates to true. |
| ILL_DEFINED_FOR_LOOP | C6294 | None | Loop whose initial test prevents its body from running. |
| reserved_not_0 | C6388 | None | Nonzero argument supplied for a reserved parameter. |
| AVOID_UNNAMED_OBJECTS | C26444 | None | Unnamed temporary object that is immediately destroyed. |
| MAY_NOT_THROW | C26439 | None | Function that can be declared noexcept. |

| Name suffix | MSVC | GCC/Clang | Diagnostic |
|---|---|---|---|
| CAST_4BYTES_TO8BYTES | C26451 | None | Arithmetic performed in a narrower type before conversion to a wider type. |
| ALWAYS_INITIALIZE_MEMBER | C26495 | -Wmissing-field-initializers | Uninitialized member; missing aggregate field initializer on GCC/Clang. |
| MARK_CONSTEXPR | C26498 | None | Variable that can be declared constexpr. |
| GetTickCount64 | C28159 | None | Use of an API for which code analysis recommends a safer alternative. |
| ASSIGMENT_IN_CONDITION | C4706 | Not defined | Assignment used as a condition. |
| DECLARATION_HIDES_MEMBER | C4458 | None | Declaration that hides a class member. |
| DECLARATION_HIDES_PREVIOUS | C4456 | None | Declaration that hides a previous local declaration. |
| OVERFLOW | C4463 | None | Value outside the representable range of a bit field. |
| COMPARISON_IS_ALWAYS | None | -Wtype-limits | Comparison whose result is fixed by the operand type's limits. |
| SIGN_COMPARE | None | -Wsign-compare | Comparison between signed and unsigned values. |
| ARITHMETIC_OVERFLOW | Not defined | None | Compatibility placeholder for arithmetic-overflow diagnostics. |

See: https://learn.microsoft.com/en-us/cpp/preprocessor/warning
See: https://gcc.gnu.org/onlinedocs/gcc/Diagnostic-Pragmas.html

## 5.6 Experimental API annotations

experimental/KYVPExperimental.h defines compiler annotations for experimental SDK declarations. Experimental APIs may change without deprecation. These annotations control compiler diagnostics and do not enable or disable APIs at run time.

### 5.6.1 KYVPExperimental.h API reference

Experimental APIs may change without deprecation. These annotations request compiler diagnostics; they do not enable or disable an API at run time. Place KYVP_EXPERIMENTAL_PREFIX before the return type and KYVP_EXPERIMENTAL_SUFFIX after the parameter list. Both positions are needed for compiler coverage.

#### 5.6.1.1 Macros

##### 5.6.1.1.1 KYVP_EXPERIMENTAL_PREFIX

Annotation placed before an experimental function return type.

Expands to an MSVC deprecation declaration without prior opt-in; empty in the other branches.

##### 5.6.1.1.2 KYVP_EXPERIMENTAL_SUFFIX

Annotation placed after an experimental function parameter list.

Expands to a supported Clang or GCC warning attribute; empty in the other branches.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# 6 KYVPLibTL library

## 6.1 Function Call Sequence

KYVPLibTL library is low level library similar to GenTL Producer. KYVPLibTL library implements the transport layer function. This library provides a transport layer interface to acquire images or other data and facilitate communication with a device. Its role is not to configure the device, except for transport-related settings, though it may be used indirectly to transmit configuration information to and from the device.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

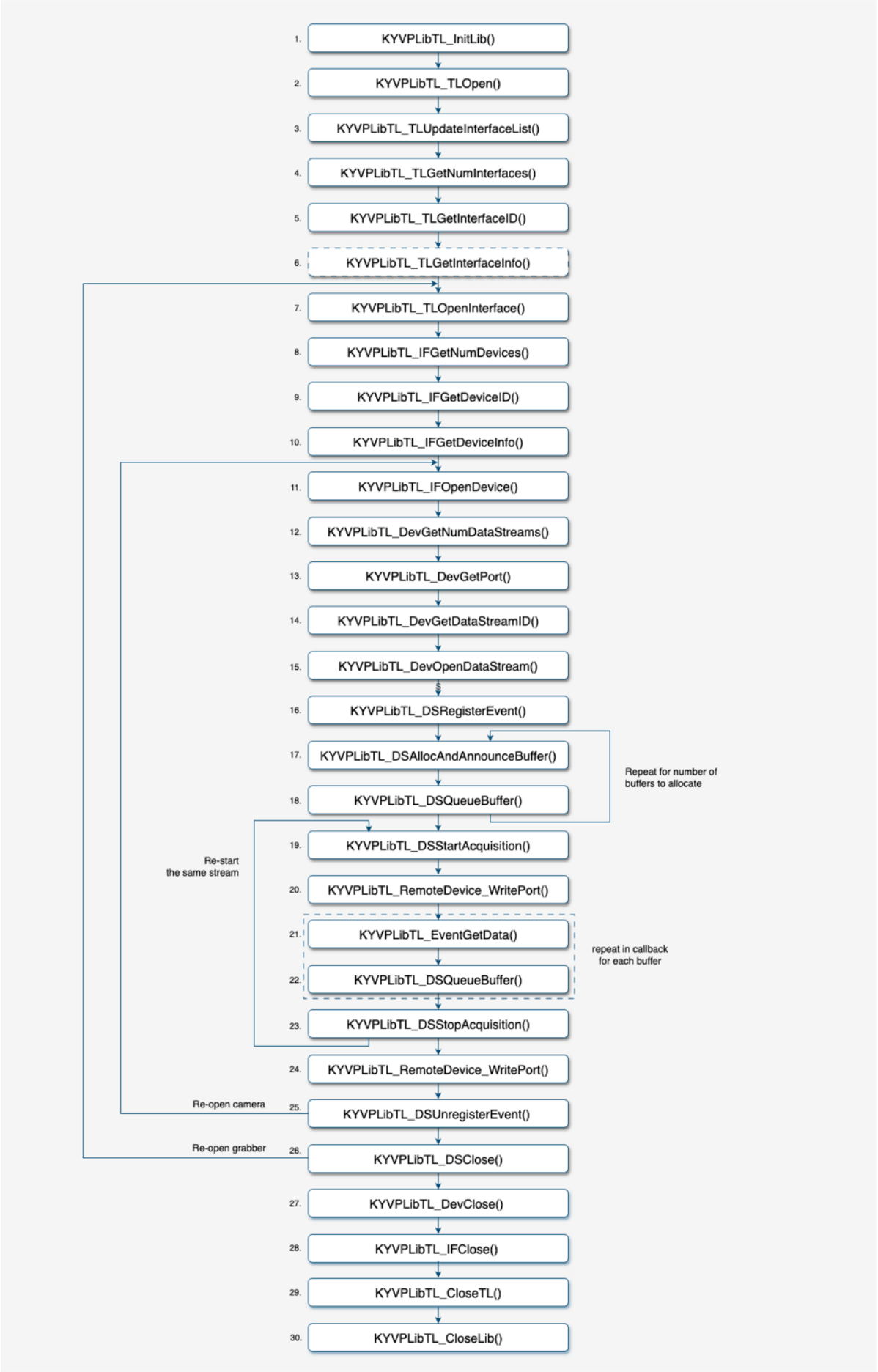



Figure 1 – KYVPLibTL function call sequence

1. KYVPLibTL_InitLib() – Initialize the library.
2. KYVPLibTL_TLOpen() – Open the system module.
3. KYVPLibTL_TLUpdateInterfaceList() – Update the internal list of available interfaces.
4. KYVPLibTL_TLGetNumInterfaces() – Get the number of available Devices (Frame Grabbers).
5. KYVPLibTL_TLGetInterfaceID() – Get the unique ID of the Device. To obtain the HANDLE required for operating on the System module's functions, the ID of the Device must be called.
6. KYVPLibTL_TLGetInterfaceInfo() – (Optional) Get information about Device.
7. KYVPLibTL_TLOpenInterface() – Open a control to a selected Device.
8. KYVPLibTL_IFGetNumDevices() – Get the number of available Local Devices on the current Device.
9. KYVPLibTL_IFGetDeviceID() – Get the unique ID of the Local Device. To obtain the HANDLE required for operating on the Local Device functions, the ID of the Device must be called.
10. KYVPLibTL_IFGetDeviceInfo() – (Optional) Get information about Local Device.
11. KYVPLibTL_IFOpenDevice() – Open a connection to the selected Local Device.
12. KYVPLibTL_DevGetNumDataStreams() – Get the number of available streams on this Local device.
13. KYVPLibTL_DevGetPort() – Retrieve the Handle for the associated Remote Device.
14. KYVPLibTL_DevGetDataStreamID() – Queries the unique ID of the data stream.
15. KYVPLibTL_DevOpenDataStream() – Open the given stream on the given Remote Device.
16. KYVPLibTL_DSRegisterEvent() – Register an event for KYVP_EVENT_TYPE_NEW_BUFFER().
17. KYVPLibTL_DSAllocAndAnnounceBuffer() – Allocate and announce a buffer and bind it to a specific stream. In most cases, it is advised to allocate the buffer memory size corresponding to a full frame. For continues acquisition, several buffers should be allocated in order to prevent from hardware dropping any incoming data frames.
18. KYVPLibTL_DSQueueBuffer() – This function queues a specific buffer for acquisition. A buffer can be queued at any time after it has been announced, whether before or after the acquisition process has begun, as long as it isn't already in the queue. The order in which buffers are delivered might differ from the order in which they were queued.
19. KYVPLibTL_DSStartAcquisition() – Start the acquisition for the specified Remote Device.
20. KYVPLibTL_RemoteDevice_WritePort() – Send command to the Remote Device to start acquisition.
21. KYVPLibTL_EventGetData() – Retrieves the next event data entry from the event data queue associated with the KYVP_EVENT_HANDLE.
22. KYVPLibTL_DSQueueBuffer() – This function queues a specific buffer for acquisition during new buffer Event. A buffer can be queued during the Data Stream.
23. KYVPLibTL_DSStopAcquisition() – Stop the acquisition for the specified Remote Device.
24. KYVPLibTL_RemoteDevice_WritePort() – Send command to the Remote Device to stop acquisition.
25. KYVPLibTL_DSUnregisterEvent() – Unregister the given event.
26. KYVPLibTL_DSClose() – Close the Data Stream. Any memory allocated by the user is NOT freed by this function. All memory allocated by the library is freed and all API handles bound to the stream became invalid.
27. KYVPLibTL_DevClose() – Close the connection to the chosen Remote Device.
28. KYVPLibTL_IFClose() – Close the connection to the PCI Interface, this will clear all relevant resources.
29. KYVPLibTL_CloseTL() – Shutdown the system module.
30. KYVPLibTL_CloseLib() – Shutdown the library.

## 6.2 System Modules

The KYVPLibTL standard defines a layered structure for libraries implementing the KYVPLibTL Interface. Each layer is defined in a module. The modules are presented in a tree structure with the System module as its root.

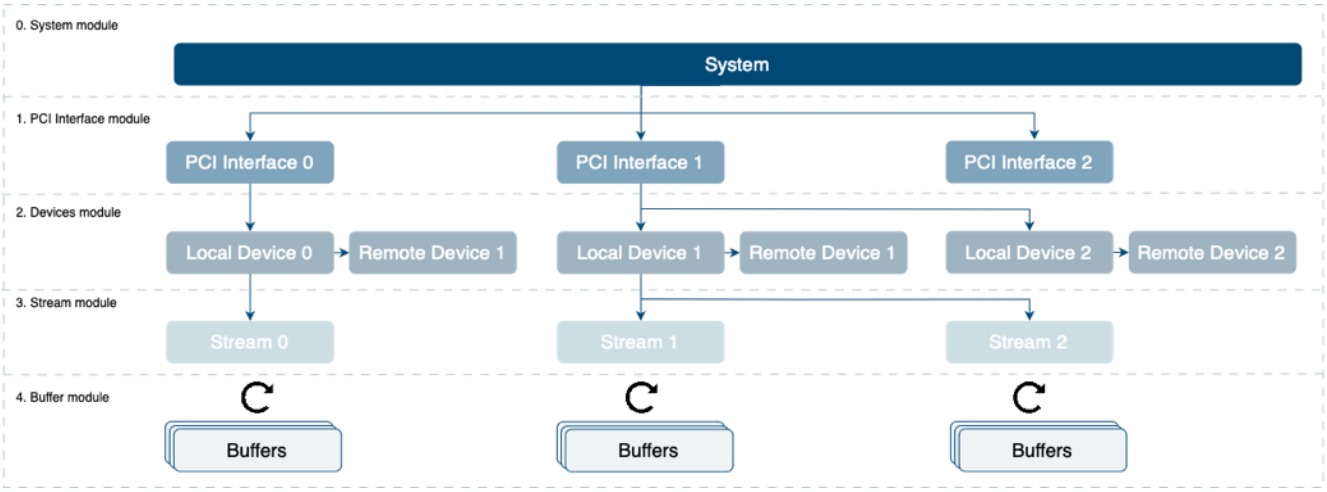



Figure 2 – Module hierarchy

### 6.2.1 System Module

The System module in KYVPLibTL serves as the entry point for any KYVPLibTL consumer, acting as the root of the hierarchy for accessing a KYVPLibTL Producer software driver. It represents the entire system from the perspective of the KYVPLibTL libraries and is responsible for enumerating and instantiating available interfaces. Additionally, the System module handles signaling and internal configuration.

### 6.2.2 PCI Interface Module

The PCI Interface module in a KYVPLibTL system represents a single physical interface – a Frame Grabber. Its main function is to enumerate and instantiate available devices on the interface, also signaling and module configuration capabilities to the KYVPLibTL Consumer. Each PCI Interface module supports only one transport layer technology, so multiple technologies require separate PCI Interfaces. The number of devices on an PCI Interface is not limited by the system but depends on the hardware capabilities.

### 6.2.3 Local Device Module

The Local Device module in a KYVPLibTL system serves as a proxy for one physical remote device on the PCI Interafce side, enabling communication and managing Data Stream modules. Local Device is a certain set of parameters that are actually on the grabber's side, but logically relate to a remote camera. Local Device is a port that can be read and written to. When reading or writing to it, parameters that are actually related to the camera, but are on the Frame Grabber side, change. It also provides signaling and configuration options to the KYVPLibTL Consumer. Each PCI Interface module can support 0, one or multiple Local Device modules, each tied to a single transport layer technology. The number of Local Devices connected to an interface is not limited by the system but depends on the hardware used.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

### 6.2.4 Remote Device Module

The Remote Device module in a KYVPLibTL system represents one physical Remote Device. Remote Device is a port that can be read and written to. During write or read to the Remote Device, a command directly sent to the camera. Each PCI Interface module can support 0, one or multiple Remote Device modules, each tied to a single transport layer technology. The number of devices connected to an interface is not limited by the system but depends on the hardware used.

### 6.2.5 Data Stream Module

The Data Stream module represents a single image data stream from a Remote Device, serving as the acquisition engine and managing the internal buffer pool. It also offers signaling and configuration options to the KYVPLibTL Consumer. A device can support zero, one or multiple data streams, it limited only by the hardware and implementation.

### 6.2.6 Buffer Module

The Buffer module represents a single memory buffer intended for acquisition. This buffer can be allocated either by the user or by the KYVPLibTL Producer, such as pre-allocated system memory. The module also offers signaling and configuration options to the KYVPLibTL Consumer. To enable data streaming, at least one buffer must be announced to the Data Stream module and placed in the input buffer pool. The KYVPLibTL Producer may preprocess the image data which changes image format and/or buffer size.

## 6.3 Module Enumeration and Instantiation

The behavior described below is observed from the perspective of a single process. A KYVPLibTL Producer implementation must ensure that each process accessing the resources has an independent view of the hardware, without needing to be aware that other processes are also involved.

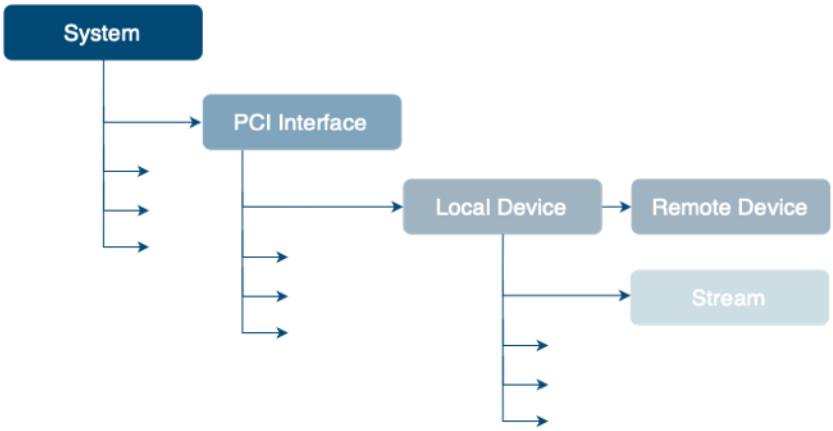



Figure 3 – Enumeration hierarchy

Before the System module can be opened or any operation performed on the KYVPLibTL Producer driver, the KYVPLibTL_InitLib() must be called (once per process). After closing the System module (e.g., when the KYVPLibTL Consumer is closed), the KYVPLibTL_CloseLib() function should be invoked to free all resources. If KYVPLibTL_InitLib() is called again without an accompanying KYVPLibTL_CloseLib(), the second call will result in an error. Likewise, multiple calls to KYVPLibTL_CloseLib() without reinitializing with KYVPLibTL_InitLib() will also free resources improperly.

### 6.3.1 System

The System module serves as the entry point for a KYVPLibTL Consumer to interact with a KYVPLibTL Producer. It enables the enumeration of hardware interfaces and requires a KYVP_TL_HANDLE to perform operations. The module manages communication between processes and ensures that system resources are properly allocated and freed. Interface modules can be enumerated and accessed via unique IDs, and updates to the interface list should be managed carefully. Proper closure of the System module ensures resources are released, and the module can be re-opened if needed.

### 6.3.2 Interface

The Interface module in KYVPLibTL represents a PCI Interface (Frame Grabber). It allows enumeration of attached devices, with each interface identified by a unique ID. The module list can be updated using KYVPLibTL_IFUpdateDeviceList(), and interfaces can be closed in any order. Devices on an interface can be enumerated without opening them, and devices can be opened directly using their unique IDs. The PCI Interface and device lists are not thread-safe and must be accessed carefully to avoid conflicts.
After closing an PCI Interface module, it can be reopened again and the handle to the module may be different from the first instantiation.

### 6.3.3 Local and Remote Devices

A Device module represents the KYVPLibTL Producer driver's view on a remote device. It handles the enumeration of available Data Streams, which is limited by the device and the KYVPLibTL implementation. Devices are identified by unique IDs, and the module can manage multiple Data Streams. Only one Local Device can exist for the same Remote Device within the same process. The module does not track references within a process, so closing it frees all related resources.
After closing a Device module, it can be reopened again and the handle to the module may be different from the first instantiation.

### 6.3.4 Data Stream

The Data Stream module is primarily focused on acquisition and does not enumerate its child modules. Each stream is identified by a unique ID within the Device module, interpreted solely by the KYVPLibTL Producer. When no longer needed, the KYVPLibTL_DSClose() function must be called to free resources, stop acquisitions, flush buffers, and revoke them. The module does not support access from different processes and lacks reference counting.
After closing a Data Stream module, it can be reopened again and the handle to the module may be different from the first instantiation.

Stream interface functions are used to handle received data. Only the memory buffer of frames that were placed in the *Input Queue* can be filled by Hardware. When an individual frame memory is filled, it is moved to *Output Queue* and becomes available for the user application via KYVPLibTL_EventGetData(). This frame memory will not be affected until it is returned to *Input Queue* using KYVPLibTL_DSQueueBuffer() function call. The user application is responsible for putting frames to *Input Queue* for each frame supplied to the host application through Stream event. If the host application fails to do so, then *Input Queue* will eventually become empty and newly acquired data will be dropped until additional frames are moved to *Input Queue*.
This mode is used for streams created with KYVPLibTL_OpenDataStream() function.

The payload size returned by the Data Stream module is dynamically recalculated according to the current device configuration and any transformations applied to the Data Stream.
Functions such as buffer allocation and announcement (e.g., KYVPLibTL_DSAnnounceBuffer and KYVPLibTL_DSAllocAndAnnounceBuffer) do not strictly validate that the provided buffer size matches the current payload size. Therefore, it is the responsibility of the user application to ensure that all currently queued buffers are allocated with a size that correctly corresponds to the active stream configuration and the payload size reported by the Data Stream.

When the remote device image parameters (such as resolution, pixel format, or other relevant settings) are modified, or when a transformation is applied to the Data Stream, the payload size may change. In such cases, the user must ensure that only buffers with the correct payload size are queued to the Data Stream. The user may maintain multiple buffer pools, each corresponding to a specific set of image parameters. When the payload size changes, the user should dequeue buffers from the currently active pool and queue buffers from the pool that matches the updated payload size.

The Data Stream instance itself can be reused across multiple configuration changes without the need to close and reopen it, provided that the user supplies correctly sized buffers corresponding to the currently active configuration.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

### 6.3.5 Buffer

Each Buffer is uniquely identified by a handle obtained from either the KYVPLibTL_DSAnnounceBuffer() or KYVPLibTL_DSAllocAndAnnounceBuffer() functions. Buffers can be allocated by the KYVPLibTL Consumer or Producer and must be announced to the Data Stream module for use. The required Buffer size should be requested from the Data Stream via function KYVPLibTL_DSGetBufferInfo(). To enable the acquisition engine to stream data into a Buffer, the Buffer must first be added to the Input Buffer Pool by calling the KYVPLibTL_DSQueueBuffer() function using the KYVP_BUFFER_HANDLE obtained from the announcement functions.
The KYVP_BUFFER_HANDLE can be released by calling the KYVPLibTL_DSRevokeBuffer() function. However, if the buffer is still in the Input Buffer Queue or the Output Buffer Queue of the acquisition engine, it cannot be revoked, and an error will be returned if attempted. A memory buffer should only be announced once per stream.

## 6.4 KYVPLibTL API functions

The following reference documents the transport-layer functions and related declarations in KYVPLibTL.h. The GenTL_v1_5.h reference supplies the standard transport definitions included by this library.

### 6.4.1 KYVPLibTL.h API reference

Initialize the library with KYVPLibTL_InitLib() before using the API. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to inspect KY_RESULT values. Use KYFoundation_What() to obtain a result description.

Structure packing, member order, and version constants form part of the SDK ABI.

#### 6.4.1.1 Macros

##### 6.4.1.1.1 KYVPTL_EXTERNAL_DLL

##### 6.4.1.1.2 KYVPTL_API

```
KYVP_EXTERNAL_C KYVPTL_EXTERNAL_DLL
```

##### 6.4.1.1.3 KYVPTL_VAR

```
KYVP_EXTERNAL KYVPTL_EXTERNAL_DLL
```

##### 6.4.1.1.4 KYVP_CUSTOM_ID_DEBUG

```
10000
```

##### 6.4.1.1.5 GENTL_NS

```
_entry_
```

##### 6.4.1.1.6 INVALID_HANDLE_INIT_VALUE

```
((void*)-1)
```

##### 6.4.1.1.7 TL_HANDLE_IS_INVALID

```
KYVPLibTL_TLHandleIsInvalid(_hHandle)
```

##### 6.4.1.1.8 TL_HANDLE_IS_NULL

```
KYVPLibTL_TLHandleIsNull(_hHandle)
```

##### 6.4.1.1.9 TL_HANDLES_IS_EQUAL

```
KYVPLibTL_TLHandlesIsEqual(_hHandleFirst, _hHandleSecond)
```

##### 6.4.1.1.10 PCI_INTERFACE_HANDLE_IS_INVALID

```
KYVPLibTL_IFHandleIsInvalid(_hHandle)
```

##### 6.4.1.1.11 PCI_INTERFACE_HANDLE_IS_NULL

```
KYVPLibTL_IFHandleIsNull(_hHandle)
```

##### 6.4.1.1.12 PCI_INTERFACE_HANDLES_IS_EQUAL

```
    KYVPLibTL_IFHandlesAreEqual(_hHandleFirst, _hHandleSecond)
```

##### 6.4.1.1.13 DEVICE_HANDLE_IS_INVALID

```
KYVPLibTL_DevHandleIsInvalid(_hHandle)
```

##### 6.4.1.1.14 DEVICE_HANDLE_IS_NULL

```
KYVPLibTL_DevHandleIsNull(_hHandle)
```

##### 6.4.1.1.15 DEVICE_HANDLES_IS_EQUAL

```
    KYVPLibTL_DevHandlesIsEqual(_hHandleFirst, _hHandleSecond)
```

##### 6.4.1.1.16 REMOTE_DEVICE_HANDLE_IS_INVALID

```
KYVPLibTL_RemoteDeviceHandleIsInvalid(_hHandle)
```

##### 6.4.1.1.17 REMOTE_DEVICE_HANDLE_IS_NULL

```
KYVPLibTL_RemoteDeviceHandleIsNull(_hHandle)
```

##### 6.4.1.1.18 REMOTE_DEVICE_HANDLES_IS_EQUAL

```
    KYVPLibTL_RemoteDeviceHandlesIsEqual(_hHandleFirst, _hHandleSecond)
```

##### 6.4.1.1.19 STREAM_HANDLE_IS_INVALID

```
KYVPLibTL_DSHandleIsInvalid(_hHandle)
```

##### 6.4.1.1.20 STREAM_HANDLE_IS_NULL

```
KYVPLibTL_DSHandleIsNull(_hHandle)
```

##### 6.4.1.1.21 STREAM_HANDLES_IS_EQUAL

```
KYVPLibTL_DSHandlesIsEqual(_hHandleFirst, _hHandleSecond)
```

#### 6.4.1.1.22 BUFFER_HANDLE_IS_INVALID

```
KYVPLibTL_BufferHandleIsInvalid(_hHandle)
```

#### 6.4.1.1.23 BUFFER_HANDLE_IS_NULL

```
KYVPLibTL_BufferHandleIsNull(_hHandle)
```

#### 6.4.1.1.24 BUFFER_HANDLES_IS_EQUAL

```
    KYVPLibTL_BufferHandlesIsEqual(_hHandleFirst, _hHandleSecond)
```

#### 6.4.1.1.25 EVENT_HANDLE_IS_INVALID

```
KYVPLibTL_EventHandleIsInvalid(_hHandle)
```

#### 6.4.1.1.26 EVENT_HANDLE_IS_NULL

```
KYVPLibTL_EventHandleIsNull(_hHandle)
```

#### 6.4.1.1.27 EVENT_HANDLES_IS_EQUAL

```
    KYVPLibTL_EventHandlesIsEqual(_hHandleFirst, _hHandleSecond)
```

#### 6.4.1.1.28 KYVP_PCIINTERFACE_EVENT_TEMPERATURE_VERSION

```
1
```

#### 6.4.1.1.29 KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_VERSION

```
1
```

#### 6.4.1.1.30 KYVP_PCIINTERFACE_EVENT_CXP2_EVENT_VERSION

```
1
```

#### 6.4.1.1.31 KYVP_PCIINTERFACE_GENERIC_EVENT_VERSION

```
1
```

#### 6.4.1.1.32 KYVP_DEVICE_EVENT_CONNECTION_LOST_VERSION

```
1
```

#### 6.4.1.1.33 KYVP_DEVICE_GENERIC_EVENT_VERSION

```
1
```

#### 6.4.1.1.34 KYVPLIBTL_GETPORTURL_STRUCT_VERSION

```
1
```

#### 6.4.1.1.35 KYVPLIBTL_READPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.36 KYVPLIBTL_WRITEPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.37 KYVPLIBTL_PCIINTERFACEGETPORTURL_STRUCT_VERSION

```
1
```

#### 6.4.1.1.38 KYVPLIBTL_PCIINTERFACEREADPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.39 KYVPLIBTL_PCIINTERFACEWRITEPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.40 KYVPLIBTL_DEVICEGETPORTURL_STRUCT_VERSION

```
1
```

#### 6.4.1.1.41 KYVPLIBTL_DEVICEREADPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.42 KYVPLIBTL_DEVICEWRITEPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.43 KYVPLIBTL_REMOTEDEVICEGETPORTURL_STRUCT_VERSION

```
1
```

#### 6.4.1.1.44 KYVPLIBTL_REMOTEDEVICEREADPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.45 KYVPLIBTL_REMOTEDEVICEWRITEPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.46 KYVPLIBTL_STREAMGETPORTURL_STRUCT_VERSION

```
1
```

#### 6.4.1.1.47 KYVPLIBTL_STREAMREADPORT_STRUCT_VERSION

```
1
```

#### 6.4.1.1.48 KYVPLIBTL_STREAMWRITEPORT_STRUCT_VERSION

```
1
```

6.4.1.1.49 KYVPLIBTL_TLOPEN_ARGS_VERSION
1
6.4.1.1.50 KYVPLIBTL_TLCLOSE_ARGS_VERSION
1
6.4.1.1.51 KYVPLIBTL_TLGETINFO_ARGS_VERSION
1
6.4.1.1.52 KYVPLIBTL_TLGETNUMINTERFACES_ARGS_VERSION
1
6.4.1.1.53 KYVPLIBTL_TLGETINTERFACEID_ARGS_VERSION
1
6.4.1.1.54 KYVPLIBTL_TLGETPCIINTERFACEINFO_ARGS_VERSION
1
6.4.1.1.55 KYVPLIBTL_TLOPENPCIINTERFACE_STRUCT_VERSION
1
6.4.1.1.56 KYVPLIBTL_TLUPDATEPCIINTERFACELIST_ARGS_VERSION
1
6.4.1.1.57 KYVPLIBTL_IFCLOSE_ARGS_VERSION
1
6.4.1.1.58 KYVPLIBTL_IFGETINFO_ARGS_VERSION
1
6.4.1.1.59 KYVPLIBTL_IFGETNUMDEVICES_ARGS_VERSION
1
6.4.1.1.60 KYVPLIBTL_IFGETDEVICEID_ARGS_VERSION
1
6.4.1.1.61 KYVPLIBTL_IFUPDATEDEVICELIST_ARGS_VERSION
2
6.4.1.1.62 KYVPLIBTL_IFGETDEVICEINFO_ARGS_VERSION
1
6.4.1.1.63 KYVPLIBTL_IFOPENDEVICE_ARGS_VERSION
1
6.4.1.1.64 KYVPLIBTL_IFREGISTEREVENT_ARGS_VERSION
1
6.4.1.1.65 KYVPLIBTL_IFUNREGISTEREVENT_ARGS_VERSION
1
6.4.1.1.66 KYVPLIBTL_DEVGETPORT_ARGS_VERSION
1
6.4.1.1.67 KYVPLIBTL_DEVGETNUMDATASTREAMS_ARGS_VERSION
1
6.4.1.1.68 KYVPLIBTL_DEVGETDATASTREAMID_ARGS_VERSION
1
6.4.1.1.69 KYVPLIBTL_DEVOPENDATASTREAM_ARGS_VERSION
1
6.4.1.1.70 KYVPLIBTL_DEVGETINFO_ARGS_VERSION
1
6.4.1.1.71 KYVPLIBTL_DEVCLOSE_ARGS_VERSION
1
6.4.1.1.72 KYVPLIBTL_DEVREGISTEREVENT_ARGS_VERSION
1
6.4.1.1.73 KYVPLIBTL_DEVUNREGISTEREVENT_ARGS_VERSION
1
6.4.1.1.74 KYVPLIBTL_DSANNOUNCEBUFFER_ARGS_VERSION
1
6.4.1.1.75 KYVPLIBTL_DSALLOCANDANNOUNCEBUFFER_ARGS_VERSION
1

#### 6.4.1.1.76 KYVPLIBTL_DSSTARTACQUISITION_ARGS_VERSION

1

#### 6.4.1.1.77 KYVPLIBTL_DSSTOPACQUISITION_ARGS_VERSION

1

#### 6.4.1.1.78 KYVPLIBTL_DSGETINFO_ARGS_VERSION

1

#### 6.4.1.1.79 KYVPLIBTL_DSGETBUFFEREID_STRUCT_VERSION

1

#### 6.4.1.1.80 KYVPLIBTL_DSCLOSE_ARGS_VERSION

1

#### 6.4.1.1.81 KYVPLIBTL_DSREVOKEBUFFER_ARGS_VERSION

1

#### 6.4.1.1.82 KYVPLIBTL_DSQUEUEBUFFER_ARGS_VERSION

1

#### 6.4.1.1.83 KYVPLIBTL_DSFLUSHQUEUE_ARGS_VERSION

1

#### 6.4.1.1.84 KYVPLIBTL_DSGETBUFFERINFO_ARGS_VERSION

1

#### 6.4.1.1.85 KYVPLIBTL_DSREGISTEREVENT_ARGS_VERSION

1

#### 6.4.1.1.86 KYVPLIBTL_DSUNREGISTEREVENT_ARGS_VERSION

1

#### 6.4.1.1.87 KYVPLIBTL_EVENTGETDATA_ARGS_VERSION

1

#### 6.4.1.1.88 KYVPLIBTL_EVENTGETDATAINFO_ARGS_VERSION

1

#### 6.4.1.1.89 KYVPLIBTL_EVENTGETINFO_ARGS_VERSION

1

#### 6.4.1.1.90 KYVPLIBTL_EVENTFLUSH_ARGS_VERSION

1

#### 6.4.1.1.91 KYVPLIBTL_EVENTKILL_ARGS_VERSION

1

### 6.4.1.2 Enumerations

#### 6.4.1.2.1 _KYVP_INFO_DATATYPE

Data types returned by information queries.

Defines the data type possible for the various Info functions. The data type itself may define its size. For buffer or string types the piSize parameter must be used to query the actual amount of data being written.

*6.4.1.2.1.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_INFO_DATATYPE_UNKNOWN | GENTL_NS(INFO_DATATYPE_UNKNOWN) | Unknown data type. This value is never returned from a function but can be used to initialize the variable to inquire the type. |
| KYVP_INFO_DATATYPE_STRING | GENTL_NS(INFO_DATATYPE_STRING) | Null-terminated C string (encoding according to the TL_INFO_CHAR_ENCODING info command). |
| KYVP_INFO_DATATYPE_STRINGLIST | GENTL_NS(INFO_DATATYPE_STRINGLIST) | Concatenated INFO_DATATYPE_STRING list. End of list is signaled with an additional 0. |
| KYVP_INFO_DATATYPE_INT16 | GENTL_NS(INFO_DATATYPE_INT16) | Signed 16-bit integer. |

| Name | Value | Description |
|---|---|---|
| KYVP_INFO_DATATYPE_UINT16 | GENTL_NS(INFO_DATATYPE_UINT16) | Unsigned 16-bit integer. |
| KYVP_INFO_DATATYPE_INT32 | GENTL_NS(INFO_DATATYPE_INT32) | Signed 32-bit integer. |
| KYVP_INFO_DATATYPE_UINT32 | GENTL_NS(INFO_DATATYPE_UINT32) | Unsigned 32-bit integer. |
| KYVP_INFO_DATATYPE_INT64 | GENTL_NS(INFO_DATATYPE_INT64) | Signed 64-bit integer. |
| KYVP_INFO_DATATYPE_UINT64 | GENTL_NS(INFO_DATATYPE_UINT64) | Unsigned 64-bit integer. |
| KYVP_INFO_DATATYPE_FLOAT64 | GENTL_NS(INFO_DATATYPE_FLOAT64) | Signed 64-bit floating point number. |
| KYVP_INFO_DATATYPE_PTR | GENTL_NS(INFO_DATATYPE_PTR) | Pointer type (void*). Size is platform dependent (32-bit on 32-bit platforms). |
| KYVP_INFO_DATATYPE_BOOL8 | GENTL_NS(INFO_DATATYPE_BOOL8) | Boolean value occupying 8-bit. 0 for false and nonzero for true. |
| KYVP_INFO_DATATYPE_SIZET | GENTL_NS(INFO_DATATYPE_SIZET) | Platform dependent unsigned integer (32-bit on 32-bit platforms). |
| KYVP_INFO_DATATYPE_BUFFER | GENTL_NS(INFO_DATATYPE_BUFFER) | Like a INFO_DATATYPE_STRING but with arbitrary data and no null termination. |
| KYVP_INFO_DATATYPE_PTRDIFF | GENTL_NS(INFO_DATATYPE_PTRDIFF) | The type ptrdiff_t is a type that can hold the result of subtracting two pointers. |
| KYVP_INFO_DATATYPE_INVALID | 0xFFFF | SDK-specific sentinel for an invalid information data type. |

#### 6.4.1.2.2 _KYVP_TL_CHAR_ENCODING

Character encodings used by the transport layer.

##### *6.4.1.2.2.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_TL_CHAR_ENCODING_ASCII | GENTL_NS(TL_CHAR_ENCODING_ASCII) | The producer uses ASCII for character strings. |
| KYVP_TL_CHAR_ENCODING_UTF8 | GENTL_NS(TL_CHAR_ENCODING_UTF8) | The producer uses UTF-8 for character strings. |
| KYVP_TL_CHAR_ENCODING_INVALID | 0xFFFF | SDK-specific sentinel for an invalid character encoding. |

#### 6.4.1.2.3 _KYVP_TL_INFO_CMD

Transport layer information commands.

##### *6.4.1.2.3.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_TL_INFO_CMD_ID | GENTL_NS(TL_INFO_ID) | Unique ID identifying a GenTL. Returned data type: STRING. |
| KYVP_TL_INFO_CMD_VENDOR | GENTL_NS(TL_INFO_VENDOR) | Vendor name. Returned data type: STRING. |
| KYVP_TL_INFO_CMD_MODEL | GENTL_NS(TL_INFO_MODEL) | Model name. Returned data type: STRING. |
| KYVP_TL_INFO_CMD_VERSION | GENTL_NS(TL_INFO_VERSION) | Producer version. Returned data type: STRING. |
| KYVP_TL_INFO_CMD_TLTYPE | GENTL_NS(TL_INFO_TLTYPE) | Transport layer technology that is supported. Returned data type: STRING. |

| Name | Value | Description |
|---|---|---|
| KYVP_TL_INFO_CMD_NAME | GENTL_NS(TL_INFO_NAME) | File name including extension of the library. Returned data type: STRING. |
| KYVP_TL_INFO_CMD_PATHNAME | GENTL_NS(TL_INFO_PATHNAME) | Full path including file name and extension of the library. Returned data type: STRING. |
| KYVP_TL_INFO_CMD_DISPLAYNAME | GENTL_NS(TL_INFO_DISPLAYNAME) | User readable name. Returned data type: STRING. |
| KYVP_TL_INFO_CMD_CHAR_ENCODING | GENTL_NS(TL_INFO_CHAR_ENCODING) | The char encoding. Returned data type: INT32. |
| KYVP_TL_INFO_CMD_GENTL_VER_MAJOR | GENTL_NS(TL_INFO_GENTL_VER_MAJOR) | Major version number of GenTL Standard Version this Producer complies with. Returned data type: INT32. |
| KYVP_TL_INFO_CMD_GENTL_VER_MINOR | GENTL_NS(TL_INFO_GENTL_VER_MINOR) | Minor version number of GenTL Standard Version this Producer complies with. Returned data type: INT32. |
| KYVP_TL_INFO_CMD_API_VER_MAJOR | GENTL_NS(TL_INFO_CUSTOM_ID + 1) | Software API major version. Returned data type: INT32. |
| KYVP_TL_INFO_CMD_API_VER_MINOR | GENTL_NS(TL_INFO_CUSTOM_ID + 2) | Software API minor version. Returned data type: INT32. |
| KYVP_TL_INFO_CMD_API_VER_PATCH | GENTL_NS(TL_INFO_CUSTOM_ID + 3) | Software API patch version. Returned data type: INT32. |
| KYVP_TL_INFO_CMD_API_VER_BUILD | GENTL_NS(TL_INFO_CUSTOM_ID + 4) | Software API build (revision) version. Returned data type: INT32. |
| KYVP_TL_INFO_CMD_API_VER_ALPHA | GENTL_NS(TL_INFO_CUSTOM_ID + 5) | True if alpha build. Returned data type: BOOL. |
| KYVP_TL_INFO_CMD_API_VER_BETA | GENTL_NS(TL_INFO_CUSTOM_ID + 6) | True if beta build. Returned data type: BOOL. |
| KYVP_TL_INFO_CMD_API_VER_RC | GENTL_NS(TL_INFO_CUSTOM_ID + 7) | True if release candidate build. Returned data type: BOOL. |
| KYVP_TL_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid system information command. |

#### 6.4.1.2.4 _KYVP_INTERFACE_INFO_CMD

Interface information commands.

##### *6.4.1.2.4.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_INTERFACE_INFO_CMD_ID | GENTL_NS(INTERFACE_INFO_ID) | Unique ID of the interface. Returned data type: STRING. |
| KYVP_INTERFACE_INFO_CMD_DISPLAYNAME | GENTL_NS(INTERFACE_INFO_DISPLAYNAME) | User readable name of the interface. Returned data type: STRING. |

| Name | Value | Description |
|---|---|---|
| KYVP_INTERFACE_INFO_CMD_TLTYPE | GENTL_NS(INTERFACE_INFO_TLTYPE) | Transport layer technology that is supported. Returned data type: STRING. |
| KYVP_INTERFACE_INFO_CMD_PCI_SLOT | GENTL_NS(INTERFACE_INFO_CUSTOM_ID + 1) | Info about interface PCI slot. Returned data type: STRING. |
| KYVP_INTERFACE_INFO_CMD_KYVP_PCI_INTERFACE_INFO | GENTL_NS(INTERFACE_INFO_CUSTOM_ID + 2) | Pointer to KYVP_PCI_INTERFACE_INFO struct. Returned data type: PTR. |
| KYVP_INTERFACE_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid interface information command. |

#### 6.4.1.2.5 _KYVP_DEVICE_ACCESS_FLAGS

Device access modes requested when opening a device.

##### *6.4.1.2.5.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_DEVICE_ACCESS_FLAGS_UNKNOWN | GENTL_NS(DEVICE_ACCESS_UNKNOWN) | Unknown access mode, suitable for initializing a query result; not an open-device command. |
| KYVP_DEVICE_ACCESS_FLAGS_NONE | GENTL_NS(DEVICE_ACCESS_NONE) | No device access: the device has not been opened, or access was denied. |
| KYVP_DEVICE_ACCESS_FLAGS_READONLY | GENTL_NS(DEVICE_ACCESS_READONLY) | Open the device for reading only; port writes are not permitted. |
| KYVP_DEVICE_ACCESS_FLAGS_CONTROL | GENTL_NS(DEVICE_ACCESS_CONTROL) | Open with read/write control while allowing other processes or hosts to read the device. |
| KYVP_DEVICE_ACCESS_FLAGS_EXCLUSIVE | GENTL_NS(DEVICE_ACCESS_EXCLUSIVE) | Open with exclusive read/write access; other processes and hosts cannot access the device. |

#### 6.4.1.2.6 _KYVP_DEVICE_ACCESS_STATUS

Device access status reported by the producer.

##### *6.4.1.2.6.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_DEVICE_ACCESS_STATUS_UNKNOWN | GENTL_NS(DEVICE_ACCESS_STATUS_UNKNOWN) | The producer cannot determine whether the device is accessible. |
| KYVP_DEVICE_ACCESS_STATUS_READWRITE | GENTL_NS(DEVICE_ACCESS_STATUS_READWRITE) | The device is not open and can be opened for read/write access. After opening, the status becomes KYVP_DEVICE_ACCESS_STATUS_OPEN_READWRITE. |
| KYVP_DEVICE_ACCESS_STATUS_READONLY | GENTL_NS(DEVICE_ACCESS_STATUS_READONLY) | The device is not open and can be opened for reading only. Read/write availability takes precedence when both modes are possible. |

| Name | Value | Description |
|---|---|---|
| KYVP_DEVICE_ACCESS_STATUS_NOACCESS | GENTL_NS(DEVICE_ACCESS_STATUS_NOACCESS) | The producer can identify the device, but the device is unreachable and cannot be accessed. |
| KYVP_DEVICE_ACCESS_STATUS_BUSY | GENTL_NS(DEVICE_ACCESS_STATUS_BUSY) | Another process or host already owns or has opened the device. |
| KYVP_DEVICE_ACCESS_STATUS_OPEN_READWRITE | GENTL_NS(DEVICE_ACCESS_STATUS_OPEN_READWRITE) | This producer has already opened the device with read/write access. |
| KYVP_DEVICE_ACCESS_STATUS_OPEN_READONLY | GENTL_NS(DEVICE_ACCESS_STATUS_OPEN_READONLY) | This producer has already opened the device with read-only access. |

#### 6.4.1.2.7 _KYVP_DEVICE_INFO_CMD

Device information commands.

##### *6.4.1.2.7.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_DEVICE_INFO_CMD_ID | GENTL_NS(DEVICE_INFO_ID) | Unique ID of the device. Returned data type: STRING. |
| KYVP_DEVICE_INFO_CMD_VENDOR | GENTL_NS(DEVICE_INFO_VENDOR) | Device vendor name. Returned data type: STRING. |
| KYVP_DEVICE_INFO_CMD_MODEL | GENTL_NS(DEVICE_INFO_MODEL) | Device model name. Returned data type: STRING. |
| KYVP_DEVICE_INFO_CMD_TLTYPE | GENTL_NS(DEVICE_INFO_TLTYPE) | Transport layer technology that is supported. Returned data type: STRING. |
| KYVP_DEVICE_INFO_CMD_DISPLAYNAME | GENTL_NS(DEVICE_INFO_DISPLAYNAME) | User readable name of the device. Returned data type: STRING. |
| KYVP_DEVICE_INFO_CMD_ACCESS_STATUS | GENTL_NS(DEVICE_INFO_ACCESS_STATUS) | Gets the access status the Producer has on the device. Returned data type: INT32. |
| KYVP_DEVICE_INFO_CMD_USER_DEFINED_NAME | GENTL_NS(DEVICE_INFO_USER_DEFINED_NAME) | String containing the user defined name of the device. Returned data type: STRING. |

| Name | Value | Description |
|---|---|---|
| KYVP_DEVICE_INFO_CMD_SERIAL_NUMBER | GENTL_NS(DEVICE_INFO_SERIAL_NUMBER) | Serial number of the device in string format. Returned data type: STRING. |
| KYVP_DEVICE_INFO_CMD_VERSION | GENTL_NS(DEVICE_INFO_VERSION) | Device version in string format. Returned data type: STRING. |
| KYVP_DEVICE_INFO_CMD_TIMESTAMP_FREQUENCY | GENTL_NS(DEVICE_INFO_TIMESTAMP_FREQUENCY) | Tick frequency of the device's timestamp counter in ticks per second. Returned data type: UINT64. |
| KYVP_DEVICE_INFO_CMD_KYVP_DEVICE_INFO | GENTL_NS(DEVICE_INFO_CUSTOM_ID + 1) | Pointer to KYVP_DEVICE_INFO struct. The pointed data must be treated as read-only: modifying its contents is undefined behavior. Returned data type: PTR. |
| KYVP_DEVICE_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid device information command. |

#### 6.4.1.2.8 _KYVP_ACQ_START_FLAGS

Acquisition start flags.

##### *6.4.1.2.8.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_ACQ_START_FLAGS_DEFAULT | GENTL_NS(ACQ_START_FLAGS_DEFAULT) | Default behavior. |

#### 6.4.1.2.9 _KYVP_ACQ_STOP_FLAGS

Acquisition stop flags.

##### *6.4.1.2.9.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_ACQ_STOP_FLAGS_DEFAULT | GENTL_NS(ACQ_STOP_FLAGS_DEFAULT) | Stops the acquisition engine when the currently running tasks like filling a buffer are completed. |

| Name | Value | Description |
|---|---|---|
| KYVP_ACQ_STOP_FLAGS_KILL | GENTL_NS(ACQ_STOP_FLAGS_KILL) | Stop the acquisition engine immediately. In case this results in a partially filled buffer the Producer will return the buffer through the regular mechanism to the user, indicating through the info function of that buffer that this buffer is not complete. |

#### 6.4.1.2.10 _KYVP_ACQ_QUEUE_TYPE

Buffer queue operations.

##### *6.4.1.2.10.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_ACQ_QUEUE_TYPE_INPUT_TO_OUTPUT | GENTL_NS(ACQ_QUEUE_INPUT_TO_OUTPUT) | Move input-pool buffers to the output queue and generate new-buffer events as needed. Buffers currently being filled remain in place. This describes the default buffer handling mode. |
| KYVP_ACQ_QUEUE_OUTPUT_DISCARD | GENTL_NS(ACQ_QUEUE_OUTPUT_DISCARD) | Remove buffers from the output queue and remove their pending new-buffer events as needed. The buffers remain announced. |
| KYVP_ACQ_QUEUE_ALL_TO_INPUT | GENTL_NS(ACQ_QUEUE_ALL_TO_INPUT) | Return every announced buffer to the input pool, including output-queue buffers and buffers being filled, and discard pending new-buffer events. |
| KYVP_ACQ_QUEUE_UNQUEUED_TO_INPUT | GENTL_NS(ACQ_QUEUE_UNQUEUED_TO_INPUT) | Queue announced buffers that are neither in the input pool nor in the output queue nor currently being filled. |

| Name | Value | Description |
|---|---|---|
| KYVP_ACQ_QUEUE_ALL_DISCARD | GENTL_NS(ACQ_QUEUE_ALL_DISCARD) | Remove every buffer from the input pool, output queue, and active filling operation so it can be revoked or queued again. This does not itself revoke the buffers. |
| KYVP_ACQ_QUEUE_INVALID | 0xFFFF | SDK-specific sentinel for an invalid queue operation. |

#### 6.4.1.2.11 _KYVP_STREAM_INFO_CMD

Data stream information commands.

Defines the data type possible for the various Info functions. The data type itself may define its size. For buffer or string types the piSize parameter must be used to query the actual amount of data being written.

##### *6.4.1.2.11.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_STREAM_INFO_CMD_ID | GENTL_NS(STREAM_INFO_ID) | Unique ID of the data stream. |
| KYVP_STREAM_INFO_CMD_NUM_DELIVERED | GENTL_NS(STREAM_INFO_NUM_DELIVERED) | Number of delivered buffers since last acquisition start. |
| KYVP_STREAM_INFO_CMD_NUM_UNDERRUN | GENTL_NS(STREAM_INFO_NUM_UNDERRUN) | Number of lost frames due to queue underrun. |
| KYVP_STREAM_INFO_CMD_NUM_ANNOUNCED | GENTL_NS(STREAM_INFO_NUM_ANNOUNCED) | Number of announced buffers. |
| KYVP_STREAM_INFO_CMD_NUM_QUEUED | GENTL_NS(STREAM_INFO_NUM_QUEUED) | Number of buffers in the input pool plus the buffer(s) currently being filled. |
| KYVP_STREAM_INFO_CMD_NUM_AWAIT_DELIVERY | GENTL_NS(STREAM_INFO_NUM_AWAIT_DELIVERY) | Number of buffers in the output buffer queue. |
| KYVP_STREAM_INFO_CMD_NUM_STARTED | GENTL_NS(STREAM_INFO_NUM_STARTED) | Number of frames started in the acquisition engine. |
| KYVP_STREAM_INFO_CMD_PAYLOAD_SIZE | GENTL_NS(STREAM_INFO_PAYLOAD_SIZE) | Size of the expected data in bytes. |

| Name | Value | Description |
|---|---|---|
| KYVP_STREAM_INFO_CMD_IS_GRABBING | GENTL_NS(STREAM_INFO_IS_GRABBING) | Flag indicating whether the acquisition engine is started or not. |
| KYVP_STREAM_INFO_CMD_DEFINES_PAYLOADSIZE | GENTL_NS(STREAM_INFO_DEFINES_PAYLOADSIZE) | Flag indicating that this data stream defines a payload size independent from the remote device. |
| KYVP_STREAM_INFO_CMD_TLTYPE | GENTL_NS(STREAM_INFO_TLTYPE) | Transport layer technology that is supported. |
| KYVP_STREAM_INFO_CMD_NUM_CHUNKS_MAX | GENTL_NS(STREAM_INFO_NUM_CHUNKS_MAX) | Maximum number of chunks to be expected in a buffer. |
| KYVP_STREAM_INFO_CMD_BUF_ANNOUNCE_MIN | GENTL_NS(STREAM_INFO_BUF_ANNOUNCE_MIN) | Minimum number of buffers to announce. |
| KYVP_STREAM_INFO_CMD_BUF_ALIGNMENT | GENTL_NS(STREAM_INFO_BUF_ALIGNMENT) | Alignment size in bytes of the buffer passed to DSAnnounceBuffer. |
| KYVP_STREAM_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid stream information command. |

#### 6.4.1.2.12 _KYVP_BUFFER_INFO_CMD

Buffer information commands.

##### *6.4.1.2.12.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_BUFFER_INFO_CMD_BASE | GENTL_NS(BUFFER_INFO_BASE) | Base address of the buffer memory as passed to the KYVPLibTL_DSAnnounceBuffer function. |
| KYVP_BUFFER_INFO_CMD_SIZE | GENTL_NS(BUFFER_INFO_SIZE) | Size of the buffer in bytes. |
| KYVP_BUFFER_INFO_CMD_USER_PTR | GENTL_NS(BUFFER_INFO_USER_PTR) | Private data pointer casted to an integer provided at buffer announcement using KYVPLibTL_DSAnnounceBuffer or KYVPLibTL_DSAllocAndAnnounceBuffer. |
| KYVP_BUFFER_INFO_CMD_TIMESTAMP | GENTL_NS(BUFFER_INFO_TIMESTAMP) | Timestamp the buffer was acquired. |

| Name | Value | Description |
|---|---|---|
| KYVP_BUFFER_INFO_CMD_NEW_DATA | GENTL_NS(BUFFER_INFO_NEW_DATA) | Flag to indicate that the buffer contains new data since the last delivery. |
| KYVP_BUFFER_INFO_CMD_IS_QUEUED | GENTL_NS(BUFFER_INFO_IS_QUEUED) | If this flag is set to true the buffer is in the input pool, the buffer is currently being filled or the buffer is in the output buffer queue. In case this value is true the buffer is owned by the GenTL Producer and it can not be revoked. |
| KYVP_BUFFER_INFO_CMD_IS_ACQUIRING | GENTL_NS(BUFFER_INFO_IS_ACQUIRING) | Flag to indicate that the buffer is currently being filled with data. |
| KYVP_BUFFER_INFO_CMD_IS_INCOMPLETE | GENTL_NS(BUFFER_INFO_IS_INCOMPLETE) | Flag to indicate that a buffer was filled but an error occurred during that process. |
| KYVP_BUFFER_INFO_CMD_TLTYPE | GENTL_NS(BUFFER_INFO_TLTYPE) | Transport layer technology that is supported. |
| KYVP_BUFFER_INFO_CMD_SIZE_FILLED | GENTL_NS(BUFFER_INFO_SIZE_FILLED) | Number of bytes written into the buffer the last time it has been filled. This value is reset to 0 when the buffer is placed into the Input Buffer Pool.If the buffer is incomplete (such as if there are missing packets), only thenumber of bytes successfully writtento the buffer are reported. If the bufferis complete, the number equals to thesize reported throughKYVP_BUFFER_INFO_DATA_SIZE. |
| KYVP_BUFFER_INFO_CMD_WIDTH | GENTL_NS(BUFFER_INFO_WIDTH) | Width of the data in the buffer in number of pixels. This information refers for example to the width entry in the GigE Vision image stream data leader. For other technologies this is to be implemented accordingly. |

| Name | Value | Description |
|---|---|---|
| KYVP_BUFFER_INFO_CMD_HEIGHT | GENTL_NS(BUFFER_INFO_HEIGHT) | Height of the data in the buffer in number of pixels as configured. For variable size images this is the maximum height of the buffer. For example this information refers to the height entry in the GigE Vision image stream data leader. For other technologies this is to be implemented accordingly. |
| KYVP_BUFFER_INFO_CMD_XOFFSET | GENTL_NS(BUFFER_INFO_XOFFSET) | XOffset of the data in the buffer in number of pixels from the image origin to handle areas of interest. This information refers for example to the information provided in the GigE Vision image stream data leader. For other technologies this is to be implemented accordingly. |
| KYVP_BUFFER_INFO_CMD_YOFFSET | GENTL_NS(BUFFER_INFO_YOFFSET) | YOffset of the data in the buffer in number of lines from the image origin to handle areas of interest. This information refers for example to the information provided in the GigE Vision image stream data leader. For other technologies this is to be implemented accordingly. |
| KYVP_BUFFER_INFO_CMD_XPADDING | GENTL_NS(BUFFER_INFO_XPADDING) | XPadding of the data in the buffer in number of bytes. This information refers for example to the information provided in the GigE Vision image stream data leader. For other technologies this is may be implemented accordingly. |
| KYVP_BUFFER_INFO_CMD_YPADDING | GENTL_NS(BUFFER_INFO_YPADDING) | YPadding of the data in the buffer in number of bytes. This information refers for example to the information provided in the GigE Vision image stream data leader. For other thechnologies this may be implemented accordingly. |

| Name | Value | Description |
|---|---|---|
| KYVP_BUFFER_INFO_CMD_FRAMEID | GENTL_NS(BUFFER_INFO_FRAMEID) | A sequentially incremented number of the frame. This information refers for example to the information provided in the GigE Vision image stream block id. For other technologies this is to be implemented accordingly. The wrap around of this number is transportation technology dependent. |
| KYVP_BUFFER_INFO_CMD_IMAGEPRESENT | GENTL_NS(BUFFER_INFO_IMAGEPRESENT) | Flag to indicate if the current data in the buffer contains image data. |
| KYVP_BUFFER_INFO_CMD_IMAGEOFFSET | GENTL_NS(BUFFER_INFO_IMAGEOFFSET) | Offset of the image data from the beginning of the delivered buffer in bytes. |
| KYVP_BUFFER_INFO_CMD_PAYLOADTYPE | GENTL_NS(BUFFER_INFO_PAYLOADTYPE) | Payload type of the data. |
| KYVP_BUFFER_INFO_CMD_PIXELFORMAT | GENTL_NS(BUFFER_INFO_PIXELFORMAT) | Pixel format of the data. |
| KYVP_BUFFER_INFO_CMD_PIXELFORMAT_NAMESPACE | GENTL_NS(BUFFER_INFO_PIXELFORMAT_NAMESPACE) | Namespace used to interpret KYVP_BUFFER_INFO_CMD_PIXELFORMAT, expressed as a KYVP_PIXELFORMAT_NAMESPACE_IDS value. Returned data type: UINT64. |
| KYVP_BUFFER_INFO_CMD_DELIVERED_IMAGEHEIGHT | GENTL_NS(BUFFER_INFO_DELIVERED_IMAGEHEIGHT) | The number of lines in the current buffer as delivered by the transport mechanism. |
| KYVP_BUFFER_INFO_CMD_DELIVERED_CHUNKPAYLOADSIZE | GENTL_NS(BUFFER_INFO_DELIVERED_CHUNKPAYLOADSIZE) | This information refers for example to the information provided in the GigE Vision image stream data trailer. |

| Name | Value | Description |
|---|---|---|
| KYVP_BUFFER_INFO_CMD_CHUNKLAYOUTID | GENTL_NS(BUFFER_INFO_CHUNKLAYOUTID) | This information refers for example to the information provided in the GigE Vision image stream data leader. The chunk layout id serves as an indicator that the chunk layout has changed and the application should reparse the chunk layout in the buffer. When a chunk layout (availability or position of individual chunks) changes since the last buffer delivered by the device through the same stream, the device must change the chunk layout id. As long as the chunk layout remains stable, the camera must keep the chunk layout id intact. When switching back to a layout, which was already used before, the camera can use the same id again or use a new id. A chunk layout id value of 0 is invalid. It is reserved for use by cameras not supporting the layout id functionality. The algorithm used to compute the chunk layout id is left as quality of implementation. For other technologies this is to be implemented accordingly. |
| KYVP_BUFFER_INFO_CMD_FILENAME | GENTL_NS(BUFFER_INFO_FILENAME) | Filename in case the payload contains a file. |
| KYVP_BUFFER_INFO_CMD_PIXEL_ENDIANNESS | GENTL_NS(BUFFER_INFO_PIXEL_ENDIANNESS) | Endianness of the multi-byte pixel data in the buffer. |
| KYVP_BUFFER_INFO_CMD_DATA_SIZE | GENTL_NS(BUFFER_INFO_DATA_SIZE) | Size of the data intended to be written to the buffer last time it has been filled. |
| KYVP_BUFFER_INFO_CMD_TIMESTAMP_NS | GENTL_NS(BUFFER_INFO_TIMESTAMP_NS) | Timestamp the buffer was acquired, in units of 1 ns (1 000 000 000 ticks per second). |
| KYVP_BUFFER_INFO_CMD_DATA_LARGER_THAN_BUFFER | GENTL_NS(BUFFER_INFO_DATA_LARGER_THAN_BUFFER) | If this values is set to true it indicates that the payload transferred would not fit into the announced buffer and that therefore only parts of the payload or no payload. |

| Name | Value | Description |
|---|---|---|
| KYVP_BUFFER_INFO_CMD_CONTAINS_CHUNKDATA | GENTL_NS(BUFFER_INFO_CONTAINS_CHUNKDATA) | If this values is set to true it indicates that the payload transferred contains chunk data. |
| KYVP_BUFFER_INFO_CMD_BUFFERID | GENTL_NS(BUFFER_INFO_CUSTOM_ID) + 1 | Buffer ID in buffers queue. |
| KYVP_BUFFER_INFO_CMD_INSTANT_FPS | GENTL_NS(BUFFER_INFO_CUSTOM_ID) + 2 | Instant FPS counter. |
| KYVP_BUFFER_INFO_CMD_KYVP_BUFFER_INFO | GENTL_NS(BUFFER_INFO_CUSTOM_ID) + 3 | Pointer to KYVP_BUFFER_INFO struct. |
| KYVP_BUFFER_INFO_CMD_TIMESTAMP_NS_CHRONO | GENTL_NS(BUFFER_INFO_CUSTOM_ID) + 4 | Timestamp the interrupt for this buffer was received (std::chrono::steady_clock), in units of 1 ns (1 000 000 000 ticks per second). |
| KYVP_BUFFER_INFO_CMD_TIMESTAMP_NS_HW | KYVP_CUSTOM_ID_DEBUG + 1 | Timestamp the hw interrupt for this buffer was received (hardware register), in units of 1 ns (1 000 000 000 ticks per second). Used for debugging purposes; activated when the corresponding INTERNAL setting is enabled. |
| KYVP_BUFFER_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid buffer information command. |

#### 6.4.1.2.13 _KYVP_BUFFER_PART_INFO_CMD

Multipart buffer information commands.

##### *6.4.1.2.13.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_BUFFER_PART_INFO_CMD_BASE | GENTL_NS(BUFFER_PART_INFO_BASE) | Address of the first valid byte in this part, excluding padding before the part. Returned data type: PTR. |
| KYVP_BUFFER_PART_INFO_CMD_DATA_SIZE | GENTL_NS(BUFFER_PART_INFO_DATA_SIZE) | Number of valid data bytes in this part, excluding padding between parts. For variable-size data, reports the actual valid size. Returned data type: SIZET. |
| KYVP_BUFFER_PART_INFO_CMD_DATA_TYPE | GENTL_NS(BUFFER_PART_INFO_DATA_TYPE) | Kind of data in this part, expressed as a KYVP_PARTDATATYPE_IDS value. Returned data type: SIZET. |
| KYVP_BUFFER_PART_INFO_CMD_DATA_FORMAT | GENTL_NS(BUFFER_PART_INFO_DATA_FORMAT) | Format identifier for items such as pixels in this part. Interpret it using KYVP_BUFFER_PART_INFO_CMD_DATA_FORMAT_NAMESPACE and the part data type. Returned data type: UINT64. |

| Name | Value | Description |
|---|---|---|
| KYVP_BUFFER_PART_INFO_CMD_DATA_FORMAT_NAMESPACE | GENTL_NS(BUFFER_PART_INFO_DATA_FORMAT_NAMESPACE) | Namespace of the data-format identifier, expressed as a KYVP_PIXELFORMAT_NAMESPACE_IDS value. Returned data type: UINT64. |
| KYVP_BUFFER_PART_INFO_CMD_WIDTH | GENTL_NS(BUFFER_PART_INFO_WIDTH) | Width of the part image in pixels, when applicable to its data type. Returned data type: SIZET. |
| KYVP_BUFFER_PART_INFO_CMD_HEIGHT | GENTL_NS(BUFFER_PART_INFO_HEIGHT) | Expected height of the part image in pixels, when applicable to its data type. Returned data type: SIZET. |
| KYVP_BUFFER_PART_INFO_CMD_XOFFSET | GENTL_NS(BUFFER_PART_INFO_XOFFSET) | Horizontal offset of this part image from the image origin, in pixels. Returned data type: SIZET. |
| KYVP_BUFFER_PART_INFO_CMD_YOFFSET | GENTL_NS(BUFFER_PART_INFO_YOFFSET) | Vertical offset of this part image from the image origin, in pixels. Returned data type: SIZET. |
| KYVP_BUFFER_PART_INFO_CMD_XPADDING | GENTL_NS(BUFFER_PART_INFO_XPADDING) | Horizontal padding of the part image, measured in pixels. Returned data type: SIZET. |
| KYVP_BUFFER_PART_INFO_CMD_SOURCE_ID | GENTL_NS(BUFFER_PART_INFO_SOURCE_ID) | Identifier used to associate parts from the same source, such as an image and its confidence map. Source IDs need not form a contiguous sequence. Returned data type: UINT64. |
| KYVP_BUFFER_PART_INFO_CMD_DELIVERED_IMAGEHEIGHT | GENTL_NS(BUFFER_PART_INFO_DELIVERED_IMAGEHEIGHT) | Number of image lines actually delivered in this part. Variable-height line-scan data can contain fewer lines than the configured height. Returned data type: SIZET. |
| KYVP_BUFFER_PART_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid buffer-part information command. |

#### 6.4.1.2.14 _KYVP_PAYLOADTYPE_INFO_IDS

Buffer payload types.

##### *6.4.1.2.14.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_PAYLOADTYPE_INFO_IDS_UNKNOWN | GENTL_NS(PAYLOAD_TYPE_UNKNOWN) | The producer does not recognize the payload type; the consumer can treat its contents as raw data. |
| KYVP_PAYLOADTYPE_INFO_IDS_IMAGE | GENTL_NS(PAYLOAD_TYPE_IMAGE) | Image payload, which can also contain chunk data. Query KYVP_BUFFER_INFO_CMD_CONTAINS_CHUNKDATA to check for chunks. |
| KYVP_PAYLOADTYPE_INFO_IDS_RAW_DATA | GENTL_NS(PAYLOAD_TYPE_RAW_DATA) | Unspecified raw bytes, for example acquisition statistics, without an implied image format. |
| KYVP_PAYLOADTYPE_INFO_IDS_FILE | GENTL_NS(PAYLOAD_TYPE_FILE) | File contents carried in the buffer. KYVP_BUFFER_INFO_CMD_FILENAME provides a filename that can help identify the format. |

| Name | Value | Description |
|---|---|---|
| KYVP_PAYLOADTYPE_INFO_IDS_CHUNK_DATA | GENTL_NS(PAYLOAD_TYPE_CHUNK_DATA) | DeprecatedSince GenTL 1.5, chunk data may accompany other payload types. Query KYVP_BUFFER_INFO_CMD_CONTAINS_CHUNKDATA instead. |
| KYVP_PAYLOADTYPE_INFO_IDS_JPEG | GENTL_NS(PAYLOAD_TYPE_JPEG) | JPEG-encoded payload using the GigE Vision 2.0 representation; buffer information commands provide additional metadata. |
| KYVP_PAYLOADTYPE_INFO_IDS_JPEG2000 | GENTL_NS(PAYLOAD_TYPE_JPEG2000) | JPEG 2000-encoded payload using the GigE Vision 2.0 representation; buffer information commands provide additional metadata. |
| KYVP_PAYLOADTYPE_INFO_IDS_H264 | GENTL_NS(PAYLOAD_TYPE_H264) | H.264-encoded payload using the GigE Vision 2.0 representation; buffer information commands provide additional metadata. |
| KYVP_PAYLOADTYPE_INFO_IDS_CHUNK_ONLY | GENTL_NS(PAYLOAD_TYPE_CHUNK_ONLY) | A payload consisting entirely of chunk data, without an accompanying image or other payload. |
| KYVP_PAYLOADTYPE_INFO_IDS_DEVICE_SPECIFIC | GENTL_NS(PAYLOAD_TYPE_DEVICE_SPECIFIC) | Payload whose format is defined by the device; buffer information commands can provide additional metadata. |
| KYVP_PAYLOADTYPE_INFO_IDS_MULTI_PART | GENTL_NS(PAYLOAD_TYPE_MULTI_PART) | A payload containing multiple data parts, each with its own data type and format. See KYVP_BUFFER_PART_INFO_CMD for part metadata. |
| KYVP_PAYLOADTYPE_INFO_IDS_INVALID | 0xFFFF | SDK-specific sentinel for an invalid payload type. |

#### 6.4.1.2.15 _KYVP_PIXELFORMAT_NAMESPACE_IDS

Pixel format namespaces.

##### *6.4.1.2.15.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_PIXELFORMAT_NAMESPACE_IDS_UNKNOWN | GENTL_NS(PIXELFORMAT_NAMESPACE_UNKNOWN) | The producer does not know how to interpret the pixel-format identifier. |
| KYVP_PIXELFORMAT_NAMESPACE_IDS_GEV | GENTL_NS(PIXELFORMAT_NAMESPACE_GEV) | Pixel-format identifiers follow the GigE Vision 1.x definitions. |

| Name | Value | Description |
|---|---|---|
| KYVP_PIXELFORMAT_NAMESPACE_IDS_IIDC | GENTL_NS(PIXELFORMAT_NAMESPACE_IIDC) | Pixel-format identifiers follow the IIDC 1.x definitions. |
| KYVP_PIXELFORMAT_NAMESPACE_IDS_PFNC_16BIT | GENTL_NS(PIXELFORMAT_NAMESPACE_PFNC_16BIT) | Pixel-format identifiers use the 16-bit PFNC namespace. GenTL recommends the 32-bit PFNC namespace when possible for broader consumer support. |
| KYVP_PIXELFORMAT_NAMESPACE_IDS_PFNC_32BIT | GENTL_NS(PIXELFORMAT_NAMESPACE_PFNC_32BIT) | Pixel-format identifiers use the 32-bit PFNC namespace. |
| KYVP_PIXELFORMAT_NAMESPACE_IDS_INVALID | 0xFFFF | SDK-specific sentinel for an invalid pixel-format namespace. |

#### 6.4.1.2.16 _KYVP_PIXELENDIANNESS_IDS

Pixel data byte orders.

##### *6.4.1.2.16.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_PIXELENDIANNESS_IDS_UNKNOWN | GENTL_NS(PIXELENDIANNESS_UNKNOWN) | The producer does not know the byte order of multibyte pixel data. |
| KYVP_PIXELENDIANNESS_IDS_LITTLE | GENTL_NS(PIXELENDIANNESS_LITTLE) | Multibyte pixel values store the least significant byte first. |
| KYVP_PIXELENDIANNESS_IDS_BIG | GENTL_NS(PIXELENDIANNESS_BIG) | Multibyte pixel values store the most significant byte first. |
| KYVP_PIXELENDIANNESS_IDS_INVALID | 0xFFFF | SDK-specific sentinel for an invalid pixel byte order. |

#### 6.4.1.2.17 _KYVP_PARTDATATYPE_IDS

Multipart buffer data types.

#### *6.4.1.2.17.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_PARTDATATYPE_IDS_UNKNOWN | GENTL_NS(PART_DATATYPE_UNKNOWN) | The producer does not recognize the part data type; the consumer can treat the part as raw data. |
| KYVP_PARTDATATYPE_IDS_2D_IMAGE | GENTL_NS(PART_DATATYPE_2D_IMAGE) | A complete monochrom e or color 2D image, including all of its pixel data. |
| KYVP_PARTDATATYPE_IDS_2D_PLANE_BIPLANAR | GENTL_NS(PART_DATATYPE_2D_PLANE_BIPLANAR) | One color plane of a 2D image made of two planes. The planes occupy consecutive buffer parts. |
| KYVP_PARTDATATYPE_IDS_2D_PLANE_TRIPLANAR | GENTL_NS(PART_DATATYPE_2D_PLANE_TRIPLANAR) | One color plane of a 2D image made of three planes. The planes occupy consecutive buffer parts. |
| KYVP_PARTDATATYPE_IDS_2D_PLANE_QUADPLANAR | GENTL_NS(PART_DATATYPE_2D_PLANE_QUADPLANAR) | One color plane of a 2D image made of four planes. The planes occupy consecutive buffer parts. |

| Name | Value | Description |
|---|---|---|
| KYVP_PARTDATATYPE_IDS_3D_IMAGE | GENTL_NS(PART_DATATYPE_3D_IMAGE) | Complete 3D image data, such as pixel coordinates or a depth map, contained in one part. |
| KYVP_PARTDATATYPE_IDS_3D_PLANE_BIPLANAR | GENTL_NS(PART_DATATYPE_3D_PLANE_BIPLANAR) | One coordinate plane of a 3D image made of two planes. The planes occupy consecutive buffer parts. |
| KYVP_PARTDATATYPE_IDS_3D_PLANE_TRIPLANAR | GENTL_NS(PART_DATATYPE_3D_PLANE_TRIPLANAR) | One coordinate plane of a 3D image made of three planes. The planes occupy consecutive buffer parts. |
| KYVP_PARTDATATYPE_IDS_3D_PLANE_QUADPLANAR | GENTL_NS(PART_DATATYPE_3D_PLANE_QUADPLANAR) | One coordinate plane of a 3D image made of four planes. The planes occupy consecutive buffer parts. |

| Name | Value | Description |
|---|---|---|
| KYVP_PARTDATATYPE_IDS_CONFIDENCE_MAP | GENTL_NS(PART_DATATYPE_CONFIDENCE_MAP) | Per-pixel validity or confidence values paired with image parts of matching dimensions. Each value describes the correspondin g image pixel; use a PFNC Confidence format. |
| KYVP_PARTDATATYPE_IDS_INVALID | 0xFFFF | SDK-specific sentinel for an invalid buffer-part data type. |

#### 6.4.1.2.18 _KYVP_PORT_INFO_CMD

Port information commands.

##### *6.4.1.2.18.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_PORT_INFO_CMD_ID | GENTL_NS(PORT_INFO_ID) | Identifier of the module associated with the port. A remote device uses its local device ID; a buffer uses its address in hexadecimal without a 0x prefix. Returned data type: STRING. |
| KYVP_PORT_INFO_CMD_VENDOR | GENTL_NS(PORT_INFO_VENDOR) | Vendor of the underlying module, or the producer vendor when the module has no separate vendor. Returned data type: STRING. |
| KYVP_PORT_INFO_CMD_MODEL | GENTL_NS(PORT_INFO_MODEL) | Model of the underlying module, or the producer model when the module has no separate model. Returned data type: STRING. |
| KYVP_PORT_INFO_CMD_TLTYPE | GENTL_NS(PORT_INFO_TLTYPE) | Name of the transport technology supported by the port. Returned data type: STRING. |
| KYVP_PORT_INFO_CMD_MODULE | GENTL_NS(PORT_INFO_MODULE) | Module category: TLSystem, TLInterface, TLDevice, TLDataStream, TLBuffer, or Device for a remote device. Returned data type: STRING. |

| Name | Value | Description |
|---|---|---|
| KYVP_PORT_INFO_CMD_LITTLE_ENDIAN | GENTL_NS(PORT_INFO_LITTLE_ENDIAN) | Whether the port data uses little-endian byte order. Returned data type: BOOL8. |
| KYVP_PORT_INFO_CMD_BIG_ENDIAN | GENTL_NS(PORT_INFO_BIG_ENDIAN) | Whether the port data uses big-endian byte order. Returned data type: BOOL8. |
| KYVP_PORT_INFO_CMD_ACCESS_READ | GENTL_NS(PORT_INFO_ACCESS_READ) | Whether reading from the port is permitted. Returned data type: BOOL8. |
| KYVP_PORT_INFO_CMD_ACCESS_WRITE | GENTL_NS(PORT_INFO_ACCESS_WRITE) | Whether writing to the port is permitted. Returned data type: BOOL8. |
| KYVP_PORT_INFO_CMD_ACCESS_NA | GENTL_NS(PORT_INFO_ACCESS_NA) | Whether the port is currently unavailable for access. Returned data type: BOOL8. |
| KYVP_PORT_INFO_CMD_ACCESS_NI | GENTL_NS(PORT_INFO_ACCESS_NI) | Whether no port is implemented. GenTL permits this condition only for buffer modules. Returned data type: BOOL8. |
| KYVP_PORT_INFO_CMD_VERSION | GENTL_NS(PORT_INFO_VERSION) | Version identifier of the port. Returned data type: STRING. |
| KYVP_PORT_INFO_CMD_PORTNAME | GENTL_NS(PORT_INFO_PORTNAME) | Port name used in the XML description to connect the port to the module nodemap. Returned data type: STRING. |
| KYVP_PORT_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid port information command. |

#### 6.4.1.2.19 _KYVP_URL_SCHEME_IDS

Port URL schemes.

##### *6.4.1.2.19.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_URL_SCHEME_IDS_LOCAL | GENTL_NS(URL_SCHEME_LOCAL) | The XML file resides in the port register map. Query KYVP_URL_INFO_CMD_FILE_REGISTER_ADDRESS and KYVP_URL_INFO_CMD_FILE_SIZE to locate it. |
| KYVP_URL_SCHEME_IDS_HTTP | GENTL_NS(URL_SCHEME_HTTP) | Retrieve the XML file from a web server using HTTP; KYVP_URL_INFO_CMD_URL supplies the address. |
| KYVP_URL_SCHEME_IDS_FILE | GENTL_NS(URL_SCHEME_FILE) | The XML file resides on the local filesystem; KYVP_URL_INFO_CMD_FILENAME supplies its filename. |
| KYVP_URL_SCHEME_IDS_INVALID | 0xFFFF | SDK-specific sentinel for an invalid URL scheme. |

#### 6.4.1.2.20 _KYVP_URL_INFO_CMD

Port URL information commands.

#### *6.4.1.2.20.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_URL_INFO_CMD_URL | GENTL_NS(URL_INFO_URL) | URL identifying the XML description associated with the port. Returned data type: STRING. |
| KYVP_URL_INFO_CMD_SCHEMA_VER_MAJOR | GENTL_NS(URL_INFO_SCHEMA_VER_MAJOR) | Major version of the XML schema used by the referenced description. Returned data type: INT32. |
| KYVP_URL_INFO_CMD_SCHEMA_VER_MINOR | GENTL_NS(URL_INFO_SCHEMA_VER_MINOR) | Minor version of the XML schema used by the referenced description. Returned data type: INT32. |
| KYVP_URL_INFO_CMD_FILE_VER_MAJOR | GENTL_NS(URL_INFO_FILE_VER_MAJOR) | Major version component of the referenced XML file. Returned data type: INT32. |
| KYVP_URL_INFO_CMD_FILE_VER_MINOR | GENTL_NS(URL_INFO_FILE_VER_MINOR) | Minor version component of the referenced XML file. Returned data type: INT32. |
| KYVP_URL_INFO_CMD_FILE_VER_SUBMINOR | GENTL_NS(URL_INFO_FILE_VER_SUBMINOR) | Subminor version component of the referenced XML file. Returned data type: INT32. |
| KYVP_URL_INFO_CMD_FILE_SHA1_HASH | GENTL_NS(URL_INFO_FILE_SHA1_HASH) | SHA-1 digest of the referenced XML file, provided as 20 bytes (160 bits). Returned data type: BUFFER. |
| KYVP_URL_INFO_CMD_FILE_REGISTER_ADDRESS | GENTL_NS(URL_INFO_FILE_REGISTER_ADDRESS) | Register-map address of the XML file. Available only when the file is stored in the device register map. Returned data type: UINT64. |
| KYVP_URL_INFO_CMD_FILE_SIZE | GENTL_NS(URL_INFO_FILE_SIZE) | Size of the referenced XML file in bytes. Returned data type: UINT64. |

| Name | Value | Description |
|---|---|---|
| KYVP_URL_INFO_CMD_SCHEME | GENTL_NS(URL_INFO_SCHEME) | Location scheme expressed as a KYVP_URL_SCHEME_IDS value. Returned data type: INT32. |
| KYVP_URL_INFO_CMD_FILENAME | GENTL_NS(URL_INFO_FILENAME) | Filename for the file scheme, or a suggested filename for XML held in the local register map. Returned data type: STRING. |
| KYVP_URL_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid URL information command. |

#### 6.4.1.2.21 _KYVP_EVENT_TYPE

Event types available for registration.

##### *6.4.1.2.21.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_EVENT_TYPE_ERROR_ | GENTL_NS(EVENT_ERROR) | Notification on module errors. |
| KYVP_EVENT_TYPE_NEW_BUFFER | GENTL_NS(EVENT_NEW_BUFFER) | Notification on newly filled buffers. |
| KYVP_EVENT_TYPE_FEATURE_INVALIDATE | GENTL_NS(EVENT_FEATURE_INVALIDATE) | Notification if a feature was changed by the Producer driver and thus needs to be invalidated. |
| KYVP_EVENT_TYPE_FEATURE_CHANGE | GENTL_NS(EVENT_FEATURE_CHANGE) | Notification if the Producer driver wants to manually set a feature in the instance. |
| KYVP_EVENT_TYPE_REMOTE_DEVICE | GENTL_NS(EVENT_REMOTE_DEVICE) | Notification if the Producer wants to inform the instance of the remote device that a GenApi compatible event was fired. |
| KYVP_EVENT_TYPE_MODULE | GENTL_NS(EVENT_MODULE) | Notification that one GenTL Producer module wants to inform the GenICam GenApi instance of this module that a GenApi compatible event was fired. This Event is to be registered on any module handle except on the Remote Device. |
| KYVP_EVENT_TYPE_PCIINTERFACE_EVENT | GENTL_NS(EVENT_CUSTOM_ID + 1) | KAYA-specific PCI interface notification. KYVPLibTL_EventGetData() returns a KYVP_PCIINTERFACE_GENERIC_EVENT payload. |
| KYVP_EVENT_TYPE_DEVICE_EVENT | GENTL_NS(EVENT_CUSTOM_ID + 2) | KAYA-specific device notification. KYVPLibTL_EventGetData() returns a KYVP_DEVICE_GENERIC_EVENT payload. |

| Name | Value | Description |
|---|---|---|
| KYVP_EVENT_TYPE_INVALID | 0xFFFF | SDK-specific sentinel for an invalid event type. |

#### 6.4.1.2.22 _KYVP_EVENT_INFO_CMD

Event information commands.

##### *6.4.1.2.22.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_EVENT_INFO_CMD_TYPE | GENTL_NS(EVENT_EVENT_TYPE) | The event type of the event handle. |
| KYVP_EVENT_INFO_CMD_NUM_IN_QUEUE | GENTL_NS(EVENT_NUM_IN_QUEUE) | Number of events in the event data queue. |
| KYVP_EVENT_INFO_CMD_NUM_FIRED | GENTL_NS(EVENT_NUM_FIRED) | Number of events that were fired since the registration of the event. |
| KYVP_EVENT_INFO_CMD_EVENT_SIZE_MAX | GENTL_NS(EVENT_SIZE_MAX) | Maximum size in bytes of the event data provided by the event. |
| KYVP_EVENT_INFO_CMD_INFO_DATA_SIZE_MAX | GENTL_NS(EVENT_INFO_DATA_SIZE_MAX) | Maximum size in bytes of the information output buffer. |
| KYVP_EVENT_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid event information command. |

#### 6.4.1.2.23 _KYVP_EVENT_DATA_INFO_CMD

Event data information commands.

##### *6.4.1.2.23.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_EVENT_DATA_INFO_CMD_ID | GENTL_NS(EVENT_DATA_ID) | Attribute in the event data to identifythe object or feature the event refers to. |
| KYVP_EVENT_DATA_INFO_CMD_VALUE | GENTL_NS(EVENT_DATA_VALUE) | Defines additional data to an ID. |
| KYVP_EVENT_DATA_INFO_CMD_NUMID | GENTL_NS(EVENT_DATA_NUMID) | Attribute in the event data to identify the object or feature the event refers to. |
| KYVP_EVENT_DATA_INFO_CMD_INVALID | 0xFFFF | SDK-specific sentinel for an invalid event-data information command. |

#### 6.4.1.2.24 _KYVP_LINK_STATUS

Link connection states.

##### *6.4.1.2.24.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_KYVP_LINK_STATUS_DISCONNECTED | 0x00 | The link is disconnected, no link has been established. |
| KYVP_KYVP_LINK_STATUS_SYNCHRONIZED | 0x01 | The link is synchronized and awaiting connection. |
| KYVP_KYVP_LINK_STATUS_CONNECTING | 0x10 | A connection is trying to be established on link. |
| KYVP_KYVP_LINK_STATUS_CONNECTED | 0x11 | The link is connected and assigned to device. |

#### 6.4.1.2.25 _KYVP_LINK_STATUS_CMD

Link status commands.

#### *6.4.1.2.25.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_LINK_STATUS_CMD_LINK_CONNECTION_STATUS | 0x00 | Return the link connection status. |
| KYVP_LINK_STATUS_CMD_LINK_LOCK_STATUS | 0x01 | Return the link lock status. |
| KYVP_LINK_STATUS_CMD_INVALID | 0xFF | SDK-specific sentinel for an invalid link-status command. |

### 6.4.1.2.26 _KYVP_STREAM_BUFFERS_TYPE

Data stream buffer modes.

#### *6.4.1.2.26.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_STREAM_BUFFERS_TYPE_QUEUED_BUFFERS | 0x00 | Data stream buffers type queued. |
| KYVP_STREAM_BUFFERS_TYPE_CYCLIC_BUFFERS | 0x01 | Data stream buffers type cyclic. |
| KYVP_STREAM_BUFFERS_TYPE_INVALID | 0xFF | SDK-specific sentinel for an invalid stream buffer mode. |

### 6.4.1.2.27 _DS_PACKED_DATA_MODE

Packed data layouts.

#### *6.4.1.2.27.1 Values*

| Name | Value | Description |
|---|---|---|
| DS_PACKED_DATA_MODE_UNPACKED | 0 | Data unpacked. |
| DS_PACKED_DATA_MODE_LINE32ALIGNED | 1 | Data packed row aligned 32. |
| DS_PACKED_DATA_MODE_LINE32ALIGNED_REVERSE | 2 | Data packed row aligned 32 reverse. |
| DS_PACKED_DATA_MODE_INVALID | 0xFF | SDK-specific sentinel for an invalid packed data layout. |

### 6.4.1.2.28 _DS_PACKED_DATA_PROCESSING_MODE

Packed data processing modes.

#### *6.4.1.2.28.1 Values*

| Name | Value | Description |
|---|---|---|
| DS_PACKED_DATA_PROCESSING_MODE_MSB8 | 0 | Trim packed data bits to most significant 8-bits. |
| DS_PACKED_DATA_PROCESSING_MODE_EXPAND16 | 1 | Data packed bits expand to 16-bits. |
| DS_PACKED_DATA_PROCESSING_MODE__INVALID | 0xFF | SDK-specific sentinel for an invalid packed data processing mode. |

## 6.4.1.3 Type definitions

### 6.4.1.3.1 KYVP_INFO_DATATYPE

```
typedef enum _KYVP_INFO_DATATYPE KYVP_INFO_DATATYPE
```

Data types returned by information queries.

Defines the data type possible for the various Info functions. The data type itself may define its size. For buffer or string types the piSize parameter must be used to query the actual amount of data being written.

### 6.4.1.3.2 KYVP_TL_CHAR_ENCODING

```
typedef enum _KYVP_TL_CHAR_ENCODING KYVP_TL_CHAR_ENCODING
```

Character encodings used by the transport layer.

### 6.4.1.3.3 KYVP_TL_INFO_CMD

```
typedef enum _KYVP_TL_INFO_CMD KYVP_TL_INFO_CMD
```

Transport layer information commands.

### 6.4.1.3.4 KYVP_INTERFACE_INFO_CMD

```
typedef enum _KYVP_INTERFACE_INFO_CMD KYVP_INTERFACE_INFO_CMD
```

Interface information commands.

#### 6.4.1.3.5 KYVP_DEVICE_ACCESS_FLAGS

```
typedef enum _KYVP_DEVICE_ACCESS_FLAGS KYVP_DEVICE_ACCESS_FLAGS
```

Device access modes requested when opening a device.

#### 6.4.1.3.6 KYVP_DEVICE_ACCESS_STATUS

```
typedef enum _KYVP_DEVICE_ACCESS_STATUS KYVP_DEVICE_ACCESS_STATUS
```

Device access status reported by the producer.

#### 6.4.1.3.7 KYVP_DEVICE_INFO_CMD

```
typedef enum _KYVP_DEVICE_INFO_CMD KYVP_DEVICE_INFO_CMD
```

Device information commands.

#### 6.4.1.3.8 KYVP_ACQ_START_FLAGS

```
typedef enum _KYVP_ACQ_START_FLAGS KYVP_ACQ_START_FLAGS
```

Acquisition start flags.

#### 6.4.1.3.9 KYVP_ACQ_STOP_FLAGS

```
typedef enum _KYVP_ACQ_STOP_FLAGS KYVP_ACQ_STOP_FLAGS
```

Acquisition stop flags.

#### 6.4.1.3.10 KYVP_ACQ_QUEUE_TYPE

```
typedef enum _KYVP_ACQ_QUEUE_TYPE KYVP_ACQ_QUEUE_TYPE
```

Buffer queue operations.

#### 6.4.1.3.11 KYVP_STREAM_INFO_CMD

```
typedef enum _KYVP_STREAM_INFO_CMD KYVP_STREAM_INFO_CMD
```

Data stream information commands.

Defines the data type possible for the various Info functions. The data type itself may define its size. For buffer or string types the piSize parameter must be used to query the actual amount of data being written.

#### 6.4.1.3.12 KYVP_BUFFER_INFO_CMD

```
typedef enum _KYVP_BUFFER_INFO_CMD KYVP_BUFFER_INFO_CMD
```

Buffer information commands.

#### 6.4.1.3.13 KYVP_BUFFER_PART_INFO_CMD

```
typedef enum _KYVP_BUFFER_PART_INFO_CMD KYVP_BUFFER_PART_INFO_CMD
```

Multipart buffer information commands.

#### 6.4.1.3.14 KYVP_PAYLOADTYPE_INFO_IDS

```
typedef enum _KYVP_PAYLOADTYPE_INFO_IDS KYVP_PAYLOADTYPE_INFO_IDS
```

Buffer payload types.

#### 6.4.1.3.15 KYVP_PIXELFORMAT_NAMESPACE_IDS

```
typedef enum _KYVP_PIXELFORMAT_NAMESPACE_IDS KYVP_PIXELFORMAT_NAMESPACE_IDS
```

Pixel format namespaces.

#### 6.4.1.3.16 KYVP_PIXELENDIANNESS_IDS

```
typedef enum _KYVP_PIXELENDIANNESS_IDS KYVP_PIXELENDIANNESS_IDS
```

Pixel data byte orders.

#### 6.4.1.3.17 KYVP_PARTDATATYPE_IDS

```
typedef enum _KYVP_PARTDATATYPE_IDS KYVP_PARTDATATYPE_IDS
```

Multipart buffer data types.

#### 6.4.1.3.18 KYVP_PORT_INFO_CMD

```
typedef enum _KYVP_PORT_INFO_CMD KYVP_PORT_INFO_CMD
```

Port information commands.

#### 6.4.1.3.19 KYVP_URL_SCHEME_IDS

```
typedef enum _KYVP_URL_SCHEME_IDS KYVP_URL_SCHEME_IDS
```

Port URL schemes.

#### 6.4.1.3.20 KYVP_URL_INFO_CMD

`typedef enum _KYVP_URL_INFO_CMD KYVP_URL_INFO_CMD`

Port URL information commands.

#### 6.4.1.3.21 KYVP_EVENT_TYPE

`typedef enum _KYVP_EVENT_TYPE KYVP_EVENT_TYPE`

Event types available for registration.

#### 6.4.1.3.22 KYVP_EVENT_INFO_CMD

`typedef enum _KYVP_EVENT_INFO_CMD KYVP_EVENT_INFO_CMD`

Event information commands.

#### 6.4.1.3.23 KYVP_EVENT_DATA_INFO_CMD

`typedef enum _KYVP_EVENT_DATA_INFO_CMD KYVP_EVENT_DATA_INFO_CMD`

Event data information commands.

#### 6.4.1.3.24 KYVP_LINK_STATUS

`typedef enum _KYVP_LINK_STATUS KYVP_LINK_STATUS`

Link connection states.

#### 6.4.1.3.25 KYVP_LINK_STATUS_CMD

`typedef enum _KYVP_LINK_STATUS_CMD KYVP_LINK_STATUS_CMD`

Link status commands.

#### 6.4.1.3.26 KYVP_STREAM_BUFFERS_TYPE

`typedef enum _KYVP_STREAM_BUFFERS_TYPE KYVP_STREAM_BUFFERS_TYPE`

Data stream buffer modes.

#### 6.4.1.3.27 DS_PACKED_DATA_MODE

`typedef enum _DS_PACKED_DATA_MODE DS_PACKED_DATA_MODE`

Packed data layouts.

#### 6.4.1.3.28 DS_PACKED_DATA_PROCESSING_MODE

`typedef enum _DS_PACKED_DATA_PROCESSING_MODE DS_PACKED_DATA_PROCESSING_MODE`

Packed data processing modes.

#### 6.4.1.3.29 KYVP_TL_HANDLE

`typedef struct _KYVP_TL_HANDLE KYVP_TL_HANDLE`

Transport layer system handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.3.30 KYVP_PCI_INTERFACE_HANDLE

`typedef struct _KYVP_PCI_INTERFACE_HANDLE KYVP_PCI_INTERFACE_HANDLE`

PCI interface handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.3.31 KYVP_DEVICE_HANDLE

`typedef struct _KYVP_DEVICE_HANDLE KYVP_DEVICE_HANDLE`

Local device handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.3.32 KYVP_REMOTE_DEVICE_HANDLE

`typedef struct _KYVP_REMOTE_DEVICE_HANDLE KYVP_REMOTE_DEVICE_HANDLE`

Remote device handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.3.33 KYVP_STREAM_HANDLE

`typedef struct _KYVP_STREAM_HANDLE KYVP_STREAM_HANDLE`

Data stream handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.3.34 KYVP_BUFFER_HANDLE

```
typedef struct _KYVP_BUFFER_HANDLE KYVP_BUFFER_HANDLE
```

Buffer handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.3.35 KYVP_EVENT_HANDLE

```
typedef struct _KYVP_EVENT_HANDLE KYVP_EVENT_HANDLE
```

Event handle.

Treat this type as opaque. Use the handle comparison functions or macros.

### 6.4.1.4 Variables

#### 6.4.1.4.1 KYVP_TL_HANDLE_NULL

```
const KYVP_TL_HANDLE KYVP_TL_HANDLE_NULL
```

#### 6.4.1.4.2 KYVP_TL_HANDLE_INVALID

```
const KYVP_TL_HANDLE KYVP_TL_HANDLE_INVALID
```

#### 6.4.1.4.3 KYVP_PCI_INTERFACE_HANDLE_NULL

```
const KYVP_PCI_INTERFACE_HANDLE KYVP_PCI_INTERFACE_HANDLE_NULL
```

#### 6.4.1.4.4 KYVP_PCI_INTERFACE_HANDLE_INVALID

```
const KYVP_PCI_INTERFACE_HANDLE KYVP_PCI_INTERFACE_HANDLE_INVALID
```

#### 6.4.1.4.5 KYVP_DEVICE_HANDLE_NULL

```
const KYVP_DEVICE_HANDLE KYVP_DEVICE_HANDLE_NULL
```

#### 6.4.1.4.6 KYVP_DEVICE_HANDLE_INVALID

```
const KYVP_DEVICE_HANDLE KYVP_DEVICE_HANDLE_INVALID
```

#### 6.4.1.4.7 KYVP_REMOTE_DEVICE_HANDLE_NULL

```
const KYVP_REMOTE_DEVICE_HANDLE KYVP_REMOTE_DEVICE_HANDLE_NULL
```

#### 6.4.1.4.8 KYVP_REMOTE_DEVICE_HANDLE_INVALID

```
const KYVP_REMOTE_DEVICE_HANDLE KYVP_REMOTE_DEVICE_HANDLE_INVALID
```

#### 6.4.1.4.9 KYVP_STREAM_HANDLE_NULL

```
const KYVP_STREAM_HANDLE KYVP_STREAM_HANDLE_NULL
```

#### 6.4.1.4.10 KYVP_STREAM_HANDLE_INVALID

```
const KYVP_STREAM_HANDLE KYVP_STREAM_HANDLE_INVALID
```

#### 6.4.1.4.11 KYVP_BUFFER_HANDLE_NULL

```
const KYVP_BUFFER_HANDLE KYVP_BUFFER_HANDLE_NULL
```

#### 6.4.1.4.12 KYVP_BUFFER_HANDLE_INVALID

```
const KYVP_BUFFER_HANDLE KYVP_BUFFER_HANDLE_INVALID
```

#### 6.4.1.4.13 KYVP_EVENT_HANDLE_NULL

```
const KYVP_EVENT_HANDLE KYVP_EVENT_HANDLE_NULL
```

#### 6.4.1.4.14 KYVP_EVENT_HANDLE_INVALID

```
const KYVP_EVENT_HANDLE KYVP_EVENT_HANDLE_INVALID
```

### 6.4.1.5 Functions

#### 6.4.1.5.1 KYVPLibTL_InitLib

```
KY_RESULT KYVPLibTL_InitLib(void)
```

Initializes the library and the GenTL producer driver.

Call this function before using other library functions.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.2 KYVPLibTL_CloseLib

```
KY_RESULT KYVPLibTL_CloseLib(void)
```

Closes the library and releases the GenTL producer resources.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.3 KYVPLibTL_GetPortURL

```
KY_RESULT KYVPLibTL_GetPortURL(KYVPLibTL_GetPortUrl_Args *_pArgs)
```

Retrieves port URL information for the system module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_GetPortUrl_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_GetPortUrl_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.4 KYVPLibTL_ReadPort

```
KY_RESULT KYVPLibTL_ReadPort(KYVPLibTL_ReadPort_Args *_pArgs)
```

Reads bytes from the specified address on the system port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_ReadPort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_ReadPort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.5 KYVPLibTL_WritePort

```
KY_RESULT KYVPLibTL_WritePort(KYVPLibTL_WritePort_Args *_pArgs)
```

Writes bytes to the specified address on the system port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_WritePort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_WritePort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.6 KYVPLibTL_PCIInterface_GetPortURL

```
KY_RESULT KYVPLibTL_PCIInterface_GetPortURL(KYVPLibTL_PCIInterface_GetPortUrl_Args *_pArgs)
```

Retrieves port URL information for the PCI interface module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_PCIInterface_GetPortUrl_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_PCIInterface_GetPortUrl_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.7 KYVPLibTL_PCIInterface_ReadPort

```
KY_RESULT KYVPLibTL_PCIInterface_ReadPort(KYVPLibTL_PCIInterface_ReadPort_Args *_pArgs)
```

Reads bytes from the specified address on the PCI interface port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_PCIInterface_ReadPort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_PCIInterface_ReadPort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.8 KYVPLibTL_PCIInterface_WritePort

```
KY_RESULT KYVPLibTL_PCIInterface_WritePort(KYVPLibTL_PCIInterface_WritePort_Args *_pArgs)
```

Writes bytes to the specified address on the PCI interface port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_PCIInterface_WritePort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_PCIInterface_WritePort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.9 KYVPLibTL_Device_GetPortURL

```
KY_RESULT KYVPLibTL_Device_GetPortURL(KYVPLibTL_Device_GetPortUrl_Args *_pArgs)
```

Retrieves port URL information for the device module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_Device_GetPortUrl_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_Device_GetPortUrl_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.10 KYVPLibTL_Device_ReadPort

```
KY_RESULT KYVPLibTL_Device_ReadPort(KYVPLibTL_Device_ReadPort_Args *_pArgs)
```

Reads bytes from the specified address on the device port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_Device_ReadPort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_Device_ReadPort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.11 KYVPLibTL_Device_WritePort

```
KY_RESULT KYVPLibTL_Device_WritePort(KYVPLibTL_Device_WritePort_Args *_pArgs)
```

Writes bytes to the specified address on the device port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_Device_WritePort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_Device_WritePort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.12 KYVPLibTL_RemoteDevice_GetPortURL

```
KY_RESULT KYVPLibTL_RemoteDevice_GetPortURL(KYVPLibTL_RemoteDevice_GetPortUrl_Args *_pArgs)
```

Retrieves port URL information for the remote device module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_RemoteDevice_GetPortUrl_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_RemoteDevice_GetPortUrl_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.13 KYVPLibTL_RemoteDevice_ReadPort

```
KY_RESULT KYVPLibTL_RemoteDevice_ReadPort(KYVPLibTL_RemoteDevice_ReadPort_Args *_pArgs)
```

Reads bytes from the specified address on the remote device port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_RemoteDevice_ReadPort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_RemoteDevice_ReadPort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.14 KYVPLibTL_RemoteDevice_WritePort

```
KY_RESULT KYVPLibTL_RemoteDevice_WritePort(KYVPLibTL_RemoteDevice_WritePort_Args *_pArgs)
```

Writes bytes to the specified address on the remote device port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_RemoteDevice_WritePort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_RemoteDevice_WritePort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.15 KYVPLibTL_Stream_GetPortURL

```
KY_RESULT KYVPLibTL_Stream_GetPortURL(KYVPLibTL_Stream_GetPortUrl_Args *_pArgs)
```

Retrieves port URL information for the data stream module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_Stream_GetPortUrl_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_Stream_GetPortUrl_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.16 KYVPLibTL_Stream_ReadPort

```
KY_RESULT KYVPLibTL_Stream_ReadPort(KYVPLibTL_Stream_ReadPort_Args *_pArgs)
```

Reads bytes from the specified address on the data stream port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_Stream_ReadPort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_Stream_ReadPort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.17 KYVPLibTL_Stream_WritePort

```
KY_RESULT KYVPLibTL_Stream_WritePort(KYVPLibTL_Stream_WritePort_Args *_pArgs)
```

Writes bytes to the specified address on the data stream port.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_Stream_WritePort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_Stream_WritePort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.18 KYVPLibTL_TLOpen

```
KY_RESULT KYVPLibTL_TLOpen(KYVPLibTL_TLOpen_Args *_pArgs)
```

Opens the transport layer system module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_TLOpen_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_TLOpen_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

Pre: KYVPLibTL_InitLib() has completed successfully.

See: KYVPLibTL_TLClose

#### 6.4.1.5.19 KYVPLibTL_TLClose

```
KY_RESULT KYVPLibTL_TLClose(KYVPLibTL_TLClose_Args *_pArgs)
```

Closes the transport layer system module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_TLClose_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_TLClose_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.20 KYVPLibTL_TLGetInfo

```
KY_RESULT KYVPLibTL_TLGetInfo(KYVPLibTL_TLGetInfo_Args *_pArgs)
```

Queries information about the system module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_TLGetInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_TLGetInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.21 KYVPLibTL_TLGetNumInterfaces

```
KY_RESULT KYVPLibTL_TLGetNumInterfaces(KYVPLibTL_TLGetNumInterfaces_Args *_pArgs)
```

Queries the number of available interfaces on this System module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_TLGetNumInterfaces_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_TLGetNumInterfaces_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.22 KYVPLibTL_TLGetInterfaceID

```
KY_RESULT KYVPLibTL_TLGetInterfaceID(KYVPLibTL_TLGetInterfaceID_Args *_pArgs)
```

Queries the unique ID of the interface at iIndex in the internal interface list.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_TLGetInterfaceID_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_TLGetInterfaceID_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.23 KYVPLibTL_TLGetInterfaceInfo

```
KY_RESULT KYVPLibTL_TLGetInterfaceInfo(KYVPLibTL_TLGetPCIInterfaceInfo_Args *_pArgs)
```

Queries information about an interface on the system module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_TLGetPCIInterfaceInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_TLGetPCIInterfaceInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.24 KYVPLibTL_TLOpenInterface

```
KY_RESULT KYVPLibTL_TLOpenInterface(KYVPLibTL_TLOpenInterface_Args *_pArgs)
```

Opens the given interface ID.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_TLOpenInterface_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_TLOpenInterface_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.25 KYVPLibTL_TLUpdateInterfaceList

```
KY_RESULT KYVPLibTL_TLUpdateInterfaceList(KYVPLibTL_TLUpdatePCIInterfaceList_Args *_pArgs)
```

Updates the internal list of available interfaces.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_TLUpdatePCIInterfaceList_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_TLUpdatePCIInterfaceList_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.26 KYVPLibTL_IFClose

```
KY_RESULT KYVPLibTL_IFClose(KYVPLibTL_IFClose_Args *_pArgs)
```

Closes the Interface module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFClose_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFClose_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.27 KYVPLibTL_IFGetInfo

```
KY_RESULT KYVPLibTL_IFGetInfo(KYVPLibTL_IFGetInfo_Args *_pArgs)
```

Inquires information about the Interface module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFGetInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFGetInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.28 KYVPLibTL_IFGetNumDevices

```
KY_RESULT KYVPLibTL_IFGetNumDevices(KYVPLibTL_IFGetNumDevices_Args *_pArgs)
```

Queries the number of available devices on this Interface module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFGetNumDevices_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFGetNumDevices_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.29 KYVPLibTL_IFGetDeviceID

```
KY_RESULT KYVPLibTL_IFGetDeviceID(KYVPLibTL_IFGetDeviceID_Args *_pArgs)
```

Queries the unique ID of the device at iIndex in the internal device list.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFGetDeviceID_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFGetDeviceID_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.30 KYVPLibTL_IFUpdateDeviceList

```
KY_RESULT KYVPLibTL_IFUpdateDeviceList(KYVPLibTL_IFUpdateDeviceList_Args *_pArgs)
```

Updates the internal list of available devices.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFUpdateDeviceList_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFUpdateDeviceList_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.31 KYVPLibTL_IFGetDeviceInfo

```
KY_RESULT KYVPLibTL_IFGetDeviceInfo(KYVPLibTL_IFGetDeviceInfo_Args *_pArgs)
```

Inquires information about a device.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFGetDeviceInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFGetDeviceInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.32 KYVPLibTL_IFOpenDevice

```
KY_RESULT KYVPLibTL_IFOpenDevice(KYVPLibTL_IFOpenDevice_Args *_pArgs)
```

Opens the specified device with the requested access flags.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFOpenDevice_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFOpenDevice_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.33 KYVPLibTL_IFRegisterEvent

```
KY_RESULT KYVPLibTL_IFRegisterEvent(KYVPLibTL_IFRegisterEvent_Args *_pArgs)
```

Registers an event on a PCI interface.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFRegisterEvent_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFRegisterEvent_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.34 KYVPLibTL_IFUnregisterEvent

```
KY_RESULT KYVPLibTL_IFUnregisterEvent(KYVPLibTL_IFUnregisterEvent_Args *_pArgs)
```

Unregisters an event from a PCI interface.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_IFUnregisterEvent_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_IFUnregisterEvent_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.35 KYVPLibTL_DevGetPort

```
KY_RESULT KYVPLibTL_DevGetPort(KYVPLibTL_DevGetPort_Args *_pArgs)
```

Retrieves the port handle for the associated remote device.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DevGetPort_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DevGetPort_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.36 KYVPLibTL_DevGetNumDataStreams

```
KY_RESULT KYVPLibTL_DevGetNumDataStreams(KYVPLibTL_DevGetNumDataStreams_Args *_pArgs)
```

Queries the number of available data streams on this Device module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DevGetNumDataStreams_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DevGetNumDataStreams_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.37 KYVPLibTL_DevGetDataStreamID

```
KY_RESULT KYVPLibTL_DevGetDataStreamID(KYVPLibTL_DevGetDataStreamID_Args *_pArgs)
```

Queries the unique ID of the data stream at iIndex in the internal data stream list.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DevGetDataStreamID_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DevGetDataStreamID_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.38 KYVPLibTL_DevOpenDataStream

```
KY_RESULT KYVPLibTL_DevOpenDataStream(KYVPLibTL_DevOpenDataStream_Args *_pArgs)
```

Opens the specified data stream on a device.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DevOpenDataStream_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DevOpenDataStream_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.39 KYVPLibTL_DevGetInfo

```
KY_RESULT KYVPLibTL_DevGetInfo(KYVPLibTL_DevGetInfo_Args *_pArgs)
```

Inquires information about a device.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DevGetInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DevGetInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.40 KYVPLibTL_DevClose

```
KY_RESULT KYVPLibTL_DevClose(KYVPLibTL_DevClose_Args *_pArgs)
```

Closes the device module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DevClose_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DevClose_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.41 KYVPLibTL_DevRegisterEvent

```
KY_RESULT KYVPLibTL_DevRegisterEvent(KYVPLibTL_DevRegisterEvent_Args *_pArgs)
```

Registers an event on a device.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DevRegisterEvent_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DevRegisterEvent_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.42 KYVPLibTL_DevUnregisterEvent

```
KY_RESULT KYVPLibTL_DevUnregisterEvent(KYVPLibTL_DevUnregisterEvent_Args *_pArgs)
```

Unregisters an event from a device.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DevUnregisterEvent_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DevUnregisterEvent_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.43 KYVPLibTL_DSAnnounceBuffer

```
KY_RESULT KYVPLibTL_DSAnnounceBuffer(KYVPLibTL_DSAnnounceBuffer_Args *_pArgs)
```

Announces caller-allocated memory to a data stream.

The returned buffer handle references this memory until the buffer is revoked.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSAnnounceBuffer_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSAnnounceBuffer_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.44 KYVPLibTL_DSAllocAndAnnounceBuffer

```
KY_RESULT KYVPLibTL_DSAllocAndAnnounceBuffer(KYVPLibTL_DSAllocAndAnnounceBuffer_Args *_pArgs)
```

Allocates memory for a buffer and announces it to a data stream.

The returned buffer handle references this memory until the buffer is revoked.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSAllocAndAnnounceBuffer_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSAllocAndAnnounceBuffer_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.45 KYVPLibTL_DSStartAcquisition

```
KY_RESULT KYVPLibTL_DSStartAcquisition(KYVPLibTL_DSStartAcquisition_Args *_pArgs)
```

Starts acquisition on the host.

Start acquisition on the remote device separately after this call.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSStartAcquisition_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSStartAcquisition_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.46 KYVPLibTL_DSStopAcquisition

```
KY_RESULT KYVPLibTL_DSStopAcquisition(KYVPLibTL_DSStopAcquisition_Args *_pArgs)
```

Stops acquisition on the host.

Stop acquisition on the remote device separately after this call.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSStopAcquisition_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSStopAcquisition_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.47 KYVPLibTL_DSGetInfo

```
KY_RESULT KYVPLibTL_DSGetInfo(KYVPLibTL_DSGetInfo_Args *_pArgs)
```

Queries information about the data stream module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSGetInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSGetInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.48 KYVPLibTL_DSGetBufferID

```
KY_RESULT KYVPLibTL_DSGetBufferID(KYVPLibTL_DSGetBufferID_Args *_pArgs)
```

Queries a buffer handle by its index in the data stream.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSGetBufferID_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSGetBufferID_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.49 KYVPLibTL_DSClose

```
KY_RESULT KYVPLibTL_DSClose(KYVPLibTL_DSClose_Args *_pArgs)
```

Closes the data stream module.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSClose_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSClose_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.50 KYVPLibTL_DSRevokeBuffer

```
KY_RESULT KYVPLibTL_DSRevokeBuffer(KYVPLibTL_DSRevokeBuffer_Args *_pArgs)
```

Removes an announced buffer from the acquisition engine.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSRevokeBuffer_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSRevokeBuffer_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.51 KYVPLibTL_DSQueueBuffer

```
KY_RESULT KYVPLibTL_DSQueueBuffer(KYVPLibTL_DSQueueBuffer_Args *_pArgs)
```

Queues a buffer for acquisition.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSQueueBuffer_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSQueueBuffer_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.52 KYVPLibTL_DSFlushQueue

```
KY_RESULT KYVPLibTL_DSFlushQueue(KYVPLibTL_DSFlushQueue_Args *_pArgs)
```

Moves or discards queued buffers according to the requested queue operation.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSFlushQueue_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSFlushQueue_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.53 KYVPLibTL_DSGetBufferInfo

```
KY_RESULT KYVPLibTL_DSGetBufferInfo(KYVPLibTL_DSGetBufferInfo_Args *_pArgs)
```

Queries information about a buffer on a data stream.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSGetBufferInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSGetBufferInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.54 KYVPLibTL_DSRegisterEvent

```
KY_RESULT KYVPLibTL_DSRegisterEvent(KYVPLibTL_DSRegisterEvent_Args *_pArgs)
```

Registers an event on a data stream.

The implementation may differ between platforms.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSRegisterEvent_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSRegisterEvent_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.55 KYVPLibTL_DSUnregisterEvent

```
KY_RESULT KYVPLibTL_DSUnregisterEvent(KYVPLibTL_DSUnregisterEvent_Args *_pArgs)
```

Unregisters the specified event from a data stream.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_DSUnregisterEvent_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_DSUnregisterEvent_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.56 KYVPLibTL_EventGetData

```
KY_RESULT KYVPLibTL_EventGetData(KYVPLibTL_EventGetData_Args *_pArgs)
```

Retrieves the next data entry from the event queue.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_EventGetData_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_EventGetData_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.57 KYVPLibTL_EventGetDataInfo

```
KY_RESULT KYVPLibTL_EventGetDataInfo(KYVPLibTL_EventGetDataInfo_Args *_pArgs)
```

Queries information about an event data buffer.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_EventGetDataInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_EventGetDataInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.58 KYVPLibTL_EventGetInfo

```
KY_RESULT KYVPLibTL_EventGetInfo(KYVPLibTL_EventGetInfo_Args *_pArgs)
```

Queries information about an event object.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_EventGetInfo_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_EventGetInfo_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.59 KYVPLibTL_EventFlush

```
KY_RESULT KYVPLibTL_EventFlush(KYVPLibTL_EventFlush_Args *_pArgs)
```

Flushes the queue of an event object.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_EventFlush_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_EventFlush_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.60 KYVPLibTL_EventKill

```
KY_RESULT KYVPLibTL_EventKill(KYVPLibTL_EventKill_Args *_pArgs)
```

Terminates a waiting operation on a previously registered event object.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPLibTL_EventKill_Args* | IN/OUT. Pointer to the initialized KYVPLibTL_EventKill_Args arguments. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 6.4.1.5.61 KYVPLibTL_GetPortURL_V1

```
KY_RESULT KYVPLibTL_GetPortURL_V1(KYVP_TL_HANDLE _hTLHandle, char *_pszURL, size_t *_piSize)
```

Retrieves port URL information for the system module.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. TL Handle. |
| _pszURL | char* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_GetPortURL

#### 6.4.1.5.62 KYVPLibTL_ReadPort_V1

```
KY_RESULT KYVPLibTL_ReadPort_V1(KYVP_TL_HANDLE _hTLHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)
```

Reads bytes from the specified address on the system port.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. TL Handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | void* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_ReadPort

#### 6.4.1.5.63 KYVPLibTL_WritePort_V1

```
KY_RESULT KYVPLibTL_WritePort_V1(KYVP_TL_HANDLE _hTLHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)
```

Writes bytes to the specified address on the system port.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. TL Handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | const void* | IN. Pointer to caller-allocated const buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_WritePort

#### 6.4.1.5.64 KYVPLibTL_PCIInterface_GetPortURL_V1

```
KY_RESULT KYVPLibTL_PCIInterface_GetPortURL_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, char *_pszURL, size_t *_piSize)
```

Retrieves port URL information for the PCI interface module.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface Handle. |
| _pszURL | char* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_PCIInterface_GetPortURL

#### 6.4.1.5.65 KYVPLibTL_PCIInterface_ReadPort_V1

```
KY_RESULT KYVPLibTL_PCIInterface_ReadPort_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)
```

Reads bytes from the specified address on the PCI interface port.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface Handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | void* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_PCIInterface_ReadPort

#### 6.4.1.5.66 KYVPLibTL_PCIInterface_WritePort_V1

```
KY_RESULT KYVPLibTL_PCIInterface_WritePort_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)
```

Writes bytes to the specified address on the PCI interface port.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface Handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | const void* | IN. Pointer to caller-allocated const buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_PCIInterface_WritePort

#### 6.4.1.5.67 KYVPLibTL_Device_GetPortURL_V1

```
KY_RESULT KYVPLibTL_Device_GetPortURL_V1(KYVP_DEVICE_HANDLE _hDevHandle, char *_pszURL, size_t *_piSize)
```

Retrieves port URL information for the device module.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Local device handle. |
| _pszURL | char* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Device_GetPortURL

#### 6.4.1.5.68 KYVPLibTL_Device_ReadPort_V1

```
KY_RESULT KYVPLibTL_Device_ReadPort_V1(KYVP_DEVICE_HANDLE _hDevHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)
```

Reads bytes from the specified address on the device port.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Local device handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | void* | OUT. Pointer to caller-allocated buffer. |

| Parameter | Type | Description |
|---|---|---|
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Device_ReadPort

#### 6.4.1.5.69 KYVPLibTL_Device_WritePort_V1

```
KY_RESULT KYVPLibTL_Device_WritePort_V1(KYVP_DEVICE_HANDLE _hDevHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)
```

Writes bytes to the specified address on the device port.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Local device handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | const void* | IN. Pointer to caller-allocated const buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Device_WritePort

#### 6.4.1.5.70 KYVPLibTL_RemoteDevice_GetPortURL_V1

```
KY_RESULT KYVPLibTL_RemoteDevice_GetPortURL_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, char *_pszURL, size_t *_piSize)
```

Retrieves port URL information for the remote device module.

| Parameter | Type | Description |
|---|---|---|
| _hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Remote device handle. |
| _pszURL | char* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_RemoteDevice_GetPortURL

#### 6.4.1.5.71 KYVPLibTL_RemoteDevice_ReadPort_V1

```
KY_RESULT KYVPLibTL_RemoteDevice_ReadPort_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)
```

Reads bytes from the specified address on the remote device port.

| Parameter | Type | Description |
|---|---|---|
| _hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Remote device handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | void* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_RemoteDevice_ReadPort

#### 6.4.1.5.72 KYVPLibTL_RemoteDevice_WritePort_V1

```
KY_RESULT KYVPLibTL_RemoteDevice_WritePort_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)
```

Writes bytes to the specified address on the remote device port.

| Parameter | Type | Description |
|---|---|---|
| _hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Remote device handle. |
| _iAddress | uint64_t | IN. Port Address. |
| _pBuffer | const void* | IN. Pointer to caller-allocated const buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_RemoteDevice_WritePort

#### 6.4.1.5.73 KYVPLibTL_Stream_GetPortURL_V1

```
KY_RESULT KYVPLibTL_Stream_GetPortURL_V1(KYVP_STREAM_HANDLE _hStreamHandle, char *_pszURL, size_t *_piSize)
```

Retrieves port URL information for the data stream module.

| Parameter | Type | Description |
|---|---|---|
| _hStreamHandle | KYVP_STREAM_HANDLE | IN. Stream handle. |
| _pszURL | char* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Stream_GetPortURL

#### 6.4.1.5.74 KYVPLibTL_Stream_ReadPort_V1

```
KY_RESULT KYVPLibTL_Stream_ReadPort_V1(KYVP_STREAM_HANDLE _hStreamHandle, uint64_t _iAddress, void *_pBuffer,
size_t *_piSize)
```

Reads bytes from the specified address on the data stream port.

| Parameter | Type | Description |
|---|---|---|
| _hStreamHandle | KYVP_STREAM_HANDLE | IN. Stream handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | void* | OUT. Pointer to caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Stream_ReadPort

#### 6.4.1.5.75 KYVPLibTL_Stream_WritePort_V1

```
KY_RESULT KYVPLibTL_Stream_WritePort_V1(KYVP_STREAM_HANDLE _hStreamHandle, uint64_t _iAddress, const void
*_pBuffer, size_t *_piSize)
```

Writes bytes to the specified address on the data stream port.

| Parameter | Type | Description |
|---|---|---|
| _hStreamHandle | KYVP_STREAM_HANDLE | IN. Stream handle. |
| _iAddress | uint64_t | IN. Port address. |
| _pBuffer | const void* | IN. Pointer to caller-allocated const buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Stream_WritePort

#### 6.4.1.5.76 KYVPLibTL_TLOpen_V1

```
KY_RESULT KYVPLibTL_TLOpen_V1(KYVP_TL_HANDLE *_phTLHandle)
```

Opens the transport layer system module.

| Parameter | Type | Description |
|---|---|---|
| _phTLHandle | KYVP_TL_HANDLE* | OUT. Pointer to KYVP_TL_HANDLE. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLOpen

#### 6.4.1.5.77 KYVPLibTL_TLClose_V1

```
KY_RESULT KYVPLibTL_TLClose_V1(KYVP_TL_HANDLE _hTLHandle)
```

Closes the transport layer system module.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. KYVP_TL HANDLE. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLClose

#### 6.4.1.5.78 KYVPLibTL_TLGetInfo_V1

```
KY_RESULT KYVPLibTL_TLGetInfo_V1(KYVP_TL_HANDLE _hTLHandle, KYVP_TL_INFO_CMD _eInfoCmd, KYVP_INFO_DATATYPE
*_pType, void *_pBuffer, size_t *_piSize)
```

Queries information about the system module.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. KYVP_TL HANDLE. |
| _eInfoCmd | KYVP_TL_INFO_CMD | IN. Information to be retrieved as defined in KYVP_TL_INFO_CMD. |
| _pType | KYVP_INFO_DATATYPE* | OUT. Data type of the pBuffer content as defined in the KYVP_INFO_DATATYPE. |
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer to receive the requested information. If this parameter is NULL, piSize will contain the minimal size of pBuffer in bytes. If the iType is a string the size includes the terminating 0. |
| _piSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLGetInfo

#### 6.4.1.5.79 KYVPLibTL_TLGetNumInterfaces_V1

```
KY_RESULT KYVPLibTL_TLGetNumInterfaces_V1(KYVP_TL_HANDLE _hTLHandle, uint32_t *_piNumIfaces)
```

Queries the number of available interfaces on this System module.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. KYVP_TL_HANDLE. |
| _piNumIfaces | uint32_t* | OUT. Pointer to caller-allocated uint32_t. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLGetNumInterfaces

#### 6.4.1.5.80 KYVPLibTL_TLGetInterfaceID_V1

```
KY_RESULT KYVPLibTL_TLGetInterfaceID_V1(KYVP_TL_HANDLE _hTLHandle, uint32_t _iIndex, char *_pszID, size_t *_piSize)
```

Queries the unique ID of the interface at iIndex in the internal interface list.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. KYVP_TL_HANDLE. |
| _iIndex | uint32_t | IN. Interface index. |
| _pszID | char* | OUT. Pointer to caller-allocated char* buffer. |
| _piSize | size_t* | IN/OUT. Pointer to caller-allocated size_t* buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLGetInterfaceID

#### 6.4.1.5.81 KYVPLibTL_TLGetInterfaceInfo_V1

```
KY_RESULT KYVPLibTL_TLGetInterfaceInfo_V1(KYVP_TL_HANDLE _hTLHandle, const char *_pszIfaceID, KYVP_INTERFACE_INFO_CMD _eInfoCmd, KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)
```

Queries information about an interface on the system module.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. KYVP_TL_HANDLE. |
| _pszIfaceID | const char* | IN. Interface ID from KYVPLibTL_TLGetInterfaceID func. |
| _eInfoCmd | KYVP_INTERFACE_INFO_CMD | IN. Information to be retrieved. |
| _pType | KYVP_INFO_DATATYPE* | OUT. Data type of the pBuffer content. |
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Out: minimal size of pBuffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLGetInterfaceInfo

#### 6.4.1.5.82 KYVPLibTL_TLOpenInterface_V1

```
KY_RESULT KYVPLibTL_TLOpenInterface_V1(KYVP_TL_HANDLE _hTLHandle, const char *_pszIfaceID, KYVP_PCI_INTERFACE_HANDLE *_phPCIInterfaceHandle)
```

Opens the given interface ID.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. KYVP_TL_HANDLE. |
| _pszIfaceID | const char* | IN. Unique interface ID to open as a null-terminated C string. |
| _phPCIInterfaceHandle | KYVP_PCI_INTERFACE_HANDLE* | OUT. Pointer to KYVP_PCI_INTERFACE_HANDLE. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLOpenInterface

#### 6.4.1.5.83 KYVPLibTL_TLUpdateInterfaceList_V1

```
KY_RESULT KYVPLibTL_TLUpdateInterfaceList_V1(KYVP_TL_HANDLE _hTLHandle, KY_BOOL *_pbChanged, uint64_t _iTimeout)
```

Updates the internal list of available interfaces.

| Parameter | Type | Description |
|---|---|---|
| _hTLHandle | KYVP_TL_HANDLE | IN. KYVP_TL_HANDLE. |

| Parameter | Type | Description |
|---|---|---|
| _pbChanged | KY_BOOL* | OUT. Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise. If set to NULL nothing is written to this parameter. |
| _iTimeout | uint64_t | IN. Timeout in ms. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLUpdateInterfaceList

#### 6.4.1.5.84 KYVPLibTL_IFClose_V1

```
KY_RESULT KYVPLibTL_IFClose_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle)
```

Closes the Interface module.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. System module handle to close. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFClose

#### 6.4.1.5.85 KYVPLibTL_IFGetInfo_V1

```
KY_RESULT KYVPLibTL_IFGetInfo_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_INTERFACE_INFO_CMD _eInfoCmd,
KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)
```

Inquires information about the Interface module.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface module to work on. |
| _eInfoCmd | KYVP_INTERFACE_INFO_CMD | IN. Information to be retrieved. |
| _pType | KYVP_INFO_DATATYPE* | OUT. Pointer to data type of the pBuffer content. |
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFGetInfo

#### 6.4.1.5.86 KYVPLibTL_IFGetNumDevices_V1

```
KY_RESULT KYVPLibTL_IFGetNumDevices_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint32_t *_pNumDevices)
```

Queries the number of available devices on this Interface module.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface module to work on. |
| _pNumDevices | uint32_t* | OUT. Number of devices. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFGetNumDevices

#### 6.4.1.5.87 KYVPLibTL_IFGetDeviceID_V1

```
KY_RESULT KYVPLibTL_IFGetDeviceID_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint32_t _iIndex, char *_pszID, size_t
*_piSize)
```

Queries the unique ID of the device at iIndex in the internal device list.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface module to work on. |
| _iIndex | uint32_t | IN. Device index. |

| Parameter | Type | Description |
|---|---|---|
| _pszID | char* | OUT. Pointer to a caller-allocated C string buffer to receive the Device module ID at the given iIndex. |
| _piSize | size_t* | IN/OUT. Minimal size of pBuffer in bytes to hold all information. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFGetDeviceID

#### 6.4.1.5.88 KYVPLibTL_IFUpdateDeviceList_V1

```
KY_RESULT KYVPLibTL_IFUpdateDeviceList_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL *_pbChanged, uint64_t _uTimeout)
```

Updates the internal list of available devices.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface module to work on. |
| _pbChanged | KY_BOOL* | OUT. Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise. |
| _uTimeout | uint64_t | IN. Timeout in ms. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFUpdateDeviceList

#### 6.4.1.5.89 KYVPLibTL_IFUpdateDeviceList_V2

```
KY_RESULT KYVPLibTL_IFUpdateDeviceList_V2(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL *_pbChanged, uint64_t _uTimeout, KY_BOOL _bRetainOpenDevices)
```

Updates the internal list of available devices.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface module to work on. |
| _pbChanged | KY_BOOL* | OUT. Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise. |
| _uTimeout | uint64_t | IN. Timeout in ms. |
| _bRetainOpenDevices | KY_BOOL | IN. KY_TRUE preserves open devices in all processes. KY_FALSE closes this process's devices, clears its device lists and performs fresh detection. Devices opened by other processes are preserved in both modes. Since version 2; defaults to KY_TRUE. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFUpdateDeviceList

#### 6.4.1.5.90 KYVPLibTL_IFGetDeviceInfo_V1

```
KY_RESULT KYVPLibTL_IFGetDeviceInfo_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, const char *_pszIfaceID, KYVP_DEVICE_INFO_CMD _eInfoCmd, KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)
```

Inquires information about a device.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface module to work on. |
| _pszIfaceID | const char* | IN. Unique ID of the device to inquire information about. |
| _eInfoCmd | KYVP_DEVICE_INFO_CMD | IN. Information to be retrieved. |

| Parameter | Type | Description |
|---|---|---|
| _pType | KYVP_INFO_DATATYPE* | OUT. Data type of the pBuffer content. |
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFGetDeviceInfo

#### 6.4.1.5.91 KYVPLibTL_IFOpenDevice_V1

```
KY_RESULT KYVPLibTL_IFOpenDevice_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, const char *_pszDeviceID,
KYVP_DEVICE_ACCESS_FLAGS _iOpenFlags, KYVP_DEVICE_HANDLE *_phDeviceHandle)
```

Opens the specified device with the requested access flags.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Interface module to work on. |
| _pszDeviceID | const char* | IN. Unique device ID to open as a null-terminated C string. |
| _iOpenFlags | KYVP_DEVICE_ACCESS_FLAGS | IN. Configures the open process. |
| _phDeviceHandle | KYVP_DEVICE_HANDLE* | OUT. Device handle of the newly created Device module. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFOpenDevice

#### 6.4.1.5.92 KYVPLibTL_IFRegisterEvent_V1

```
KY_RESULT KYVPLibTL_IFRegisterEvent_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_EVENT_TYPE _iEventID,
KYVP_EVENT_HANDLE *_phEvent)
```

Registers an event on a PCI interface.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Module handle to access to register event to. |
| _iEventID | KYVP_EVENT_TYPE | IN. Event type to register. |
| _phEvent | KYVP_EVENT_HANDLE* | OUT. New handle to an event object to work with. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFRegisterEvent

#### 6.4.1.5.93 KYVPLibTL_IFUnregisterEvent_V1

```
KY_RESULT KYVPLibTL_IFUnregisterEvent_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_EVENT_TYPE _iEventID)
```

Unregisters an event from a PCI interface.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Module handle to access to unregister event from. |
| _iEventID | KYVP_EVENT_TYPE | IN. Event type to register. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFUnregisterEvent

#### 6.4.1.5.94 KYVPLibTL_DevGetPort_V1

```
KY_RESULT KYVPLibTL_DevGetPort_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_REMOTE_DEVICE_HANDLE
*_phRemoteDeviceHandle)
```

Retrieves the port handle for the associated remote device.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device module to work on. |
| _phRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE* | OUT. Port handle for the remote device. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevGetPort

#### 6.4.1.5.95 KYVPLibTL_DevGetNumDataStreams_V1

```
KY_RESULT KYVPLibTL_DevGetNumDataStreams_V1(KYVP_DEVICE_HANDLE _hDevHandle, uint32_t *_piNumDataStreams)
```

Queries the number of available data streams on this Device module.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device module to work on. |
| _piNumDataStreams | uint32_t* | OUT. Number of data stream on this Device module. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevGetNumDataStreams

#### 6.4.1.5.96 KYVPLibTL_DevGetDataStreamID_V1

```
KY_RESULT KYVPLibTL_DevGetDataStreamID_V1(KYVP_DEVICE_HANDLE _hDevHandle, uint32_t _iIndex, char *_sDataStreamID,
size_t *_piSize)
```

Queries the unique ID of the data stream at iIndex in the internal data stream list.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device module to work on. |
| _iIndex | uint32_t | IN. Zero-based index of the data stream on this device. |
| _sDataStreamID | char* | OUT. Pointer to a caller-allocated C string buffer. |
| _piSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevGetDataStreamID

#### 6.4.1.5.97 KYVPLibTL_DevOpenDataStream_V1

```
KY_RESULT KYVPLibTL_DevOpenDataStream_V1(KYVP_DEVICE_HANDLE _hDevHandle, const char *_pszDataStreamID,
KYVP_STREAM_HANDLE *_phDataStream)
```

Opens the specified data stream on a device.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device module to work on. |
| _pszDataStreamID | const char* | IN. Unique data stream ID to open as a null-terminated C string. |
| _phDataStream | KYVP_STREAM_HANDLE* | OUT. Data Stream module handle of the newly created stream. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevOpenDataStream

#### 6.4.1.5.98 KYVPLibTL_DevGetInfo_V1

```
KY_RESULT KYVPLibTL_DevGetInfo_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_DEVICE_INFO_CMD _iInfoCmd,
KYVP_INFO_DATATYPE *_piType, void *_pBuffer, size_t *_piSize)
```

Inquires information about a device.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device module to work on. |
| _iInfoCmd | KYVP_DEVICE_INFO_CMD | IN. Information to be retrieved. |
| _piType | KYVP_INFO_DATATYPE* | OUT. Data type of the pBuffer content. |
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevGetInfo

#### 6.4.1.5.99 KYVPLibTL_DevClose_V1

```
KY_RESULT KYVPLibTL_DevClose_V1(KYVP_DEVICE_HANDLE _hDevHandle)
```

Closes the device module.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device module handle to close. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevClose

#### 6.4.1.5.100 KYVPLibTL_DevRegisterEvent_V1

```
KY_RESULT KYVPLibTL_DevRegisterEvent_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_EVENT_TYPE _iEventID,
KYVP_EVENT_HANDLE *_phEvent)
```

Registers an event on a device.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device handle to access to register event to. |
| _iEventID | KYVP_EVENT_TYPE | IN. Event type to register. |
| _phEvent | KYVP_EVENT_HANDLE* | OUT. New handle to an event object to work with. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevRegisterEvent

#### 6.4.1.5.101 KYVPLibTL_DevUnregisterEvent_V1

```
KY_RESULT KYVPLibTL_DevUnregisterEvent_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_EVENT_TYPE _iEventID)
```

Unregisters an event from a device.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device handle to access to unregister event from. |
| _iEventID | KYVP_EVENT_TYPE | IN. Event type to unregister. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevUnregisterEvent

#### 6.4.1.5.102 KYVPLibTL_DSAnnounceBuffer_V1

```
KY_RESULT KYVPLibTL_DSAnnounceBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, void *_pBuffer, size_t _iSize, void
*_pPrivate, KYVP_BUFFER_HANDLE *_phBuffer)
```

Announces caller-allocated memory to a data stream.

The returned buffer handle references this memory until the buffer is revoked.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |

| Parameter | Type | Description |
|---|---|---|
| _pBuffer | void* | IN. Pointer to buffer memory to announce. |
| _iSize | size_t | IN. Size of the pBuffer in bytes. |
| _pPrivate | void* | IN. Pointer to private data which will be passed to the GenTL Consumer on New Buffer events. This parameter may be NULL. |
| _phBuffer | KYVP_BUFFER_HANDLE* | OUT. Buffer module handle of the newly announced buffer. I. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSAnnounceBuffer

#### 6.4.1.5.103 KYVPLibTL_DSAllocAndAnnounceBuffer_V1

```
KY_RESULT KYVPLibTL_DSAllocAndAnnounceBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, size_t _iSize, void *_pPrivate,
KYVP_BUFFER_HANDLE *_phBuffer)
```

Allocates memory for a buffer and announces it to a data stream.

The returned buffer handle references this memory until the buffer is revoked.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _iSize | size_t | IN. Size of the buffer in bytes. |
| _pPrivate | void* | IN. Pointer to private data which will be passed to the Consumer on New Buffer events. This parameter may be NULL. |
| _phBuffer | KYVP_BUFFER_HANDLE* | OUT. Buffer module handle of the newly announced buffer. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSAllocAndAnnounceBuffer

#### 6.4.1.5.104 KYVPLibTL_DSStartAcquisition_V1

```
KY_RESULT KYVPLibTL_DSStartAcquisition_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_ACQ_START_FLAGS _iStartFlags,
uint64_t _iNumToAcquire)
```

Starts acquisition on the host.

Start acquisition on the remote device separately after this call.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _iStartFlags | KYVP_ACQ_START_FLAGS | IN. As defined in KYVP_ACQ_START_FLAGS. |
| _iNumToAcquire | uint64_t | IN. Sets the number of filled/delivered buffers after which the acquisition engine stops automatically. Buffers that are internally discarded or missed are not counted. If set to 0, acquisition continues until a call to KYVPLibTL_DSStopAcquisition. To achieve precise accuracy and fine control over the number of acquired frames, the use of triggers is recommended. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSStartAcquisition

#### 6.4.1.5.105 KYVPLibTL_DSStopAcquisition_V1

```
KY_RESULT KYVPLibTL_DSStopAcquisition_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_ACQ_STOP_FLAGS _iStopFlags)
```

Stops acquisition on the host.

Stop acquisition on the remote device separately after this call.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _iStopFlags | KYVP_ACQ_STOP_FLAGS | IN. Stops the acquisition as defined in KYVP_ACQ_STOP_FLAGS. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSStopAcquisition

#### 6.4.1.5.106 KYVPLibTL_DSGetInfo_V1

```
KY_RESULT KYVPLibTL_DSGetInfo_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_STREAM_INFO_CMD _eInfoCmd,
KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)
```

Queries information about the data stream module.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _eInfoCmd | KYVP_STREAM_INFO_CMD | IN. Information to be retrieved. |
| _pType | KYVP_INFO_DATATYPE* | OUT. Data type of the pBuffer content. |
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer to receive the requested information. |
| _piSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSGetInfo

#### 6.4.1.5.107 KYVPLibTL_DSGetBufferID_V1

```
KY_RESULT KYVPLibTL_DSGetBufferID_V1(KYVP_STREAM_HANDLE _hDSHandle, uint32_t _iIndex, KYVP_BUFFER_HANDLE
*_ppBufferHandle)
```

Queries a buffer handle by its index in the data stream.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _iIndex | uint32_t | IN. Zero-based index of the buffer on this data stream. |
| _ppBufferHandle | KYVP_BUFFER_HANDLE* | OUT. Buffer module handle of the given iIndex. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSGetBufferID

#### 6.4.1.5.108 KYVPLibTL_DSClose_V1

```
KY_RESULT KYVPLibTL_DSClose_V1(KYVP_STREAM_HANDLE _hDSHandle)
```

Closes the data stream module.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSClose

#### 6.4.1.5.109 KYVPLibTL_DSRevokeBuffer_V1

```
KY_RESULT KYVPLibTL_DSRevokeBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_BUFFER_HANDLE _hBufferHandle, void **_ppBuffer, void **_ppPrivate)
```

Removes an announced buffer from the acquisition engine.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _hBufferHandle | KYVP_BUFFER_HANDLE | IN. Buffer handle to revoke. |
| _ppBuffer | void | OUT. Pointer to the buffer memory This is for convenience if consumer allocated memory is used which is to be freed. If the buffer was allocated by the GenTL Producer NULL is to be returned. If the parameter is set to NULL it is ignored. |
| _ppPrivate | void | OUT. Pointer to the user data pointer given in the announce function. If the parameter is set to NULL it is ignored. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSRevokeBuffer

#### 6.4.1.5.110 KYVPLibTL_DSQueueBuffer_V1

```
KY_RESULT KYVPLibTL_DSQueueBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_BUFFER_HANDLE _hBufferHandle)
```

Queues a buffer for acquisition.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _hBufferHandle | KYVP_BUFFER_HANDLE | IN. Buffer handle to queue. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSQueueBuffer

#### 6.4.1.5.111 KYVPLibTL_DSFlushQueue_V1

```
KY_RESULT KYVPLibTL_DSFlushQueue_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_ACQ_QUEUE_TYPE _iOperation)
```

Moves or discards queued buffers according to the requested queue operation.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _iOperation | KYVP_ACQ_QUEUE_TYPE | IN. Flush operation type. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSFlushQueue

#### 6.4.1.5.112 KYVPLibTL_DSGetBufferInfo_V1

```
KY_RESULT KYVPLibTL_DSGetBufferInfo_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_BUFFER_HANDLE _hBufferHandle, KYVP_BUFFER_INFO_CMD _iInfoCmd, KYVP_INFO_DATATYPE *_piType, void *_pBuffer, size_t *_piSize)
```

Queries information about a buffer on a data stream.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data Stream module to work on. |
| _hBufferHandle | KYVP_BUFFER_HANDLE | IN. Buffer handle to retrieve information about. |
| _iInfoCmd | KYVP_BUFFER_INFO_CMD | IN. Information to be retrieved. |
| _piType | KYVP_INFO_DATATYPE* | OUT. Data type of the pBuffer content. |

| Parameter | Type | Description |
|---|---|---|
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer to receive the requested information. |
| _piSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSGetBufferInfo

#### 6.4.1.5.113 KYVPLibTL_DSRegisterEvent_V1

```
KY_RESULT KYVPLibTL_DSRegisterEvent_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_EVENT_TYPE _iEventID,
KYVP_EVENT_HANDLE *_phEvent)
```

Registers an event on a data stream.

The implementation may differ between platforms.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Module handle to access to register event to. |
| _iEventID | KYVP_EVENT_TYPE | IN. Event type to register. |
| _phEvent | KYVP_EVENT_HANDLE* | OUT. New handle to an event object to work with. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSRegisterEvent

#### 6.4.1.5.114 KYVPLibTL_DSUnregisterEvent_V1

```
KY_RESULT KYVPLibTL_DSUnregisterEvent_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_EVENT_TYPE _iEventID)
```

Unregisters the specified event from a data stream.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Module handle to access to unregister event from. |
| _iEventID | KYVP_EVENT_TYPE | IN. Event type to register. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSUnregisterEvent

#### 6.4.1.5.115 KYVPLibTL_EventGetData_V1

```
KY_RESULT KYVPLibTL_EventGetData_V1(KYVP_EVENT_HANDLE _hEventHandle, void *_pBuffer, size_t *_piSize, uint64_t
_iTimeout)
```

Retrieves the next data entry from the event queue.

| Parameter | Type | Description |
|---|---|---|
| _hEventHandle | KYVP_EVENT_HANDLE | IN. Event handle to wait for. |
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer. |
| _piSize | size_t* | IN/OUT. Size of the provided pBuffer in bytes. |
| _iTimeout | uint64_t | IN. Timeout for the wait in ms. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventGetData

#### 6.4.1.5.116 KYVPLibTL_EventGetDataInfo_V1

```
KY_RESULT KYVPLibTL_EventGetDataInfo_V1(KYVP_EVENT_HANDLE _hEventHandle, const void *pInBuffer, size_t _iInSize,
KYVP_EVENT_DATA_INFO_CMD _iInfoCmd, KYVP_INFO_DATATYPE *_piType, void *_pOutBuffer, size_t *_piOutSize)
```

Queries information about an event data buffer.

| Parameter | Type | Description |
|---|---|---|
| _hEventHandle | KYVP_EVENT_HANDLE | IN. Event handle to parse data from. |
| pInBuffer | const void* | IN. Pointer to a buffer containing event data. |
| _iInSize | size_t | IN. Size of the provided pInBuffer in bytes. |
| _iInfoCmd | KYVP_EVENT_DATA_INFO_CMD | IN. Information to be retrieved. |
| _piType | KYVP_INFO_DATATYPE* | OUT. Data type of the pOutBuffer content. |
| _pOutBuffer | void* | OUT. Pointer to a caller-allocated buffer. |
| _piOutSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventGetDataInfo

#### 6.4.1.5.117 KYVPLibTL_EventGetInfo_V1

```
KY_RESULT KYVPLibTL_EventGetInfo_V1(KYVP_EVENT_HANDLE _hEventHandle, KYVP_EVENT_INFO_CMD _iInfoCmd,
KYVP_INFO_DATATYPE *_piType, void *_pBuffer, size_t *piSize)
```

Queries information about an event object.

| Parameter | Type | Description |
|---|---|---|
| _hEventHandle | KYVP_EVENT_HANDLE | IN. Event handle to parse data from. |
| _iInfoCmd | KYVP_EVENT_INFO_CMD | IN. Information to be retrieved. |
| _piType | KYVP_INFO_DATATYPE* | OUT. Data type of the pBuffer content. |
| _pBuffer | void* | OUT. Pointer to a caller-allocated buffer. |
| piSize | size_t* | IN/OUT. Number of bytes filled by the function. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventGetInfo

#### 6.4.1.5.118 KYVPLibTL_EventFlush_V1

```
KY_RESULT KYVPLibTL_EventFlush_V1(KYVP_EVENT_HANDLE _hEventHandle)
```

Flushes the queue of an event object.

| Parameter | Type | Description |
|---|---|---|
| _hEventHandle | KYVP_EVENT_HANDLE | IN. Event handle. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventFlush

#### 6.4.1.5.119 KYVPLibTL_EventKill_V1

```
KY_RESULT KYVPLibTL_EventKill_V1(KYVP_EVENT_HANDLE _hEventHandle)
```

Terminates a waiting operation on a previously registered event object.

| Parameter | Type | Description |
|---|---|---|
| _hEventHandle | KYVP_EVENT_HANDLE | IN. Event handle. |

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventKill

#### 6.4.1.5.120 KYVPLibTL_TLHandleIsInvalid

```
KY_BOOL KYVPLibTL_TLHandleIsInvalid(KYVP_TL_HANDLE _hHandle)
```

Reports whether the handle is invalid.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_TL_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

#### 6.4.1.5.121 KYVPLibTL_TLHandleIsNull

```
KY_BOOL KYVPLibTL_TLHandleIsNull(KYVP_TL_HANDLE _hHandle)
```

Reports whether the handle is null.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_TL_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

#### 6.4.1.5.122 KYVPLibTL_TLHandlesIsEqual

```
KY_BOOL KYVPLibTL_TLHandlesIsEqual(KYVP_TL_HANDLE _hHandleFirst, KYVP_TL_HANDLE _hHandleSecond)
```

Reports whether the handles are equal.

| Parameter | Type | Description |
|---|---|---|
| _hHandleFirst | KYVP_TL_HANDLE | IN. First handle to compare. |
| _hHandleSecond | KYVP_TL_HANDLE | IN. Second handle to compare. |

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

#### 6.4.1.5.123 KYVPLibTL_IFHandleIsInvalid

```
KY_BOOL KYVPLibTL_IFHandleIsInvalid(KYVP_PCI_INTERFACE_HANDLE _hHandle)
```

Reports whether the handle is invalid.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

#### 6.4.1.5.124 KYVPLibTL_IFHandleIsNull

```
KY_BOOL KYVPLibTL_IFHandleIsNull(KYVP_PCI_INTERFACE_HANDLE _hHandle)
```

Reports whether the handle is null.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_PCI_INTERFACE_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

#### 6.4.1.5.125 KYVPLibTL_IFHandlesAreEqual

```
KY_BOOL KYVPLibTL_IFHandlesAreEqual(KYVP_PCI_INTERFACE_HANDLE _hHandleFirst, KYVP_PCI_INTERFACE_HANDLE
_hHandleSecond)
```

Reports whether the handles are equal.

| Parameter | Type | Description |
|---|---|---|
| _hHandleFirst | KYVP_PCI_INTERFACE_HANDLE | IN. First handle to compare. |
| _hHandleSecond | KYVP_PCI_INTERFACE_HANDLE | IN. Second handle to compare. |

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

#### 6.4.1.5.126 KYVPLibTL_DevHandleIsInvalid

```
KY_BOOL KYVPLibTL_DevHandleIsInvalid(KYVP_DEVICE_HANDLE _hHandle)
```

Reports whether the handle is invalid.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_DEVICE_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

#### 6.4.1.5.127 KYVPLibTL_DevHandleIsNull

```
KY_BOOL KYVPLibTL_DevHandleIsNull(KYVP_DEVICE_HANDLE _hHandle)
```

Reports whether the handle is null.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_DEVICE_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

#### 6.4.1.5.128 KYVPLibTL_DevHandlesIsEqual

```
KY_BOOL KYVPLibTL_DevHandlesIsEqual(KYVP_DEVICE_HANDLE _hHandleFirst, KYVP_DEVICE_HANDLE _hHandleSecond)
```

Reports whether the handles are equal.

| Parameter | Type | Description |
|---|---|---|
| _hHandleFirst | KYVP_DEVICE_HANDLE | IN. First handle to compare. |
| _hHandleSecond | KYVP_DEVICE_HANDLE | IN. Second handle to compare. |

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

#### 6.4.1.5.129 KYVPLibTL_RemoteDeviceHandleIsInvalid

```
KY_BOOL KYVPLibTL_RemoteDeviceHandleIsInvalid(KYVP_REMOTE_DEVICE_HANDLE _hHandle)
```

Reports whether the handle is invalid.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

#### 6.4.1.5.130 KYVPLibTL_RemoteDeviceHandleIsNull

```
KY_BOOL KYVPLibTL_RemoteDeviceHandleIsNull(KYVP_REMOTE_DEVICE_HANDLE _hHandle)
```

Reports whether the handle is null.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

#### 6.4.1.5.131 KYVPLibTL_RemoteDeviceHandlesIsEqual

```
KY_BOOL KYVPLibTL_RemoteDeviceHandlesIsEqual(KYVP_REMOTE_DEVICE_HANDLE _hHandleFirst, KYVP_REMOTE_DEVICE_HANDLE
_hHandleSecond)
```

Reports whether the handles are equal.

| Parameter | Type | Description |
|---|---|---|
| _hHandleFirst | KYVP_REMOTE_DEVICE_HANDLE | IN. First handle to compare. |
| _hHandleSecond | KYVP_REMOTE_DEVICE_HANDLE | IN. Second handle to compare. |

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

#### 6.4.1.5.132 KYVPLibTL_DSHandleIsInvalid

```
KY_BOOL KYVPLibTL_DSHandleIsInvalid(KYVP_STREAM_HANDLE _hHandle)
```

Reports whether the handle is invalid.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_STREAM_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

#### 6.4.1.5.133 KYVPLibTL_DSHandleIsNull

```
KY_BOOL KYVPLibTL_DSHandleIsNull(KYVP_STREAM_HANDLE _hHandle)
```

Reports whether the handle is null.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_STREAM_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

#### 6.4.1.5.134 KYVPLibTL_DSHandlesIsEqual

```
KY_BOOL KYVPLibTL_DSHandlesIsEqual(KYVP_STREAM_HANDLE _hHandleFirst, KYVP_STREAM_HANDLE _hHandleSecond)
```

Reports whether the handles are equal.

| Parameter | Type | Description |
|---|---|---|
| _hHandleFirst | KYVP_STREAM_HANDLE | IN. First handle to compare. |
| _hHandleSecond | KYVP_STREAM_HANDLE | IN. Second handle to compare. |

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

#### 6.4.1.5.135 KYVPLibTL_BufferHandleIsInvalid

```
KY_BOOL KYVPLibTL_BufferHandleIsInvalid(KYVP_BUFFER_HANDLE _hHandle)
```

Reports whether the handle is invalid.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_BUFFER_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

#### 6.4.1.5.136 KYVPLibTL_BufferHandleIsNull

```
KY_BOOL KYVPLibTL_BufferHandleIsNull(KYVP_BUFFER_HANDLE _hHandle)
```

Reports whether the handle is null.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_BUFFER_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

#### 6.4.1.5.137 KYVPLibTL_BufferHandlesIsEqual

```
KY_BOOL KYVPLibTL_BufferHandlesIsEqual(KYVP_BUFFER_HANDLE _hHandleFirst, KYVP_BUFFER_HANDLE _hHandleSecond)
```

Reports whether the handles are equal.

| Parameter | Type | Description |
|---|---|---|
| _hHandleFirst | KYVP_BUFFER_HANDLE | IN. First handle to compare. |
| _hHandleSecond | KYVP_BUFFER_HANDLE | IN. Second handle to compare. |

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

#### 6.4.1.5.138 KYVPLibTL_EventHandleIsInvalid

```
KY_BOOL KYVPLibTL_EventHandleIsInvalid(KYVP_EVENT_HANDLE _hHandle)
```

Reports whether the handle is invalid.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_EVENT_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

#### 6.4.1.5.139 KYVPLibTL_EventHandleIsNull

```
KY_BOOL KYVPLibTL_EventHandleIsNull(KYVP_EVENT_HANDLE _hHandle)
```

Reports whether the handle is null.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYVP_EVENT_HANDLE | IN. Handle to inspect. |

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

#### 6.4.1.5.140 KYVPLibTL_EventHandlesIsEqual

```
KY_BOOL KYVPLibTL_EventHandlesIsEqual(KYVP_EVENT_HANDLE _hHandleFirst, KYVP_EVENT_HANDLE _hHandleSecond)
```

Reports whether the handles are equal.

| Parameter | Type | Description |
|---|---|---|
| _hHandleFirst | KYVP_EVENT_HANDLE | IN. First handle to compare. |
| _hHandleSecond | KYVP_EVENT_HANDLE | IN. Second handle to compare. |

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

#### 6.4.1.6 KYVP_TL_HANDLE

Transport layer system handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.7 KYVP_PCI_INTERFACE_HANDLE

PCI interface handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.8 KYVP_DEVICE_HANDLE

Local device handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.9 KYVP_REMOTE_DEVICE_HANDLE

Remote device handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.10 KYVP_STREAM_HANDLE

Data stream handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.11 KYVP_BUFFER_HANDLE

Buffer handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.12 KYVP_EVENT_HANDLE

Event handle.

Treat this type as opaque. Use the handle comparison functions or macros.

#### 6.4.1.13 KYVPLibTL Structs

Field reference for function arguments and event payloads.

C++ member defaults initialize structure versions and default values. C callers must set version to the constant shown above each table and initialize the fields required by the operation explicitly. Zero initialization alone does not select a valid structure version.

Each table follows the declaration order and lists the exact public field types. Handle types are opaque. Use the documented API to work with them.

See: KYVPLibTL API

##### 6.4.1.13.1 KYVPLIBTL_EVENT_NEW_BUFFER_DATA

Event data for a newly available buffer.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| hBufferHandle | KYVP_BUFFER_HANDLE | Buffer handle which contains new data. | |
| pUserPointer | void* | User pointer provided at announcement of the buffer. | |

##### 6.4.1.13.2 KYVP_PCIINTERFACE_EVENT_TEMPERATURE

PCI interface temperature event data.

Initialize version to KYVP_PCIINTERFACE_EVENT_TEMPERATURE_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| eTemperatureThresholdId | KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID | Indicates the current temperature threshold state of the PCI interface. | |

#### 6.4.1.13.3 KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT

PCI interface CXP 2.0 heartbeat event data.

Initialize version to KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| deviceHandle | KYVP_DEVICE_HANDLE | Handle of the device associated with the event. | |
| heartBeat | KYVP_CXP2_HEARTBEAT | CXP 2.0 heartbeat data structure. | |

#### 6.4.1.13.4 KYVP_PCIINTERFACE_EVENT_CXP2_EVENT

PCI interface CXP 2.0 event data.

Initialize version to KYVP_PCIINTERFACE_EVENT_CXP2_EVENT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| deviceHandle | KYVP_DEVICE_HANDLE | Handle of the device associated with the event. | |
| cxp2Event | KYVP_CXP2_EVENT | CXP 2.0 event data structure containing event messages. | |

#### 6.4.1.13.5 KYVP_PCIINTERFACE_GENERIC_EVENT

PCI interface event identifier and payload.

Initialize version to KYVP_PCIINTERFACE_GENERIC_EVENT_VERSION.

Note: The data union contains one of the following structures depending on eEventId :

KYVP_PCIINTERFACE_EVENT_TEMPERATURE

KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT

KYVP_PCIINTERFACE_EVENT_CXP2_EVENT

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| eEventId | KYVP_PCIINTERFACE_EVENT_ID | Identifier of the event type. | |
| data | union | Event-specific data. The active member is determined by eEventId. | |

#### 6.4.1.13.6 KYVP_DEVICE_EVENT_CONNECTION_LOST

Device connection lost event data.

Initialize version to KYVP_DEVICE_EVENT_CONNECTION_LOST_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Handle of the device associated with the event. | |

#### 6.4.1.13.7 KYVP_DEVICE_GENERIC_EVENT

Device event identifier and payload.

Initialize version to KYVP_DEVICE_GENERIC_EVENT_VERSION.

Note: The data union contains one of the following structures depending on eEventId :

KYVP_DEVICE_EVENT_CONNECTION_LOST

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| eEventId | KYVP_DEVICE_EVENT_ID | Identifier of the event type. | |
| data | union | Event-specific data. The active member is determined by eEventId. | |

#### 6.4.1.13.8 KYVPLibTL_GetPortUrl_Args

Arguments for KYVPLibTL_GetPortURL().

Initialize version to KYVPLIBTL_GETPORTURL_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| pszURL | char* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.9 KYVPLibTL_ReadPort_Args

Arguments for KYVPLibTL_ReadPort().

Initialize version to KYVPLIBTL_READPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| iAddress | uint64_t | Port address. | |
| pBuffer | void* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.10 KYVPLibTL_WritePort_Args

Arguments for KYVPLibTL_WritePort().

Initialize version to KYVPLIBTL_WRITEPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| iAddress | uint64_t | Port address. | |
| pBuffer | const void* | Pointer to caller-allocated const buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.11 KYVPLibTL_PCIInterface_GetPortUrl_Args

Arguments for KYVPLibTL_PCIInterface_GetPortURL().

Initialize version to KYVPLIBTL_PCIINTERFACEGETPORTURL_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| pszURL | char* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.12 KYVPLibTL_PCIInterface_ReadPort_Args

Arguments for KYVPLibTL_PCIInterface_ReadPort().

Initialize version to KYVPLIBTL_PCIINTERFACEREADPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| iAddress | uint64_t | Port address. | |
| pBuffer | void* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.13 KYVPLibTL_PCIInterface_WritePort_Args

Arguments for KYVPLibTL_PCIInterface_WritePort().

Initialize version to KYVPLIBTL_PCIINTERFACEWRITEPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| iAddress | uint64_t | Port address. | |
| pBuffer | const void* | Pointer to caller-allocated const buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.14 KYVPLibTL_Device_GetPortUrl_Args

Arguments for KYVPLibTL_Device_GetPortURL().

Initialize version to KYVPLIBTL_DEVICEGETPORTURL_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| hDevHandle | KYVP_DEVICE_HANDLE | Local device handle. | |
| pszURL | char* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.15 KYVPLibTL_Device_ReadPort_Args

Arguments for KYVPLibTL_Device_ReadPort().

Initialize version to KYVPLIBTL_DEVICEREADPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Local device handle. | |
| iAddress | uint64_t | Port address. | |
| pBuffer | void* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.16 KYVPLibTL_Device_WritePort_Args

Arguments for KYVPLibTL_Device_WritePort().

Initialize version to KYVPLIBTL_DEVICEWRITEPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Local device handle. | |
| iAddress | uint64_t | Port address. | |
| pBuffer | const void* | Pointer to caller-allocated const buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.17 KYVPLibTL_RemoteDevice_GetPortUrl_Args

Arguments for KYVPLibTL_RemoteDevice_GetPortURL().

Initialize version to KYVPLIBTL_REMOTEDEVICEGETPORTURL_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | Remote device handle. | |
| pszURL | char* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.18 KYVPLibTL_RemoteDevice_ReadPort_Args

Arguments for KYVPLibTL_RemoteDevice_ReadPort().

Initialize version to KYVPLIBTL_REMOTEDEVICEREADPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | Remote device handle. | |
| iAddress | uint64_t | Port address. | |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pBuffer | void* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.19 KYVPLibTL_RemoteDevice_WritePort_Args

Arguments for KYVPLibTL_RemoteDevice_WritePort().

Initialize version to KYVPLIBTL_REMOTEDEVICEWRITEPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | Remote device handle. | |
| iAddress | uint64_t | Port Address. | |
| pBuffer | const void* | Pointer to caller-allocated const buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.20 KYVPLibTL_Stream_GetPortUrl_Args

Arguments for KYVPLibTL_Stream_GetPortURL().

Initialize version to KYVPLIBTL_STREAMGETPORTURL_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hStreamHandle | KYVP_STREAM_HANDLE | Stream handle. | |
| pszURL | char* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.21 KYVPLibTL_Stream_ReadPort_Args

Arguments for KYVPLibTL_Stream_ReadPort().

Initialize version to KYVPLIBTL_STREAMREADPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hStreamHandle | KYVP_STREAM_HANDLE | Stream handle. | |
| iAddress | uint64_t | Port address. | |
| pBuffer | void* | Pointer to caller-allocated buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.22 KYVPLibTL_Stream_WritePort_Args

Arguments for KYVPLibTL_Stream_WritePort().

Initialize version to KYVPLIBTL_STREAMWRITEPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hStreamHandle | KYVP_STREAM_HANDLE | Stream handle. | |
| iAddress | uint64_t | Port address. | |
| pBuffer | const void* | Pointer to caller-allocated const buffer. | |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| piSize | size_t* | Pointer to caller-allocated size_t buffer. | |

#### 6.4.1.13.23 KYVPLibTL_TLOpen_Args

Arguments for KYVPLibTL_TLOpen().

Initialize version to KYVPLIBTL_TLOPEN_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| phTLHandle | KYVP_TL_HANDLE * | Pointer to KYVP_TL_HANDLE. | |

#### 6.4.1.13.24 KYVPLibTL_TLClose_Args

Arguments for KYVPLibTL_TLClose().

Initialize version to KYVPLIBTL_TLCLOSE_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |

#### 6.4.1.13.25 KYVPLibTL_TLGetInfo_Args

Arguments for KYVPLibTL_TLGetInfo().

Initialize version to KYVPLIBTL_TLGETINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| eInfoCmd | KYVP_TL_INFO_CMD | Information to be retrieved as defined in KYVP_TL_INFO_CMD. | |
| pType | KYVP_INFO_DATATYPE * | Data type of the pBuffer content as defined in the KYVP_INFO_DATATYPE. | |
| pBuffer | void* | Pointer to a caller-allocated buffer to receive the requested information. If this parameter is NULL, piSize will contain the minimal size of pBuffer in bytes. If the iType is a string the size includes the terminating 0. | |
| piSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.26 KYVPLibTL_TLGetNumInterfaces_Args

Arguments for KYVPLibTL_TLGetNumInterfaces().

Initialize version to KYVPLIBTL_TLGETNUMINTERFACES_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| piNumIfaces | uint32_t* | Pointer to caller-allocated uint32_t. | |

#### 6.4.1.13.27 KYVPLibTL_TLGetInterfaceID_Args

Arguments for KYVPLibTL_TLGetInterfaceID().

Initialize version to KYVPLIBTL_TLGETINTERFACEID_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| iIndex | uint32_t | Interface index. | |
| pszID | char* | Pointer to caller-allocated char* buffer. | |
| piSize | size_t* | Pointer to caller-allocated size_t* buffer. | |

#### 6.4.1.13.28 KYVPLibTL_TLGetPCIInterfaceInfo_Args

Arguments for KYVPLibTL_TLGetInterfaceInfo().

Initialize version to KYVPLIBTL_TLGETPCIINTERFACEINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| pszIfaceID | const char* | Interface ID from KYVPLibTL_TLGetInterfaceID func. | |
| eInfoCmd | KYVP_INTERFACE_INFO_CMD | Information to be retrieved. | |
| pType | KYVP_INFO_DATATYPE * | Data type of the pBuffer content. | |
| pBuffer | void* | Pointer to a caller-allocated buffer. | |
| piSize | size_t* | Out: minimal size of pBuffer. | |

#### 6.4.1.13.29 KYVPLibTL_TLOpenInterface_Args

Arguments for KYVPLibTL_TLOpenInterface().

Initialize version to KYVPLIBTL_TLOPENPCIINTERFACE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| pszIfaceID | const char* | Unique interface ID to open as a null-terminated C string. | |
| phPCIInterfaceHandle | KYVP_PCI_INTERFACE_HANDLE * | Pointer to KYVP_PCI_INTERFACE_HANDLE. | |

#### 6.4.1.13.30 KYVPLibTL_TLUpdatePCIInterfaceList_Args

Arguments for KYVPLibTL_TLUpdateInterfaceList().

Initialize version to KYVPLIBTL_TLUPDATEPCIINTERFACELIST_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hTLHandle | KYVP_TL_HANDLE | Transport layer system handle. | |
| pbChanged | KY_BOOL * | Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise. If set to NULL nothing is written to this parameter. | |
| iTimeout | uint64_t | Timeout in ms. | |

#### 6.4.1.13.31 KYVPLibTL_IFClose_Args

Arguments for KYVPLibTL_IFClose().

Initialize version to KYVPLIBTL_IFCLOSE_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle to close. | |

#### 6.4.1.13.32 KYVPLibTL_IFGetInfo_Args

Arguments for KYVPLibTL_IFGetInfo().

Initialize version to KYVPLIBTL_IFGETINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| eInfoCmd | KYVP_INTERFACE_INFO_CMD | Information to be retrieved. | |
| pType | KYVP_INFO_DATATYPE * | Pointer to data type of the pBuffer content. | |
| pBuffer | void* | Pointer to a caller-allocated buffer. | |
| piSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.33 KYVPLibTL_IFGetNumDevices_Args

Arguments for KYVPLibTL_IFGetNumDevices().

Initialize version to KYVPLIBTL_IFGETNUMDEVICES_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| pNumDevices | uint32_t* | Number of devices. | |

#### 6.4.1.13.34 KYVPLibTL_IFGetDeviceID_Args

Arguments for KYVPLibTL_IFGetDeviceID().

Initialize version to KYVPLIBTL_IFGETDEVICEID_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| iIndex | uint32_t | Device index. | |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pszID | char* | Pointer to a caller-allocated C string buffer to receive the Device module ID at the given iIndex. | |
| piSize | size_t* | Minimum size of pszID in bytes to hold the device ID. | |

#### 6.4.1.13.35 KYVPLibTL_IFUpdateDeviceList_Args

Arguments for KYVPLibTL_IFUpdateDeviceList().

Initialize version to KYVPLIBTL_IFUPDATEDEVICELIST_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| pbChanged | KY_BOOL * | Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise. | |
| uTimeout | uint64_t | Timeout in ms. | |
| bRetainOpenDevices | KY_BOOL | KY_TRUE preserves open devices in all processes. KY_FALSE closes this process's devices, clears its device lists and performs fresh detection. Devices opened by other processes are preserved in both modes. | Since version 2; defaults to KY_TRUE. |

#### 6.4.1.13.36 KYVPLibTL_IFGetDeviceInfo_Args

Arguments for KYVPLibTL_IFGetDeviceInfo().

Initialize version to KYVPLIBTL_IFGETDEVICEINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| pszIfaceID | const char* | Unique ID of the device to inquire information about. | |
| eInfoCmd | KYVP_DEVICE_INFO_CMD | Information to be retrieved. | |
| pType | KYVP_INFO_DATATYPE * | Data type of the pBuffer content. | |
| pBuffer | void* | Pointer to a caller-allocated buffer. | |
| piSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.37 KYVPLibTL_IFOpenDevice_Args

Arguments for KYVPLibTL_IFOpenDevice().

Initialize version to KYVPLIBTL_IFOPENDEVICE_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface handle. | |
| pszDeviceID | const char* | Unique device ID to open as a null-terminated C string. | |
| iOpenFlags | KYVP_DEVICE_ACCESS_FLAGS | Configures the open process. | |
| phDeviceHandle | KYVP_DEVICE_HANDLE * | Device handle of the newly created Device module. | |

#### 6.4.1.13.38 KYVPLibTL_IFRegisterEvent_Args

Arguments for KYVPLibTL_IFRegisterEvent().

Initialize version to KYVPLIBTL_IFREGISTEREVENT_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | Handle of the module on which to register the event. | |
| iEventID | KYVP_EVENT_TYPE | Event type to register. | |
| phEvent | KYVP_EVENT_HANDLE * | Pointer that receives the new event handle. | |

#### 6.4.1.13.39 KYVPLibTL_IFUnregisterEvent_Args

Arguments for KYVPLibTL_IFUnregisterEvent().

Initialize version to KYVPLIBTL_IFUNREGISTEREVENT_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | Handle of the module from which to unregister the event. | |
| iEventID | KYVP_EVENT_TYPE | Event type to unregister. | |

#### 6.4.1.13.40 KYVPLibTL_DevGetPort_Args

Arguments for KYVPLibTL_DevGetPort().

Initialize version to KYVPLIBTL_DEVGETPORT_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Device handle. | |
| phRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE * | Port handle for the remote device. | |

#### 6.4.1.13.41 KYVPLibTL_DevGetNumDataStreams_Args

Arguments for KYVPLibTL_DevGetNumDataStreams().

Initialize version to KYVPLIBTL_DEVGETNUMDATASTREAMS_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Device handle. | |
| piNumDataStreams | uint32_t* | Pointer that receives the number of data streams on this device. | |

#### 6.4.1.13.42 KYVPLibTL_DevGetDataStreamID_Args

Arguments for KYVPLibTL_DevGetDataStreamID().

Initialize version to KYVPLIBTL_DEVGETDATASTREAMID_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Device handle. | |
| iIndex | uint32_t | Zero-based index of the data stream on this device. | |
| sDataStreamID | char* | Pointer to a caller-allocated C string buffer. | |
| piSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.43 KYVPLibTL_DevOpenDataStream_Args

Arguments for KYVPLibTL_DevOpenDataStream().

Initialize version to KYVPLIBTL_DEVOPENDATASTREAM_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Device handle. | |
| pszDataStreamID | const char* | Unique data stream ID to open as a null-terminated C string. | |
| phDataStream | KYVP_STREAM_HANDLE * | Data Stream module handle of the newly created stream. | |

#### 6.4.1.13.44 KYVPLibTL_DevGetInfo_Args

Arguments for KYVPLibTL_DevGetInfo().

Initialize version to KYVPLIBTL_DEVGETINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Device handle. | |
| iInfoCmd | KYVP_DEVICE_INFO_CMD | Information to be retrieved. | |
| piType | KYVP_INFO_DATATYPE * | Data type of the pBuffer content. | |
| pBuffer | void* | Pointer to a caller-allocated buffer. | |
| piSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.45 KYVPLibTL_DevClose_Args

Arguments for KYVPLibTL_DevClose().

Initialize version to KYVPLIBTL_DEVCLOSE_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Device module handle to close. | |

#### 6.4.1.13.46 KYVPLibTL_DevRegisterEvent_Args

Arguments for KYVPLibTL_DevRegisterEvent().

Initialize version to KYVPLIBTL_DEVREGISTEREVENT_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Handle of the device on which to register the event. | |
| iEventID | KYVP_EVENT_TYPE | Event type to register. | |
| phEvent | KYVP_EVENT_HANDLE * | Pointer that receives the new event handle. | |

#### 6.4.1.13.47 KYVPLibTL_DevUnregisterEvent_Args

Arguments for KYVPLibTL_DevUnregisterEvent().

Initialize version to KYVPLIBTL_DEVUNREGISTEREVENT_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDevHandle | KYVP_DEVICE_HANDLE | Handle of the device from which to unregister the event. | |
| iEventID | KYVP_EVENT_TYPE | Event type to unregister. | |

#### 6.4.1.13.48 KYVPLibTL_DSAnnounceBuffer_Args

Arguments for KYVPLibTL_DSAnnounceBuffer().

Initialize version to KYVPLIBTL_DSANNOUNCEBUFFER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| pBuffer | void* | Pointer to buffer memory to announce. | |
| iSize | size_t | Size of the pBuffer in bytes. | |
| pPrivate | void* | Pointer to private data which will be passed to the GenTL Consumer on New Buffer events. This parameter may be NULL. | |
| phBuffer | KYVP_BUFFER_HANDLE * | Buffer module handle of the newly announced buffer. | |

#### 6.4.1.13.49 KYVPLibTL_DSAllocAndAnnounceBuffer_Args

Arguments for KYVPLibTL_DSAllocAndAnnounceBuffer().

Initialize version to KYVPLIBTL_DSALLOCANDANNOUNCEBUFFER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| iSize | size_t | Size of the buffer in bytes. | |
| pPrivate | void* | Pointer to private data which will be passed to the Consumer on New Buffer events. This parameter may be NULL. | |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| phBuffer | KYVP_BUFFER_HANDLE * | Buffer module handle of the newly announced buffer. | |

#### 6.4.1.13.50 KYVPLibTL_DSStartAcquisition_Args

Arguments for KYVPLibTL_DSStartAcquisition().

Initialize version to KYVPLIBTL_DSSTARTACQUISITION_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| iStartFlags | KYVP_ACQ_START_FLAGS | As defined in KYVP_ACQ_START_FLAGS. | |
| iNumToAcquire | uint64_t | Sets the number of filled/delivered buffers after which the acquisition engine stops automatically. Buffers that are internally discarded or missed are not counted. If set to 0, acquisition continues until a call to KYVPLibTL_DSStopAcquisition. To achieve precise accuracy and fine control over the number of acquired frames, the use of triggers is recommended. | |

#### 6.4.1.13.51 KYVPLibTL_DSStopAcquisition_Args

Arguments for KYVPLibTL_DSStopAcquisition().

Initialize version to KYVPLIBTL_DSSTOPACQUISITION_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| iStopFlags | KYVP_ACQ_STOP_FLAGS | Stops the acquisition as defined in KYVP_ACQ_STOP_FLAGS. | |

#### 6.4.1.13.52 KYVPLibTL_DSGetInfo_Args

Arguments for KYVPLibTL_DSGetInfo().

Initialize version to KYVPLIBTL_DSGETINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| eInfoCmd | KYVP_STREAM_INFO_CMD | Information to be retrieved. | |
| pType | KYVP_INFO_DATATYPE * | Data type of the pBuffer content. | |
| pBuffer | void* | Pointer to a caller-allocated buffer to receive the requested information. | |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| piSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.53 KYVPLibTL_DSGetBufferID_Args

Arguments for KYVPLibTL_DSGetBufferID().

Initialize version to KYVPLIBTL_DSGETBUFFEREID_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| iIndex | uint32_t | Zero-based index of the buffer on this data stream. | |
| ppBufferHandle | KYVP_BUFFER_HANDLE * | Buffer module handle of the given iIndex. | |

#### 6.4.1.13.54 KYVPLibTL_DSClose_Args

Arguments for KYVPLibTL_DSClose().

Initialize version to KYVPLIBTL_DSCLOSE_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |

#### 6.4.1.13.55 KYVPLibTL_DSRevokeBuffer_Args

Arguments for KYVPLibTL_DSRevokeBuffer().

Initialize version to KYVPLIBTL_DSREVOKEBUFFER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| hBufferHandle | KYVP_BUFFER_HANDLE | Buffer handle to revoke. | |
| ppBuffer | void** | Pointer to the buffer memory This is for convenience if consumer allocated memory is used which is to be freed. If the buffer was allocated by the GenTL Producer NULL is to be returned. If the parameter is set to NULL it is ignored. | |
| ppPrivate | void** | Pointer to the user data pointer given in the announce function. If the parameter is set to NULL it is ignored. | |

#### 6.4.1.13.56 KYVPLibTL_DSQueueBuffer_Args

Arguments for KYVPLibTL_DSQueueBuffer().

Initialize version to KYVPLIBTL_DSQUEUEBUFFER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| hBufferHandle | KYVP_BUFFER_HANDLE | Buffer handle to queue. | |

#### 6.4.1.13.57 KYVPLibTL_DSFlushQueue_Args

Arguments for KYVPLibTL_DSFlushQueue().

Initialize version to KYVPLIBTL_DSFLUSHQUEUE_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| iOperation | KYVP_ACQ_QUEUE_TYPE | Flush operation type. | |

#### 6.4.1.13.58 KYVPLibTL_DSGetBufferInfo_Args

Arguments for KYVPLibTL_DSGetBufferInfo().

Initialize version to KYVPLIBTL_DSGETBUFFERINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream handle. | |
| hBufferHandle | KYVP_BUFFER_HANDLE | Buffer handle to retrieve information about. | |
| iInfoCmd | KYVP_BUFFER_INFO_CMD | Information to be retrieved. | |
| piType | KYVP_INFO_DATATYPE * | Data type of the pBuffer content. | |
| pBuffer | void* | Pointer to a caller-allocated buffer to receive the requested information. | |
| piSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.59 KYVPLibTL_DSRegisterEvent_Args

Arguments for KYVPLibTL_DSRegisterEvent().

Initialize version to KYVPLIBTL_DSREGISTEREVENT_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Handle of the module on which to register the event. | |
| iEventID | KYVP_EVENT_TYPE | Event type to register. | |
| phEvent | KYVP_EVENT_HANDLE * | Pointer that receives the new event handle. | |

#### 6.4.1.13.60 KYVPLibTL_DSUnregisterEvent_Args

Arguments for KYVPLibTL_DSUnregisterEvent().

Initialize version to KYVPLIBTL_DSUNREGISTEREVENT_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hDSHandle | KYVP_STREAM_HANDLE | Handle of the module from which to unregister the event. | |
| iEventID | KYVP_EVENT_TYPE | Event type to unregister. | |

#### 6.4.1.13.61 KYVPLibTL_EventGetData_Args

Arguments for KYVPLibTL_EventGetData().

Initialize version to KYVPLIBTL_EVENTGETDATA_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hEventHandle | KYVP_EVENT_HANDLE | Event handle to wait for. | |
| pBuffer | void* | Pointer to a caller-allocated buffer. | |
| piSize | size_t* | Size of the provided pBuffer in bytes. | |
| iTimeout | uint64_t | Timeout for the wait in ms. | |

#### 6.4.1.13.62 KYVPLibTL_EventGetDataInfo_Args

Arguments for KYVPLibTL_EventGetDataInfo().

Initialize version to KYVPLIBTL_EVENTGETDATAINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hEventHandle | KYVP_EVENT_HANDLE | Event handle to parse data from. | |
| pInBuffer | const void* | Pointer to a buffer containing event data. | |
| iInSize | size_t | Size of the provided pInBuffer in bytes. | |
| iInfoCmd | KYVP_EVENT_DATA_INFO_CMD | Information to be retrieved. | |
| piType | KYVP_INFO_DATATYPE * | Data type of the pOutBuffer content. | |
| pOutBuffer | void* | Pointer to a caller-allocated buffer. | |
| piOutSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.63 KYVPLibTL_EventGetInfo_Args

Arguments for KYVPLibTL_EventGetInfo().

Initialize version to KYVPLIBTL_EVENTGETINFO_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hEventHandle | KYVP_EVENT_HANDLE | Event handle to parse data from. | |
| iInfoCmd | KYVP_EVENT_INFO_CMD | Information to be retrieved. | |
| piType | KYVP_INFO_DATATYPE * | Data type of the pBuffer content. | |
| pBuffer | void* | Pointer to a caller-allocated buffer. | |
| piSize | size_t* | Number of bytes filled by the function. | |

#### 6.4.1.13.64 KYVPLibTL_EventFlush_Args

Arguments for KYVPLibTL_EventFlush().

Initialize version to KYVPLIBTL_EVENTFLUSH_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hEventHandle | KYVP_EVENT_HANDLE | Event handle. | |

#### 6.4.1.13.65 KYVPLibTL_EventKill_Args

Arguments for KYVPLibTL_EventKill().

Initialize version to KYVPLIBTL_EVENTKILL_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | Required. |
| hEventHandle | KYVP_EVENT_HANDLE | Event handle. | |

# 7 KYVPLibExtension library

KYVPLibExtension is a low-level extension library designed to enhance the functionality of the KYVPLibTL library. It provides additional functionalities not available via TL Library. For example, firmware update, direct access to hardware registers etc. For more details see reference.

## 7.1 KYVPLibExtension API functions

The following reference documents the extension functions in KYVPLibExtension.h, the linkage and visibility macros in KYVPLibExtensionDefines.h, and the experimental logging API in experimental/KYVPLogging.h. Applications using the logging API must link with the extension library.

### 7.1.1 KYVPLibExtension.h API reference

Use the argument-structure entry points or their versioned convenience functions. Initialize each argument version explicitly in C; C++ member defaults supply the declared versions. Structure field tables preserve the packed public layouts and document input, output, and ownership requirements. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to inspect status values.

#### 7.1.1.1 Macros

##### 7.1.1.1.1 KYVPEXTENSION_PCIINTERFACE_FIRMWAREUPDATE_ARGS_VERSION

1

Structure layout version 1.

##### 7.1.1.1.2 KYVPEXTENSION_PCIINTERFACE_CHECKUPDATEFILE_ARGS_VERSION

1

Structure layout version 1.

##### 7.1.1.1.3 KYVPEXTENSION_PCIINTERFACE_EVENTCALLBACKREGISTER_ARGS_VERSION

1

Structure layout version 1.

##### 7.1.1.1.4 KYVPEXTENSION_PCIINTERFACE_EVENTCALLBACKUNREGISTER_ARGS_VERSION

1

Structure layout version 1.

##### 7.1.1.1.5 KYVPEXTENSION_DEVICE_EVENTCALLBACKREGISTER_ARGS_VERSION

1

Structure layout version 1.

##### 7.1.1.1.6 KYVPEXTENSION_DEVICE_EVENTCALLBACKUNREGISTER_ARGS_VERSION

1

Structure layout version 1.

##### 7.1.1.1.7 KYVPEXTENSION_REMOTEDEVICEDIRECTHWREADREG_STRUCT_VERSION

1

Structure layout version 1.

##### 7.1.1.1.8 KYVPEXTENSION_REMOTEDEVICEDIRECTHWWRITEREG_STRUCT_VERSION

1

Structure layout version 1.

##### 7.1.1.1.9 KYVPEXTENSION_REMOTEDEVICE_SETIMAGEDETAILS_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.10 KYVPEXTENSION_PCIINTERFACEDIRECTHWREADREG_STRUCT_VERSION

1

Structure layout version 1.

#### 7.1.1.1.11 KYVPEXTENSION_PCIINTERFACEDIRECTHWWRITEREG_STRUCT_VERSION

1

Structure layout version 1.

#### 7.1.1.1.12 KYVPEXTENSION_PCIINTERFACELOCKLINK_STRUCT_VERSION

1

Structure layout version 1.

#### 7.1.1.1.13 KYVPEXTENSION_PCIINTERFACEUNLOCKLINK_STRUCT_VERSION

1

Structure layout version 1.

#### 7.1.1.1.14 KYVPEXTENSION_PCIINTERFACEREADLINK_STRUCT_VERSION

1

Structure layout version 1.

#### 7.1.1.1.15 KYVPEXTENSION_PCIINTERFACEWRITELINK_STRUCT_VERSION

1

Structure layout version 1.

#### 7.1.1.1.16 KYVPEXTENSION_PCIINTERFACESENDLINKEVENT_STRUCT_VERSION

1

Structure layout version 1.

#### 7.1.1.1.17 KYVPEXTENSION_PCIINTERFACELINKGETSTATUS_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.18 KYVPEXTENSION_AUX_DATA_CALLBACK_REGISTER_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.19 KYVPEXTENSION_AUX_DATA_CALLBACK_UNREGISTER_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.20 KYVPEXTENSION_DATASTREAMBUFFERCHUNK_STRUCT_VERSION

1

Structure layout version 1.

#### 7.1.1.1.21 KYVPEXTENSION_DATASTREAMCREATE_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.22 KYVPEXTENSION_DATASTREAMANNOUNCEBUFFERCHUNKS_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.23 KYVPEXTENSION_DATASTREAM_CLOSE_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.24 KYVPEXTENSION_DATASTREAMCALLBACK_REGISTER_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.25 KYVPEXTENSION_DATASTREAMCALLBACKUNREGISTER_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.26 KYVPEXTENSION_SERIALCALLBACK_REGISTER_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.27 KYVPEXTENSION_SERIALCALLBACKUNREGISTER_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.28 KYVPEXTENSION_SERIAL_IPCSERVER_START_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.29 KYVPEXTENSION_SERIAL_IPCSERVER_STOP_ARGS_VERSION

1

Structure layout version 1.

#### 7.1.1.1.30 KYVPEXTENSION_REMOTEDEVICE_GETPARAMSCOLLECTIONHANDLE_ARGS

1

Structure layout version 1.

#### 7.1.1.1.31 KYVPEXTENSION_DEVICELINKDEFINITION_VERSION

1

Structure layout version 1.

#### 7.1.1.1.32 KYVPEXTENSION_DEVICEDEFINITION_VERSION

1

Structure layout version 1.

#### 7.1.1.1.33 KYVPEXTENSION_DETECTDEVICESMANUALLY_ARGS

3

Structure layout version 3.

### 7.1.1.2 Type definitions

#### 7.1.1.2.1 KYVP_UPDATE_CALLBACK

```
typedef KY_BOOL(* KYVP_UPDATE_CALLBACK) (const KYVP_UPDATE_STATUS *UpdateStatus, void *context))(const
KYVP_UPDATE_STATUS *UpdateStatus, void *context)
```

Receive firmware-update progress.

| Parameter | Type | Description |
|---|---|---|
| UpdateStatus | const KYVP_UPDATE_STATUS* | IN. Non-NULL progress snapshot; valid for the callback duration. |
| context | void* | IN. Application context supplied with the update request; may be NULL. |

Returns: Reserved Boolean result. Current firmware backends ignore this value; it does not cancel the update.

See: KYVP_UPDATE_STATUS KYVPExtension_PCIInterface_FirmwareUpdate

#### 7.1.1.2.2 KYVP_DATA_STREAM_CALLBACK

```
typedef void(* KYVP_DATA_STREAM_CALLBACK) (KYVP_BUFFER_HANDLE _phBufferHandle, void
*_pContext))(KYVP_BUFFER_HANDLE _phBufferHandle, void *_pContext)
```

Receive a data-stream buffer notification.

| Parameter | Type | Description |
|---|---|---|
| _phBufferHandle | KYVP_BUFFER_HANDLE | IN. Handle of the buffer associated with the notification. |

| Parameter | Type | Description |
|---|---|---|
| _pContext | void* | IN. Application context supplied at registration; may be NULL. |

See: KYVPExtension_DataStream_Callback_Register

#### 7.1.1.2.3 KYVP_AUX_DATA_CALLBACK

```
typedef void(* KYVP_AUX_DATA_CALLBACK) (const KYVP_AUX_DATA *_pData, void *_pContext))(const KYVP_AUX_DATA *_pData, void *_pContext)
```

Receive auxiliary data generated by a PCI interface.

Interpret the data according to its context and message identifier. Copy any payload needed after the callback returns.

| Parameter | Type | Description |
|---|---|---|
| _pData | const KYVP_AUX_DATA* | IN. Auxiliary-data record; valid for the callback duration. |
| _pContext | void* | IN. Application context supplied at registration; may be NULL. |

See: KYVPExtension_PCIInterface_AuxDataCallback_Register

#### 7.1.1.2.4 KYVP_SERIAL_CALLBACK

```
typedef void(* KYVP_SERIAL_CALLBACK) (uint32_t _uChannel, uint16_t _uEventId, const void *_pData, uint32_t _uSize, void *_pContext))(uint32_t _uChannel, uint16_t _uEventId, const void *_pData, uint32_t _uSize, void *_pContext)
```

Receive a serial-port event and its payload.

| Parameter | Type | Description |
|---|---|---|
| _uChannel | uint32_t | IN. Serial channel reporting the event. |
| _uEventId | uint16_t | IN. Transport-specific serial event identifier. |
| _pData | const void* | IN. Event payload; interpret according to the event identifier. Valid during the callback. |
| _uSize | uint32_t | IN. Payload size in bytes. |
| _pContext | void* | IN. Application context supplied at registration; may be NULL. |

See: KYVPExtension_Serial_Callback_Register

#### 7.1.1.2.5 KYVP_PCI_INTERFACE_EVENT_CALLBACK

```
typedef void(* KYVP_PCI_INTERFACE_EVENT_CALLBACK) (KYVP_PCIINTERFACE_GENERIC_EVENT *_pGenericEventData, void *_pUserContext))(KYVP_PCIINTERFACE_GENERIC_EVENT *_pGenericEventData, void *_pUserContext)
```

Receive a PCI interface event.

| Parameter | Type | Description |
|---|---|---|
| _pGenericEventData | KYVP_PCIINTERFACE_GENERIC_EVENT* | IN. Event record; use its event identifier to interpret the payload. Valid during the callback. |
| _pUserContext | void* | IN. Application context supplied at registration; may be NULL. |

See: KYVPExtension_PCIInterface_EventCallBackRegister

#### 7.1.1.2.6 KYVP_DEVICE_EVENT_CALLBACK

```
typedef void( * KYVP_DEVICE_EVENT_CALLBACK) (KYVP_DEVICE_GENERIC_EVENT *_pGenericEventData, void *_pUserContext))(KYVP_DEVICE_GENERIC_EVENT *_pGenericEventData, void *_pUserContext)
```

Receive a device event.

| Parameter | Type | Description |
|---|---|---|
| _pGenericEventData | KYVP_DEVICE_GENERIC_EVENT* | IN. Event record; use its event identifier to interpret the payload. Valid during the callback. |
| _pUserContext | void* | IN. Application context supplied at registration; may be NULL. |

See: KYVPExtension_Device_EventCallBackRegister

#### 7.1.1.2.7 GENTL_DS_HANDLE

```
typedef DS_HANDLE GENTL_DS_HANDLE
```

GenTL data-stream handle used by the experimental diagnostics API.

#### 7.1.1.2.8 GENTL_BUFFER_HANDLE

```
typedef BUFFER_HANDLE GENTL_BUFFER_HANDLE
```

GenTL buffer handle used by the experimental diagnostics API.

### 7.1.1.3 Functions

#### 7.1.1.3.1 KYVPExtension_PCIInterface_FirmwareUpdate

```
KY_RESULT KYVPExtension_PCIInterface_FirmwareUpdate(KYVPExtension_PCIInterface_FirmwareUpdate_Args *_pArgs)
```

Update PCI interface firmware from a file.

The optional callback reports write and validation progress. Its return value is ignored by the current firmware backends.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_FirmwareUpdate_Args * | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_FirmwareUpdate_Args . |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

#### 7.1.1.3.2 KYVPExtension_PCIInterface_CheckUpdateFile

```
KY_RESULT KYVPExtension_PCIInterface_CheckUpdateFile(KYVPExtension_PCIInterface_CheckUpdateFile_Args *_pArgs)
```

Check a firmware update file and return file and installed-flash metadata.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_CheckUpdateFile_Args * | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_CheckUpdateFile_Args . |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

#### 7.1.1.3.3 KYVPExtension_PCIInterface_EventCallBackRegister

```
KY_RESULT KYVPExtension_PCIInterface_EventCallBackRegister(KYVPExtension_PCIInterface_EventCallBackRegister_Args *_pArgs)
```

Register a callback for PCI interface events.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_EventCallBackRegister_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_EventCallBackRegister_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_EventCallBackRegister_V1

#### 7.1.1.3.4 KYVPExtension_PCIInterface_EventCallBackUnregister

```
KY_RESULT
KYVPExtension_PCIInterface_EventCallBackUnregister(KYVPExtension_PCIInterface_EventCallBackUnregister_Args
*_pArgs)
```

Remove a PCI interface event callback registration.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_EventCallBackUnregister_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_EventCallBackUnregister_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_EventCallBackUnregister_V1

#### 7.1.1.3.5 KYVPExtension_PCIInterface_DirectHWReadReg

```
KY_RESULT KYVPExtension_PCIInterface_DirectHWReadReg(KYVPExtension_PCIInterface_DirectHWReadReg_Args *_pArgs)
```

Read PCI interface registers through direct hardware access.

BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_DirectHWReadReg_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_DirectHWReadReg_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_DirectHWReadReg_V1

#### 7.1.1.3.6 KYVPExtension_PCIInterface_DirectHWWriteReg

```
KY_RESULT KYVPExtension_PCIInterface_DirectHWWriteReg(KYVPExtension_PCIInterface_DirectHWWriteReg_Args *_pArgs)
```

Write PCI interface registers through direct hardware access.

BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_DirectHWWriteReg_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_DirectHWWriteReg_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_DirectHWWriteReg_V1

#### 7.1.1.3.7 KYVPExtension_PCIInterface_LockLink

```
KY_RESULT KYVPExtension_PCIInterface_LockLink(KYVPExtension_PCIInterface_LockLink_Args *_pArgs)
```

Lock a PCI interface link for exclusive access.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_LockLink_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_LockLink_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_LockLink_V1

#### 7.1.1.3.8 KYVPExtension_PCIInterface_UnlockLink

```
KY_RESULT KYVPExtension_PCIInterface_UnlockLink(KYVPExtension_PCIInterface_UnlockLink_Args *_pArgs)
```

Release a PCI interface link lock.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_UnlockLink_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_UnlockLink_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_UnlockLink_V1

#### 7.1.1.3.9 KYVPExtension_PCIInterface_ReadLink

```
KY_RESULT KYVPExtension_PCIInterface_ReadLink(KYVPExtension_PCIInterface_ReadLink_Args *_pArgs)
```

Read registers through the selected PCI interface link.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_ReadLink_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_ReadLink_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_ReadLink_V1

#### 7.1.1.3.10 KYVPExtension_PCIInterface_WriteLink

```
KY_RESULT KYVPExtension_PCIInterface_WriteLink(KYVPExtension_PCIInterface_WriteLink_Args *_pArgs)
```

Write registers through the selected PCI interface link.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_WriteLink_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_WriteLink_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_WriteLink_V1

#### 7.1.1.3.11 KYVPExtension_PCIInterface_SendLinkEvent

```
KY_RESULT KYVPExtension_PCIInterface_SendLinkEvent(KYVPExtension_PCIInterface_SendLinkEvent_Args *_pArgs)
```

Send an event through the selected PCI interface link.

Support depends on the link transport; an unsupported transport returns an error.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_SendLinkEvent_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_SendLinkEvent_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

#### 7.1.1.3.12 KYVPExtension_PCIInterface_GetLinkStatus

```
KY_RESULT KYVPExtension_PCIInterface_GetLinkStatus(KYVPExtension_PCIInterface_LinkGetStatus_Args *_pArgs)
```

Query connection or lock status for a PCI interface link.

Connection status is returned as int32_t ; lock status is returned as KY_BOOL. Set the buffer to NULL to query the required size.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_PCIInterface_LinkGetStatus_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_PCIInterface_LinkGetStatus_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_GetLinkStatus_V1

#### 7.1.1.3.13 KYVPExtension_PCIInterface_AuxDataCallback_Register

```
KY_RESULT KYVPExtension_PCIInterface_AuxDataCallback_Register(KYVPExtension_AuxDataCallback_Register_Args
*_pArgs)
```

Register a callback for auxiliary data from a PCI interface.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_AuxDataCallback_Register_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_AuxDataCallback_Register_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_AuxDataCallback_Register_V1

#### 7.1.1.3.14 KYVPExtension_PCIInterface_AuxDataCallback_Unregister

```
KY_RESULT KYVPExtension_PCIInterface_AuxDataCallback_Unregister(KYVPExtension_AuxDataCallback_Unregister_Args *_pArgs)
```

Remove a PCI interface auxiliary-data callback registration.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_AuxDataCallback_Unregister_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_AuxDataCallback_Unregister_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_AuxDataCallback_Unregister_V1

#### 7.1.1.3.15 KYVPExtension_Device_EventCallBackRegister

```
KY_RESULT KYVPExtension_Device_EventCallBackRegister(KYVPExtension_Device_EventCallBackRegister_Args *_pArgs)
```

Register a callback for device events.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_Device_EventCallBackRegister_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Device_EventCallBackRegister_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Device_EventCallBackRegister_V1

#### 7.1.1.3.16 KYVPExtension_Device_EventCallBackUnregister

```
KY_RESULT KYVPExtension_Device_EventCallBackUnregister(KYVPExtension_Device_EventCallBackUnregister_Args *_pArgs)
```

Remove a device event callback registration.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_Device_EventCallBackUnregister_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Device_EventCallBackUnregister_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Device_EventCallBackUnregister_V1

#### 7.1.1.3.17 KYVPExtension_RemoteDevice_DirectHWReadReg

```
KY_RESULT KYVPExtension_RemoteDevice_DirectHWReadReg(KYVPExtension_RemoteDevice_DirectHWReadReg_Args *_pArgs)
```

Read remote-device registers through direct hardware access.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_RemoteDevice_DirectHWReadReg_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_RemoteDevice_DirectHWReadReg_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_DirectHWReadReg_V1

#### 7.1.1.3.18 KYVPExtension_RemoteDevice_DirectHWWriteReg

```
KY_RESULT KYVPExtension_RemoteDevice_DirectHWWriteReg(KYVPExtension_RemoteDevice_DirectHWWriteReg_Args *_pArgs)
```

Write remote-device registers through direct hardware access.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_RemoteDevice_DirectHWWriteReg_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_RemoteDevice_DirectHWWriteReg_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_DirectHWWriteReg_V1

#### 7.1.1.3.19 KYVPExtension_RemoteDevice_SetImageDetails

```
KY_RESULT KYVPExtension_RemoteDevice_SetImageDetails(KYVPExtension_RemoteDevice_SetImageDetails_Args *_pArgs)
```

Set the remote-device image geometry and pixel format used by the SDK.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_RemoteDevice_SetImageDetails_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_RemoteDevice_SetImageDetails_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_SetImageDetails_V1

#### 7.1.1.3.20 KYVPExtension_RemoteDevice_GetParamsCollectionHandle

```
KY_RESULT
KYVPExtension_RemoteDevice_GetParamsCollectionHandle(KYVPExtension_RemoteDevice_GetParamsCollectionHandle_Args
*_pArgs)
```

Retrieve the parameter collection associated with a remote device.

The collection belongs to the remote device. Use the parameter-handler API to access it; do not delete it as an application-created collection.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_RemoteDevice_GetParamsCollectionHandle_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_RemoteDevice_GetParamsCollectionHandle_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_GetParamsCollectionHandle_V1

#### 7.1.1.3.21 KYVPExtension_DataStream_Create

```
KY_RESULT KYVPExtension_DataStream_Create(KYVPExtension_DataStream_Create_Args *_pArgs)
```

Open a device data stream with the selected buffer-management mode.

The stream identifier must identify a device stream known to the initialized SDK. Close the returned handle with KYVPExtension_DataStream_Close().

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_DataStream_Create_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_DataStream_Create_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Create_V1

#### 7.1.1.3.22 KYVPExtension_DataStream_Close

```
KY_RESULT KYVPExtension_DataStream_Close(KYVPExtension_DataStream_Close_Args *_pArgs)
```

Close and release a data stream.

The stream handle becomes invalid on success.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_DataStream_Close_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_DataStream_Close_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Close_V1

#### 7.1.1.3.23 KYVPExtension_DSAnnounceBufferChunks

```
KY_RESULT KYVPExtension_DSAnnounceBufferChunks(KYVPExtension_DSAnnounceBufferChunks_Args *_pArgs)
```

Announce a buffer backed by multiple application-allocated memory chunks.

The chunk memory belongs to the application and must remain valid while the announced buffer is in use. Revoke the buffer before releasing its memory.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_DSAnnounceBufferChunks_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_DSAnnounceBufferChunks_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DSAnnounceBufferChunks_V1

#### 7.1.1.3.24 KYVPExtension_DataStream_Callback_Register

```
KY_RESULT KYVPExtension_DataStream_Callback_Register(KYVPExtension_DataStream_Callback_Register_Args *_pArgs)
```

Register a callback for data-stream buffer notifications.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_DataStream_Callback_Register_Args * | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_DataStream_Callback_Register_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Callback_Register_V1

#### 7.1.1.3.25 KYVPExtension_DataStream_Callback_Unregister

```
KY_RESULT KYVPExtension_DataStream_Callback_Unregister(KYVPExtension_DataStream_Callback_Unregister_Args *_pArgs)
```

Remove a data-stream callback registration.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_DataStream_Callback_Unregister_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_DataStream_Callback_Unregister_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Callback_Unregister_V1

#### 7.1.1.3.26 KYVPExtension_Serial_Callback_Register

```
KY_RESULT KYVPExtension_Serial_Callback_Register(KYVPExtension_Serial_Callback_Register_Args *_pArgs)
```

Register a serial-event callback for a PCI interface link.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_Serial_Callback_Register_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Serial_Callback_Register_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_Callback_Register_V1

#### 7.1.1.3.27 KYVPExtension_Serial_Callback_Unregister

```
KY_RESULT KYVPExtension_Serial_Callback_Unregister(KYVPExtension_Serial_Callback_Unregister_Args *_pArgs)
```

Request removal of a serial-event callback registration.

Warning: The current implementation returns success without removing the registered callback. Do not use this call as confirmation that callback delivery has stopped.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_Serial_Callback_Unregister_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Serial_Callback_Unregister_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_Callback_Unregister_V1

#### 7.1.1.3.28 KYVPExtension_Serial_IPCServer_Start

```
KY_RESULT KYVPExtension_Serial_IPCServer_Start(KYVPExtension_Serial_IPCServer_Start_Args *_pArgs)
```

Start the serial-port IPC server in the calling process.

Starting an already running server succeeds without creating another instance.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_Serial_IPCServer_Start_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Serial_IPCServer_Start_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_IPCServer_Start_V1

#### 7.1.1.3.29 KYVPExtension_Serial_IPCServer_Stop

```
KY_RESULT KYVPExtension_Serial_IPCServer_Stop(KYVPExtension_Serial_IPCServer_Stop_Args *_pArgs)
```

Stop the serial-port IPC server in the calling process.

Stopping an already stopped server succeeds.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_Serial_IPCServer_Stop_Args* | IN. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Serial_IPCServer_Stop_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_IPCServer_Stop_V1

#### 7.1.1.3.30 KYVPExtension_DetectDevicesManually

```
KY_RESULT KYVPExtension_DetectDevicesManually(KYVPExtension_DetectDevicesManually_Args *_pArgs)
```

Detect devices using an application-supplied link topology.

Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. Version 1 verifies link IDs and retains open devices. Version 2 adds optional link-ID verification suppression. Version 3 adds control over retaining open devices. With bRetainOpenDevices set to KY_FALSE, this process closes its devices and clears its device lists before fresh detection; existing handles and streams must be discarded. Devices opened by other processes are preserved in both modes.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYVPExtension_DetectDevicesManually_Args* | IN/OUT. Non-NULL argument block; initialize its version and fields as described in KYVPExtension_DetectDevicesManually_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DetectDevicesManually_V3

#### 7.1.1.3.31 KYVPExtension_PCIInterface_DirectHWReadReg_V1

```
KY_RESULT KYVPExtension_PCIInterface_DirectHWReadReg_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iBar, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)
```

Read PCI interface registers through direct hardware access.

BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration. Convenience entry point for KYVPExtension_PCIInterface_DirectHWReadReg() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _iBar | uint64_t | IN. PCI Base Address Register index. Normally 0. |
| _iAddress | uint64_t | IN. Starting register address in the selected address space. |
| _pBuffer | void* | OUT. Destination buffer for register data. Non-NULL for a nonzero transfer. |
| _piSize | size_t* | IN/OUT. Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_DirectHWReadReg

#### 7.1.1.3.32 KYVPExtension_PCIInterface_DirectHWWriteReg_V1

```
KY_RESULT KYVPExtension_PCIInterface_DirectHWWriteReg_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iBar, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)
```

Write PCI interface registers through direct hardware access.

BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration. Convenience entry point for KYVPExtension_PCIInterface_DirectHWWriteReg() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _iBar | uint64_t | IN. PCI Base Address Register index. Normally 0. |
| _iAddress | uint64_t | IN. Starting register address in the selected address space. |
| _pBuffer | const void* | IN. Source buffer containing bytes to write or send. Non-NULL for a nonzero transfer. |
| _piSize | size_t* | IN/OUT. Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_DirectHWWriteReg

#### 7.1.1.3.33 KYVPExtension_PCIInterface_LockLink_V1

```
KY_RESULT KYVPExtension_PCIInterface_LockLink_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink)
```

Lock a PCI interface link for exclusive access.

Convenience entry point for KYVPExtension_PCIInterface_LockLink() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _iLink | uint64_t | IN. Zero-based host link index on the PCI interface. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_LockLink

#### 7.1.1.3.34 KYVPExtension_PCIInterface_UnlockLink_V1

```
KY_RESULT KYVPExtension_PCIInterface_UnlockLink_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink)
```

Release a PCI interface link lock.

Convenience entry point for KYVPExtension_PCIInterface_UnlockLink() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _iLink | uint64_t | IN. Zero-based host link index on the PCI interface. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_UnlockLink

#### 7.1.1.3.35 KYVPExtension_PCIInterface_ReadLink_V1

```
KY_RESULT KYVPExtension_PCIInterface_ReadLink_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink, uint64_t
_iAddress, void *_pBuffer, size_t *_piSize)
```

Read registers through the selected PCI interface link.

Convenience entry point for KYVPExtension_PCIInterface_ReadLink() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _iLink | uint64_t | IN. Zero-based host link index on the PCI interface. |
| _iAddress | uint64_t | IN. Starting register address in the selected address space. |
| _pBuffer | void* | OUT. Destination buffer for register data. Non-NULL for a nonzero transfer. |
| _piSize | size_t* | IN/OUT. Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_ReadLink

#### 7.1.1.3.36 KYVPExtension_PCIInterface_WriteLink_V1

```
KY_RESULT KYVPExtension_PCIInterface_WriteLink_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink, uint64_t
_iAddress, const void *_pBuffer, size_t *_piSize)
```

Write registers through the selected PCI interface link.

Convenience entry point for KYVPExtension_PCIInterface_WriteLink() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _iLink | uint64_t | IN. Zero-based host link index on the PCI interface. |
| _iAddress | uint64_t | IN. Starting register address in the selected address space. |
| _pBuffer | const void* | IN. Source buffer containing bytes to write or send. Non-NULL for a nonzero transfer. |
| _piSize | size_t* | IN/OUT. Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_WriteLink

#### 7.1.1.3.37 KYVPExtension_PCIInterface_GetLinkStatus_V1

```
KY_RESULT KYVPExtension_PCIInterface_GetLinkStatus_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink,
KYVP_LINK_STATUS_CMD _iInfoCmd, KYVP_INFO_DATATYPE *_piType, void *_pBuffer, size_t *_piSize)
```

Query connection or lock status for a PCI interface link.

Connection status is returned as int32_t ; lock status is returned as KY_BOOL. Set the buffer to NULL to query the required size. Convenience entry point for KYVPExtension_PCIInterface_GetLinkStatus() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _iLink | uint64_t | IN. Zero-based host link index on the PCI interface. |
| _iInfoCmd | KYVP_LINK_STATUS_CMD | IN. Link-status property to query. |
| _piType | KYVP_INFO_DATATYPE* | OUT. Receives the data type of the returned status value. Non-NULL, including size-only queries. |
| _pBuffer | void* | OUT. Receives the selected link-status value. NULL queries the required size without copying data. |
| _piSize | size_t* | IN/OUT. Buffer capacity on input; required value size on output, in bytes. Non-NULL. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_GetLinkStatus

#### 7.1.1.3.38 KYVPExtension_PCIInterface_AuxDataCallback_Register_V1

```
KY_RESULT KYVPExtension_PCIInterface_AuxDataCallback_Register_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle,
KYVP_AUX_DATA_CALLBACK _pCallbackFunction, void *_pUserContext)
```

Register a callback for auxiliary data from a PCI interface.

Convenience entry point for KYVPExtension_PCIInterface_AuxDataCallback_Register() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _pCallbackFunction | KYVP_AUX_DATA_CALLBACK | IN. Callback function to invoke. Non-NULL. |

| Parameter | Type | Description |
|---|---|---|
| _pUserContext | void* | IN. Application context passed back to the callback. Optional, may be NULL. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_AuxDataCallback_Register

#### 7.1.1.3.39 KYVPExtension_PCIInterface_AuxDataCallback_Unregister_V1

```
KY_RESULT KYVPExtension_PCIInterface_AuxDataCallback_Unregister_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle,
KYVP_AUX_DATA_CALLBACK _pCallbackFunction)
```

Remove a PCI interface auxiliary-data callback registration.

Convenience entry point for KYVPExtension_PCIInterface_AuxDataCallback_Unregister() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _pCallbackFunction | KYVP_AUX_DATA_CALLBACK | IN. Callback function to remove. Non-NULL; pass the registered function. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_AuxDataCallback_Unregister

#### 7.1.1.3.40 KYVPExtension_PCIInterface_EventCallBackRegister_V1

```
KY_RESULT KYVPExtension_PCIInterface_EventCallBackRegister_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle,
KYVP_PCI_INTERFACE_EVENT_CALLBACK _pUserFunc, void *_pUserContext)
```

Register a callback for PCI interface events.

Convenience entry point for KYVPExtension_PCIInterface_EventCallBackRegister() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _pUserFunc | KYVP_PCI_INTERFACE_EVENT_CALLBACK | IN. Callback function to invoke. Non-NULL. |
| _pUserContext | void* | IN. Application context passed back to the callback. Optional, may be NULL. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_EventCallBackRegister

#### 7.1.1.3.41 KYVPExtension_PCIInterface_EventCallBackUnregister_V1

```
KY_RESULT KYVPExtension_PCIInterface_EventCallBackUnregister_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle,
KYVP_PCI_INTERFACE_EVENT_CALLBACK _pUserFunc)
```

Remove a PCI interface event callback registration.

Convenience entry point for KYVPExtension_PCIInterface_EventCallBackUnregister() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _pUserFunc | KYVP_PCI_INTERFACE_EVENT_CALLBACK | IN. Callback function to remove. Non-NULL; pass the registered function. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_EventCallBackUnregister

#### 7.1.1.3.42 KYVPExtension_Device_EventCallBackRegister_V1

```
KY_RESULT KYVPExtension_Device_EventCallBackRegister_V1(KYVP_DEVICE_HANDLE _hDevHandle,
KYVP_DEVICE_EVENT_CALLBACK _pUserFunc, void *_pUserContext)
```

Register a callback for device events.

Convenience entry point for KYVPExtension_Device_EventCallBackRegister() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device to receive event notifications from. Valid device handle. |
| _pUserFunc | KYVP_DEVICE_EVENT_CALLBACK | IN. Callback function to invoke. Non-NULL. |
| _pUserContext | void* | IN. Application context passed back to the callback. Optional, may be NULL. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Device_EventCallBackRegister

#### 7.1.1.3.43 KYVPExtension_Device_EventCallBackUnregister_V1

```
KY_RESULT KYVPExtension_Device_EventCallBackUnregister_V1(KYVP_DEVICE_HANDLE _hDevHandle,
KYVP_DEVICE_EVENT_CALLBACK _pUserFunc)
```

Remove a device event callback registration.

Convenience entry point for KYVPExtension_Device_EventCallBackUnregister() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hDevHandle | KYVP_DEVICE_HANDLE | IN. Device to receive event notifications from. Valid device handle. |
| _pUserFunc | KYVP_DEVICE_EVENT_CALLBACK | IN. Callback function to remove. Non-NULL; pass the registered function. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Device_EventCallBackUnregister

#### 7.1.1.3.44 KYVPExtension_RemoteDevice_DirectHWReadReg_V1

```
KY_RESULT KYVPExtension_RemoteDevice_DirectHWReadReg_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, uint64_t
_iAddress, void *_pBuffer, size_t *_piSize)
```

Read remote-device registers through direct hardware access.

Convenience entry point for KYVPExtension_RemoteDevice_DirectHWReadReg() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Remote device to operate on. Valid remote-device handle. |
| _iAddress | uint64_t | IN. Starting register address in the selected address space. |
| _pBuffer | void* | OUT. Destination buffer for register data. Non-NULL for a nonzero transfer. |
| _piSize | size_t* | IN/OUT. Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_DirectHWReadReg

#### 7.1.1.3.45 KYVPExtension_RemoteDevice_DirectHWWriteReg_V1

```
KY_RESULT KYVPExtension_RemoteDevice_DirectHWWriteReg_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, uint64_t
_iAddress, const void *_pBuffer, size_t *_piSize)
```

Write remote-device registers through direct hardware access.

Convenience entry point for KYVPExtension_RemoteDevice_DirectHWWriteReg() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Remote device to operate on. Valid remote-device handle. |
| _iAddress | uint64_t | IN. Starting register address in the selected address space. |
| _pBuffer | const void* | IN. Source buffer containing bytes to write or send. Non-NULL for a nonzero transfer. |
| _piSize | size_t* | IN/OUT. Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_DirectHWWriteReg

#### 7.1.1.3.46 KYVPExtension_RemoteDevice_SetImageDetails_V1

```
KY_RESULT KYVPExtension_RemoteDevice_SetImageDetails_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle,
KYVP_DEVICE_IMAGE_DETAILS _imageDetails)
```

Set the remote-device image geometry and pixel format used by the SDK.

Convenience entry point for KYVPExtension_RemoteDevice_SetImageDetails() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Remote device to operate on. Valid remote-device handle. |
| _imageDetails | KYVP_DEVICE_IMAGE_DETAILS | IN. Image dimensions and pixel format to apply. Initialize the nested structure version and fields. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_SetImageDetails

#### 7.1.1.3.47 KYVPExtension_RemoteDevice_GetParamsCollectionHandle_V1

```
KY_RESULT KYVPExtension_RemoteDevice_GetParamsCollectionHandle_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle,
KYVP_COLLECTION_HANDLE *_phParamsCollectionHandle)
```

Retrieve the parameter collection associated with a remote device.

The collection belongs to the remote device. Use the parameter-handler API to access it; do not delete it as an application-created collection. Convenience entry point for KYVPExtension_RemoteDevice_GetParamsCollectionHandle() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | IN. Remote device to operate on. Valid remote-device handle. |
| _phParamsCollectionHandle | KYVP_COLLECTION_HANDLE* | OUT. Receives the remote-device parameter collection handle. Non-NULL; owned by the remote device. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_GetParamsCollectionHandle

#### 7.1.1.3.48 KYVPExtension_DataStream_Create_V1

```
KY_RESULT KYVPExtension_DataStream_Create_V1(const char *_pszDevDataStreamID, KYVP_STREAM_BUFFERS_TYPE
_eStreamBufferType, KYVP_STREAM_HANDLE *_phDSHandle)
```

Open a device data stream with the selected buffer-management mode.

The stream identifier must identify a device stream known to the initialized SDK. Close the returned handle with KYVPExtension_DataStream_Close(). Convenience entry point for KYVPExtension_DataStream_Create() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _pszDevDataStreamID | const char* | IN. Identifier of the device data stream to open. Non-NULL, null-terminated string. |
| _eStreamBufferType | KYVP_STREAM_BUFFERS_TYPE | IN. Buffer-management mode for the stream. |
| _phDSHandle | KYVP_STREAM_HANDLE* | OUT. Receives the newly opened data-stream handle. Non-NULL. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Create

#### 7.1.1.3.49 KYVPExtension_DataStream_Close_V1

```
KY_RESULT KYVPExtension_DataStream_Close_V1(KYVP_STREAM_HANDLE _hDSHandle)
```

Close and release a data stream.

The stream handle becomes invalid on success. Convenience entry point for KYVPExtension_DataStream_Close() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data stream to operate on. Valid stream handle. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Close

#### 7.1.1.3.50 KYVPExtension_DSAnnounceBufferChunks_V1

```
KY_RESULT KYVPExtension_DSAnnounceBufferChunks_V1(KYVP_STREAM_HANDLE _hDSHandle, const
KYVPExtension_DataStreamBufferChunk *_pChunks, size_t _iNumChunks, void *_pPrivate, KYVP_BUFFER_HANDLE
*_phBuffer)
```

Announce a buffer backed by multiple application-allocated memory chunks.

The chunk memory belongs to the application and must remain valid while the announced buffer is in use. Revoke the buffer before releasing its memory. Convenience entry point for KYVPExtension_DSAnnounceBufferChunks() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data stream to operate on. Valid stream handle. |
| _pChunks | const KYVPExtension_DataStreamBufferChunk* | IN. Array describing the memory chunks that make up one buffer. Non-NULL, with _iNumChunks entries. |
| _iNumChunks | size_t | IN. Number of entries in _pChunks . Greater than zero. |
| _pPrivate | void* | IN. Application data associated with the announced buffer. Optional, may be NULL. |
| _phBuffer | KYVP_BUFFER_HANDLE* | OUT. Receives the announced buffer handle. Optional, may be NULL. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DSAnnounceBufferChunks

#### 7.1.1.3.51 KYVPExtension_DataStream_Callback_Register_V1

```
KY_RESULT KYVPExtension_DataStream_Callback_Register_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_DATA_STREAM_CALLBACK
_pCallbackFunction, void *_pContext)
```

Register a callback for data-stream buffer notifications.

Convenience entry point for KYVPExtension_DataStream_Callback_Register() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data stream to operate on. Valid stream handle. |
| _pCallbackFunction | KYVP_DATA_STREAM_CALLBACK | IN. Callback function to invoke. Non-NULL. |
| _pContext | void* | IN. Application context passed back to the callback. Optional, may be NULL. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Callback_Register

#### 7.1.1.3.52 KYVPExtension_DataStream_Callback_Unregister_V1

```
KY_RESULT KYVPExtension_DataStream_Callback_Unregister_V1(KYVP_STREAM_HANDLE _hDSHandle,
KYVP_DATA_STREAM_CALLBACK _pCallbackFunction)
```

Remove a data-stream callback registration.

Convenience entry point for KYVPExtension_DataStream_Callback_Unregister() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data stream to operate on. Valid stream handle. |
| _pCallbackFunction | KYVP_DATA_STREAM_CALLBACK | IN. Callback function to remove. Non-NULL; pass the registered function. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Callback_Unregister

#### 7.1.1.3.53 KYVPExtension_Serial_Callback_Register_V1

```
KY_RESULT KYVPExtension_Serial_Callback_Register_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink,
KYVP_SERIAL_CALLBACK _pCallbackFunction, void *_pContext)
```

Register a serial-event callback for a PCI interface link.

Convenience entry point for KYVPExtension_Serial_Callback_Register() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _iLink | uint64_t | IN. Zero-based host link index on the PCI interface. |
| _pCallbackFunction | KYVP_SERIAL_CALLBACK | IN. Callback function to invoke. Non-NULL. |
| _pContext | void* | IN. Application context passed back to the callback. Optional, may be NULL. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_Callback_Register

#### 7.1.1.3.54 KYVPExtension_Serial_Callback_Unregister_V1

```
KY_RESULT KYVPExtension_Serial_Callback_Unregister_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink,
KYVP_SERIAL_CALLBACK _pCallbackFunction)
```

Request removal of a serial-event callback registration.

Warning: The current implementation returns success without removing the registered callback. Do not use this call as confirmation that callback delivery has stopped. Convenience entry point for KYVPExtension_Serial_Callback_Unregister() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |

| Parameter | Type | Description |
|---|---|---|
| _iLink | uint64_t | IN. Zero-based host link index on the PCI interface. |
| _pCallbackFunction | KYVP_SERIAL_CALLBACK | IN. Callback function to remove. Non-NULL; pass the registered function. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_Callback_Unregister

#### 7.1.1.3.55 KYVPExtension_Serial_IPCServer_Start_V1

```
KY_RESULT KYVPExtension_Serial_IPCServer_Start_V1()
```

Start the serial-port IPC server in the calling process.

Starting an already running server succeeds without creating another instance. Convenience entry point for KYVPExtension_Serial_IPCServer_Start(). The argument version is supplied internally.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_IPCServer_Start

#### 7.1.1.3.56 KYVPExtension_Serial_IPCServer_Stop_V1

```
KY_RESULT KYVPExtension_Serial_IPCServer_Stop_V1()
```

Stop the serial-port IPC server in the calling process.

Stopping an already stopped server succeeds. Convenience entry point for KYVPExtension_Serial_IPCServer_Stop(). The argument version is supplied internally.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_IPCServer_Stop

#### 7.1.1.3.57 KYVPExtension_DetectDevicesManually_V1

```
KY_RESULT KYVPExtension_DetectDevicesManually_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL _bNoDevicesAccess, KY_BOOL *_pbChanged, DeviceDefinition *_pDeviceDefinitionArray, size_t _uDeviceDefinitionArrayCount)
```

Detect devices using an application-supplied link topology.

Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. Convenience entry point for KYVPExtension_DetectDevicesManually() with explicit parameters. This entry point verifies link IDs and retains open devices.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _bNoDevicesAccess | KY_BOOL | IN. Skip detection operations that require remote-device access. KY_TRUE enables this mode; SDK configuration can override it. |
| _pbChanged | KY_BOOL* | OUT. Receives whether the device list changed. Optional, may be NULL. |
| _pDeviceDefinitionArray | DeviceDefinition* | IN/OUT. Device topologies to detect, with detection results written back. Non-NULL array. |
| _uDeviceDefinitionArrayCount | size_t | IN. Number of entries in _pDeviceDefinitionArray . Greater than zero. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DetectDevicesManually

#### 7.1.1.3.58 KYVPExtension_DetectDevicesManually_V2

```
KY_RESULT KYVPExtension_DetectDevicesManually_V2(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL _bNoDevicesAccess, KY_BOOL _bSuppressLinkIDVerification, KY_BOOL *_pbChanged, DeviceDefinition *_pDeviceDefinitionArray, size_t _uDeviceDefinitionArrayCount)
```

Detect devices using an application-supplied link topology.

Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. Convenience entry point for KYVPExtension_DetectDevicesManually() with explicit parameters. This entry point retains open devices.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _bNoDevicesAccess | KY_BOOL | IN. Skip detection operations that require remote-device access. KY_TRUE enables this mode; SDK configuration can override it. |
| _bSuppressLinkIDVerification | KY_BOOL | IN. Allow detection without verifying the configured device link-ID order. Since version 2. |
| _pbChanged | KY_BOOL* | OUT. Receives whether the device list changed. Optional, may be NULL. |
| _pDeviceDefinitionArray | DeviceDefinition* | IN/OUT. Device topologies to detect, with detection results written back. Non-NULL array. |
| _uDeviceDefinitionArrayCount | size_t | IN. Number of entries in _pDeviceDefinitionArray . Greater than zero. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DetectDevicesManually

#### 7.1.1.3.59 KYVPExtension_DetectDevicesManually_V3

```
KY_RESULT KYVPExtension_DetectDevicesManually_V3(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL _bNoDevicesAccess, KY_BOOL _bSuppressLinkIDVerification, KY_BOOL *_pbChanged, DeviceDefinition *_pDeviceDefinitionArray, size_t _uDeviceDefinitionArrayCount, KY_BOOL _bRetainOpenDevices)
```

Detect devices using an application-supplied link topology.

Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. With _bRetainOpenDevices set to KY_FALSE, this process closes its devices and clears its device lists before fresh detection; existing handles and streams must be discarded. Devices opened by other processes are preserved in both modes. Convenience entry point for KYVPExtension_DetectDevicesManually() with explicit parameters.

| Parameter | Type | Description |
|---|---|---|
| _hIFHandle | KYVP_PCI_INTERFACE_HANDLE | IN. PCI interface to operate on. Valid interface handle. |
| _bNoDevicesAccess | KY_BOOL | IN. Skip detection operations that require remote-device access. KY_TRUE enables this mode; SDK configuration can override it. |
| _bSuppressLinkIDVerification | KY_BOOL | IN. Allow detection without verifying the configured device link-ID order. Since version 2. |
| _pbChanged | KY_BOOL* | OUT. Receives whether the device list changed. Optional, may be NULL. |

| Parameter | Type | Description |
|---|---|---|
| _pDeviceDefinitionArray | DeviceDefinition* | IN/OUT. Device topologies to detect, with detection results written back. Non-NULL array. |
| _uDeviceDefinitionArrayCount | size_t | IN. Number of entries in _pDeviceDefinitionArray . Greater than zero. |
| _bRetainOpenDevices | KY_BOOL | IN. Preserve open devices; KY_FALSE closes this process's devices before fresh detection. Since version 3. Other processes' devices are preserved. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DetectDevicesManually

#### 7.1.1.3.60 KYVPExtension_KYVPLibTL_DebugBuffer_V1

```
KY_RESULT KYVPExtension_KYVPLibTL_DebugBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_BUFFER_HANDLE _hBuffer)
```

Log diagnostic details for a KYVPLibTL stream buffer.

Warning: Experimental API. Its interface and behavior may change without deprecation.

| Parameter | Type | Description |
|---|---|---|
| _hDSHandle | KYVP_STREAM_HANDLE | IN. Data stream to operate on. Valid stream handle. |
| _hBuffer | KYVP_BUFFER_HANDLE | IN. Buffer handle belonging to the selected stream. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

#### 7.1.1.3.61 KYVPExtension_GentTL_DebugBuffer_V1

```
KY_RESULT KYVPExtension_GentTL_DebugBuffer_V1(GENTL_DS_HANDLE _hDataStream, GENTL_BUFFER_HANDLE _hBuffer)
```

Log diagnostic details for a buffer from the KAYA GenTL producer.

Warning: Experimental API. Its interface and behavior may change without deprecation.

| Parameter | Type | Description |
|---|---|---|
| _hDataStream | GENTL_DS_HANDLE | IN. Data-stream handle from the KAYA GenTL producer. |
| _hBuffer | GENTL_BUFFER_HANDLE | IN. Buffer handle belonging to the selected stream. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

### 7.1.1.4 KYVPExtension Structs

Field reference for extension API records and argument structures.

Tables list fields in declaration order. Set each version to the documented constant before calling the API. C++ default initialization does not replace required handles, buffers, and callback pointers.

#### 7.1.1.4.1 KYVP_UPDATE_STATUS

Progress snapshot supplied to the firmware-update callback.

The library supplies this structure. Read fields only for a supported struct_version ; copy any data needed after the callback returns.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| struct_version | int | Firmware-progress structure layout version. | Output; currently 1. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| bytes_sent | uint64_t | Progress byte count reported for the current firmware operation. | Output; interpret together with total_size and is_writing . |
| total_size | uint64_t | Total byte count used for progress reporting. | Output; firmware payload or phase size reported by the backend. |
| is_writing | KY_BOOL | Firmware operation phase. | Output; KY_TRUE for writing, KY_FALSE for validation. |

#### 7.1.1.4.2 KYVPExtension_PCIInterface_FirmwareUpdate_Args

Arguments: Update PCI interface firmware from a file.

Initialize version to KYVPEXTENSION_PCIINTERFACE_FIRMWAREUPDATE_ARGS_VERSION. The optional callback reports write and validation progress. Its return value is ignored by the current firmware backends.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| pszFilePath | char* | Path to the firmware update file. | Input; non-NULL, null-terminated string. |
| pCallbackFunction | KYVP_UPDATE_CALLBACK | Callback function to invoke. | Input; optional, may be NULL. |
| pUserContext | void* | Application context passed back to the callback. | Input; optional, may be NULL. |

#### 7.1.1.4.3 KYVPExtension_PCIInterface_CheckUpdateFile_Args

Arguments: Check a firmware update file and return file and installed-flash metadata.

Initialize version to KYVPEXTENSION_PCIINTERFACE_CHECKUPDATEFILE_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| fileMinorRev | uint16_t | Minor revision from the update file. | Output; valid on success. |
| fileMajorRev | uint16_t | Major revision from the update file. | Output; valid on success. |
| fileVendorId | uint16_t | Vendor identifier from the update file. | Output; valid on success. |
| fileBoardId | uint16_t | Board identifier from the update file. | Output; valid on success. |
| fileVersion | uint32_t | Firmware version from the update file. | Output; valid on success. |
| fileTimeStamp | uint32_t | Firmware timestamp value from the update file. | Output; valid on success. |
| flashMinorRev | uint16_t | Minor revision from the installed flash. | Output; valid on success. |
| flashMajorRev | uint16_t | Major revision from the installed flash. | Output; valid on success. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| flashVendorId | uint32_t | Vendor identifier from the installed flash. | Output; valid on success. |
| flashBoardId | uint32_t | Board identifier from the installed flash. | Output; valid on success. |
| flashVersion | uint32_t | Firmware version from the installed flash. | Output; valid on success. |
| flashTimeStamp | uint32_t | Firmware timestamp value from the installed flash. | Output; valid on success. |
| pszFilePath | char* | Path to the firmware update file. | Input; non-NULL, null-terminated string. |

#### 7.1.1.4.4 KYVPExtension_PCIInterface_EventCallBackRegister_Args

Arguments: Register a callback for PCI interface events.

Initialize version to KYVPEXTENSION_PCIINTERFACE_EVENTCALLBACKREGISTER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| pUserFunc | KYVP_PCI_INTERFACE_EVENT_CALLBACK | Callback function to invoke. | Input; non-NULL. |
| pUserContext | void* | Application context passed back to the callback. | Input; optional, may be NULL. |

#### 7.1.1.4.5 KYVPExtension_PCIInterface_EventCallBackUnregister_Args

Arguments: Remove a PCI interface event callback registration.

Initialize version to KYVPEXTENSION_PCIINTERFACE_EVENTCALLBACKUNREGISTER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| pUserFunc | KYVP_PCI_INTERFACE_EVENT_CALLBACK | Callback function to remove. | Input; non-NULL; pass the registered function. |

#### 7.1.1.4.6 KYVPExtension_Device_EventCallBackRegister_Args

Arguments: Register a callback for device events.

Initialize version to KYVPEXTENSION_DEVICE_EVENTCALLBACKREGISTER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hDevHandle | KYVP_DEVICE_HANDLE | Device to receive event notifications from. | Input; valid device handle. |
| pUserFunc | KYVP_DEVICE_EVENT_CALLBACK | Callback function to invoke. | Input; non-NULL. |
| pUserContext | void* | Application context passed back to the callback. | Input; optional, may be NULL. |

#### 7.1.1.4.7 KYVPExtension_Device_EventCallBackUnregister_Args

Arguments: Remove a device event callback registration.

Initialize version to KYVPEXTENSION_DEVICE_EVENTCALLBACKUNREGISTER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hDevHandle | KYVP_DEVICE_HANDLE | Device to receive event notifications from. | Input; valid device handle. |
| pUserFunc | KYVP_DEVICE_EVENT_CALLBACK | Callback function to remove. | Input; non-NULL; pass the registered function. |

#### 7.1.1.4.8 KYVPExtension_RemoteDevice_DirectHWReadReg_Args

Arguments: Read remote-device registers through direct hardware access.

Initialize version to KYVPEXTENSION_REMOTEDEVICEDIRECTHWREADREG_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | Remote device to operate on. | Input; valid remote-device handle. |
| iAddress | uint64_t | Starting register address in the selected address space. | Input. |
| pBuffer | void* | Destination buffer for register data. | Output; non-NULL for a nonzero transfer. |
| piSize | size_t* | Requested transfer size in bytes; may be updated by the transport. | Input/output; non-NULL; buffer must cover the requested size. |

#### 7.1.1.4.9 KYVPExtension_RemoteDevice_DirectHWWriteReg_Args

Arguments: Write remote-device registers through direct hardware access.

Initialize version to KYVPEXTENSION_REMOTEDEVICEDIRECTHWWRITEREG_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | Remote device to operate on. | Input; valid remote-device handle. |
| iAddress | uint64_t | Starting register address in the selected address space. | Input. |
| pBuffer | const void* | Source buffer containing bytes to write or send. | Input; non-NULL for a nonzero transfer. |
| piSize | size_t* | Requested transfer size in bytes; may be updated by the transport. | Input/output; non-NULL; buffer must cover the requested size. |

#### 7.1.1.4.10 KYVPExtension_RemoteDevice_SetImageDetails_Args

Arguments: Set the remote-device image geometry and pixel format used by the SDK.

Initialize version to KYVPEXTENSION_REMOTEDEVICE_SETIMAGEDETAILS_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | Remote device to operate on. | Input; valid remote-device handle. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| imageDetails | KYVP_DEVICE_IMAGE_DETAILS | Image dimensions and pixel format to apply. | Input; initialize the nested structure version and fields. |

#### 7.1.1.4.11 KYVPExtension_PCIInterface_DirectHWReadReg_Args

Arguments: Read PCI interface registers through direct hardware access.

Initialize version to KYVPEXTENSION_PCIINTERFACEDIRECTHWREADREG_STRUCT_VERSION. BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iBar | uint64_t | PCI Base Address Register index. | Input; normally 0. |
| iAddress | uint64_t | Starting register address in the selected address space. | Input. |
| pBuffer | void* | Destination buffer for register data. | Output; non-NULL for a nonzero transfer. |
| piSize | size_t* | Requested transfer size in bytes; may be updated by the transport. | Input/output; non-NULL; buffer must cover the requested size. |

#### 7.1.1.4.12 KYVPExtension_PCIInterface_DirectHWWriteReg_Args

Arguments: Write PCI interface registers through direct hardware access.

Initialize version to KYVPEXTENSION_PCIINTERFACEDIRECTHWWRITEREG_STRUCT_VERSION. BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iBar | uint64_t | PCI Base Address Register index. | Input; normally 0. |
| iAddress | uint64_t | Starting register address in the selected address space. | Input. |
| pBuffer | const void* | Source buffer containing bytes to write or send. | Input; non-NULL for a nonzero transfer. |
| piSize | size_t* | Requested transfer size in bytes; may be updated by the transport. | Input/output; non-NULL; buffer must cover the requested size. |

#### 7.1.1.4.13 KYVPExtension_PCIInterface_LockLink_Args

Arguments: Lock a PCI interface link for exclusive access.

Initialize version to KYVPEXTENSION_PCIINTERFACELOCKLINK_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iLink | uint64_t | Zero-based host link index on the PCI interface. | Input. |

#### 7.1.1.4.14 KYVPExtension_PCIInterface_UnlockLink_Args

Arguments: Release a PCI interface link lock.

Initialize version to KYVPEXTENSION_PCIINTERFACEUNLOCKLINK_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iLink | uint64_t | Zero-based host link index on the PCI interface. | Input. |

#### 7.1.1.4.15 KYVPExtension_PCIInterface_ReadLink_Args

Arguments: Read registers through the selected PCI interface link.

Initialize version to KYVPEXTENSION_PCIINTERFACEREADLINK_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iLink | uint64_t | Zero-based host link index on the PCI interface. | Input. |
| iAddress | uint64_t | Starting register address in the selected address space. | Input. |
| pBuffer | void* | Destination buffer for register data. | Output; non-NULL for a nonzero transfer. |
| piSize | size_t* | Requested transfer size in bytes; may be updated by the transport. | Input/output; non-NULL; buffer must cover the requested size. |

#### 7.1.1.4.16 KYVPExtension_PCIInterface_WriteLink_Args

Arguments: Write registers through the selected PCI interface link.

Initialize version to KYVPEXTENSION_PCIINTERFACEWRITELINK_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iLink | uint64_t | Zero-based host link index on the PCI interface. | Input. |
| iAddress | uint64_t | Starting register address in the selected address space. | Input. |
| pBuffer | const void* | Source buffer containing bytes to write or send. | Input; non-NULL for a nonzero transfer. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| piSize | size_t* | Requested transfer size in bytes; may be updated by the transport. | Input/output; non-NULL; buffer must cover the requested size. |

#### 7.1.1.4.17 KYVPExtension_PCIInterface_SendLinkEvent_Args

Arguments: Send an event through the selected PCI interface link.

Initialize version to KYVPEXTENSION_PCIINTERFACEWRITELINK_STRUCT_VERSION. Support depends on the link transport; an unsupported transport returns an error.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iLink | uint64_t | Zero-based host link index on the PCI interface. | Input. |
| iEventId | uint16_t | Transport-specific event identifier. | Input. |
| pBuffer | const void* | Source buffer containing bytes to write or send. | Input; non-NULL for a nonzero transfer. |
| piSize | size_t* | Requested transfer size in bytes; may be updated by the transport. | Input/output; non-NULL; buffer must cover the requested size. |

#### 7.1.1.4.18 KYVPExtension_PCIInterface_LinkGetStatus_Args

Arguments: Query connection or lock status for a PCI interface link.

Initialize version to KYVPEXTENSION_PCIINTERFACELINKGETSTATUS_ARGS_VERSION. Connection status is returned as int32_t ; lock status is returned as KY_BOOL. Set the buffer to NULL to query the required size.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iLink | uint64_t | Zero-based host link index on the PCI interface. | Input. |
| iInfoCmd | KYVP_LINK_STATUS_CMD | Link-status property to query. | Input. |
| piType | KYVP_INFO_DATATYPE* | Receives the data type of the returned status value. | Output; non-NULL, including size-only queries. |
| pBuffer | void* | Receives the selected link-status value. | Output; NULL queries the required size without copying data. |
| piSize | size_t* | Buffer capacity on input; required value size on output, in bytes. | Input/output; non-NULL. |

#### 7.1.1.4.19 KYVPExtension_AuxDataCallback_Register_Args

Arguments: Register a callback for auxiliary data from a PCI interface.

Initialize version to KYVPEXTENSION_AUX_DATA_CALLBACK_REGISTER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| pCallbackFunction | KYVP_AUX_DATA_CALLBACK | Callback function to invoke. | Input; non-NULL. |
| pUserContext | void* | Application context passed back to the callback. | Input; optional, may be NULL. |

#### 7.1.1.4.20 KYVPExtension_AuxDataCallback_Unregister_Args

Arguments: Remove a PCI interface auxiliary-data callback registration.

Initialize version to KYVPEXTENSION_AUX_DATA_CALLBACK_UNREGISTER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| pCallbackFunction | KYVP_AUX_DATA_CALLBACK | Callback function to remove. | Input; non-NULL; pass the registered function. |

#### 7.1.1.4.21 KYVPExtension_DataStreamBufferChunk

One memory segment of a buffer announced with KYVPExtension_DSAnnounceBufferChunks().

Initialize version to KYVPEXTENSION_DATASTREAMBUFFERCHUNK_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| pBuffer | void* | Virtual address of application-allocated chunk memory. | Input; retain the memory while the buffer is announced. |
| iSize | size_t | Size of this memory chunk in bytes. | Input. |

#### 7.1.1.4.22 KYVPExtension_DataStream_Create_Args

Arguments: Open a device data stream with the selected buffer-management mode.

Initialize version to KYVPEXTENSION_DATASTREAMCREATE_ARGS_VERSION. The stream identifier must identify a device stream known to the initialized SDK. Close the returned handle with KYVPExtension_DataStream_Close().

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| pszDevDataStreamID | const char* | Identifier of the device data stream to open. | Input; non-NULL, null-terminated string. |
| eStreamBufferType | KYVP_STREAM_BUFFERS_TYPE | Buffer-management mode for the stream. | Input. |
| phDSHandle | KYVP_STREAM_HANDLE* | Receives the newly opened data-stream handle. | Output; non-NULL. |

#### 7.1.1.4.23 KYVPExtension_DSAnnounceBufferChunks_Args

Arguments: Announce a buffer backed by multiple application-allocated memory chunks.

Initialize version to KYVPEXTENSION_DATASTREAMANNOUNCEBUFFERCHUNKS_ARGS_VERSION. The chunk memory belongs to the application and must remain valid while the announced buffer is in use. Revoke the buffer before releasing its memory.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream to operate on. | Input; valid stream handle. |
| pChunks | const KYVPExtension_DataStreamBufferChunk* | Array describing the memory chunks that make up one buffer. | Input; non-NULL, with iNumChunks entries. |
| iNumChunks | size_t | Number of entries in pChunks . | Input; greater than zero. |
| pPrivate | void* | Application data associated with the announced buffer. | Input; optional, may be NULL. |
| phBuffer | KYVP_BUFFER_HANDLE* | Receives the announced buffer handle. | Output; optional, may be NULL. |

#### 7.1.1.4.24 KYVPExtension_DataStream_Close_Args

Arguments: Close and release a data stream.

Initialize version to KYVPEXTENSION_DATASTREAM_CLOSE_ARGS_VERSION. The stream handle becomes invalid on success.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream to operate on. | Input; valid stream handle. |

#### 7.1.1.4.25 KYVPExtension_DataStream_Callback_Register_Args

Arguments: Register a callback for data-stream buffer notifications.

Initialize version to 1. The C constant is KYVPEXTENSION_DATASTREAMCALLBACK_REGISTER_ARGS_VERSION ; the C++ constant is KYVPEXTENSION_DATASTREAMCALLBACKREGISTER_ARGS_VERSION .

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream to operate on. | Input; valid stream handle. |
| pCallbackFunction | KYVP_DATA_STREAM_CALLBACK | Callback function to invoke. | Input; non-NULL. |
| pContext | void* | Application context passed back to the callback. | Input; optional, may be NULL. |

#### 7.1.1.4.26 KYVPExtension_DataStream_Callback_Unregister_Args

Arguments: Remove a data-stream callback registration.

Initialize version to KYVPEXTENSION_DATASTREAMCALLBACKUNREGISTER_ARGS_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hDSHandle | KYVP_STREAM_HANDLE | Data stream to operate on. | Input; valid stream handle. |
| pCallbackFunction | KYVP_DATA_STREAM_CALLBACK | Callback function to remove. | Input; non-NULL; pass the registered function. |

#### 7.1.1.4.27 KYVPExtension_Serial_Callback_Register_Args

Arguments: Register a serial-event callback for a PCI interface link.

Initialize version to 1. The C constant is KYVPEXTENSION_SERIALCALLBACK_REGISTER_ARGS_VERSION ; the C++ constant is KYVPEXTENSION_SERIALCALLBACKREGISTER_ARGS_VERSION .

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iLink | uint64_t | Zero-based host link index on the PCI interface. | Input. |
| pCallbackFunction | KYVP_SERIAL_CALLBACK | Callback function to invoke. | Input; non-NULL. |
| pContext | void* | Application context passed back to the callback. | Input; optional, may be NULL. |

#### 7.1.1.4.28 KYVPExtension_Serial_Callback_Unregister_Args

Arguments: Request removal of a serial-event callback registration.

Initialize version to KYVPEXTENSION_SERIALCALLBACKUNREGISTER_ARGS_VERSION.

Warning: The current implementation returns success without removing the registered callback. Do not use this call as confirmation that callback delivery has stopped.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| iLink | uint64_t | Zero-based host link index on the PCI interface. | Input. |
| pCallbackFunction | KYVP_SERIAL_CALLBACK | Callback function to remove. | Input; non-NULL; pass the registered function. |

#### 7.1.1.4.29 KYVPExtension_Serial_IPCServer_Start_Args

Arguments: Start the serial-port IPC server in the calling process.

Initialize version to KYVPEXTENSION_SERIAL_IPCSERVER_START_ARGS_VERSION. Starting an already running server succeeds without creating another instance.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |

#### 7.1.1.4.30 KYVPExtension_Serial_IPCServer_Stop_Args

Arguments: Stop the serial-port IPC server in the calling process.

Initialize version to KYVPEXTENSION_SERIAL_IPCSERVER_STOP_ARGS_VERSION. Stopping an already stopped server succeeds.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |

#### 7.1.1.4.31 KYVPExtension_RemoteDevice_GetParamsCollectionHandle_Args

Arguments: Retrieve the parameter collection associated with a remote device.

Initialize version to KYVPEXTENSION_REMOTEDEVICE_GETPARAMSCOLLECTIONHANDLE_ARGS. The collection belongs to the remote device. Use the parameter-handler API to access it; do not delete it as an application-created collection.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 1. |
| hRemoteDeviceHandle | KYVP_REMOTE_DEVICE_HANDLE | Remote device to operate on. | Input; valid remote-device handle. |
| phParamsCollectionHandle | KYVP_COLLECTION_HANDLE* | Receives the remote-device parameter collection handle. | Output; non-NULL; owned by the remote device. |

#### 7.1.1.4.32 DeviceLinkDefinition

Mapping for one device link in a manually configured device.

Initialize uVersion to KYVPEXTENSION_DEVICELINKDEFINITION_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Argument structure layout version. | Input; set to 1. |
| uHostLink | uint32_t | Zero-based host link connected to this device link. | Input; replace the unassigned default with a valid index. |

#### 7.1.1.4.33 DeviceDefinition

Expected device topology and the result of manual detection.

Initialize uVersion to KYVPEXTENSION_DEVICEDEFINITION_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Argument structure layout version. | Input; set to 1. |
| eConnectionSpeed | KYVP_CXP_LINK_SPEED | Expected current CoaXPress link speed of the device. | Input. |
| pDeviceLinkDefinitionArray | DeviceLinkDefinition* | Array mapping device link indices to host link indices. | Input; non-NULL; array position identifies the device link. |
| uDeviceLinkDefinitionArrayCount | size_t | Number of entries in pDeviceLinkDefinitionArray. | Input. |
| bFound | KY_BOOL | Whether the configured device was found. | Output; KY_TRUE when found. |
| szID | char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1] | Transport-layer device identifier returned by detection. | Output; null-terminated when the device is found. |

#### 7.1.1.4.34 KYVPExtension_DetectDevicesManually_Args

Arguments: Detect devices using an application-supplied link topology.

Initialize version to KYVPEXTENSION_DETECTDEVICESMANUALLY_ARGS. Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. Version 1 verifies link IDs and retains open devices. Version 2 adds optional link-ID verification suppression. Version 3 adds control over retaining open devices. With bRetainOpenDevices set to KY_FALSE, this

process closes its devices and clears its device lists before fresh detection; existing handles and streams must be discarded. Devices opened by other processes are preserved in both modes.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Argument structure layout version. | Input; set to 3. |
| hIFHandle | KYVP_PCI_INTERFACE_HANDLE | PCI interface to operate on. | Input; valid interface handle. |
| pbChanged | KY_BOOL* | Receives whether the device list changed. | Output; optional, may be NULL. |
| pDeviceDefinitionArray | DeviceDefinition* | Device topologies to detect, with detection results written back. | Input/output; non-NULL array. |
| uDeviceDefinitionArrayCount | size_t | Number of entries in pDeviceDefinitionArray . | Input; greater than zero. |
| bNoDevicesAccess | KY_BOOL | Skip detection operations that require remote-device access. | Input; KY_TRUE enables this mode; SDK configuration can override it. |
| bSuppressLinkIDVerification | KY_BOOL | Allow detection without verifying the configured device link-ID order. | Input; since version 2; defaults to KY_FALSE. |
| bRetainOpenDevices | KY_BOOL | Preserve open devices; KY_FALSE closes this process's devices before fresh detection. | Input; since version 3; defaults to KY_TRUE. Other processes' devices are preserved. |

## 7.1.2 KYVPLibExtensionDefines.h API reference

### 7.1.2.1 Macros

#### 7.1.2.1.1 KYVP_EXTENSION_EXTERNAL_DLL

#### 7.1.2.1.2 KYVP_EXTENSION_API

```
KYVP_EXTERNAL_C KYVP_EXTENSION_EXTERNAL_DLL
```

## 7.1.3 KYVPLogging.h API reference

Use KYVPLOG() or a severity-specific macro to attach the source file, function, and line to a printf-style message. Logger configuration controls which messages are emitted. Applications using these macros must link with the extension library. See Experimental API annotations for compiler diagnostics and the MSVC opt-in include order.

### 7.1.3.1 Macros

#### 7.1.3.1.1 KYVPLOG_FILE

```
(kyvp_current_source_filename(__FILE__))
```

Source basename with its final filename extension removed.

The result uses reusable internal storage and may be truncated to fit. Copy it if it must survive another call. Storage is thread-local only when the active KYVP_TLS branch supplies a thread-local specifier; the fallback uses shared static storage.

#### 7.1.3.1.2 KYVP_USE_EXPERIMENTAL_APIS

Legacy flag defined by this logging header.

This definition enables the following logging macro branch. It occurs after the experimental annotations are included, so it does not select MSVC opt-in retroactively. Define the flag before the first include to select that diagnostic mode. This header defines the flag unconditionally; it is not a caller-controlled logging disable switch.

#### 7.1.3.1.3 KYVPLOG

```
KYVPExtension_AppLog_V1(_log_priority, KYVPLOG_FILE, KY_FUNC, __LINE__, _fmt, ##__VA_ARGS__)
```

Log a formatted message with automatic source location.

| Parameter | Description |
|---|---|
| _log_priority | Message severity from KYVPLOG_PRIORITY. |
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

#### 7.1.3.1.4 KYVPLOG_TRACE

```
KYVPLOG(KYVPLOG_TRACE_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at TRACE severity.

| Parameter | Description |
|---|---|
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

#### 7.1.3.1.5 KYVPLOG_DEBUG

```
KYVPLOG(KYVPLOG_DEBUG_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at DEBUG severity.

| Parameter | Description |
|---|---|
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

#### 7.1.3.1.6 KYVPLOG_INFO

```
KYVPLOG(KYVPLOG_INFO_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at INFO severity.

| Parameter | Description |
|---|---|
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

#### 7.1.3.1.7 KYVPLOG_NOTICE

```
KYVPLOG(KYVPLOG_NOTICE_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at NOTICE severity.

| Parameter | Description |
|---|---|
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

#### 7.1.3.1.8 KYVPLOG_WARN

```
KYVPLOG(KYVPLOG_WARN_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at WARN severity.

| Parameter | Description |
|---|---|
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

#### 7.1.3.1.9 KYVPLOG_ERROR

```
KYVPLOG(KYVPLOG_ERROR_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at ERROR severity.

| Parameter | Description |
|---|---|
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

#### 7.1.3.1.10 KYVPLOG_CRITICAL

```
KYVPLOG(KYVPLOG_CRITICAL_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at CRITICAL severity.

| Parameter | Description |
|---|---|
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

#### 7.1.3.1.11 KYVPLOG_FATAL

```
KYVPLOG(KYVPLOG_FATAL_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at FATAL severity.

| Parameter | Description |
|---|---|
| _fmt | Non-NULL printf-style format string. |
| ... | Optional formatting arguments matching _fmt . |

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

#### 7.1.3.1.12 KYVPLOG_AND_RETURN_ERROR

```
    {                                          \
       KYVPLOG_ERROR("returning error %d", _err); \
       return _err;                            \
    }
```

Log an error and return its value from the enclosing function.

Expands to a statement block. Use braces around surrounding conditional branches. The error expression is evaluated once for logging and again for return; pass a stable value without side effects.

| Parameter | Description |
|---|---|
| _err | Integer error value compatible with d and the enclosing return type. |

#### 7.1.3.1.13 KYVPLOG_AND_RETURN_ERROR_MSG

```
    {                                                          \
       KYVPLOG_ERROR("returning error %d, " _fmt, _err, ##__VA_ARGS__); \
       return _err;                                            \
    }
```

Log an error with a message and return its value from the enclosing function.

Expands to a statement block. Use braces around surrounding conditional branches. The error expression is evaluated once for logging and again for return; pass a stable value without side effects.

| Parameter | Description |
|---|---|
| _err | Integer error value compatible with d and the enclosing return type. |
| _fmt | String literal or macro expanding to one, concatenated with the error prefix. |
| ... | Optional formatting arguments matching _fmt . |

### 7.1.3.2 Enumerations

#### 7.1.3.2.1 _KYVPLOG_PRIORITY

Message severities accepted by the application logger.

##### *7.1.3.2.1.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVPLOG_TRACE_LEVEL | 8 | Detailed execution tracing; least severe priority. |
| KYVPLOG_DEBUG_LEVEL | 7 | Diagnostic details for debugging. |
| KYVPLOG_INFO_LEVEL | 6 | Information about normal application activity. |
| KYVPLOG_NOTICE_LEVEL | 5 | Significant events during normal operation. |
| KYVPLOG_WARN_LEVEL | 4 | A condition that may require attention. |
| KYVPLOG_ERROR_LEVEL | 3 | An operation failed or encountered an error. |
| KYVPLOG_CRITICAL_LEVEL | 2 | A serious condition affecting application operation. |
| KYVPLOG_FATAL_LEVEL | 1 | The highest severity classification; logging alone does not terminate the process. |

### 7.1.3.3 Type definitions

#### 7.1.3.3.1 KYVPLOG_PRIORITY

```
typedef enum _KYVPLOG_PRIORITY KYVPLOG_PRIORITY
```

Message severities accepted by the application logger.

### 7.1.3.4 Functions

#### 7.1.3.4.1 KYVPExtension_AppLog_V1

```
KY_RESULT KYVPExtension_AppLog_V1(KYVPLOG_PRIORITY _log_priority, const char *_pszLoggerName, const char *_pszFuncName, int _line, const char *_pszFmt,...)
```

Forward a formatted application message to the SDK logger.

Warning: Experimental backend entry point. Use KYVPLOG() or its severity-specific macros; the function signature may change without notice. The logger name is placed under the KYVPExt logging namespace. The function prefixes the message with the source line. A success result does not guarantee that logger filters emitted the message.

| Parameter | Type | Description |
|---|---|---|
| _log_priority | KYVPLOG_PRIORITY | IN. One of the documented KYVPLOG_PRIORITY values. |
| _pszLoggerName | const char* | IN. Non-NULL, null-terminated logger name, normally the source basename. |
| _pszFuncName | const char* | IN. Non-NULL, null-terminated calling-function name. |
| _line | int | IN. Calling source line number. |
| _pszFmt | const char* | IN. Non-NULL, null-terminated printf-style format string. |
| ... | ... | IN. Formatting arguments matching the format string. |

Returns: The current implementation returns SDK success after submitting the message. Inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

# 8 KYVPParametersHandler library

## 8.1 Function Call Sequence

This KYVPParametersHandler is a high level library that loads Gen<i>Cam XML and provides software interface for reading and writing values by a named parameter, and also queuering various attributes of parameter such as min, max, description etc (similar to what Gen<i>Cam reference library implementation provides). It translates high level API calls with parameter names into low level transport requests with register addresses. It does not implement transport functionality and should be used in conjunction with a separate transport implementation.

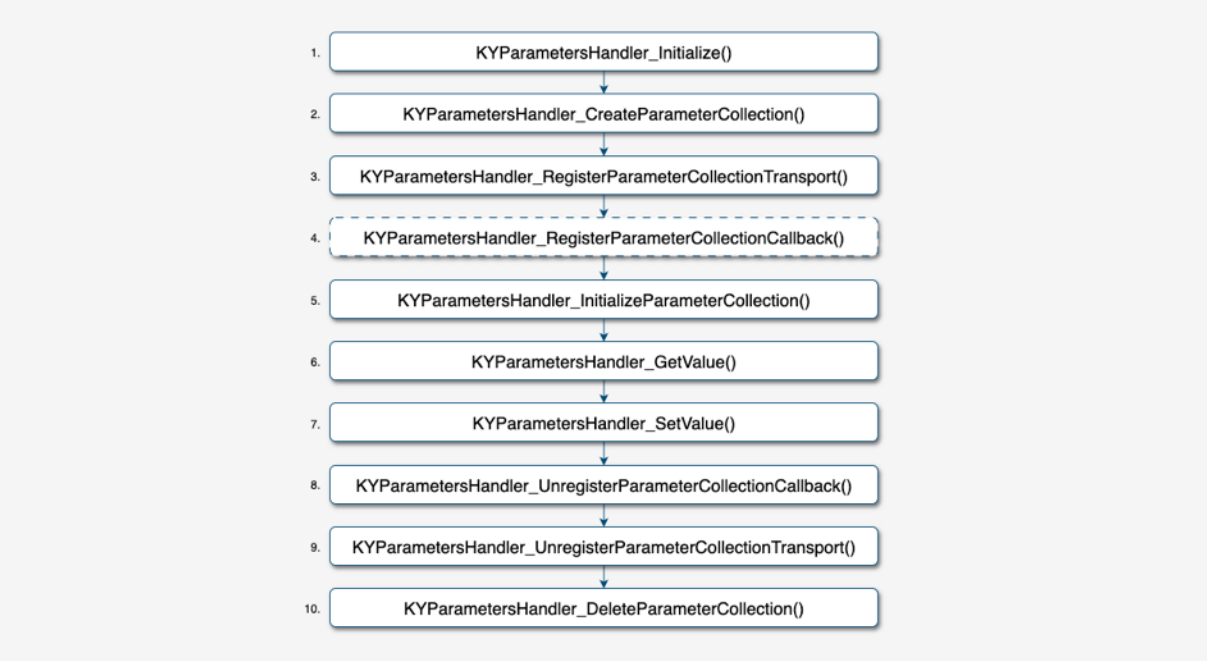



Figure 4 – KYVPParameterHandler function call sequence

1. KYParametersHandler_Initialize() – Handle the allocation of necessary resources and prepare the library for opening.
2. KYParametersHandler_CreateParameterCollection() – This function initializes and returns a handle to a new instance of the library. The handle can then be used for subsequent operations provided by the library. The function ensures that all necessary resources are allocated, and that the library is ready for use.
3. KYParametersHandler_RegisterParameterCollectionTransport() – Register and configure a get and set function collection of parameters within a system.
4. KYParametersHandler_RegisterParameterCollectionCallback() – (Optional) The function called after setting new parameter.
5. KYParametersHandler_InitializeParameterCollection() – The function is generally used to set up or initialize a collection of parameters within a system or application. This function is typically employed to prepare a set of parameters for use, ensuring that they are properly configured and ready for subsequent operations.
6. KYParametersHandler_GetValue() – Retrieve the value associated with a specific key, parameter, or identifier.
7. KYParametersHandler_SetValue() – Assign or update the value associated with a specific key, parameter, or identifier.
8. KYParametersHandler_UnregisterParameterCollectionCallback() – Unregister the function called after setting new parameter.

9. KYParametersHandler_UnregisterParameterCollectionTransport() – Unregister a get or set function collection of parameters within a system.
10. KYParametersHandler_DeleteParameterCollection() – Delete a handle to a new instance of the library.

## 8.2 KYVPParametersHandler API functions

The following reference documents the parameter-collection functions in KYVPParametersHandler.h and the node metadata, types, and callback definitions in KYVPNodeDescriptor.h.

### 8.2.1 KYVPParametersHandler.h API reference

Initialize the library, create a collection, register its transport, and initialize the collection before reading or writing parameter nodes. Delete each collection when finished. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to inspect returned status values. Public declarations, version defaults, and packed layouts form part of the SDK ABI.

#### 8.2.1.1 Macros

##### 8.2.1.1.1 KYVPPARAMLIB_EXTERNAL_DLL

Parameters Handler DLL import/export attribute on Microsoft compilers.

##### 8.2.1.1.2 KYVPPARAMLIB_API

```
KYVP_EXTERNAL_C KYVPPARAMLIB_EXTERNAL_DLL
```

Public Parameters Handler function linkage and visibility.

##### 8.2.1.1.3 KYVPPARAMLIB_VAR

```
KYVP_EXTERNAL KYVPPARAMLIB_EXTERNAL_DLL
```

Public Parameters Handler variable linkage and visibility.

##### 8.2.1.1.4 KYVP_MAX_ERROR_STRING_BUFFER_SIZE

```
1024
```

Convenience capacity in bytes for an error-string buffer.

##### 8.2.1.1.5 COLLECTION_INVALID_HANDLE_INIT_VALUE

```
((void*)-1)
```

Internal invalid-handle initializer; applications should use the public sentinel.

##### 8.2.1.1.6 COLLECTION_HANDLE_IS_INVALID

```
KYParametersHandler_CollectionHandleIsInvalid(_hHandle)
```

Test a collection handle against the invalid sentinel.

##### 8.2.1.1.7 COLLECTION_HANDLE_IS_NULL

```
KYParametersHandler_CollectionHandleIsNull(_hHandle)
```

Test a collection handle against the null sentinel.

##### 8.2.1.1.8 COLLECTION_HANDLES_IS_EQUAL

```
    KYParametersHandler_CollectionHandlesIsEqual(_hHandleFirst, _hHandleSecond)
```

Compare two collection handle identities.

##### 8.2.1.1.9 KYPARAMETERSHANDLER_TRANSPORTCALLPARAMS_STRUCT_VERSION

```
1
```

Version 1 layout identifier for the corresponding Parameters Handler arguments.

##### 8.2.1.1.10 KYPARAMETERSHANDLER_INITPARAMETERS_STRUCT_VERSION

```
1
```

Version 1 layout identifier for the corresponding Parameters Handler arguments.

##### 8.2.1.1.11 KYPARAMETERSHANDLER_INITIALIZE_STRUCT_VERSION

```
1
```

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.12 KYPARAMETERSHANDLER_CREATEPARAMETERCOLLECTION_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.13 KYPARAMETERSHANDLER_DELETEPARAMETERCOLLECTION_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.14 KYPARAMETERSHANDLER_REGISTERPARAMETERCOLLECTIONTRANSPORT_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.15 KYPARAMETERSHANDLER_UNREGISTERPARAMETERCOLLECTIONTRANSPORT_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.16 KYPARAMETERSHANDLER_REGISTERPARAMETERCOLLECTIONCALLBACK_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.17 KYPARAMETERSHANDLER_UNREGISTERPARAMETERCOLLECTIONCALLBACK_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.18 KYPARAMETERSHANDLER_INITIALIZEPARAMETERCOLLECTION_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.19 KYPARAMETERSHANDLER_GETNODEDESCRIPTOR_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.20 KYPARAMETERSHANDLER_GETNODEDESCRIPTORSARRAY_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.21 KYPARAMETERSHANDLER_GETVALUE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.22 KYPARAMETERSHANDLER_SETVALUE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.23 KYPARAMETERSHANDLER_GETVALUEENUMASSTRING_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.24 KYPARAMETERSHANDLER_SETVALUEENUMBYVALUENAME_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.25 KYPARAMETERSHANDLER_INVALIDATEPARAMETERCOLLECTION_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.26 KYPARAMETERSHANDLER_INVALIDATEPARAMETER_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.27 KYPARAMETERSHANDLER_SAVETOFILE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.28 KYVP_PARAMETER_NAME_REMAP_MAX_CHARS

128

Capacity of each remap-name array, including the NUL terminator.

#### 8.2.1.1.29 KYPARAMETERSHANDLER_SETPARAMETERNAME_REMAPTABLE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.30 KYPARAMETERSHANDLER_CLEARPARAMETERNAME_REMAPTABLE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.31 KYPARAMETERSHANDLER_GETPARAMETERNAME_REMAPTABLESIZE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.32 KYPARAMETERSHANDLER_ADDPARAMETERNAME_REMAPENTRY_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.33 KYPARAMETERSHANDLER_REMOVEPARAMETERNAME_REMAPENTRY_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

#### 8.2.1.1.34 KYPARAMETERSHANDLER_GETPARAMETERNAME_REMAPENTRYBYINDEX_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

### 8.2.1.2 Enumerations

#### 8.2.1.2.1 _KYParametersHandler_TransportOperationType

Operations requested from the transport callback.

##### *8.2.1.2.1.1 Values*

| Name | Value | Description |
|---|---|---|
| KYParametersHandler_TransportOperationType_Read | 0 | Read iSize bytes at iAddress into the supplied pData buffer. |
| KYParametersHandler_TransportOperationType_Write | 1 | Write iSize bytes from the supplied pData buffer to iAddress. |
| KYParametersHandler_TransportOperationType_GetResourceData | 2 | Supply the GenICam XML resource through pData, iSize, and bResourceDataZipped. |

#### 8.2.1.2.2 _KYParametersHandler_TransportOperationResult

Result supplied by the transport callback.

##### *8.2.1.2.2.1 Values*

| Name | Value | Description |
|---|---|---|
| KYParametersHandler_TransportOperationResult_Success | 0 | The requested transport operation completed successfully. |
| KYParametersHandler_TransportOperationResult_Failed | 1 | The requested transport operation failed. |
| KYParametersHandler_TransportOperationResult_Timeout | 2 | The requested transport operation timed out. |

### 8.2.1.3 Type definitions

#### 8.2.1.3.1 KYParametersHandler_TransportOperationType

```
typedef enum _KYParametersHandler_TransportOperationType KYParametersHandler_TransportOperationType
```

Operations requested from the transport callback.

#### 8.2.1.3.2 KYParametersHandler_TransportOperationResult

```
typedef enum _KYParametersHandler_TransportOperationResult KYParametersHandler_TransportOperationResult
```

Result supplied by the transport callback.

#### 8.2.1.3.3 TRANSPORT_FUNC

```
typedef void(* TRANSPORT_FUNC) (KYParametersHandler_TransportCallParams_Args
*_pArgs))(KYParametersHandler_TransportCallParams_Args *_pArgs)
```

Implements synchronous register access and XML-resource retrieval for a collection.

Read eOperation and fill the outputs documented in KYParametersHandler_TransportCallParams_Args. Set eResult before returning. Do not let C++ exceptions cross the callback boundary.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_TransportCallParams_Args* | IN/OUT. Transport request and callback outputs. |

### 8.2.1.4 Variables

#### 8.2.1.4.1 KYVP_COLLECTION_HANDLE_NULL

```
const KYVP_COLLECTION_HANDLE KYVP_COLLECTION_HANDLE_NULL
```

Null collection-handle sentinel.

#### 8.2.1.4.2 KYVP_COLLECTION_HANDLE_INVALID

```
const KYVP_COLLECTION_HANDLE KYVP_COLLECTION_HANDLE_INVALID
```

Invalid collection-handle sentinel.

### 8.2.1.5 Functions

#### 8.2.1.5.1 KYParametersHandler_Initialize

```
KY_RESULT KYParametersHandler_Initialize(KYParametersHandler_Initialize_Args *_pArgs)
```

Initialize the Parameters Handler library.

Call before creating a parameter collection.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_Initialize_Args* | IN/OUT. Required pointer to KYParametersHandler_Initialize_Args; initialize its version and required input fields. |

See: KYParametersHandler_Initialize_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.2 KYParametersHandler_CreateParameterCollection

```
KY_RESULT KYParametersHandler_CreateParameterCollection(KYParametersHandler_CreateParameterCollection_Args
*_pArgs)
```

Create an empty parameter collection.

Register its transport and initialize the collection before accessing nodes.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_CreateParameterCollection_Args* | IN/OUT. Required pointer to KYParametersHandler_CreateParameterCollection_Args; initialize its version and required input fields. |

See: KYParametersHandler_CreateParameterCollection_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.3 KYParametersHandler_RegisterParameterCollectionTransport

```
KY_RESULT
KYParametersHandler_RegisterParameterCollectionTransport(KYParametersHandler_RegisterParameterCollectionTransport
_Args *_pArgs)
```

Register the transport callback and its application context.

Registration replaces the previously registered transport callback.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_RegisterParameterCollectionTransport_Args* | IN/OUT. Required pointer to KYParametersHandler_RegisterParameterCollectionTransport_Args; initialize its version and required input fields. |

See: KYParametersHandler_RegisterParameterCollectionTransport_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.4 KYParametersHandler_RegisterParameterCollectionCallback

```
KY_RESULT
KYParametersHandler_RegisterParameterCollectionCallback(KYParametersHandler_RegisterParameterCollectionCallback_A
rgs *_pArgs)
```

Register a callback for parameter-node updates.

Registration replaces the previously registered node callback.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_RegisterParameterCollectionCallback_Args* | IN/OUT. Required pointer to KYParametersHandler_RegisterParameterCollectionCallback_Args; initialize its version and required input fields. |

See: KYParametersHandler_RegisterParameterCollectionCallback_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.5 KYParametersHandler_InitializeParameterCollection

```
KY_RESULT
KYParametersHandler_InitializeParameterCollection(KYParametersHandler_InitializeParameterCollection_Args *_pArgs)
```

Load the transport-supplied GenICam XML and construct the node collection.

A transport callback must be registered. It supplies the XML resource using GetResourceData.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_InitializeParameterCollection_Args* | IN/OUT. Required pointer to KYParametersHandler_InitializeParameterCollection_Args; initialize its version and required input fields. |

See: KYParametersHandler_InitializeParameterCollection_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.6 KYParametersHandler_DeleteParameterCollection

```
KY_RESULT KYParametersHandler_DeleteParameterCollection(KYParametersHandler_DeleteParameterCollection_Args
*_pArgs)
```

Delete a collection and release its nodes and cached resource.

All descriptors and other library-owned pointers from this collection become invalid.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_DeleteParameterCollection_Args* | IN/OUT. Required pointer to KYParametersHandler_DeleteParameterCollection_Args; initialize its version and required input fields. |

See: KYParametersHandler_DeleteParameterCollection_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.7 KYParametersHandler_UnregisterParameterCollectionTransport

```
KY_RESULT KYParametersHandler_UnregisterParameterCollectionTransport(KYParametersHandler_UnregisterParameterCollectionTransport_Args *_pArgs)
```

Remove the transport callback and its context.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_UnregisterParameterCollectionTransport_Args* | IN/OUT. Required pointer to KYParametersHandler_UnregisterParameterCollectionTransport_Args; initialize its version and required input fields. |

See: KYParametersHandler_UnregisterParameterCollectionTransport_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.8 KYParametersHandler_UnregisterParameterCollectionCallback

```
KY_RESULT KYParametersHandler_UnregisterParameterCollectionCallback(KYParametersHandler_UnregisterParameterCollectionCallback_Args *_pArgs)
```

Remove the parameter-node callback and its context.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_UnregisterParameterCollectionCallback_Args* | IN/OUT. Required pointer to KYParametersHandler_UnregisterParameterCollectionCallback_Args; initialize its version and required input fields. |

See: KYParametersHandler_UnregisterParameterCollectionCallback_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.9 KYParametersHandler_GetNodeDescriptor

```
KY_RESULT KYParametersHandler_GetNodeDescriptor(KYParametersHandler_GetNodeDescriptor_Args *_pArgs)
```

Return the library-owned descriptor for a named node.

Treat the returned descriptor as read-only. Its lifetime is tied to the initialized collection.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_GetNodeDescriptor_Args* | IN/OUT. Required pointer to KYParametersHandler_GetNodeDescriptor_Args; initialize its version and required input fields. |

See: KYParametersHandler_GetNodeDescriptor_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.10 KYParametersHandler_GetNodeDescriptorsArray

```
KY_RESULT KYParametersHandler_GetNodeDescriptorsArray(KYParametersHandler_GetNodeDescriptorsArray_Args *_pArgs)
```

Return the library-owned contiguous descriptor array and its element count.

This returns an array of descriptors, not an array of descriptor pointers. Do not free or modify the array; its lifetime is tied to the initialized collection.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_GetNodeDescriptorsArray_Args* | IN/OUT. Required pointer to KYParametersHandler_GetNodeDescriptorsArray_Args; initialize its version and required input fields. |

See: KYParametersHandler_GetNodeDescriptorsArray_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.11 KYParametersHandler_GetValue

```
KY_RESULT KYParametersHandler_GetValue(KYParametersHandler_GetValue_Args *_pArgs)
```

Read a parameter value or query its required storage size.

Pass a NULL value buffer to query the required byte count for a supported value node. Use storage of the type and size described in Parameter value storage.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_GetValue_Args* | IN/OUT. Required pointer to KYParametersHandler_GetValue_Args; initialize its version and required input fields. |

See: KYParametersHandler_GetValue_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.12 KYParametersHandler_SetValue

```
KY_RESULT KYParametersHandler_SetValue(KYParametersHandler_SetValue_Args *_pArgs)
```

Write a parameter value, or execute a command node.

Supply a non-NULL value pointer and byte-count pointer. See Parameter value storage.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_SetValue_Args* | IN/OUT. Required pointer to KYParametersHandler_SetValue_Args; initialize its version and required input fields. |

See: KYParametersHandler_SetValue_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.13 KYParametersHandler_GetValueEnumAsString

```
KY_RESULT KYParametersHandler_GetValueEnumAsString(KYParametersHandler_GetValueEnumAsString_Args *_pArgs)
```

Read the symbolic name of the current enumeration entry.

Pass a NULL destination to query the byte count including the terminator. An undersized destination reports the required count without copying the string.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_GetValueEnumAsString_Args* | IN/OUT. Required pointer to KYParametersHandler_GetValueEnumAsString_Args; initialize its version and required input fields. |

See: KYParametersHandler_GetValueEnumAsString_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.14 KYParametersHandler_SetValueEnumByValueName

```
KY_RESULT KYParametersHandler_SetValueEnumByValueName(KYParametersHandler_SetValueEnumByValueName_Args *_pArgs)
```

Select an enumeration entry by its symbolic name.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_SetValueEnumByValueName_Args* | IN/OUT. Required pointer to KYParametersHandler_SetValueEnumByValueName_Args; initialize its version and required input fields. |

See: KYParametersHandler_SetValueEnumByValueName_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.15 KYParametersHandler_InvalidateParameterCollection

```
KY_RESULT
KYParametersHandler_InvalidateParameterCollection(KYParametersHandler_InvalidateParameterCollection_Args *_pArgs)
```

Invalidate cached values throughout a collection.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_InvalidateParameterCollection_Args* | IN/OUT. Required pointer to KYParametersHandler_InvalidateParameterCollection_Args; initialize its version and required input fields. |

See: KYParametersHandler_InvalidateParameterCollection_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.16 KYParametersHandler_InvalidateParameter

```
KY_RESULT KYParametersHandler_InvalidateParameter(KYParametersHandler_InvalidateParameter_Args *_pArgs)
```

Invalidate the cached value of a named parameter.

A valid, non-NULL parameter name is required. Use KYParametersHandler_InvalidateParameterCollection() to invalidate all nodes.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_InvalidateParameter_Args* | IN/OUT. Required pointer to KYParametersHandler_InvalidateParameter_Args; initialize its version and required input fields. |

See: KYParametersHandler_InvalidateParameter_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.17 KYParametersHandler_SaveToFile

```
KY_RESULT KYParametersHandler_SaveToFile(KYParametersHandler_SaveToFile_Args *_pArgs)
```

Write the collection's original XML or ZIP resource to a file.

The original resource bytes are saved, preserving their compressed or uncompressed form. This does not serialize current parameter values. An existing destination file is overwritten.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_SaveToFile_Args* | IN/OUT. Required pointer to KYParametersHandler_SaveToFile_Args; initialize its version and required input fields. |

See: KYParametersHandler_SaveToFile_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.18 KYParametersHandler_GetParameterAttributeValue

```
KY_RESULT KYParametersHandler_GetParameterAttributeValue(KYParametersHandler_GetParameterAttributeValueArgs *_pArgs)
```

Read a supported GenApi property as text.

Pass a NULL property buffer first to obtain its required byte count, including the terminator. Allocate at least that many bytes before retrieving the text; the caller is responsible for sufficient capacity.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_GetParameterAttributeValueArgs* | IN/OUT. Required pointer to KYParametersHandler_GetParameterAttributeValueArgs; initialize its version and required input fields. |

See: KYParametersHandler_GetParameterAttributeValue_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.19 KYParametersHandler_IsParameterAvailable

```
KY_RESULT KYParametersHandler_IsParameterAvailable(KYParametersHandler_IsParameterAvailableArgs *_pArgs)
```

Check whether a named node exists in the collection.

This checks collection membership. Inspect the descriptor access flags for current device availability and read/write access.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_IsParameterAvailableArgs* | IN/OUT. Required pointer to KYParametersHandler_IsParameterAvailableArgs; initialize its version and required input fields. |

See: KYParametersHandler_IsParameterAvailable_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.20 KYParametersHandler_SetParameterNameRemapTable

```
KY_RESULT KYParametersHandler_SetParameterNameRemapTable(KYParametersHandler_SetParameterNameRemapTable_Args *_pArgs)
```

Replace the complete parameter-name remapping table.

Entries are copied. A zero count clears the table. Validate every entry before calling: replacement clears the old table before processing the new entries, and an invalid entry can leave a partial table.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_SetParameterNameRemapTable_Args* | IN/OUT. Required pointer to KYParametersHandler_SetParameterNameRemapTable_Args; initialize its version and required input fields. |

See: KYParametersHandler_SetParameterNameRemapTable_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.21 KYParametersHandler_AddParameterNameRemapEntry

```
KY_RESULT KYParametersHandler_AddParameterNameRemapEntry(KYParametersHandler_AddParameterNameRemapEntry_Args *_pArgs)
```

Add a parameter-name mapping or replace the first matching source name.

The entry is copied into the collection.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_AddParameterNameRemapEntry_Args* | IN/OUT. Required pointer to KYParametersHandler_AddParameterNameRemapEntry_Args; initialize its version and required input fields. |

See: KYParametersHandler_AddParameterNameRemapEntry_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.22 KYParametersHandler_RemoveParameterNameRemapEntry

```
KY_RESULT
KYParametersHandler_RemoveParameterNameRemapEntry(KYParametersHandler_RemoveParameterNameRemapEntry_Args *_pArgs)
```

Remove the first mapping for a source parameter name.

A missing source name is not an error. The optional output reports whether an entry was removed.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_RemoveParameterNameRemapEntry_Args* | IN/OUT. Required pointer to KYParametersHandler_RemoveParameterNameRemapEntry_Args; initialize its version and required input fields. |

See: KYParametersHandler_RemoveParameterNameRemapEntry_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.23 KYParametersHandler_GetParameterNameRemapEntryByIndex

```
KY_RESULT
KYParametersHandler_GetParameterNameRemapEntryByIndex(KYParametersHandler_GetParameterNameRemapEntryByIndex_Args
*_pArgs)
```

Copy one parameter-name mapping by its zero-based index.

Query the table size first; the index must be less than the current entry count.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_GetParameterNameRemapEntryByIndex_Args* | IN/OUT. Required pointer to KYParametersHandler_GetParameterNameRemapEntryByIndex_Args; initialize its version and required input fields. |

See: KYParametersHandler_GetParameterNameRemapEntryByIndex_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.24 KYParametersHandler_GetParameterNameRemapTableSize

```
KY_RESULT
KYParametersHandler_GetParameterNameRemapTableSize(KYParametersHandler_GetParameterNameRemapTableSize_Args
*_pArgs)
```

Return the number of parameter-name mappings.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_GetParameterNameRemapTableSize_Args* | IN/OUT. Required pointer to KYParametersHandler_GetParameterNameRemapTableSize_Args; initialize its version and required input fields. |

See: KYParametersHandler_GetParameterNameRemapTableSize_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.25 KYParametersHandler_ClearParameterNameRemapTable

```
KY_RESULT KYParametersHandler_ClearParameterNameRemapTable(KYParametersHandler_ClearParameterNameRemapTable_Args *_pArgs)
```

Remove all parameter-name mappings from a collection.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYParametersHandler_ClearParameterNameRemapTable_Args* | IN/OUT. Required pointer to KYParametersHandler_ClearParameterNameRemapTable_Args; initialize its version and required input fields. |

See: KYParametersHandler_ClearParameterNameRemapTable_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.26 KYParametersHandler_CollectionHandleIsInvalid

```
KY_BOOL KYParametersHandler_CollectionHandleIsInvalid(KYVP_COLLECTION_HANDLE _hCollectionHandle)
```

Test whether a handle equals the invalid sentinel.

These comparisons do not validate whether a collection is still alive.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection handle to compare. |

Returns: KY_TRUE when the comparison matches; KY_FALSE otherwise.

#### 8.2.1.5.27 KYParametersHandler_CollectionHandleIsNull

```
KY_BOOL KYParametersHandler_CollectionHandleIsNull(KYVP_COLLECTION_HANDLE _hCollectionHandle)
```

Test whether a handle equals the null sentinel.

These comparisons do not validate whether a collection is still alive.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection handle to compare. |

Returns: KY_TRUE when the comparison matches; KY_FALSE otherwise.

#### 8.2.1.5.28 KYParametersHandler_CollectionHandlesIsEqual

```
KY_BOOL KYParametersHandler_CollectionHandlesIsEqual(KYVP_COLLECTION_HANDLE _hCollectionHandleFirst, KYVP_COLLECTION_HANDLE _hCollectionHandleSecond)
```

Test whether two collection handles have the same native identity.

These comparisons do not validate whether a collection is still alive.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandleFirst | KYVP_COLLECTION_HANDLE | IN. Collection handle to compare. |
| _hCollectionHandleSecond | KYVP_COLLECTION_HANDLE | IN. Collection handle to compare. |

Returns: KY_TRUE when the comparison matches; KY_FALSE otherwise.

#### 8.2.1.5.29 KYParametersHandler_Initialize_V1

```
KY_RESULT KYParametersHandler_Initialize_V1(KYParametersHandler_InitParameters *_pKYParametersHandler_InitParameters, char *_pszErrorString, size_t *_piErrorStringSize)
```

Initialize the Parameters Handler library.

Call before creating a parameter collection.

| Parameter | Type | Description |
|---|---|---|
| _pKYParametersHandler_InitParameters | KYParametersHandler_InitParameters* | IN. Required initialization options; initialize the nested version. |

| Parameter | Type | Description |
|---|---|---|
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_Initialize()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.30 KYParametersHandler_CreateParameterCollection_V1

```
KY_RESULT KYParametersHandler_CreateParameterCollection_V1(KYVP_COLLECTION_HANDLE *_phCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)
```

Create an empty parameter collection.

Register its transport and initialize the collection before accessing nodes.

| Parameter | Type | Description |
|---|---|---|
| _phCollectionHandle | KYVP_COLLECTION_HANDLE* | OUT. Receives the new collection handle. Required. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_CreateParameterCollection()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.31 KYParametersHandler_DeleteParameterCollection_V1

```
KY_RESULT KYParametersHandler_DeleteParameterCollection_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)
```

Delete a collection and release its nodes and cached resource.

All descriptors and other library-owned pointers from this collection become invalid.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_DeleteParameterCollection()

Note: The handle is passed by value; discard the caller's copy after successful deletion.

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.32 KYParametersHandler_RegisterParameterCollectionTransport_V1

```
KY_RESULT KYParametersHandler_RegisterParameterCollectionTransport_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, TRANSPORT_FUNC _pTRANSPORT_FUNC, void *_pContext, char *_pszErrorString, size_t *_piErrorStringSize)
```

Register the transport callback and its application context.

Registration replaces the previously registered transport callback.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pTRANSPORT_FUNC | TRANSPORT_FUNC | IN. Callback implementing register and XML-resource transport. Required. |
| _pContext | void* | IN. Application context passed to the registered callback. May be NULL. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_RegisterParameterCollectionTransport()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.33 KYParametersHandler_UnregisterParameterCollectionTransport_V1

```
KY_RESULT KYParametersHandler_UnregisterParameterCollectionTransport_V1(KYVP_COLLECTION_HANDLE
_hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)
```

Remove the transport callback and its context.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_UnregisterParameterCollectionTransport()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.34 KYParametersHandler_RegisterParameterCollectionCallback_V1

```
KY_RESULT KYParametersHandler_RegisterParameterCollectionCallback_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle,
PARAMETER_CALLBACK_FUNC _pPARAMETER_CALLBACK_FUNC, void *_pContext, char *_pszErrorString, size_t
*_piErrorStringSize)
```

Register a callback for parameter-node updates.

Registration replaces the previously registered node callback.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pPARAMETER_CALLBACK_FUNC | PARAMETER_CALLBACK_FUNC | IN. Callback receiving node updates. Required. |
| _pContext | void* | IN. Application context passed to the registered callback. May be NULL. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_RegisterParameterCollectionCallback()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.35 KYParametersHandler_UnregisterParameterCollectionCallback_V1

```
KY_RESULT KYParametersHandler_UnregisterParameterCollectionCallback_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)
```

Remove the parameter-node callback and its context.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_UnregisterParameterCollectionCallback()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.36 KYParametersHandler_InitializeParameterCollection_V1

```
KY_RESULT KYParametersHandler_InitializeParameterCollection_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)
```

Load the transport-supplied GenICam XML and construct the node collection.

A transport callback must be registered. It supplies the XML resource using GetResourceData.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_InitializeParameterCollection()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.37 KYParametersHandler_GetNodeDescriptor_V1

```
KY_RESULT KYParametersHandler_GetNodeDescriptor_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, KYVP_NodeDescriptor **_ppNodeDescriptor, char *_pszErrorString, size_t *_piErrorStringSize)
```

Return the library-owned descriptor for a named node.

Treat the returned descriptor as read-only. Its lifetime is tied to the initialized collection.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _ppNodeDescriptor | KYVP_NodeDescriptor | OUT. Receives a pointer to the library-owned node descriptor. Required; treat the descriptor as read-only. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |

| Parameter | Type | Description |
|---|---|---|
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_GetNodeDescriptor()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.38 KYParametersHandler_GetNodeDescriptorsArray_V1

```
KY_RESULT KYParametersHandler_GetNodeDescriptorsArray_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const KYVP_NodeDescriptor **_ppArray, size_t *_piSize, char *_pszErrorString, size_t *_piErrorStringSize)
```

Return the library-owned contiguous descriptor array and its element count.

This returns an array of descriptors, not an array of descriptor pointers. Do not free or modify the array; its lifetime is tied to the initialized collection.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _ppArray | const KYVP_NodeDescriptor | OUT. Receives the base address of the library-owned descriptor array. Required; elements are descriptors, not pointers. |
| _piSize | size_t* | OUT. Required pointer receiving the number of descriptors. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_GetNodeDescriptorsArray()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.39 KYParametersHandler_GetValue_V1

```
KY_RESULT KYParametersHandler_GetValue_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, void *_pParamValue, size_t *_piParamValueSize, char *_pszErrorString, size_t *_piErrorStringSize)
```

Read a parameter value or query its required storage size.

Pass a NULL value buffer to query the required byte count for a supported value node. Use storage of the type and size described in Parameter value storage.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _pParamValue | void* | OUT. Value buffer; NULL queries required size when reading. See Parameter value storage. |
| _piParamValueSize | size_t* | IN/OUT. Required pointer to value-buffer size in bytes; receives size on a NULL-buffer query. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |

| Parameter | Type | Description |
|---|---|---|
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_GetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.40 KYParametersHandler_SetValue_V1

```
KY_RESULT KYParametersHandler_SetValue_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, const void *_pParamValue, size_t *_piParamValueSize, char *_pszErrorString, size_t *_piErrorStringSize)
```

Write a parameter value, or execute a command node.

Supply a non-NULL value pointer and byte-count pointer. See Parameter value storage.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _pParamValue | const void* | IN. Required value buffer. See Parameter value storage. |
| _piParamValueSize | size_t* | IN. Required pointer to input value size in bytes; the count is not updated. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_SetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.41 KYParametersHandler_GetValueEnumAsString_V1

```
KY_RESULT KYParametersHandler_GetValueEnumAsString_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszEnumerationName, char *_pszParamValueString, size_t *_piParamValueStringSize, char *_pszErrorString, size_t *_piErrorStringSize)
```

Read the symbolic name of the current enumeration entry.

Pass a NULL destination to query the byte count including the terminator. An undersized destination reports the required count without copying the string.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszEnumerationName | const char* | IN. Name of the enumeration to read. Non-NULL, NUL-terminated string; remapping applies. |
| _pszParamValueString | char* | OUT. Buffer receiving the current enumeration entry symbolic name. NULL queries the required size. |

| Parameter | Type | Description |
|---|---|---|
| _piParamValueStringSize | size_t* | IN/OUT. Pointer to destination capacity in bytes, including the terminator. Required; updated on a size query or insufficient capacity. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_GetValueEnumAsString()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.42 KYParametersHandler_SetValueEnumByValueName_V1

```
KY_RESULT KYParametersHandler_SetValueEnumByValueName_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, const char *_pszParamValue, char *_pszErrorString, size_t *_piErrorStringSize)
```

Select an enumeration entry by its symbolic name.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _pszParamValue | const char* | IN. Symbolic name of the enumeration entry to select. Non-NULL, NUL-terminated string. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_SetValueEnumByValueName()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.43 KYParametersHandler_SetValueEnum_V1

```
KY_RESULT KYParametersHandler_SetValueEnum_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, int64_t _Value, char *_pszErrorString, size_t *_piErrorStringSize)
```

Set an enumeration to its numeric entry value.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _Value | int64_t | IN. Value to write. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_SetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.44 KYParametersHandler_SetValueInt_V1

```
KY_RESULT KYParametersHandler_SetValueInt_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char
*_pszParamName, int64_t _Value, char *_pszErrorString, size_t *_piErrorStringSize)
```

Write a signed 64-bit integer parameter value.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _Value | int64_t | IN. Value to write. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_SetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.45 KYParametersHandler_SetValueBool_V1

```
KY_RESULT KYParametersHandler_SetValueBool_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char
*_pszParamName, KY_BOOL _Value, char *_pszErrorString, size_t *_piErrorStringSize)
```

Write a Boolean parameter value.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _Value | KY_BOOL | IN. Value to write. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_SetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.46 KYParametersHandler_InvalidateParameterCollection_V1

```
KY_RESULT KYParametersHandler_InvalidateParameterCollection_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char
*_pszErrorString, size_t *_piErrorStringSize)
```

Invalidate cached values throughout a collection.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |

| Parameter | Type | Description |
|---|---|---|
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_InvalidateParameterCollection()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.47 KYParametersHandler_InvalidateParameter_V1

```
KY_RESULT KYParametersHandler_InvalidateParameter_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, char *_pszErrorString, size_t *_piErrorStringSize)
```

Invalidate the cached value of a named parameter.

A valid, non-NULL parameter name is required. Use KYParametersHandler_InvalidateParameterCollection() to invalidate all nodes.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_InvalidateParameter()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.48 KYParametersHandler_SaveToFile_V1

```
KY_RESULT KYParametersHandler_SaveToFile_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszFilePath, char *_pszErrorString, size_t *_piErrorStringSize)
```

Write the collection's original XML or ZIP resource to a file.

The original resource bytes are saved, preserving their compressed or uncompressed form. This does not serialize current parameter values. An existing destination file is overwritten.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszFilePath | const char* | IN. Destination path for the original XML or ZIP resource. Non-NULL, NUL-terminated string. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_SaveToFile()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.49 KYParametersHandler_GetParameterAttributeValue_V1

```
KY_RESULT KYParametersHandler_GetParameterAttributeValue_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, KYVP_ParameterAttributeType _eAttributeType, char *_pszPropertyValue, size_t *_piPropertyValueSize, char *_pszErrorString, size_t *_piErrorStringSize)
```

Read a supported GenApi property as text.

Pass a NULL property buffer first to obtain its required byte count, including the terminator. Allocate at least that many bytes before retrieving the text; the caller is responsible for sufficient capacity.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _eAttributeType | KYVP_ParameterAttributeType | IN. GenApi property to query. Supports Invalidator and Value. |
| _pszPropertyValue | char* | OUT. Buffer receiving the property text. NULL queries the required byte count. |
| _piPropertyValueSize | size_t* | OUT. Receives the property text size, including its terminator. Required; query before allocating the text buffer. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_GetParameterAttributeValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.50 KYParametersHandler_IsParameterAvailable_V1

```
KY_RESULT KYParametersHandler_IsParameterAvailable_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *pszParamName, KY_BOOL *_pbIsAvailable, char *_pszErrorString, size_t *_piErrorStringSize)
```

Check whether a named node exists in the collection.

This checks collection membership. Inspect the descriptor access flags for current device availability and read/write access.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| pszParamName | const char* | IN. Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies. |
| _pbIsAvailable | KY_BOOL* | OUT. Required pointer receiving whether the node exists in the collection. |
| _pszErrorString | char* | OUT. Optional diagnostic text buffer. See Optional diagnostic buffers. |
| _piErrorStringSize | size_t* | IN. Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied. |

See: KYParametersHandler_IsParameterAvailable()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.51 KYParametersHandler_SetParameterNameRemapTable_V1

```
KY_RESULT KYParametersHandler_SetParameterNameRemapTable_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const KYVP_ParameterNameRemapEntry *_pEntries, size_t _iNumEntries)
```

Replace the complete parameter-name remapping table.

Entries are copied. A zero count clears the table. Validate every entry before calling: replacement clears the old table before processing the new entries, and an invalid entry can leave a partial table.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pEntries | const KYVP_ParameterNameRemapEntry* | IN. Array of mappings to copy into the collection. Required when iNumEntries is nonzero. |
| _iNumEntries | size_t | IN. Number of mappings in pEntries . Zero clears the table. |

See: KYParametersHandler_SetParameterNameRemapTable()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.52 KYParametersHandler_AddParameterNameRemapEntry_V1

```
KY_RESULT KYParametersHandler_AddParameterNameRemapEntry_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const KYVP_ParameterNameRemapEntry *_pEntry, KY_BOOL *_pbReplaced)
```

Add a parameter-name mapping or replace the first matching source name.

The entry is copied into the collection.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pEntry | const KYVP_ParameterNameRemapEntry* | IN. Required pointer to the mapping to copy. |
| _pbReplaced | KY_BOOL* | OUT. Receives KY_TRUE when an existing source mapping was replaced. Optional. |

See: KYParametersHandler_AddParameterNameRemapEntry()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.53 KYParametersHandler_RemoveParameterNameRemapEntry_V1

```
KY_RESULT KYParametersHandler_RemoveParameterNameRemapEntry_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszFromName, KY_BOOL *_pbRemoved)
```

Remove the first mapping for a source parameter name.

A missing source name is not an error. The optional output reports whether an entry was removed.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _pszFromName | const char* | IN. Source name whose mapping should be removed. Non-NULL, NUL-terminated string; case-sensitive. |
| _pbRemoved | KY_BOOL* | OUT. Receives KY_TRUE when a mapping was removed. Optional. |

See: KYParametersHandler_RemoveParameterNameRemapEntry()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.54 KYParametersHandler_GetParameterNameRemapEntryByIndex_V1

```
KY_RESULT KYParametersHandler_GetParameterNameRemapEntryByIndex_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, size_t _iIndex, KYVP_ParameterNameRemapEntry *_pEntry)
```

Copy one parameter-name mapping by its zero-based index.

Query the table size first; the index must be less than the current entry count.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _iIndex | size_t | IN. Zero-based index of the mapping to retrieve. Must be smaller than the table size. |
| _pEntry | KYVP_ParameterNameRemapEntry* | OUT. Required pointer receiving a copy of the mapping. |

See: KYParametersHandler_GetParameterNameRemapEntryByIndex()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.55 KYParametersHandler_GetParameterNameRemapTableSize_V1

```
KY_RESULT KYParametersHandler_GetParameterNameRemapTableSize_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, size_t *_piSize)
```

Return the number of parameter-name mappings.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |
| _piSize | size_t* | OUT. Required pointer receiving the number of remap entries. |

See: KYParametersHandler_GetParameterNameRemapTableSize()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

#### 8.2.1.5.56 KYParametersHandler_ClearParameterNameRemapTable_V1

```
KY_RESULT KYParametersHandler_ClearParameterNameRemapTable_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle)
```

Remove all parameter-name mappings from a collection.

| Parameter | Type | Description |
|---|---|---|
| _hCollectionHandle | KYVP_COLLECTION_HANDLE | IN. Collection to operate on. Valid collection handle required. |

See: KYParametersHandler_ClearParameterNameRemapTable()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

### 8.2.1.6 Parameters Handler Library Structs

Field reference for collection handles, transport callbacks, and function arguments.

Initialize each version field to the constant shown above its table. C++ defaults provide these values; C callers must initialize them explicitly. Version zero is not the declared argument layout. Tables list fields in declaration order and distinguish required inputs, outputs, and optional pointers. Descriptor pointers and arrays returned by the library remain owned by their collection. See Optional diagnostic buffers for optional diagnostic buffers and Parameter value storage for value storage.

#### 8.2.1.6.1 KYVP_COLLECTION_HANDLE

Opaque collection handle. Use the create/delete functions and public comparison helpers. The C++ default constructor selects the invalid sentinel. In C, use the exported KYVP_COLLECTION_HANDLE_INVALID or KYVP_COLLECTION_HANDLE_NULL constants. The native pointer is an implementation detail; do not dereference or synthesize it.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pNativePointer | void* | Opaque native collection identity. | Managed by the SDK; protected in C++. |

#### 8.2.1.6.2 KYParametersHandler_TransportCallParams_Args

KYParametersHandler_TransportCallParams_Args

Arguments passed to TRANSPORT_FUNC. Complete the requested operation before returning. For Read, fill the supplied buffer. For Write, consume the supplied bytes without modifying them. For GetResourceData, return a pointer to the XML or ZIP resource and its size. The resource must remain valid until collection initialization returns; the library copies the bytes.

The library sets version to KYPARAMETERSHANDLER_TRANSPORTCALLPARAMS_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Transport argument layout version. | IN. Supplied by the library. |
| eOperation | KYParametersHandler_TransportOperationType | Requested transport operation. | IN. |
| pData | void* | Transfer buffer or returned XML/ZIP resource address. | Read: destination; Write: source; GetResourceData: OUT pointer. |
| pContext | void* | Application context registered with the transport callback. | IN. |
| iAddress | uint64_t | Starting device register address. | IN. Used for Read and Write. |
| iSize | size_t | Number of transfer or resource bytes. | Read/Write: IN; GetResourceData: OUT. |
| bResourceDataZipped | KY_BOOL | Whether the returned XML resource is ZIP-compressed. | OUT for GetResourceData; ignored for Read/Write. |
| eResult | KYParametersHandler_TransportOperationResult | Result of the requested transport operation. | OUT. Set before returning; defaults to Failed. |

#### 8.2.1.6.3 KYParametersHandler_InitParameters

Library initialization options. The current layout contains only its version field.

Initialize version to KYPARAMETERSHANDLER_INITPARAMETERS_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |

#### 8.2.1.6.4 KYParametersHandler_Initialize_Args

Arguments for KYParametersHandler_Initialize(). Call before creating a parameter collection.

Initialize version to KYPARAMETERSHANDLER_INITIALIZE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| pInitParameters | KYParametersHandler_InitParameters* | Library initialization parameters. | IN. Required; initialize the nested version. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.5 KYParametersHandler_CreateParameterCollection_Args

KYParametersHandler_CreateParameterCollection_Args

Arguments for KYParametersHandler_CreateParameterCollection(). Register its transport and initialize the collection before accessing nodes.

Initialize version to KYPARAMETERSHANDLER_CREATEPARAMETERCOLLECTION_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| phCollectionHandle | KYVP_COLLECTION_HANDLE* | Receives the new collection handle. | OUT. Required. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.6 KYParametersHandler_DeleteParameterCollection_Args

KYParametersHandler_DeleteParameterCollection_Args

Arguments for KYParametersHandler_DeleteParameterCollection(). All descriptors and other library-owned pointers from this collection become invalid.

Initialize version to KYPARAMETERSHANDLER_DELETEPARAMETERCOLLECTION_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to delete. | IN/OUT. Set to the invalid sentinel on success. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.7 KYParametersHandler_RegisterParameterCollectionTransport_Args

KYParametersHandler_RegisterParameterCollectionTransport_Args

Arguments for KYParametersHandler_RegisterParameterCollectionTransport(). Registration replaces the previously registered transport callback.

Initialize version to KYPARAMETERSHANDLER_REGISTERPARAMETERCOLLECTIONTRANSPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pTRANSPORT_FUNC | TRANSPORT_FUNC | Callback implementing register and XML-resource transport. | IN. Required. |
| pContext | void* | Application context passed to the registered callback. | IN. May be NULL. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.8 KYParametersHandler_UnregisterParameterCollectionTransport_Args

KYParametersHandler_UnregisterParameterCollectionTransport_Args

Arguments for KYParametersHandler_UnregisterParameterCollectionTransport().

Initialize version to KYPARAMETERSHANDLER_UNREGISTERPARAMETERCOLLECTIONTRANSPORT_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.9 KYParametersHandler_RegisterParameterCollectionCallback_Args

KYParametersHandler_RegisterParameterCollectionCallback_Args

Arguments for KYParametersHandler_RegisterParameterCollectionCallback(). Registration replaces the previously registered node callback.

Initialize version to KYPARAMETERSHANDLER_REGISTERPARAMETERCOLLECTIONCALLBACK_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pPARAMETER_CALLBACK_FUNC | PARAMETER_CALLBACK_FUNC | Callback receiving node updates. | IN. Required. |
| pContext | void* | Application context passed to the registered callback. | IN. May be NULL. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.10 KYParametersHandler_UnregisterParameterCollectionCallback_Args

KYParametersHandler_UnregisterParameterCollectionCallback_Args

Arguments for KYParametersHandler_UnregisterParameterCollectionCallback().

Initialize version to KYPARAMETERSHANDLER_UNREGISTERPARAMETERCOLLECTIONCALLBACK_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.11 KYParametersHandler_InitializeParameterCollection_Args

KYParametersHandler_InitializeParameterCollection_Args

Arguments for KYParametersHandler_InitializeParameterCollection(). A transport callback must be registered. It supplies the XML resource using GetResourceData.

Initialize version to KYPARAMETERSHANDLER_INITIALIZEPARAMETERCOLLECTION_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.12 KYParametersHandler_GetNodeDescriptor_Args

Arguments for KYParametersHandler_GetNodeDescriptor(). Treat the returned descriptor as read-only. Its lifetime is tied to the initialized collection.

Initialize version to KYPARAMETERSHANDLER_GETNODEDESCRIPTOR_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszParamName | const char* | Name of the parameter to access. | IN. Non-NULL, NUL-terminated string; remapping applies. |
| ppNodeDescriptor | KYVP_NodeDescriptor** | Receives a pointer to the library-owned node descriptor. | OUT. Required; treat the descriptor as read-only. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.13 KYParametersHandler_GetNodeDescriptorsArray_Args

KYParametersHandler_GetNodeDescriptorsArray_Args

Arguments for KYParametersHandler_GetNodeDescriptorsArray(). This returns an array of descriptors, not an array of descriptor pointers. Do not free or modify the array; its lifetime is tied to the initialized collection.

Initialize version to KYPARAMETERSHANDLER_GETNODEDESCRIPTORSARRAY_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| ppArray | const KYVP_NodeDescriptor** | Receives the base address of the library-owned descriptor array. | OUT. Required; elements are descriptors, not pointers. |
| piSize | size_t* | Receives the number of descriptors. | OUT. Required; count is in elements, not bytes. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.14 KYParametersHandler_GetValue_Args

Arguments for KYParametersHandler_GetValue(). Pass a NULL value buffer to query the required byte count for a supported value node. Use storage of the type and size described in Parameter value storage.

Initialize version to KYPARAMETERSHANDLER_GETVALUE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszParamName | const char* | Name of the parameter to access. | IN. Non-NULL, NUL-terminated string; remapping applies. |
| pParamValue | void* | Destination for the parameter value. | OUT. NULL queries size; see Parameter value storage. |
| piParamValueSize | size_t* | Pointer to value-buffer size in bytes. | IN/OUT. Required; receives the required size on a NULL-buffer query. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.15 KYParametersHandler_SetValue_Args

Arguments for KYParametersHandler_SetValue(). Supply a non-NULL value pointer and byte-count pointer. See Parameter value storage.

Initialize version to KYPARAMETERSHANDLER_SETVALUE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszParamName | const char* | Name of the parameter to access. | IN. Non-NULL, NUL-terminated string; remapping applies. |
| pParamValue | const void* | Value to write, interpreted by node interface type. | IN. Required; see Parameter value storage. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| piParamValueSize | size_t* | Pointer to the input value size in bytes. | IN. Required; the pointed count is not updated. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.16 KYParametersHandler_GetValueEnumAsString_Args

KYParametersHandler_GetValueEnumAsString_Args

Arguments for KYParametersHandler_GetValueEnumAsString(). Pass a NULL destination to query the byte count including the terminator. An undersized destination reports the required count without copying the string.

Initialize version to KYPARAMETERSHANDLER_GETVALUEENUMASSTRING_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszEnumerationName | const char* | Name of the enumeration to read. | IN. Non-NULL, NUL-terminated string; remapping applies. |
| pszParamValueString | char* | Buffer receiving the current enumeration entry symbolic name. | OUT. NULL queries the required size. |
| piParamValueStringSize | size_t* | Pointer to destination capacity in bytes, including the terminator. | IN/OUT. Required; updated on a size query or insufficient capacity. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.17 KYParametersHandler_SetValueEnumByValueName_Args

KYParametersHandler_SetValueEnumByValueName_Args

Arguments for KYParametersHandler_SetValueEnumByValueName().

Initialize version to KYPARAMETERSHANDLER_SETVALUE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszParamName | const char* | Name of the parameter to access. | IN. Non-NULL, NUL-terminated string; remapping applies. |
| pszParamValue | const char* | Symbolic name of the enumeration entry to select. | IN. Non-NULL, NUL-terminated string. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.18 KYParametersHandler_InvalidateParameterCollection_Args

KYParametersHandler_InvalidateParameterCollection_Args

Arguments for KYParametersHandler_InvalidateParameterCollection().

Initialize version to KYPARAMETERSHANDLER_INVALIDATEPARAMETERCOLLECTION_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.19 KYParametersHandler_InvalidateParameter_Args

KYParametersHandler_InvalidateParameter_Args

Arguments for KYParametersHandler_InvalidateParameter(). A valid, non-NULL parameter name is required. Use KYParametersHandler_InvalidateParameterCollection() to invalidate all nodes.

Initialize version to KYPARAMETERSHANDLER_INVALIDATEPARAMETER_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszParamName | const char* | Name of the parameter to access. | IN. Non-NULL, NUL-terminated string; remapping applies. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.20 KYParametersHandler_SaveToFile_Args

Arguments for KYParametersHandler_SaveToFile(). The original resource bytes are saved, preserving their compressed or uncompressed form. This does not serialize current parameter values. An existing destination file is overwritten.

Initialize version to KYPARAMETERSHANDLER_SAVETOFILE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszFilePath | const char* | Destination path for the original XML or ZIP resource. | IN. Non-NULL, NUL-terminated string. |
| piFilePathSize | size_t* | Reserved path-size pointer. | Unused by this implementation; leave NULL. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.21 KYParametersHandler_GetParameterAttributeValueArgs

KYParametersHandler_GetParameterAttributeValueArgs

Arguments for KYParametersHandler_GetParameterAttributeValue(). Pass a NULL property buffer first to obtain its required byte count, including the terminator. Allocate at least that many bytes before retrieving the text; the caller is responsible for sufficient capacity.

Initialize version to KYPARAMETERSHANDLER_SAVETOFILE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszParamName | const char* | Name of the parameter to access. | IN. Non-NULL, NUL-terminated string; remapping applies. |
| eAttributeType | KYVP_ParameterAttributeType | GenApi property to query. | IN. Supports Invalidator and Value. |
| pszPropertyValue | char* | Buffer receiving the property text. | OUT. NULL queries the required byte count. |
| piPropertyValueSize | size_t* | Receives the property text size, including its terminator. | OUT. Required; query before allocating the text buffer. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.22 KYParametersHandler_IsParameterAvailableArgs

KYParametersHandler_IsParameterAvailableArgs

Arguments for KYParametersHandler_IsParameterAvailable(). This checks collection membership. Inspect the descriptor access flags for current device availability and read/write access.

Initialize version to KYPARAMETERSHANDLER_SAVETOFILE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszParamName | const char* | Name of the parameter to access. | IN. Non-NULL, NUL-terminated string; remapping applies. |
| bIsAvailable | KY_BOOL | Whether the named node exists in the collection. | OUT. Does not report the node access mode. |
| pszErrorString | char* | Optional diagnostic text buffer. | OUT. See Optional diagnostic buffers. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| piErrorStringSize | size_t* | Pointer to diagnostic buffer capacity in bytes. | IN. Optional; required when pszErrorString is supplied. |

#### 8.2.1.6.23 KYVP_ParameterNameRemapEntry

One application-name to XML-node-name mapping. Both names must be nonempty and NUL-terminated within their arrays. The capacity is KYVP_PARAMETER_NAME_REMAP_MAX_CHARS bytes per name, allowing at most 127 characters plus the terminator. Names are matched case-sensitively.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pszFrom | char [KYVP_PARAMETER_NAME_REMAP_MAX_CHARS] | Application-facing source name. | IN. Nonempty, NUL-terminated string. |
| pszTo | char [KYVP_PARAMETER_NAME_REMAP_MAX_CHARS] | Target node name in the collection. | IN. Nonempty, NUL-terminated string. |

#### 8.2.1.6.24 KYParametersHandler_SetParameterNameRemapTable_Args

KYParametersHandler_SetParameterNameRemapTable_Args

Arguments for KYParametersHandler_SetParameterNameRemapTable(). Entries are copied. A zero count clears the table. Validate every entry before calling: replacement clears the old table before processing the new entries, and an invalid entry can leave a partial table.

Initialize version to KYPARAMETERSHANDLER_SETPARAMETERNAME_REMAPTABLE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pEntries | const KYVP_ParameterNameRemapEntry* | Array of mappings to copy into the collection. | IN. Required when iNumEntries is nonzero. |
| iNumEntries | size_t | Number of mappings in pEntries . | IN. Zero clears the table. |

#### 8.2.1.6.25 KYParametersHandler_ClearParameterNameRemapTable_Args

KYParametersHandler_ClearParameterNameRemapTable_Args

Arguments for KYParametersHandler_ClearParameterNameRemapTable().

Initialize version to KYPARAMETERSHANDLER_CLEARPARAMETERNAME_REMAPTABLE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |

#### 8.2.1.6.26 KYParametersHandler_GetParameterNameRemapTableSize_Args

KYParametersHandler_GetParameterNameRemapTableSize_Args

Arguments for KYParametersHandler_GetParameterNameRemapTableSize().

Initialize version to KYPARAMETERSHANDLER_GETPARAMETERNAME_REMAPTABLESIZE_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| piSize | size_t* | Receives the number of name-remap entries. | OUT. Required; count is in elements, not bytes. |

#### 8.2.1.6.27 KYParametersHandler_AddParameterNameRemapEntry_Args

KYParametersHandler_AddParameterNameRemapEntry_Args

Arguments for KYParametersHandler_AddParameterNameRemapEntry(). The entry is copied into the collection.

Initialize version to KYPARAMETERSHANDLER_ADDPARAMETERNAME_REMAPENTRY_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| entry | KYVP_ParameterNameRemapEntry | Mapping to copy into the collection. | IN. Both names must be valid. |
| pbReplaced | KY_BOOL* | Receives KY_TRUE when an existing source mapping was replaced. | OUT. Optional. |

#### 8.2.1.6.28 KYParametersHandler_RemoveParameterNameRemapEntry_Args

KYParametersHandler_RemoveParameterNameRemapEntry_Args

Arguments for KYParametersHandler_RemoveParameterNameRemapEntry(). A missing source name is not an error. The optional output reports whether an entry was removed.

Initialize version to KYPARAMETERSHANDLER_REMOVEPARAMETERNAME_REMAPENTRY_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| pszFromName | const char* | Source name whose mapping should be removed. | IN. Non-NULL, NUL-terminated string; case-sensitive. |
| pbRemoved | KY_BOOL* | Receives KY_TRUE when a mapping was removed. | OUT. Optional. |

#### 8.2.1.6.29 KYParametersHandler_GetParameterNameRemapEntryByIndex_Args

KYParametersHandler_GetParameterNameRemapEntryByIndex_Args

Arguments for KYParametersHandler_GetParameterNameRemapEntryByIndex(). Query the table size first; the index must be less than the current entry count.

Initialize version to KYPARAMETERSHANDLER_GETPARAMETERNAME_REMAPENTRYBYINDEX_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure version. | IN. Required. |
| hCollectionHandle | KYVP_COLLECTION_HANDLE | Collection to operate on. | IN. Valid collection handle required. |
| iIndex | size_t | Zero-based index of the mapping to retrieve. | IN. Must be smaller than the table size. |
| entry | KYVP_ParameterNameRemapEntry | Mapping copied from the requested table index. | OUT. |

### 8.2.2 KYVPNodeDescriptor.h API reference

Public enum values, structure packing, member order, and C++ special members form part of the SDK ABI.

### 8.2.2.1 Enumerations

#### 8.2.2.1.1 _KYVP_NodeDescriptorType

Descriptor notification classifications.

##### *8.2.2.1.1.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_NodeDescriptorType_Invalid | 0 | No valid descriptor notification classification. |
| KYVP_NodeDescriptorType_NewNode | 1 | Descriptor classification for a newly introduced parameter node. |
| KYVP_NodeDescriptorType_NewEnumEntry | 2 | Descriptor classification for a newly introduced enumeration entry. |
| KYVP_NodeDescriptorType_UpdateNode | 3 | Descriptor classification for an update to an existing node. |

#### 8.2.2.1.2 _KYVP_ParameterInterfaceType

GenApi principal interface types exposed by a parameter.

##### *8.2.2.1.2.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_ParameterInterfaceType_IValue | 0 | Generic value interface with textual conversion. |
| KYVP_ParameterInterfaceType_IBase | 1 | Base interface exposing access mode. |
| KYVP_ParameterInterfaceType_IInteger | 2 | Integer value interface. |
| KYVP_ParameterInterfaceType_IBoolean | 3 | Boolean value interface. |
| KYVP_ParameterInterfaceType_ICommand | 4 | Command execution and completion interface. |
| KYVP_ParameterInterfaceType_IFloat | 5 | Floating-point value interface. |
| KYVP_ParameterInterfaceType_IString | 6 | String value interface. |
| KYVP_ParameterInterfaceType_IRegister | 7 | Raw register data interface. |
| KYVP_ParameterInterfaceType_ICategory | 8 | Category containing related feature nodes. |
| KYVP_ParameterInterfaceType_IEnumeration | 9 | Enumeration with named entries. |
| KYVP_ParameterInterfaceType_IEnumEntry | 10 | One named entry of an enumeration. |
| KYVP_ParameterInterfaceType_IPort | 11 | Addressed device-port access interface. |
| KYVP_ParameterInterfaceType_IUnknown | 12 | Unknown or unrecognized principal interface. |

#### 8.2.2.1.3 _KYVP_ParameterRepresentation

Hints for displaying numeric parameter values.

##### *8.2.2.1.3.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_ParameterRepresentation_Linear | 0 | Display with a linear scale. |
| KYVP_ParameterRepresentation_Logarithmic | 1 | Display with a logarithmic scale. |
| KYVP_ParameterRepresentation_Boolean | 2 | Display as a Boolean control. |
| KYVP_ParameterRepresentation_PureNumber | 3 | Display as a decimal number. |
| KYVP_ParameterRepresentation_HexNumber | 4 | Display as a hexadecimal number. |
| KYVP_ParameterRepresentation_IPV4Address | 5 | Display in IPv4 address notation. |
| KYVP_ParameterRepresentation_MACAddress | 6 | Display in MAC address notation. |
| KYVP_ParameterRepresentation_UndefinedRepresentation | 7 | No defined display representation. |

#### 8.2.2.1.4 _KYVP_ParameterVisibility

User-interface visibility levels; these do not grant access rights.

##### *8.2.2.1.4.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_ParameterVisibility_Beginner | 0 | Visible at the beginner level and above. |
| KYVP_ParameterVisibility_Expert | 1 | Visible at the expert level and above. |

| Name | Value | Description |
|---|---|---|
| KYVP_ParameterVisibility_Guru | 2 | Visible at the guru level. |
| KYVP_ParameterVisibility_Invisible | 3 | Hidden from the normal feature view. |
| KYVP_ParameterVisibility_Undefined | 99 | Undefined visibility; SDK value 99 (decimal). |
| KYVP_ParameterVisibility_Developer | 200 | KAYA developer visibility extension; SDK value 200 (decimal). |

#### 8.2.2.1.5 _KYVP_ParameterAttributeType

Parameter properties available through the attribute query API.

##### *8.2.2.1.5.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_ParameterAttributeType_Invalidator | 0 | The pInvalidator property naming a node whose changes invalidate this node cache. |
| KYVP_ParameterAttributeType_Value | 1 | The pValue property naming the node that supplies this node's value. |
| KYVP_ParameterAttributeType_Invalid | 0xFF | Invalid or unsupported attribute selector. |

### 8.2.2.2 Type definitions

#### 8.2.2.2.1 KYVP_NodeDescriptorType

```
typedef enum _KYVP_NodeDescriptorType KYVP_NodeDescriptorType
```

Descriptor notification classifications.

#### 8.2.2.2.2 KYVP_ParameterInterfaceType

```
typedef enum _KYVP_ParameterInterfaceType KYVP_ParameterInterfaceType
```

GenApi principal interface types exposed by a parameter.

#### 8.2.2.2.3 KYVP_ParameterRepresentation

```
typedef enum _KYVP_ParameterRepresentation KYVP_ParameterRepresentation
```

Hints for displaying numeric parameter values.

#### 8.2.2.2.4 KYVP_ParameterVisibility

```
typedef enum _KYVP_ParameterVisibility KYVP_ParameterVisibility
```

User-interface visibility levels; these do not grant access rights.

#### 8.2.2.2.5 KYVP_ParameterAttributeType

```
typedef enum _KYVP_ParameterAttributeType KYVP_ParameterAttributeType
```

Parameter properties available through the attribute query API.

#### 8.2.2.2.6 PARAMETER_CALLBACK_FUNC

```
typedef void(* PARAMETER_CALLBACK_FUNC) (const KYVP_NodeDescriptor *_pNodeDescriptor, void *_pContext))(const KYVP_NodeDescriptor *_pNodeDescriptor, void *_pContext)
```

Receives a notification with the current parameter descriptor.

The callback runs synchronously in the node-notification path. Keep it short and do not let C++ exceptions cross the callback boundary.

| Parameter | Type | Description |
|---|---|---|
| _pNodeDescriptor | const KYVP_NodeDescriptor* | IN. Library-owned descriptor; inspect or copy the required data during the callback. |
| _pContext | void* | IN. Application context supplied when the callback was registered. |

### 8.2.2.3 KYVP Node Descriptor

#### 8.2.2.3.1 KYVP_NodeDescriptor

Library-owned metadata and cached values for a parameter node. Interpret value and range fields using m_eInterfaceType ; unrelated fields can retain defaults. The descriptor and its referenced strings belong to the

collection. Treat returned data as read-only. Values and string addresses can change after node updates. Copy any information you need to retain and do not use these pointers after the collection is deleted or reinitialized.

The C++ constructor stores the supplied string pointers and implementation flag; it does not copy strings. C++ supports move construction. Copy construction and both assignment operators are deleted.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_nVersion | int | Descriptor layout version. | SDK default: 0; populated by the library. |
| m_szParamName | const char* | Node name used by parameter APIs. | Library-owned string. |
| m_szParamDisplayName | const char* | Human-readable feature name. | Falls back to the node name when no display name is supplied. |
| m_szToolTip | const char* | Short feature help text. | Library-owned string. |
| m_szDescription | const char* | Detailed feature description. | Library-owned string. |
| m_szPhysicalUnitName | const char* | Physical unit label for a numeric feature. | May be empty. |
| m_eDescriptorType | KYVP_NodeDescriptorType | Descriptor notification classification. | May retain the Invalid default. |
| m_eInterfaceType | KYVP_ParameterInterfaceType | Principal interface determining how to interpret value fields. | |
| m_eRepresentation | KYVP_ParameterRepresentation | Suggested numeric display format. | Presentation hint. |
| m_eVisibility | KYVP_ParameterVisibility | Suggested feature visibility level. | Independent of read/write access. |
| m_nLogicalIndex | int64_t | Logical index reserved for enumeration-entry presentation. | SDK default: 0. |
| m_pParentNode | const struct _KYVP_NodeDescriptor* | Parent category or enumeration descriptor. | Library-owned; NULL when no parent is assigned. |
| m_bIsSelector | KY_BOOL | Whether this feature selects other features. | |
| m_szSelectorName | const char* | Name of a feature selecting this node. | May be empty; represents one selector. |
| m_nCurSelectorValue | int64_t | Cached selector value slot. | May retain its default when not populated. |
| m_bIsImplemented | const KY_BOOL | Implementation flag fixed when the descriptor is constructed. | Read-only member. |
| m_bIsAvailable | KY_BOOL | Cached availability of the node. | May change with device state. |
| m_bIsWritable | KY_BOOL | Cached write-access flag. | |
| m_bIsReadable | KY_BOOL | Cached read-access flag. | |
| m_nMinIntValue | int64_t | Integer minimum or enumeration entry index. | For an enumeration, the selected entry index; for an entry, its index. |
| m_nMaxIntValue | int64_t | Integer maximum or enumeration entry count. | Interpret according to m_eInterfaceType . |

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| m_nIncIntValue | int64_t | Integer value increment. | Applies to integer nodes. |
| m_nCurIntValue | int64_t | Current integer value or numeric enumeration value. | For an enum entry, its assigned value. |
| m_fMinFloatValue | double | Minimum floating-point value. | Applies to float nodes. |
| m_fMaxFloatValue | double | Maximum floating-point value. | Applies to float nodes. |
| m_fIncFloatValue | double | Floating-point increment when supplied by the node. | A default value does not establish a supported increment. |
| m_nFloatDisplayPrecision | int64_t | Suggested floating-point display precision. | Formatting hint supplied by GenApi. |
| m_fCurFloatValue | double | Current floating-point value. | Applies to float nodes. |
| m_bCurBoolValue | KY_BOOL | Current Boolean value. | Applies to Boolean nodes. |
| m_szCurStringValue | const char* | Current string, enum-entry symbolic name, or register data address. | Register data is binary; use the value API to obtain its length. |
| m_bTracing | KY_BOOL | Tracing flag reserved for diagnostics. | SDK default: KY_FALSE. |

# 9 KYVPImageProcessing library

The KYVPImageProcessing is a library designed to perform some image processing tasks within application. This library offers robust functionality for converting video formats, saving various types of data, including raw data, etc.

## 9.1 KYVPImageProcessing API functions

The following reference documents the image conversion, pixel-format lookup, image-saving, and recording functions in KYVPImageProcessing.h. The accompanying KYIMGPROC_PixelFormatDecoder_Defines.h reference describes pixel-format encodings and decoder metadata.

### 9.1.1 KYVPImageProcessing.h API reference

See Introduction for operation lifecycles and Initializing image-processing records for C/C++ initialization requirements. Structure versions, declarations, and packed layouts are part of the SDK ABI.

#### 9.1.1.1 Macros

##### 9.1.1.1.1 KYVPIMAGEPROCESSING_EXTERNAL_DLL

Image-processing DLL import/export annotation on MSVC; empty elsewhere.

##### 9.1.1.1.2 KYIMGPROC_API

```
KYVP_EXTERNAL_C KYVPIMAGEPROCESSING_EXTERNAL_DLL
```

Linkage and DLL visibility for image-processing functions.

##### 9.1.1.1.3 KYIMGPROC_VAR

```
KYVP_EXTERNAL KYVPIMAGEPROCESSING_EXTERNAL_DLL
```

Linkage and DLL visibility for exported image-processing variables.

##### 9.1.1.1.4 DECL_PIXELIF

```
KYVP_VIDEO_PIXELIF _name;
```

Declare one CoaXPress pixel-field record.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

##### 9.1.1.1.5 INIT_PIXEL_IF

```
    _name.data_width = KYVP_DATA_WIDTH_UNKNOWN;     \
   _name.data_subtype = KYVP_DATA_SUBTYPE_INVALID; \
   _name.data_type = KYVP_DATA_TYPE_INVALID;
```

Set the pixel-field record to unknown width and invalid type/subtype.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

##### 9.1.1.1.6 DECL_VP_PIXEL_FORMAT

```
    KYVP_PIXEL_FORMAT _name;        \
   DECL_PIXELIF(_name##pVPPixelIF)
```

Declare a metadata record and its helper pixel-field record.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.7 INIT_VP_PIXEL_FORMAT

```
INIT_PIXEL_IF(_name##pVPPixelIF)                                \
_name.version = KYIMGPROC_KYVP_PIXEL_FORMAT_STRUCT_VERSION; \
_name.pVPPixelIF = _name##pVPPixelIF;
```

Initialize metadata version and copy the helper pixel fields.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.8 DECL_PIXEL_DECODER

```
DECL_VP_PIXEL_FORMAT(_name##_pVP_PIXEL_FORMAT) \
KYIMGPROC_PIXEL_DECODER _name;
```

Declare a decoder and the metadata storage it will reference.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.9 INIT_PIXEL_DECODER

```
INIT_VP_PIXEL_FORMAT(_name##_pVP_PIXEL_FORMAT)          \
_name.version = KYIMGPROC_PIXEL_DECODER_STRUCT_VERSION; \
_name.pVP_PIXEL_FORMAT = &_name##_pVP_PIXEL_FORMAT;
```

Initialize decoder version and attach the declared metadata storage.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.10 DECL_VPIMAGE

```
DECL_PIXEL_DECODER(_name##_pixelFormat) \
VPImage _name;
```

Declare an image descriptor and its decoder/metadata storage.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.11 INIT_VPIMAGE

```
INIT_PIXEL_DECODER(_name##_pixelFormat)           \
_name.uVersion = KYIMGPROC_VPIMAGE_STRUCT_VERSION; \
_name.pixelFormat = &_name##_pixelFormat;
```

Initialize image version and attach its declared decoder storage.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.12 DECL_CONVERT_PARAMS

```
DECL_VPIMAGE(_name##_srcImage) \
DECL_VPIMAGE(_name##_dstImage) \
KYIMGPROC_ConvertParam_Args _name;
```

Declare conversion arguments and both nested image descriptors.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.13 INIT_CONVERT_PARAMS

```
    _name.srcImage = &_name##_srcImage;                    \
   _name.dstImage = &_name##_dstImage;                     \
   _name.uVersion = KYIMGPROC_CONVERTPARAM_STRUCT_VERSION; \
   INIT_VPIMAGE(_name##_srcImage)                          \
   INIT_VPIMAGE(_name##_dstImage)
```

Initialize conversion versions and connect source/destination descriptors.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.14 CONVERT_PARAMS_DECL_AND_INIT

```
DECL_CONVERT_PARAMS(params) INIT_CONVERT_PARAMS(params)
```

Declare and connect storage for a conversion argument block.

| Parameter | Description |
|---|---|
| params | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.15 DECL_SAVE_IMAGE_PARAMS

```
    DECL_VPIMAGE(_name##_srcImage)    \
   KYIMGPROC_SaveImageArgs _name;
```

Declare image-saving arguments and the source image storage.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.16 INIT_SAVE_IMAGE_PARAMS

```
    _name.srcImage = &_name##_srcImage;                  \
   _name.uVersion = KYIMGPROC_SAVEIMAGE_STRUCT_VERSION; \
   INIT_VPIMAGE(_name##_srcImage)
```

Initialize image-saving versions and connect the source image descriptor.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.17 SAVE_IMAGE_PARAMS_DECL_AND_INIT

```
DECL_SAVE_IMAGE_PARAMS(params) INIT_SAVE_IMAGE_PARAMS(params)
```

Declare and connect storage for image-saving arguments.

| Parameter | Description |
|---|---|
| params | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.18 DECL_SAVE_VIDEO_PARAMS

```
    DECL_PIXEL_DECODER(_name##_pixelFormat) \
   KYIMGPROC_SaveVideoArgs _name;
```

Declare video-recording arguments and source-format storage.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.19 INIT_SAVE_VIDEO_PARAMS

```
    _name.uVersion = KYIMGPROC_SAVEVIDEO_STRUCT_VERSION; \
   INIT_PIXEL_DECODER(_name##_pixelFormat)              \
   _name.srcPixelFormat = &_name##_pixelFormat;
```

Initialize recording versions and connect the source decoder.

| Parameter | Description |
|---|---|
| _name | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.20 SAVE_VIDEO_PARAMS_DECL_AND_INIT

```
DECL_SAVE_VIDEO_PARAMS(params) INIT_SAVE_VIDEO_PARAMS(params)
```

Declare and connect storage for video-recording arguments.

| Parameter | Description |
|---|---|
| params | Identifier used for the argument record and generated helper names. |

See: kyimgproc_initialization

#### 9.1.1.1.21 KYIMGPROC_VIDEO_RECORDER_HANDLE_INVALID_INIT_VALUE

```
((void*)-1)
```

Native-pointer sentinel for a default invalid handle.

#### 9.1.1.1.22 KYIMGPROC_VIDEO_RECORDER_HANDLE_IS_INVALI

```
KYImgProc_VideoRecorderHandleIsInvalid(_hHandle)
```

Forward a handle comparison to KYImgProc_VideoRecorderHandleIsInvalid().

| Parameter | Description |
|---|---|
| _hHandle | Handle value to compare. |

Returns: KY_TRUE if the comparison succeeds; otherwise KY_FALSE. The historical macro spelling is retained for source compatibility.

#### 9.1.1.1.23 KYIMGPROC_VIDEO_RECORDER_HANDLE_IS_NULL

```
KYImgProc_VideoRecorderHandleIsNull(_hHandle)
```

Forward a handle comparison to KYImgProc_VideoRecorderHandleIsNull().

| Parameter | Description |
|---|---|
| _hHandle | Handle value to compare. |

Returns: KY_TRUE if the comparison succeeds; otherwise KY_FALSE.

#### 9.1.1.1.24 KYIMGPROC_VIDEO_RECORDER_HANDLE_IS_EQUAL

```
    KYImgProc_VideoRecorderHandleIsEqual(_hHandleFirst, _hHandleSecond)
```

Forward a handle comparison to KYImgProc_VideoRecorderHandleIsEqual().

| Parameter | Description |
|---|---|
| _hHandleFirst | Handle value to compare. |
| _hHandleSecond | Handle value to compare. |

Returns: KY_TRUE if the comparison succeeds; otherwise KY_FALSE.

#### 9.1.1.1.25 KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID_INIT_VALUE

```
((void*)-1)
```

Native-pointer sentinel for a default invalid handle.

#### 9.1.1.1.26 KYIMGPROC_IMAGE_CONVERTER_HANDLE_IS_INVALID

```
KYImgProc_ImageConverterHandleIsInvalid(_hHandle)
```

Forward a handle comparison to KYImgProc_ImageConverterHandleIsInvalid().

| Parameter | Description |
|---|---|
| _hHandle | Handle value to compare. |

Returns: KY_TRUE if the comparison succeeds; otherwise KY_FALSE.

#### 9.1.1.1.27 KYIMGPROC_IMAGE_CONVERTER_HANDLE_IS_NULL

```
KYImgProc_ImageConverterHandleIsNull(_hHandle)
```

Forward a handle comparison to KYImgProc_ImageConverterHandleIsNull().

| Parameter | Description |
|---|---|
| _hHandle | Handle value to compare. |

Returns: KY_TRUE if the comparison succeeds; otherwise KY_FALSE.

#### 9.1.1.1.28 KYIMGPROC_IMAGE_CONVERTER_HANDLES_IS_EQUAL

```
KYImgProc_ImageConverterHandlesIsEqual(_hHandleFirst, _hHandleSecond)
```

#### 9.1.1.1.29 KYIMGPROC_VPIMAGE_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.30 KYIMGPROC_PACKEDDATAINFO_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.31 KYIMGPROC_INITVIDEORECORDER_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.32 KYIMGPROC_DELETEVIDEORECORDER_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.33 KYIMGPROC_INITIMAGECONVERTER_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.34 KYIMGPROC_DELETEIMAGECONVERTER_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.35 KYIMGPROC_CONVERTPARAM_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.36 KYIMGPROC_SAVEIMAGE_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.37 KYIMGPROC_SAVEIMAGEMULTI_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.38 KYIMGPROC_SAVERAW_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.39 KYIMGPROC_SAVERAWMULTI_STRUCT_VERSION

```
1
```

Declared structure layout version 1.

#### 9.1.1.1.40 KYIMGPROC_SAVEVIDEO_STRUCT_VERSION

1

Declared structure layout version 1.

#### 9.1.1.1.41 KYIMGPROC_WRITEVIDEOFRAME_STRUCT_VERSION

1

Declared structure layout version 1.

#### 9.1.1.1.42 KYIMGPROC_STOPVIDEORECORD_STRUCT_VERSION

1

Declared structure layout version 1.

#### 9.1.1.1.43 KYIMGPROC_GETPACKEDDATAINFO_STRUCT_VERSION

1

Declared structure layout version 1.

#### 9.1.1.1.44 KYIMGPROC_PREPROCCESSPACKEDDATA_STRUCT_VERSION

1

Declared structure layout version 1.

#### 9.1.1.1.45 KYIMGPROC_GETPIXELFORMAT_STRUCT_VERSION

1

Declared structure layout version 1.

#### 9.1.1.1.46 KYIMGPROC_GETPIXELFORMATBYNAME_STRUCT_VERSION

1

Declared structure layout version 1.

### 9.1.1.2 Enumerations

#### 9.1.1.2.1 _BITS_DEPTH_SHIFT_MODE

Alignment requested when saving samples in a 16-bit channel container.

##### *9.1.1.2.1.1 Values*

| Name | Value | Description |
|---|---|---|
| BITS_DEPTH_SHIFT_MODE_UNKNOWN | 0x00 | Unspecified alignment; rejected by the image-saving API. |
| BITS_DEPTH_SHIFT_MODE_LSB | 0x01 | Keep meaningful bits in the least significant positions of each channel. |
| BITS_DEPTH_SHIFT_MODE_MSB | 0x02 | Shift meaningful bits to the most significant positions of each 16-bit channel. |

#### 9.1.1.2.2 _SAVED_IMAGE_FORMAT

Output file formats supported by the image-saving API.

##### *9.1.1.2.2.1 Values*

| Name | Value | Description |
|---|---|---|
| SAVED_IMAGE_FORMAT_UNKNOWN | 0x00 | Unspecified image format; rejected by the image-saving API. |
| SAVED_IMAGE_FORMAT_BMP | 0x01 | Windows bitmap file; output filename uses the .bmp extension. |
| SAVED_IMAGE_FORMAT_PNG | 0x02 | PNG file; output filename uses the .png extension. |
| SAVED_IMAGE_FORMAT_TIF | 0x03 | TIFF file; output filename uses the .tif extension. |

### 9.1.1.3 Type definitions

#### 9.1.1.3.1 BITS_DEPTH_SHIFT_MODE

```
typedef enum _BITS_DEPTH_SHIFT_MODE BITS_DEPTH_SHIFT_MODE
```

Alignment requested when saving samples in a 16-bit channel container.

#### 9.1.1.3.2 SAVED_IMAGE_FORMAT

```
typedef enum _SAVED_IMAGE_FORMAT SAVED_IMAGE_FORMAT
```

Output file formats supported by the image-saving API.

### 9.1.1.4 Variables

#### 9.1.1.4.1 KYIMGPROC_VIDEO_RECORDER_HANDLE_NULL

```
const KYIMGPROC_VIDEO_RECORDER_HANDLE KYIMGPROC_VIDEO_RECORDER_HANDLE_NULL
```

Exported null video-recorder handle value.

#### 9.1.1.4.2 KYIMGPROC_VIDEO_RECORDER_HANDLE_INVALID

```
const KYIMGPROC_VIDEO_RECORDER_HANDLE KYIMGPROC_VIDEO_RECORDER_HANDLE_INVALID
```

Exported invalid video-recorder handle value.

#### 9.1.1.4.3 KYIMGPROC_IMAGE_CONVERTER_HANDLE_NULL

```
const KYIMGPROC_IMAGE_CONVERTER_HANDLE KYIMGPROC_IMAGE_CONVERTER_HANDLE_NULL
```

Exported null image-converter handle value.

#### 9.1.1.4.4 KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID

```
const KYIMGPROC_IMAGE_CONVERTER_HANDLE KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID
```

Exported invalid image-converter handle value.

### 9.1.1.5 Functions

#### 9.1.1.5.1 KYImgProc_InitImageConverter

```
KY_RESULT KYImgProc_InitImageConverter(KYIMGPROC_InitImageConverterArgs *_pArgs)
```

Create and register an image converter.

Delete the returned handle with KYImgProc_DeleteImageConverter() when finished. The current version-1 conversion API does not use a caller-supplied converter handle.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_InitImageConverterArgs* | IN/OUT. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_InitImageConverterArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_DeleteImageConverter

#### 9.1.1.5.2 KYImgProc_InitVideoRecorder

```
KY_RESULT KYImgProc_InitVideoRecorder(KYIMGPROC_InitVideoRecorderArgs *_pArgs)
```

Create and register a video recorder.

Configure it with KYImgProc_StartVideoRecord() before submitting frames.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_InitVideoRecorderArgs* | IN/OUT. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_InitVideoRecorderArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_StartVideoRecord

#### 9.1.1.5.3 KYImgProc_DeleteImageConverter

```
KY_RESULT KYImgProc_DeleteImageConverter(KYIMGPROC_DeleteImageConverterArgs *_pArgs)
```

Delete an image converter and remove its handle registration.

The existing C++ member default is zero. Set uVersion explicitly to KYIMGPROC_DELETEIMAGECONVERTER_STRUCT_VERSION before use.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_DeleteImageConverterArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_DeleteImageConverterArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

#### 9.1.1.5.4 KYImgProc_DeleteVideoRecorder

```
KY_RESULT KYImgProc_DeleteVideoRecorder(KYIMGPROC_DeleteVideoRecorderArgs *_pArgs)
```

Remove a video-recorder handle registration.

For an active recorder, stop it first with KYImgProc_StopVideoRecord(). This call removes its handle registration; it does not stop recording or destroy the recorder object.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_DeleteVideoRecorderArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_DeleteVideoRecorderArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

#### 9.1.1.5.5 KYImgProc_GetPixelFormat

```
KY_RESULT KYImgProc_GetPixelFormat(KYImgProc_GetPixelFormat_Args *_pArgs)
```

Look up a PFNC or CoaXPress numeric pixel-format code.

Initialize the decoder version and point pVP_PIXEL_FORMAT at a writable KYVP_PIXEL_FORMAT before calling. Zero is rejected by numeric lookup, including the all-zero raw code; lookup by the name Raw is available. Lookup returns the first matching entry in the SDK table. A shared CoaXPress code can map to several PFNC storage formats, so use an exact PFNC numeric value when packed/unpacked identity matters.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYImgProc_GetPixelFormat_Args* | IN/OUT. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_GetPixelFormat_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_GetPixelFormatByName

#### 9.1.1.5.6 KYImgProc_GetPixelFormatByName

```
KY_RESULT KYImgProc_GetPixelFormatByName(KYImgProc_GetPixelFormatByName_Args *_pArgs)
```

Look up a PFNC-style or CoaXPress pixel-format name.

Initialize the decoder version and point pVP_PIXEL_FORMAT at a writable KYVP_PIXEL_FORMAT before calling. Names are matched case-insensitively against both SDK PFNC and CoaXPress names; the first matching table entry is returned.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYImgProc_GetPixelFormatByName_Args* | IN/OUT. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_GetPixelFormatByName_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_GetPixelFormat

#### 9.1.1.5.7 KYImgProc_ConvertImage

```
KY_RESULT KYImgProc_ConvertImage(KYIMGPROC_ConvertParam_Args *_convertStruct)
```

Convert or resize an image into application-provided destination storage.

The public KYImgProc_ConvertImage() entry point currently accepts version 1 only. Although the layout includes h_ImgProcHandle for version 2, setting uVersion to 2 is currently rejected. Initialize h_ImgProcHandle to KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID, including when using version 1. Initialize both image descriptors and their format metadata. Supported conversions depend on the configured image-processing backend.

| Parameter | Type | Description |
|---|---|---|
| _convertStruct | KYIMGPROC_ConvertParam_Args* | IN/OUT. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_ConvertParam_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: CONVERT_PARAMS_DECL_AND_INIT

#### 9.1.1.5.8 KYImgProc_GetPackedDataInfo

`KY_RESULT KYImgProc_GetPackedDataInfo(KYImgProc_GetPackedDataInfo_Args *_pArgs)`

Calculate a packed-data processing plan and destination allocation size.

Supply a decoded format and positive image dimensions. The result reports the destination allocation size using SDK row-alignment rules; it does not allocate memory.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYImgProc_GetPackedDataInfo_Args* | IN/OUT. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_GetPackedDataInfo_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_PreproccessPackedData

#### 9.1.1.5.9 KYImgProc_PreproccessPackedData

`KY_RESULT KYImgProc_PreproccessPackedData(KYImgProc_PreproccessPackedData_Args *_pArgs)`

Process packed samples according to a prepared processing plan.

Obtain the plan with KYImgProc_GetPackedDataInfo() first. Allocate sufficient non-overlapping source and destination storage. The caller must ensure both buffer capacities are sufficient; buffer-size errors may not be reported. Packed source rows must follow the SDK four-byte row-alignment rule. Provide at least three additional readable padding bytes after the source image data.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYImgProc_PreproccessPackedData_Args* | IN/OUT. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_PreproccessPackedData_Args. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_GetPackedDataInfo

#### 9.1.1.5.10 KYImgProc_SaveImage

`KY_RESULT KYImgProc_SaveImage(KYIMGPROC_SaveImageArgs *_pArgs)`

Save one image in the selected image-file format.

Select a non-UNKNOWN format and bit alignment. The filename extension is added or replaced to match the selected format. Encoded-image saving requires OpenCV support in the SDK build.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_SaveImageArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveImageArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_SaveImageMulti

#### 9.1.1.5.11 KYImgProc_SaveImageMulti

```
KY_RESULT KYImgProc_SaveImageMulti(KYIMGPROC_SaveImageMultiArgs *_pArgs)
```

Save an array of images as one vertically stacked image.

Images are stacked in array order, top to bottom, into a single image. All entries must have matching dimensions, decoded pixel fields, and RGB channel order. This is not a multipage TIFF operation. Select a non-UNKNOWN format and bit alignment. The filename extension is added or replaced. Encoded-image saving requires OpenCV support in the SDK build.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_SaveImageMultiArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveImageMultiArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

#### 9.1.1.5.12 KYImgProc_SaveRawData

```
KY_RESULT KYImgProc_SaveRawData(KYIMGPROC_SaveRawArgs *_pArgs)
```

Write one image buffer to a raw binary file.

The output filename uses the .raw extension. The buffer is written unchanged, with no image header or pixel conversion. Raw saving does not require OpenCV.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_SaveRawArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveRawArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_SaveRawDataMulti

#### 9.1.1.5.13 KYImgProc_SaveRawDataMulti

```
KY_RESULT KYImgProc_SaveRawDataMulti(KYIMGPROC_SaveRawMultiArgs *_pArgs)
```

Concatenate image buffers into a raw binary file.

The buffers are written unchanged in array order with no headers or separators. Every buffer must have the same nonzero byte count. The output filename uses the .raw extension; raw saving does not require OpenCV.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_SaveRawMultiArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveRawMultiArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

#### 9.1.1.5.14 KYImgProc_StartVideoRecord

```
KY_RESULT KYImgProc_StartVideoRecord(KYIMGPROC_SaveVideoArgs *_pArgs)
```

Start an AVI recording with the configured MPEG-4 encoder.

Initialize every field; several members have no C++ default initializer. The recorder retains a pointer to this structure, so keep it and its referenced filename and pixel metadata valid until recording stops. The current backend writes MPEG-4 video in an AVI container.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_SaveVideoArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveVideoArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_WriteVideoFrame

#### 9.1.1.5.15 KYImgProc_StopVideoRecord

```
KY_RESULT KYImgProc_StopVideoRecord(KYImgProc_StopVideoRecordArgs *_pArgs)
```

Finalize recording and destroy the recorder object.

Stop waits for the frame worker and finalizes the file, then destroys the recorder object. The handle registration remains until KYImgProc_DeleteVideoRecorder() is called. After stop, use the handle only to remove that registration.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYImgProc_StopVideoRecordArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_StopVideoRecordArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_DeleteVideoRecorder

#### 9.1.1.5.16 KYImgProc_WriteVideoFrame

```
KY_RESULT KYImgProc_WriteVideoFrame(KYIMGPROC_WriteVideoFrameArgs *_pArgs)
```

Copy a frame into the recorder queue for asynchronous encoding.

Submit only after successful start. The recorder copies image bytes and pixel-format records before queuing the frame. The caller retains ownership of its source storage. A successful return does not guarantee that the frame was queued, encoded, or written to disk. Queueing and asynchronous encoding failures are not reported by this return value.

| Parameter | Type | Description |
|---|---|---|
| _pArgs | KYIMGPROC_WriteVideoFrameArgs* | IN. Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_WriteVideoFrameArgs. |

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_StopVideoRecord

#### 9.1.1.5.17 KYImgProc_ImageConverterHandleIsInvalid

```
KY_BOOL KYImgProc_ImageConverterHandleIsInvalid(KYIMGPROC_IMAGE_CONVERTER_HANDLE _hHandle)
```

Test whether a handle equals the invalid sentinel.

This is a stored-value comparison, not a check that the underlying object is live or registered.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYIMGPROC_IMAGE_CONVERTER_HANDLE | IN. First handle value. |

Returns: KY_TRUE when the handle equals the invalid sentinel; otherwise KY_FALSE.

#### 9.1.1.5.18 KYImgProc_ImageConverterHandleIsNull

```
KY_BOOL KYImgProc_ImageConverterHandleIsNull(KYIMGPROC_IMAGE_CONVERTER_HANDLE _hHandle)
```

Test whether a handle equals the null sentinel.

This is a stored-value comparison, not a check that the underlying object is live or registered.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYIMGPROC_IMAGE_CONVERTER_HANDLE | IN. First handle value. |

Returns: KY_TRUE when the handle equals the null sentinel; otherwise KY_FALSE.

#### 9.1.1.5.19 KYImgProc_ImageConverterHandlesIsEqual

```
KY_BOOL KYImgProc_ImageConverterHandlesIsEqual(KYIMGPROC_IMAGE_CONVERTER_HANDLE _hFirstHandle,
KYIMGPROC_IMAGE_CONVERTER_HANDLE _hOtherHandle)
```

Compare two handle values for equality.

This is a stored-value comparison, not a check that the underlying object is live or registered.

| Parameter | Type | Description |
|---|---|---|
| _hFirstHandle | KYIMGPROC_IMAGE_CONVERTER_HANDLE | IN. First handle value. |
| _hOtherHandle | KYIMGPROC_IMAGE_CONVERTER_HANDLE | IN. Second handle value. |

Returns: KY_TRUE when the stored handle values are equal; otherwise KY_FALSE.

#### 9.1.1.5.20 KYImgProc_VideoRecorderHandleIsInvalid

```
KY_BOOL KYImgProc_VideoRecorderHandleIsInvalid(KYIMGPROC_VIDEO_RECORDER_HANDLE _hHandle)
```

Test whether a handle equals the invalid sentinel.

This is a stored-value comparison, not a check that the underlying object is live or registered.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYIMGPROC_VIDEO_RECORDER_HANDLE | IN. First handle value. |

Returns: KY_TRUE when the handle equals the invalid sentinel; otherwise KY_FALSE.

#### 9.1.1.5.21 KYImgProc_VideoRecorderHandleIsNull

```
KY_BOOL KYImgProc_VideoRecorderHandleIsNull(KYIMGPROC_VIDEO_RECORDER_HANDLE _hHandle)
```

Test whether a handle equals the null sentinel.

This is a stored-value comparison, not a check that the underlying object is live or registered.

| Parameter | Type | Description |
|---|---|---|
| _hHandle | KYIMGPROC_VIDEO_RECORDER_HANDLE | IN. First handle value. |

Returns: KY_TRUE when the handle equals the null sentinel; otherwise KY_FALSE.

#### 9.1.1.5.22 KYImgProc_VideoRecorderHandleIsEqual

```
KY_BOOL KYImgProc_VideoRecorderHandleIsEqual(KYIMGPROC_VIDEO_RECORDER_HANDLE _hFirstHandle,
KYIMGPROC_VIDEO_RECORDER_HANDLE _hOtherHandle)
```

Compare two handle values for equality.

This is a stored-value comparison, not a check that the underlying object is live or registered.

| Parameter | Type | Description |
|---|---|---|
| _hFirstHandle | KYIMGPROC_VIDEO_RECORDER_HANDLE | IN. First handle value. |
| _hOtherHandle | KYIMGPROC_VIDEO_RECORDER_HANDLE | IN. Second handle value. |

Returns: KY_TRUE when the stored handle values are equal; otherwise KY_FALSE.

### 9.1.1.6 Image Processing Library Structs

Field reference for handles, images, and operation arguments.

Tables list fields in declaration order, including exact pointer types. Set versions explicitly in C. C++ defaults do not initialize every member; follow each table's input and ownership requirements.

Pixel-format records are described in Pixel Format Decoder Structs.

#### 9.1.1.6.1 KYIMGPROC_VIDEO_RECORDER_HANDLE

Opaque handle identifying a video-recorder registration.

Treat the native pointer as opaque. The C++ default constructor selects the invalid sentinel; initialize C handles explicitly with the exported null or invalid value. Handle comparisons test stored values only; they do not establish that a recorder is still alive.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pNativePointer | void* | Native object identity used by the SDK. | Opaque; do not dereference or modify directly. |

#### 9.1.1.6.2 KYIMGPROC_IMAGE_CONVERTER_HANDLE

Opaque handle identifying an image converter.

Treat the native pointer as opaque. The C++ default constructor selects the invalid sentinel; initialize C handles explicitly with the exported null or invalid value. Do not use the C++ call operator in application code. Use the documented handle functions.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pNativePointer | void* | Native object identity used by the SDK. | Opaque; do not dereference or modify directly. |

#### 9.1.1.6.3 VPImage

Application-owned image storage and its decoded pixel format.

Initialize uVersion to KYIMGPROC_VPIMAGE_STRUCT_VERSION.

The API does not allocate this image buffer. Supply valid storage and decoded format metadata for conversion, encoded-image saving, and video recording. Raw saving only reads pBuffer and uBufferSize . This structure has no row-stride field. Supply the memory layout required by the operation; image-file saving expects a tightly packed, unpacked-channel image.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| uWidth | uint32_t | Image width in pixels. | Input; greater than zero for processing. |
| uHeight | uint32_t | Image height in pixels. | Input; greater than zero for processing. |
| pBuffer | void* | Address of the image pixel bytes. | Application-owned; readable for source images, writable for destination images. |
| uBufferSize | size_t | Number of image-buffer bytes available to the operation. | Input; nonzero; encoded-image saving requires the exact expected image byte count. |
| pixelFormat | KYIMGPROC_PIXEL_DECODER* | Decoded pixel-format description for this image. | Input; initialized decoder and metadata required for processing. |

#### 9.1.1.6.4 PackedDataInfo

Packed-data processing plan populated by KYImgProc_GetPackedDataInfo().

Initialize uVersion to KYIMGPROC_PACKEDDATAINFO_STRUCT_VERSION.

Initialize uVersion before the query; KYImgProc_GetPackedDataInfo() fills the other fields without writing the version. The result borrows the source decoder pointer, which must remain valid while this plan is used.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| ePackedModeType | DS_PACKED_DATA_MODE | Packing mode of the source stream data. | Output; copied from the query inputs. |
| eProcessingModeType | DS_PACKED_DATA_PROCESSING_MODE | Requested processing: extract 8-bit samples or expand to 16-bit samples. | Output; copied from the query inputs. |
| pPixelFormat | const KYIMGPROC_PIXEL_DECODER* | Decoded source pixel format used for size and sample calculations. | Output; copied from the query inputs; borrowed pointer. |
| uWidth | size_t | Image width in pixels. | Output; copied from the query inputs. |
| uHeight | size_t | Image height in pixels. | Output; copied from the query inputs. |
| uUnpackedBufferSize | size_t | Destination allocation size reported by the SDK, in bytes. | Output; includes the size calculation's row-alignment allowance. |

#### 9.1.1.6.5 KYIMGPROC_InitVideoRecorderArgs

Arguments for creating a video-recorder handle.

Initialize uVersion to KYIMGPROC_INITVIDEORECORDER_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| phRecordHandle | KYIMGPROC_VIDEO_RECORDER_HANDLE* | Receives the newly created video-recorder handle. | Output; non-NULL. |

#### 9.1.1.6.6 KYIMGPROC_DeleteVideoRecorderArgs

Arguments for removing a video-recorder handle registration.

Initialize uVersion to KYIMGPROC_DELETEVIDEORECORDER_STRUCT_VERSION.

For an active recorder, stop it first with KYImgProc_StopVideoRecord(). This call removes its handle registration; it does not stop recording or destroy the recorder object.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| phRecordHandle | KYIMGPROC_VIDEO_RECORDER_HANDLE | Recorder handle whose registration is to be removed. | Input; handle value, despite the ph prefix; stop active recording first. |

#### 9.1.1.6.7 KYIMGPROC_InitImageConverterArgs

Arguments for creating an image-converter handle.

Initialize uVersion to KYIMGPROC_INITIMAGECONVERTER_STRUCT_VERSION.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| phImageProcHandle | KYIMGPROC_IMAGE_CONVERTER_HANDLE* | Receives the newly created image-converter handle. | Output; non-NULL. |

#### 9.1.1.6.8 KYIMGPROC_DeleteImageConverterArgs

Arguments for deleting an image converter.

Initialize uVersion to KYIMGPROC_DELETEIMAGECONVERTER_STRUCT_VERSION.

The existing C++ member default is zero. Set uVersion explicitly to KYIMGPROC_DELETEIMAGECONVERTER_STRUCT_VERSION before use.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. Existing C++ default: 0. |
| hHandle | KYIMGPROC_IMAGE_CONVERTER_HANDLE | Image-converter handle to delete. | Input; becomes unusable after deletion. |

#### 9.1.1.6.9 KYIMGPROC_ConvertParam_Args

Source and destination images for conversion.

Initialize uVersion to KYIMGPROC_CONVERTPARAM_STRUCT_VERSION.

The public KYImgProc_ConvertImage() entry point currently accepts version 1 only. Although the layout includes h_ImgProcHandle for version 2, setting uVersion to 2 is currently rejected. Initialize h_ImgProcHandle to KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID, including when using version 1. Initialize both image descriptors and their format metadata.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| srcImage | VPImage* | Source image descriptor and readable pixel buffer. | Input; non-NULL; application-owned. |
| dstImage | VPImage* | Destination dimensions, decoded format, and writable pixel buffer. | Input/output; non-NULL; buffer allocated by the caller. |
| h_ImgProcHandle | KYIMGPROC_IMAGE_CONVERTER_HANDLE | Converter handle field reserved for the version-2 conversion path. | Initialize to the invalid sentinel; public conversion currently accepts version 1 only. |

#### 9.1.1.6.10 KYIMGPROC_SaveImageArgs

Arguments for saving one image in a selected file format.

Initialize uVersion to KYIMGPROC_SAVEIMAGE_STRUCT_VERSION.

Select a non-UNKNOWN format and bit alignment. The filename extension is added or replaced to match the selected format. Encoded-image saving requires OpenCV support in the SDK build.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| eImageFormat | SAVED_IMAGE_FORMAT | Output image-file format. | Input; UNKNOWN is not accepted. |
| eBitsDepthShiftMode | BITS_DEPTH_SHIFT_MODE | Bit alignment of saved channel samples. | Input; use LSB or MSB, including for 8-bit images. |
| srcImage | const VPImage* | Source image descriptor and readable pixel buffer. | Input; non-NULL; application-owned. |
| szFileName | const char* | Output file path. | Input; non-NULL, null-terminated string; existing output may be overwritten. |

#### 9.1.1.6.11 KYIMGPROC_SaveImageMultiArgs

Arguments for saving equally sized images as one vertically stacked image.

Initialize uVersion to KYIMGPROC_SAVEIMAGEMULTI_STRUCT_VERSION.

Images are stacked in array order, top to bottom, into a single image. All entries must have matching dimensions, decoded pixel fields, and RGB channel order. This is not a multipage TIFF operation. Select a non-UNKNOWN format and bit alignment. The filename extension is added or replaced. Encoded-image saving requires OpenCV support in the SDK build.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| eImageFormat | SAVED_IMAGE_FORMAT | Output image-file format. | Input; UNKNOWN is not accepted. |
| eBitsDepthShiftMode | BITS_DEPTH_SHIFT_MODE | Bit alignment of saved channel samples. | Input; use LSB or MSB, including for 8-bit images. |
| ppSrcImages | const VPImage** | Array of pointers to source image descriptors. | Input; non-NULL array with no NULL entries. |
| uSrcImageCount | size_t | Number of image pointers in ppSrcImages . | Input; greater than zero. |
| szFileName | const char* | Output file path. | Input; non-NULL, null-terminated string; existing output may be overwritten. |

#### 9.1.1.6.12 KYIMGPROC_SaveRawArgs

Arguments for writing one image buffer as raw bytes.

Initialize uVersion to KYIMGPROC_SAVERAW_STRUCT_VERSION.

The output filename uses the .raw extension. The buffer is written unchanged, with no image header or pixel conversion. Raw saving does not require OpenCV.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| srcImage | const VPImage* | Source image descriptor and readable pixel buffer. | Input; non-NULL; application-owned. |
| szFileName | const char* | Output file path. | Input; non-NULL, null-terminated string; existing output may be overwritten. |

#### 9.1.1.6.13 KYIMGPROC_SaveRawMultiArgs

Arguments for concatenating image buffers into one raw file.

Initialize uVersion to KYIMGPROC_SAVERAWMULTI_STRUCT_VERSION.

The buffers are written unchanged in array order with no headers or separators. Every buffer must have the same nonzero byte count. The output filename uses the .raw extension; raw saving does not require OpenCV.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| ppSrcImages | const VPImage** | Array of pointers to source image descriptors. | Input; non-NULL array with no NULL entries. |
| uSrcImageCount | size_t | Number of image pointers in ppSrcImages . | Input; greater than zero. |
| szFileName | const char* | Output file path. | Input; non-NULL, null-terminated string; existing output may be overwritten. |

#### 9.1.1.6.14 KYIMGPROC_SaveVideoArgs

Video-recorder configuration retained by KYImgProc_StartVideoRecord().

Initialize uVersion to KYIMGPROC_SAVEVIDEO_STRUCT_VERSION.

Initialize every field; several members have no C++ default initializer. The recorder retains a pointer to this structure, so keep it and its referenced filename and pixel metadata valid until recording stops. The current backend writes MPEG-4 video in an AVI container.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| vHandler | KYIMGPROC_VIDEO_RECORDER_HANDLE | Video-recorder handle for the operation. | Input; valid registration; observe the recording lifecycle. |
| szFileName | const char* | Output file path. | Input; non-NULL, null-terminated string; existing output may be overwritten. |
| srcPixelFormat | KYIMGPROC_PIXEL_DECODER* | Decoded pixel format expected for submitted frames. | Input; non-NULL; keep metadata valid while recording. |
| bitRate | uint32_t | Target encoder bit rate in bits per second. | Input; explicitly configure before starting. |
| dFps | double | Recording frame rate in frames per second. | Input; greater than zero. |
| uWidth | uint32_t | Image width in pixels. | Input; greater than zero for processing. |
| uHeight | uint32_t | Image height in pixels. | Input; greater than zero for processing. |

#### 9.1.1.6.15 KYIMGPROC_WriteVideoFrameArgs

Arguments for submitting one frame to the recording queue.

Initialize uVersion to KYIMGPROC_WRITEVIDEOFRAME_STRUCT_VERSION.

Submit only after successful start. The recorder copies image bytes and pixel-format records before queuing the frame. The caller retains ownership of its source storage. A successful return does not guarantee that the frame was queued, encoded, or written to disk. Queueing and asynchronous encoding failures are not reported by this return value.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| vHandler | KYIMGPROC_VIDEO_RECORDER_HANDLE | Video-recorder handle for the operation. | Input; valid registration; observe the recording lifecycle. |
| pSrcImage | VPImage* | Image frame to copy into the recording queue. | Input; non-NULL; dimensions and format must match the recording configuration. |

#### 9.1.1.6.16 KYImgProc_StopVideoRecordArgs

Arguments for finalizing a recording and destroying its recorder object.

Initialize uVersion to KYIMGPROC_STOPVIDEORECORD_STRUCT_VERSION.

Stop waits for the frame worker and finalizes the file, then destroys the recorder object. The handle registration remains until KYImgProc_DeleteVideoRecorder() is called. After stop, use the handle only to remove that registration.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| vHandler | KYIMGPROC_VIDEO_RECORDER_HANDLE | Video-recorder handle for the operation. | Input; valid registration; observe the recording lifecycle. |

#### 9.1.1.6.17 KYImgProc_GetPackedDataInfo_Args

Arguments for calculating packed-data processing information.

Initialize uVersion to KYIMGPROC_GETPACKEDDATAINFO_STRUCT_VERSION.

Supply a decoded format and positive image dimensions. The result reports the destination allocation size using SDK row-alignment rules; it does not allocate memory.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| pPixelFormat | const KYIMGPROC_PIXEL_DECODER* | Decoded source pixel format used for size and sample calculations. | Input; non-NULL decoder with valid metadata. |
| ePackedModeType | DS_PACKED_DATA_MODE | Packing mode of the source stream data. | Input; select the mode matching the actual buffer layout. |
| eProcessingModeType | DS_PACKED_DATA_PROCESSING_MODE | Requested processing: extract 8-bit samples or expand to 16-bit samples. | Input; use MSB8 or EXPAND16. |
| uWidth | size_t | Image width in pixels. | Input; greater than zero for processing. |
| uHeight | size_t | Image height in pixels. | Input; greater than zero for processing. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| pPackedDataInfoStruct | PackedDataInfo* | Processing plan to populate. | Input/output; non-NULL; initialize its version first. |

#### 9.1.1.6.18 KYImgProc_PreproccessPackedData_Args

Source and destination buffers for packed-data processing.

Initialize uVersion to KYIMGPROC_PREPROCCESSPACKEDDATA_STRUCT_VERSION.

Obtain the plan with KYImgProc_GetPackedDataInfo() first. Allocate sufficient non-overlapping source and destination storage. The caller must ensure both buffer capacities are sufficient; buffer-size errors may not be reported. Packed source rows must follow the SDK four-byte row-alignment rule. Provide at least three additional readable padding bytes after the source image data.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| pPackedDataInfoStruct | const PackedDataInfo* | Processing plan previously populated by the query API. | Input; non-NULL; referenced decoder must remain valid. |
| pSourceData | const void* | Readable source buffer containing packed image bytes. | Input; non-NULL; include row alignment and readable end padding. |
| uSourceDataSize | size_t | Source data byte count supplied to the processing operation. | Input; nonzero; must cover the image data. |
| pDestinationData | void* | Writable buffer receiving processed samples. | Output; non-NULL; must not overlap the source buffer. |
| uDestinationDataSize | size_t | Destination capacity in bytes. | Input; allocate at least uUnpackedBufferSize from the processing plan. |

#### 9.1.1.6.19 KYImgProc_GetPixelFormat_Args

Arguments for looking up a numeric pixel-format code.

Initialize uVersion to KYIMGPROC_GETPIXELFORMAT_STRUCT_VERSION.

Initialize the decoder version and point pVP_PIXEL_FORMAT at a writable KYVP_PIXEL_FORMAT before calling. Zero is rejected by numeric lookup, including the all-zero raw code; lookup by the name Raw is available. Lookup returns the first matching entry in the SDK table. A shared CoaXPress code can map to several PFNC storage formats, so use an exact PFNC numeric value when packed/unpacked identity matters.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| uPixelFormatValue | uint32_t | PFNC 32-bit value or CoaXPress 16-bit code carried in a uint32_t. | Input; nonzero; exact numeric PFNC codes distinguish storage variants. |
| pPixelFormatDecoder | KYIMGPROC_PIXEL_DECODER* | Decoder and attached metadata storage to populate. | Input/output; non-NULL; initialize version and pVP_PIXEL_FORMAT first. |

#### 9.1.1.6.20 KYImgProc_GetPixelFormatByName_Args

Arguments for looking up a pixel format by name.

Initialize uVersion to KYIMGPROC_GETPIXELFORMATBYNAME_STRUCT_VERSION.

Initialize the decoder version and point pVP_PIXEL_FORMAT at a writable KYVP_PIXEL_FORMAT before calling. Names are matched case-insensitively against both SDK PFNC and CoaXPress names; the first matching table entry is returned.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| uVersion | uint32_t | Structure layout version. | Input; set to 1. |
| pszPixelFormatName | const char* | PFNC-style or CoaXPress name to look up. | Input; non-NULL, nonempty, null-terminated string; case-insensitive match. |
| pPixelFormatDecoder | KYIMGPROC_PIXEL_DECODER* | Decoder and attached metadata storage to populate. | Input/output; non-NULL; initialize version and pVP_PIXEL_FORMAT first. |

### 9.1.2 KYIMGPROC_PixelFormatDecoder_Defines.h API reference

The CoaXPress fields describe transport-format components; PFNC values describe named pixel formats. The SDK also retains legacy aliases and sentinels. Use the lookup API to populate application-owned decoder records. Standard encodings were checked against JIIA CXP-001-2013 (CoaXPress 1.1), section 9.4.1, tables 24 and 26-34. SDK-specific names are identified separately. See the EMVA pixel-format names and values for PFNC identifiers.

#### 9.1.2.1 Macros

##### 9.1.2.1.1 KYIMGPROC_KYVP_PIXEL_FORMAT_STRUCT_VERSION

1

Declared structure layout version 1.

##### 9.1.2.1.2 KYIMGPROC_PIXEL_DECODER_STRUCT_VERSION

1

Declared structure layout version 1.

#### 9.1.2.2 Enumerations

##### 9.1.2.2.1 _kyvp_video_data_width

Encoded width of one monochrome sample or one color component.

These values are encodings, not literal bit counts. Packing in host memory is described separately by the format and stream packed-data mode.

*9.1.2.2.1.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_DATA_WIDTH_UNKNOWN | 0x00 | Unspecified sample width; SDK sentinel. |
| KYVP_DATA_WIDTH_8BIT | 0x01 | 8-bit samples; encoded width value 1. |
| KYVP_DATA_WIDTH_10BIT | 0x02 | 10-bit samples; encoded width value 2. |
| KYVP_DATA_WIDTH_12BIT | 0x03 | 12-bit samples; encoded width value 3. |
| KYVP_DATA_WIDTH_14BIT | 0x04 | 14-bit samples; encoded width value 4. |
| KYVP_DATA_WIDTH_16BIT | 0x05 | 16-bit samples; encoded width value 5. |

##### 9.1.2.2.2 _kyvp_video_data_subtype

Subtype interpreted together with the pixel data type.

Values intentionally overlap across planar, Bayer, YUV, and YCbCr families. Always inspect data_type before interpreting data_subtype . SDK legacy names and invalid sentinels are not additional CoaXPress wire-format definitions.

*9.1.2.2.2.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_DATA_SUBTYPE_NONE | 0x00 | No subtype; used by formats such as Mono, RGB, and RGBA. |

| Name | Value | Description |
|---|---|---|
| KYVP_DATA_SUBTYPE_PLANAR_RY | 0x01 | Plane 1: normally R for RGB or Y for YUV/YCbCr. |
| KYVP_DATA_SUBTYPE_PLANAR_GUCB | 0x02 | Plane 2: normally G for RGB, U for YUV, or Cb for YCbCr. |
| KYVP_DATA_SUBTYPE_PLANAR_BVCR | 0x03 | Plane 3: normally B for RGB, V for YUV, or Cr for YCbCr. |
| KYVP_DATA_SUBTYPE_PLANAR_RGB | 0x04 | Legacy SDK name for planar RGB; numeric code 0x04 denotes plane 4 in CoaXPress. |
| KYVP_DATA_SUBTYPE_BAYER_GR | 0x01 | Bayer GR pattern: first row G,R; second row B,G. |
| KYVP_DATA_SUBTYPE_BAYER_RG | 0x02 | Bayer RG pattern: first row R,G; second row G,B. |
| KYVP_DATA_SUBTYPE_BAYER_GB | 0x03 | Bayer GB pattern: first row G,B; second row R,G. |
| KYVP_DATA_SUBTYPE_BAYER_BG | 0x04 | Bayer BG pattern: first row B,G; second row G,R. |
| KYVP_DATA_SUBTYPE_YUV_411 | 0x01 | YUV 4:1:1 with CoaXPress component order Y,Y,U,Y,Y,V. |
| KYVP_DATA_SUBTYPE_YUV_422 | 0x02 | YUV 4:2:2 with CoaXPress component order Y,U,Y,V. |
| KYVP_DATA_SUBTYPE_YUV_444 | 0x03 | YUV 4:4:4 with CoaXPress component order Y,U,V. |
| KYVP_DATA_SUBTYPE_YCBCR_411 | 0x01 | YCbCr 4:1:1 with CoaXPress component order Y,Y,Cb,Y,Y,Cr. |
| KYVP_DATA_SUBTYPE_YCBCR_422 | 0x02 | YCbCr 4:2:2 with CoaXPress component order Y,Cb,Y,Cr. |
| KYVP_DATA_SUBTYPE_YCBCR_444 | 0x03 | YCbCr 4:4:4 with CoaXPress component order Y,Cb,Cr. |
| KYVP_DATA_SUBTYPE_INVALID | 0x0F | SDK invalid-subtype sentinel; CoaXPress also uses 0x0F for planar plane 15. |

#### 9.1.2.2.3 _kyvp_video_data_type

Pixel data families represented by the SDK CoaXPress format fields.

##### *9.1.2.2.3.1 Values*

| Name | Value | Description |
|---|---|---|
| KYVP_DATA_TYPE_RAW | 0x00 | Raw or application-specific data; the all-zero CoaXPress format code denotes raw data. |
| KYVP_DATA_TYPE_MONO | 0x01 | Monochrome luminance samples; no subtype. |
| KYVP_DATA_TYPE_PLANAR | 0x02 | Separate component planes; the subtype identifies a plane or an SDK planar convention. |
| KYVP_DATA_TYPE_BAYER | 0x03 | Bayer color-filter samples; the subtype selects the two-row mosaic pattern. |
| KYVP_DATA_TYPE_RGB | 0x04 | Interleaved red, green, and blue components in that order; no subtype. |
| KYVP_DATA_TYPE_RGBA | 0x05 | Interleaved red, green, blue, and alpha components in that order; no subtype. |
| KYVP_DATA_TYPE_YUV | 0x06 | YUV components; the subtype selects 4:1:1, 4:2:2, or 4:4:4 sampling. |
| KYVP_DATA_TYPE_YCBCR601 | 0x07 | YCbCr using ITU-R BT.601 colorimetry; the subtype selects chroma sampling. |
| KYVP_DATA_TYPE_YCBCR709 | 0x08 | YCbCr using ITU-R BT.709 colorimetry; the subtype selects chroma sampling. |
| KYVP_DATA_TYPE_INVALID | 0xFF | Invalid or unsupported SDK data-type sentinel. |

### 9.1.2.3 Type definitions

#### 9.1.2.3.1 KYVP_VIDEO_DATA_WIDTH

```
typedef enum _kyvp_video_data_width KYVP_VIDEO_DATA_WIDTH
```

Encoded width of one monochrome sample or one color component.

These values are encodings, not literal bit counts. Packing in host memory is described separately by the format and stream packed-data mode.

#### 9.1.2.3.2 KYVP_VIDEO_DATA_SUBTYPE

`typedef enum _kyvp_video_data_subtype KYVP_VIDEO_DATA_SUBTYPE`
Subtype interpreted together with the pixel data type.

Values intentionally overlap across planar, Bayer, YUV, and YCbCr families. Always inspect data_type before interpreting data_subtype . SDK legacy names and invalid sentinels are not additional CoaXPress wire-format definitions.

#### 9.1.2.3.3 KYVP_VIDEO_DATA_TYPE

`typedef enum _kyvp_video_data_type KYVP_VIDEO_DATA_TYPE`
Pixel data families represented by the SDK CoaXPress format fields.

### 9.1.2.4 Pixel Format Decoder Structs

#### 9.1.2.4.1 KYVP_VIDEO_PIXELIF

Decoded CoaXPress pixel-format fields.

In the numeric CoaXPress code, width occupies bits 3:0, subtype bits 7:4, and type bits 15:8. The all-zero code denotes raw or user-specific data. This C/C++ bit-field structure is an SDK representation. Its allocation and bit ordering depend on the compiler; do not serialize it by copying its object representation.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| data_width | KYVP_VIDEO_DATA_WIDTH : 4 | Encoded sample or color-component width. | 4-bit field; corresponds to numeric code bits 3:0. |
| data_subtype | KYVP_VIDEO_DATA_SUBTYPE : 4 | Subtype within the selected data family. | 4-bit field; corresponds to numeric code bits 7:4. |
| data_type | KYVP_VIDEO_DATA_TYPE : 8 | Pixel data family. | 8-bit field; corresponds to numeric code bits 15:8. |

#### 9.1.2.4.2 KYVP_PIXEL_FORMAT

Pixel-format metadata populated in application-provided storage.

Initialize version to KYIMGPROC_KYVP_PIXEL_FORMAT_STRUCT_VERSION.

The application owns this structure. Returned name pointers refer to library-owned strings; do not modify or free them. A successful lookup does not guarantee that every conversion or saving backend supports the format. uPFNCValue preserves the SDK lookup-table value, which may be a legacy CoaXPress alias. uCxpValue equal to zero may mean raw data or no corresponding CoaXPress mapping.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure layout version. | Input; set to 1. |
| pszPFNCName | const char* | PFNC-style name of the matched SDK table entry. | Output; library-owned, null-terminated string. |
| uPFNCValue | uint32_t | Numeric value of the matched SDK table entry. | Output; may be a legacy CoaXPress alias rather than a PFNC identifier. |
| pszCxpName | const char* | CoaXPress name or alias associated with the matched entry. | Output; library-owned string; may be empty. |
| uCxpValue | uint32_t | CoaXPress pixel-format code stored in a uint32_t. | Output; low 16 bits encode width, subtype, and type; zero is ambiguous. |

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| bIsPacked | KY_BOOL | Packed-data flag recorded in the SDK format table. | Output; does not establish the actual host-buffer layout or conversion support. |
| bRGBReordered | KY_BOOL | Whether the matched RGB-family entry uses reversed red/blue order. | Output; KY_TRUE identifies BGR-family ordering. |
| uChannelsPerPixel | uint8_t | Channel count recorded for the matched format. | Output; use with format-specific packing and subsampling rules. |
| pVPPixelIF | KYVP_VIDEO_PIXELIF | Decoded CoaXPress width, subtype, and data-family fields. | Output; embedded structure, not a pointer. |

#### 9.1.2.4.3 KYIMGPROC_PIXEL_DECODER

Lookup result and pointer to application-provided pixel-format metadata.

Initialize version to KYIMGPROC_PIXEL_DECODER_STRUCT_VERSION.

Initialize version and attach a writable KYVP_PIXEL_FORMAT through pVP_PIXEL_FORMAT . Pass this decoder through the argument block of KYImgProc_GetPixelFormat() or KYImgProc_GetPixelFormatByName(). On success, the decoder contains the matched table value and a library-owned name. Both records remain application-owned; their returned name strings must not be freed.

| Structure Field | Type | Description | Remarks |
|---|---|---|---|
| version | uint32_t | Structure layout version. | Input; set to 1. |
| uValue | uint32_t | Numeric value of the matched SDK format-table entry. | Output; query input comes from the lookup argument block. |
| pszPixelFormatName | const char* | Pixel-format name selected by lookup. | Output; library-owned string on success. |
| pVP_PIXEL_FORMAT | KYVP_PIXEL_FORMAT* | Application-provided storage for decoded pixel metadata. | Input/output; non-NULL; library fills fields without allocating the structure. |

# 10 KYFoundation library

The KYFoundation library includes tools and utilities for efficient handling of KY_RESULT, promoting consistent status management and efficient error processing across the system.

## 10.1 KYFoundation API functions

The following reference documents the result inspection functions and related declarations in KYFoundation.h. Shared result codes, foundation types, and compiler-warning controls are described in Common include headers.

### 10.1.1 KYFoundation.h API reference

Include this header to use KY_RESULT, common result codes, and the result inspection functions. Test completion with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED(); use the accessors for the numeric code, severity, and diagnostic text. A result code or severity alone is not a success test.

#### 10.1.1.1 Macros

##### 10.1.1.1.1 KY_CALLCONV

Foundation calling convention; __cdecl with MSVC and empty otherwise.

##### 10.1.1.1.2 KYFOUNDATION_EXTERNAL_C

C linkage in C++; empty in C.

##### 10.1.1.1.3 KYFOUNDATION_EXTERNAL_DLL

Foundation DLL import or export attribute on MSVC; empty otherwise.

Application builds use the import declaration. Do not define KY_FOUNDATION_EXPORTS.

##### 10.1.1.1.4 KYFOUNDATION_API

```
KYFOUNDATION_EXTERNAL_C KYFOUNDATION_EXTERNAL_DLL
```

Linkage and symbol visibility for Foundation API declarations.

##### 10.1.1.1.5 KY_PRINTF_PARAM

Annotation for a printf-style format-string parameter.

Place before the parameter type. Enables MSVC format checking; empty otherwise.

##### 10.1.1.1.6 KY_PRINTF_ATTR

```
__attribute__((format(printf, _fmt_index, _args_index)))
```

Annotation for printf-style argument checking on GNU-compatible compilers.

Place after the function parameter list. Empty on MSVC. For non-static C++ member functions, the implicit this parameter counts as argument one.

| Parameter | Description |
|---|---|
| _fmt_index | One-based index of the format-string parameter. |
| _args_index | One-based index of the first argument to check; zero for va_list functions. |

##### 10.1.1.1.7 KY_RESULT_SUCCEEDED

```
(0 == ((result.uResult >> 31) & 0x01))
```

Test whether an SDK result reports success.

| Parameter | Description |
|---|---|
| result | Result returned by an SDK operation. |

Returns: Nonzero for success; zero for failure.

See: KY_RESULT_FAILED()

#### 10.1.1.1.8 KY_RESULT_FAILED

`(!KY_RESULT_SUCCEEDED(result))`

Test whether an SDK result reports failure.

| Parameter | Description |
|---|---|
| result | Result returned by an SDK operation. |

Returns: Nonzero for failure; zero for success.

See: KY_RESULT_SUCCEEDED()

### 10.1.1.2 Functions

#### 10.1.1.2.1 KYFoundation_What

```
const char * KYFoundation_What(KY_RESULT _result)
```

Retrieve diagnostic text for an SDK result.

| Parameter | Type | Description |
|---|---|---|
| _result | KY_RESULT | IN. Result returned by an SDK operation. |

Returns: Library-owned, null-terminated diagnostic string. Returns "Unknown KY_RESULT" when no description is available.

Remark: Do not modify or free the returned string. Retrieve the description on the thread that performed the operation to obtain any available operation-specific detail. Copy the text before further SDK calls or before retaining it beyond the calling thread's lifetime. Diagnostic wording may vary; do not parse it to determine status.

See: KYFoundation_GetErrorCode()

#### 10.1.1.2.2 KYFoundation_GetErrorCode

```
uint32_t KYFoundation_GetErrorCode(KY_RESULT _result)
```

Retrieve the numeric code from an SDK result.

| Parameter | Type | Description |
|---|---|---|
| _result | KY_RESULT | IN. Result returned by an SDK operation. |

Returns: Code component of the result. Compare with KY_RESULT_CODE_* constants or the additional result codes documented by the called API.

Remark: The code alone does not identify the complete result or its success state. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() first.

See: Common result codes

#### 10.1.1.2.3 KYFoundation_GetSeverity

```
uint32_t KYFoundation_GetSeverity(KY_RESULT _result)
```

Retrieve the severity associated with an SDK result.

| Parameter | Type | Description |
|---|---|---|
| _result | KY_RESULT | IN. Result returned by an SDK operation. |

Returns: One of the KY_SEVERITY_* values.

Remark: Severity describes the diagnostic category; it is independent of the success test performed by KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: Diagnostic severities

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# 11 KY_RESULT

All Vision Point II SDK functions return a KY_RESULT, which is a generic status code indicating the outcome of the function execution.
Some error statuses are general, while others point to a specific condition or failure.

To determine whether a function call succeeded or failed, a user might use macros from the KYFoundation library:

- KY_RESULT_SUCCEEDED(result)
- KY_RESULT_FAILED(result)

In certain cases, it may be useful to determine the exact severity of a KY_RESULT. Use the GetSeverity() function to retrieve the corresponding severity level (such as error, warning, or information).

For a descriptive error code call the KYFoundation_GetErrorCode() function.
Generic error codes are described in the public header KYFoundation_Errors.h.

To obtain an extended description of the result, use:
const char* KYFoundation_What(KY_RESULT result);
This returns a string with additional information about the status.

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

# 12 Library Exiting

During its operation, our KYVPLibTL library and KYVPParametersHandler library allocate certain resources that must be freed before the library is unloaded from the process:

## 12.1 Memory buffers

When KYVPLibTL_DSAllocAndAnnounceBuffer() used and memory buffers are allocated by our library, they are marked as such. These buffers are released by our library when the user calls KYVPLibTL_DSClose().

## 12.2 Background monitoring thread

There is a background monitoring thread started by our library when an application calls KYVPLibTL_TLOpen(). This thread is stopped when KYVPLibTL_TLClose() is called. Unloading the library without closing all grabber handles may lead to undefined behaviour (uncaught exceptions, etc.). The application must call KYVPLibTL_TLClose() before unloading the library.

NOTE about DllMain's DLL_PROCESS_DETACH in Windows

When our library is unloaded from the process it tries to release allocated resources, e.g., stop threads, unpin and release memory, etc. But there are significant limitations on what can be done at this stage – as it is stated in the "DllMain entry point" documentation", "There are significant limits on what you can safely do in a DLL entry point". Therefore, it is still responsibility of the user application to properly close the library and all used resourses before the DLL is unloaded.

# 13 API Examples

## 13.1 VPII_KYVPLibTL_Example_Acquisition_Queued_Buffers

Demonstrate low-level transport-layer acquisition without XML-based parameter collections.

### 13.1.1 Purpose and workflow

This C sample opens the transport layer, enumerates PCI interfaces and devices, opens a data stream, allocates 16 buffers and handles new-buffer events on a worker thread. It returns processed buffers to the input queue and can listen for PCI-interface events. Camera acquisition is started and stopped by writing remote registers directly.

### 13.1.2 Camera configuration

Before running, set CAMERA_XML_ACQUISITION_START_REG, CAMERA_XML_ACQUISITION_STOP_REG and their corresponding VALUE constants from the camera's XML/register description. The supplied address 0x1005C and values 1/0 are for KAYA Iron cameras; they are not universal camera settings. Use VPII_Example_Acquisition_Queued_Buffers when you want named AcquisitionStart/AcquisitionStop parameters instead.

### 13.1.3 Interactive console options

Select the grabber and camera by number, then enter console controls:

| Control | Action |
|---|---|
| s | Start or stop acquisition. |
| t | Start or stop listening for PCI-interface events. |
| e | Exit and release resources. |

These menu controls are available on Windows and Linux.

### 13.1.4 Launch and help

Launch the executable without arguments and follow its console prompts. All operations are selected interactively. The only command-line option is --help , which prints usage and exits before opening hardware. Other arguments are rejected.

The sample requires the native transport-layer runtime and a compatible camera.

## 13.2 VPII_Example_Acquisition_Queued_Buffers

Acquire images with native Vision Point II transport, parameter and queued-buffer APIs.

### 13.2.1 Purpose and workflow

Use this C sample as a starting point for native acquisition. It enumerates PCI interfaces and cameras, opens a selected device, loads XML parameter descriptions, obtains image and stream requirements, and prepares 16 queued buffers. It controls the camera through AcquisitionStart/AcquisitionStop parameters, processes completed buffers and returns them to the input queue. It also demonstrates grabber, AUX and device events and releases streams, collections and handles on exit.

### 13.2.2 Launch and help

Launch the executable without arguments and follow its console prompts. All operations are selected interactively. The only command-line option is --help , which prints usage and exits before opening hardware. Other arguments are rejected.

### 13.2.3 Interactive console options

Select the PCI interface and camera by number, then enter menu controls:

| Control | Action and condition |
|---|---|
| m | Switch continuous/limited-frame mode while the mode selector is available (before starting or after releasing the stream). |
| s | In continuous mode, start or stop the stream. |

| Control | Action and condition |
|---|---|
| d | In limited-frame mode, request LIMITED_FRAMES_NUMBER frames (10 by default); create the stream on the first request. |
| f | In limited-frame mode, stop acquisition and release the created stream, allowing mode selection again. |
| t | Start or stop listening for PCI-interface events. |
| a | Start or stop listening for PCI-interface AUX events. |
| o | Start or stop listening for device events. |
| e | Exit and release resources. |

These menu controls are available on Windows and Linux. In this sample f releases a stream and a toggles AUX events; they are unrelated to the Linux crash tests in the adapter samples.

### 13.2.4 Build configuration

KYVP_USE_STREAM_DIRECT_CALLBACK is enabled; disabling it selects new-buffer event processing on a thread. Buffers are library allocated by default. KYVP_ENABLE_USER_MANAGED_BUFFER_MEMORY enables application allocation and announcement. KYVP_CUDA_BUFFERS enables the CUDA memory path when user-managed memory is selected, requiring compatible CUDA headers, libraries and hardware. NUMBER_OF_BUFFERS controls queue size.

## 13.3 VPII_Example_Grabber_Firmware_Update

Check and update KAYA grabber firmware through the Vision Point II extension API.

### 13.3.1 Purpose and workflow

This C console tool enumerates PCI interfaces, opens a selected grabber and accepts a firmware-file path. KYVPExtension_PCIInterface_CheckUpdateFile() reports board, vendor and version information. KYVPExtension_PCIInterface_FirmwareUpdate() performs the update and reports loading and validation progress. Use firmware intended for that board and perform the full PC power cycle requested by the application after a successful update.

### 13.3.2 Interactive console options

Select the PCI interface by number, and enter the firmware-file path at the prompt. The ? character terminates path entry. Enter the following controls at the firmware-update menu:

| Control | Action |
|---|---|
| c | Check the selected firmware file and display its metadata. |
| s | Check the file and start the firmware update. |
| t | Enter another firmware-file path. |
| e | Exit. |

### 13.3.3 Launch and help

Launch the executable without arguments and follow its console prompts. All operations are selected interactively. The only command-line option is --help , which prints usage and exits before opening hardware. Other arguments are rejected.

## 13.4 VPII_Example_Manual_Detection

Define camera connections using Vision Point II parameters before discovery and image acquisition.

### 13.4.1 Purpose and workflow

This native C sample opens the transport layer and a PCI interface, loads its parameter collection, and sets ManualDetectionDevicesCount and the per-device links-speed, links-count and link-mapping parameters. It then discovers and opens the configured camera, loads its remote parameter collection and runs a queued stream. The source uses the direct stream callback API from the extension library; it also contains an event-thread alternative.

### 13.4.2 Manual setup and interactive console options

Select the PCI interface by number. At the link-speed prompt, enter 1, 2, 3, 5 or 6 for 1.25, 2.5, 3.125, 5 or 6.25 Gbps; a/A selects 10 Gbps and c/C selects 12 Gbps. Enter the link count (1, 2, 4 or 8) and the host link index (0 through 7) for each device link. Answer y/n to add another device before discovery. Choose settings matching the connected camera, then select a detected device by number. The acquisition menu accepts:

| Control | Action |
|---|---|
| s | Start or stop acquisition. |
| t | Start or stop listening for PCI-interface events. |
| e | Exit and release resources. |

These console controls and setup choices are available on Windows and Linux.

### 13.4.3 Launch and help

Launch the executable without arguments and follow its console prompts. All operations are selected interactively. The only command-line option is --help , which prints usage and exits before opening hardware. Other arguments are rejected.

## 13.5 VPII_Example_Parameters_Handle

Inspect grabber and camera features through KYVPParametersHandler collections.

### 13.5.1 Purpose and workflow

This C sample obtains the grabber and remote-device XML descriptions, creates parameter collections, and connects them to transport-layer read/write callbacks. It demonstrates feature enumeration, cache invalidation and saving a collection's XML. Grabber features can be inspected even when no camera is detected. This is a parameter-access example, not an image-acquisition example.

### 13.5.2 Interactive console options

Select the grabber and, when detected, the camera by number. Enter the following controls at the feature menu:

| Control | Action |
|---|---|
| i | Invalidate the grabber parameter cache. |
| o | Print the grabber feature list. |
| g | Save the grabber XML description. |
| h | Export grabber features to CSV when ENABLE_SAVE_CSV_FILE is defined. |
| n | Invalidate the camera parameter cache. |
| p | Print the camera feature list. |
| c | Save the camera XML description. |
| v | Export camera features to CSV when ENABLE_SAVE_CSV_FILE is defined. |
| e | Exit and release collections and handles. |

Camera controls are offered only when a camera was detected and opened. CSV export is enabled in the supplied source by ENABLE_SAVE_CSV_FILE; it is a build choice, not a Linux-only feature. XML files are written as Grabber_saved_XML.xml or .zip and Camera_saved_XML.xml or .zip in the working directory, according to whether the original description is zipped.

### 13.5.3 Launch and help

Launch the executable without arguments and follow its console prompts. All operations are selected interactively. The only command-line option is --help , which prints usage and exits before opening hardware. Other arguments are rejected.

## 13.6 VPII_Example_SerialPort_API

Open camera serial ports using native Vision Point II device discovery and extension APIs.

### 13.6.1 Purpose and workflow

This C console sample opens the transport layer, enumerates PCI interfaces and cameras, and opens a native device before using the clserkyi serial-port API. It lists port identifiers, initializes a selected port, sends text commands and reads responses. It does not acquire image buffers.

### 13.6.2 Interactive console options

Select the PCI interface and camera by number, then use l to list ports before v to initialize one. Enter these controls at the serial-port menu:

| Control | Action |
|---|---|
| l | List available serial ports and their identifiers; populate the port count used by v. |
| v | Prompt for a listed port index and initialize that serial port. |
| b | Read a line of command text from the console and transmit it through the initialized port. |
| n | Retrieve queued received bytes. |
| e | Exit and release resources. |

The local bUseCallback variable defaults to true, registering asynchronous receive handling. Change it in the source for manual-only reading. Port selection is made at the v prompt.

### 13.6.3 Launch and help

Launch the executable without arguments and follow its console prompts. All operations are selected interactively. The only command-line option is --help , which prints usage and exits before opening hardware. Other arguments are rejected.

## 13.7 KYFGLibA_Example_GenICam_Camera_Firmware_Update

Transfer and verify remote-camera firmware through GenICam file-access features.

### 13.7.1 Purpose and workflow

This utility uses KYFGLib-style discovery and camera access to transfer a firmware binary to a camera. It opens the requested grabber and camera, discovers the supported FirmwareUpdate/FirmwareUpdate2 file type, transfers chunks, checks data according to the selected mode and writes a timestamped log.

### 13.7.2 Command-line arguments

Run at startup: KYFGLibA_Example_GenICam_Camera_Firmware_Update GRABBER CAMERA FILE [UPDATE_MODE [CHECK_MODE]] .

| Argument | Meaning |
|---|---|
| GRABBER | Required grabber index; the current parser reads one digit. |
| CAMERA | Required camera index; the current parser reads one digit. |
| FILE | Required firmware binary path; quote paths containing spaces. |
| UPDATE_MODE | 0: write and check; 1: write only; 2: check only; 3: write the first section last (default). |
| CHECK_MODE | 0: check the complete file (default); 1: check the previous section during transfer. |

When the camera uses FirmwareUpdate2, the implementation forces write-only mode regardless of UPDATE_MODE and skips read-back checking. The modes in the table apply to the legacy FirmwareUpdate file type. The supplied .bat and .sh launchers can be edited to select the firmware path and indices.

### 13.7.3 Requirements and completion

The camera must implement the file-access features used by this utility. Close applications using the same grabber before updating and select firmware intended for that camera. The source sets ConnectionConfig to 0x10038 for a 3.125 Gbps connection during the update. Check the console and

KYFGLibA_Example_GenICam_Camera_Firmware_Update timestamped log for completion, then power-cycle the camera as described in the supplied Readme.txt. Build against the SDK libraries selected by the supplied project.

## 13.8 KYFGLibA_Example_SerialPort_API

Use the adapter implementation of the Camera Link serial API with legacy camera handles.

### 13.8.1 Purpose and workflow

This C console project links clserkyi functions from KYFGLibA and defines KYCLSER_ADAPTER. It uses legacy discovery and camera handles, enumerates serial ports, opens a port, sends commands and reads responses either through a callback or explicit reads. Per-camera clGetNumSerialPortsEx() and clSerialInitEx() are used when a specific camera index is selected.

### 13.8.2 Launch and help

Launch the executable without arguments and follow its console prompts. All operations are selected interactively. The only command-line option is --help , which prints usage and exits before opening hardware. Other arguments are rejected.

### 13.8.3 Interactive console options

After startup, enter the following controls in order to select a grabber, open cameras and initialize the serial port before sending or reading data:

| Control | Action |
|---|---|
| Grabber number | Select and open a grabber using its displayed single-digit index. |
| c | Detect cameras, then prompt for a camera index; enter -1 to open all detected cameras. |
| v | Enumerate serial ports, then prompt for a port index and initialize that port. |
| b | Enter and transmit a serial command. |
| n | Read queued incoming bytes. |
| e | Close the serial port and grabbers, then wait at the final exit prompt. |

The local bUseCallback variable defaults to true, registering asynchronous receive handling. Set it to false in the source to read data manually with n. By default b reads a text command; the disabled LENS_COMMAND_ECHO and LENS_COMMAND_CALIBRATE_MOTORS source switches replace it with fixed lens commands. Commands and port indices must match the connected camera. No image stream is acquired.

### 13.8.4 Project variant

The source is shared with KYFGLibA_KYVPLibExtension_Example_SerialPort_API. Keep KYCLSER_ADAPTER defined for this adapter variant and use its supplied linker settings. The example requires a camera/bridge exposing a supported serial port and the installed native adapter runtime.

## 13.9 KYFGLibA_KYVPLibExtension_Example_SerialPort_API

Combine legacy camera discovery with serial-port functions from the native Vision Point II extension.

### 13.9.1 Purpose and workflow

This migration variant builds the same C source as KYFGLibA_Example_SerialPort_API without KYCLSER_ADAPTER. Its linker inputs place KYVPLibExtension before KYFGLibA so clserkyi calls resolve to the native extension. Legacy camera handles are converted through KYFG_Camera_Get_KYVP_DEVICE_HANDLE() where the native device handle is needed. Serial ports are enumerated globally and opened with clSerialInit(); the adapter-only per-camera Ex calls are bypassed. Port identifiers are also displayed.

### 13.9.2 Launch and help

Launch the executable without arguments and follow its console prompts. All operations are selected interactively. The only command-line option is --help , which prints usage and exits before opening hardware. Other arguments are rejected.

### 13.9.3 Interactive console options

After startup, enter the following controls in order to select a grabber, open cameras and initialize the serial port before sending or reading data:

| Control | Action |
|---|---|
| Grabber number | Select and open a grabber using its displayed single-digit index. |
| c | Detect cameras, then prompt for a camera index; enter -1 to open all detected cameras. |
| v | Enumerate serial ports, then prompt for a port index and initialize that port. |
| b | Enter and transmit a serial command. |
| n | Read queued incoming bytes. |
| e | Close the serial port and grabbers, then wait at the final exit prompt. |

The local bUseCallback variable defaults to true, registering asynchronous receive handling. Set it to false in the source to read data manually with n. By default b reads a text command; the disabled LENS_COMMAND_ECHO and LENS_COMMAND_CALIBRATE_MOTORS source switches replace it with fixed lens commands. Commands and port indices must match the connected camera. No image stream is acquired.

### 13.9.4 Build requirements

Use the supplied variant project to preserve library order and compile definitions. Both the native extension and adapter runtime libraries are required. This example demonstrates incremental migration; VPII_Example_SerialPort_API uses native discovery as well.

## 13.10 KYFGLibA_Example_Manual_Detection

Configure cameras manually through the legacy API adapter, then acquire images with queued buffers.

### 13.10.1 Purpose and workflow

Use this C console sample when camera links must be defined explicitly before discovery. It opens a selected grabber, configures one or more manual cameras, calls KYFG_UpdateCameraList(), opens the selected camera and registers a stream-buffer callback. Completed buffers are returned to the input queue for reuse. The application closes the stream, camera and grabber on exit.

### 13.10.2 Manual camera setup

Select a grabber by number, then configure each camera at the setup prompts. For link speed, enter 1, 2, 3, 5 or 6 for 1.25, 2.5, 3.125, 5 or 6.25 Gbps; a/A selects 10 Gbps and c/C selects 12 Gbps. These setup choices apply on Windows and Linux. Enter the grabber link index for each camera link and answer y/n to add another link. The prompts allow 1, 2, 4 or 8 links. Answer y/n to supply a camera XML manifest override and to add another manual camera. The XML-path prompt reads one whitespace-delimited path. After setup, discovery runs automatically; select a detected camera by number. The sample sets CameraSelector, ManualCameraMode, ManualCameraConnectionConfig, ManualCameraChannelSelector and ManualCameraFGLink before detection. Use settings and an XML description appropriate to the connected camera.

### 13.10.3 Interactive console options

After manual setup and camera selection, enter the following controls at the acquisition menu:

| Control | Action | Availability |
|---|---|---|
| s | Start or stop acquisition. | Windows and Linux |

| Control | Action | Availability |
|---|---|---|
| v | Toggle device-event output. | Windows and Linux |
| h | Toggle heartbeat output. | Windows and Linux |
| e | Exit and release resources. | Windows and Linux |
| f | Execute the deliberate division-by-zero crash test intended to trigger SIGFPE. | Linux only |
| a | Call abort() to trigger SIGABRT for crash-handler testing. | Linux only |

Both the menu entries and handlers for the f and a crash tests are guarded by __linux__ ; they are unavailable on Windows. The a/A link-speed choice belongs to the earlier setup prompt and selects 10 Gbps on either platform.

### 13.10.4 Build configuration

FGLIB_ALLOCATED_BUFFERS is enabled in the source; disabling it selects application-allocated aligned buffers. Build with the installed SDK and KYFGLibA adapter.

## 13.11 KYFGLibA_Example_Acquisition_Queued_Buffers

Acquire images through the legacy KYFGLib adapter using explicitly queued stream buffers.

### 13.11.1 Purpose and workflow

This C console sample demonstrates migration of a legacy acquisition application to KYFGLibA. It enumerates grabbers, detects cameras, opens a selected camera and creates a stream with 16 buffers. It queries payload size and alignment, announces or allocates the buffers, queues them for acquisition, and requeues completed buffers from the callback. It reports frame information and can display device events.

### 13.11.2 Interactive console options

Run the executable and respond to its console prompts. Select a grabber by number, enter d to detect cameras, then select a detected camera by number.

| Control | Action | Availability |
|---|---|---|
| s | Start or stop acquisition. | Windows and Linux |
| v | Toggle CoaXPress event output. | Windows and Linux |
| h | Toggle heartbeat output. | Windows and Linux |
| e | Exit and release resources. | Windows and Linux |
| f | Execute the deliberate division-by-zero crash test intended to trigger SIGFPE. | Linux only |
| a | Call abort() to trigger SIGABRT for crash-handler testing. | Linux only |

Both the menu entries and handlers for the f and a crash tests are guarded by __linux__ ; they are unavailable on Windows. These controls are entered while the program is running.

### 13.11.3 Buffer configuration

The default uses application-allocated aligned memory. Enable FGLIB_ALLOCATED_BUFFERS to let the library allocate buffers, or CUDA_BUFFERS to use CUDA host allocations. These two switches are alternatives and must not be enabled together. CUDA mode needs the CUDA headers and libraries. The selected grabber must support queued buffers. Link the project with the installed KYFGLibA adapter.

## 13.12 KYFGLibA_Example_Acquisition_Cyclic_Buffers

Acquire images with a cyclic buffer stream through the legacy-compatible KYFGLibA API.

### 13.12.1 Purpose and workflow

This C console sample scans for grabbers and cameras, opens a selected camera, creates a stream, registers a buffer callback and starts continuous acquisition. It demonstrates the cyclic-buffer approach used by existing

KYFGLib applications. The callback receives successive frames without the explicit input-queue recycling used by the queued-buffer sample.

### 13.12.2 Interactive console options

Run the executable and enter options at its console prompts. Select a grabber by number, enter d at the detection prompt, then select a detected camera by number. The acquisition menu accepts the following controls on Windows and Linux:

| Control | Action |
|---|---|
| s | Start the stream when stopped, or stop it when running. |
| e | Leave the menu and close the camera and grabber. |

### 13.12.3 Build configuration

FGLIB_ALLOCATED_BUFFERS is defined inside main(), selecting library allocation of 16 buffers. Removing that definition enables the alternative aligned allocation, buffer announcement and KYFG_StreamLinkFramesContinuously() path. MINIMAL_CALLBACK is enabled; remove it to print additional per-frame metadata. These are source definitions, not console menu controls. The installed KYFGLibA library and a compatible camera are required.

## 13.13 KYFGLibA.NET

Build the C++/CLI wrapper that exposes the native adapter to managed applications.

### 13.13.1 Purpose and workflow

This directory contains a managed library project, not a standalone acquisition executable. It builds KYFGLibA.NET.dll and exposes the KAYA_ADAPTER namespace, including library discovery, device and camera objects, parameter access, streams and buffer operations. Native FGSTATUS failures are translated into managed exceptions. The companion C# queued-buffer sample consumes this assembly.

### 13.13.2 Build and use

Build KYFGLib_Adapter.NET.vcxproj with the Windows C++/CLI toolchain and the installed native SDK. The project targets .NET Framework 4.0. Build this wrapper before its managed client and make the KYFGLibA.NET.dll assembly and matching native runtime libraries available to that client.

### 13.13.3 Application interface

This project produces a DLL and has no executable entry point or interactive console menu. Client applications use Lib.Scan(), Lib.Open() or Lib.OpenEx(), then operate on the returned managed objects. Close camera and device resources when finished. The companion C# sample provides the interactive console workflow and demonstrates delegate lifetime and stream-buffer handling.

## 13.14 KYFGLibA.NET_Example_Acquisition_Queued_Buffers

Acquire queued image buffers from C# through the KAYA_ADAPTER managed wrapper.

### 13.14.1 Purpose and workflow

The sample queries the SDK version, enumerates grabbers and asks which grabber to open. After Enter triggers discovery, it opens the first detected camera, sets Width to 640 and Height to 480, creates a stream and prepares 16 buffers. A retained callback delegate prints frame metadata and returns completed buffers to the input queue. Acquisition runs until console input requests stopping; callbacks, camera, grabber and user memory are then released.

### 13.14.2 Interactive console options

Run the executable and use its console prompts in order:

| Prompt | Input and action |
|---|---|
| Which grabber to use | Enter the displayed grabber index and press Enter. |

| Prompt | Input and action |
|---|---|
| Detect and connect cameras | Press Enter to discover cameras, or enter e to exit before discovery. |
| Press any key to continue | Supply console input to stop acquisition and release resources. |
| Press some key to exit script | Supply console input to finish the program. |

The last two prompts use Console.ReadLine() followed by Console.Read(); input can require Enter and an additional character. The first detected camera is selected automatically. The commented-out r prompt for registering device events is not an active option.

### 13.14.3 Build configuration

The camera must support the configured 640 by 480 image size. The local userAllocatedBuffers variable defaults to true: aligned unmanaged memory is allocated and announced. Set it to false to use BufferAllocAndAnnounce(). The buffers variable controls the buffer count. These settings are changed in source. Build the KYFGLibA.NET wrapper first, reference KYFGLibA.NET.dll, and make it and the matching native SDK runtime libraries available to the sample. Use a compatible Windows/.NET runtime.

## 13.15 Building API example for Windows

1. Open the “Vision Point II API Samples” shortcut on the desktop or using Windows quick search. In the “Vision Point API” subfolder, open an example project for Microsoft Visual Studio.
2. Select the x64 solution platform in Visual Studio.
   Note: *The Vision Point software stack does not support Win32 platform with OS x64.*
3. Build the project. The executable uses the same name as the project.
4. Run the application and follow the console prompts described for that sample above.

## 13.16 Building API example for Linux

API examples are installed in `/opt/KAYA_Instruments/Vision_Point_II/Examples`. This system directory is normally not writable by regular users. Before modifying or building an example, run prepare_examples.sh from the Examples directory to copy all examples to a writable location in your home directory.

1. Open a terminal, change to the installed Examples directory, and run the preparation script:

   ```
   cd /opt/KAYA_Instruments/Vision_Point_II/Examples
   bash prepare_examples.sh
   ```

2. Wait for the script to finish. It prints the exact destination after "Examples copied to:". Use that directory to modify and build the copied examples.
3. Open the VPII_KYVPLibTL_Example_Acquisition_Queued_Buffers directory in the copied examples. Replace <copied-examples-directory> below with the destination reported by the script:

   ```
   cd "<copied-examples-directory>/Vision Point API"
   ```

   ```
   cd VPII_KYVPLibTL_Example_Acquisition_Queued_Buffers
   ```

4. Build the example. A successful build creates the VPII_KYVPLibTL_Example_Acquisition_Queued_Buffers executable in the same directory:

   ```
   make
   ```

+1 (561) 689-2899
info@kaya.vision
www.kaya.vision

5. Run the executable from that directory:

```
./VPII_KYVPLibTL_Example_Acquisition_Queued_Buffers
```

# 14 Collect Diagnostic Info

Use **Collect diagnostic info** when reporting an issue to KAYA support. The command runs the KYInfo collection script, gathers system information and log files, and creates an archive that can be sent to KAYA support for diagnosis.

## 14.1 Windows Operating System

Windows provides three ways to start diagnostic collection. Use any one of the following methods.

### KAYA Service Controller II tray menu (recommended)

Right-click the KAYA Service Controller II icon in the notification area (system tray), then select Collect diagnostic info.

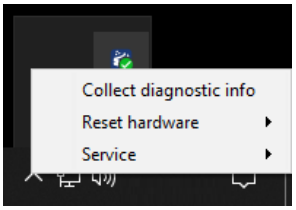



Figure 5 – Collect diagnostic info from the KAYA Service Controller II tray menu

### KAYA Vision Studio

In KAYA Vision Studio, open the Help menu and select Collect diagnostic info.

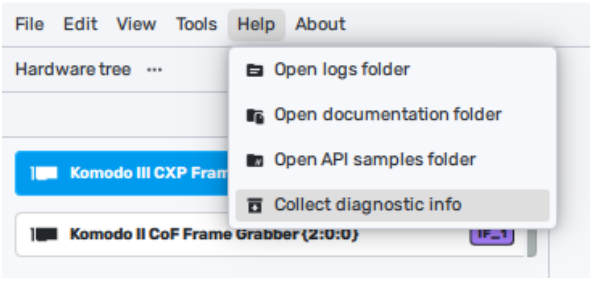



Figure 6 – Collect diagnostic info from the KAYA Vision Studio Help menu

### Windows Start menu

Open the Start menu, expand KAYA Instruments, and select Collect diagnostic info.

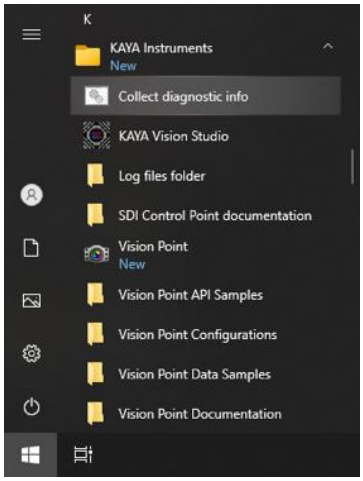



Figure 7 – Collect diagnostic info from the Windows Start menu

Wait for collection to finish. File Explorer selects the generated archive, C:\ProgramData\KAYA Instruments\KAYA.zip. Send this archive to KAYA support.

The default installation folder is C:\Program Files\KAYA Instruments. Installation logs are stored in C:\Program Files\KAYA Instruments\Log\Installer. If you selected a different installation folder, use that location instead.

## 14.2 Linux Operating System

Linux provides two ways to start diagnostic collection, illustrated below.

### KAYA Service Controller II tray menu (recommended)

Right-click the KAYA Service Controller II icon in the system tray and select Collect diagnostic info.

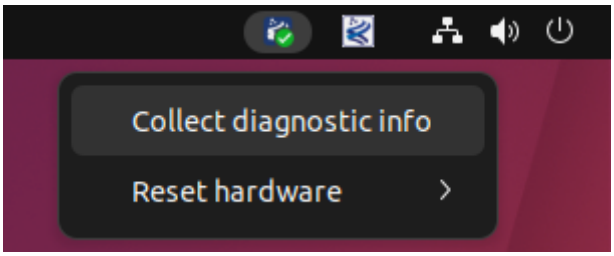



Figure 8 – Collect diagnostic info from the KAYA Service Controller II tray menu on Linux

### KAYA Vision Studio

In KAYA Vision Studio, open the Help menu and select Collect diagnostic info.

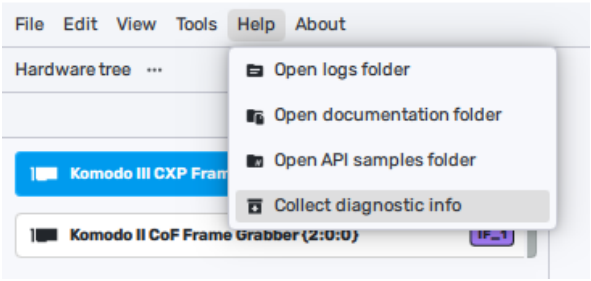



Figure 9 – Collect diagnostic info from the KAYA Vision Studio Help menu

Wait for collection to finish. The collection script creates ~/Documents/KAYA_Info/KAYA.tar.gz in your home directory and opens the containing folder. Send this archive to KAYA support.
Linux log files are stored in /var/log/KAYA_Instruments/.

# REFERENCES

Supported vision standards:

CoaXPress GEN<i>CAM

Vision Point documentation:

Vision Point

# TECHNICAL SUPPORT AND PROFESSIONAL SERVICE

If you searched the documents and could not find the answers you need, contact KAYA Vision support service:

- Create a support request on the web: support.kaya.vision
- Our knowledge base is available on: kb.kaya.vision

Visit us at www.kaya.vision for comprehensive information.

# SUBMITTING A SUPPORT REQUEST

When opening a support request, please provide the following information when applicable:

| For Frame Grabbers: | For Camera: | For Range Extender: |
|---|---|---|
| • Vision Point Diagnostic Info*<br>• Serial number of Frame Grabber<br>• Camera model<br>• SFP+ module model<br>• CoaXPress/Fiber cable model and length<br>• External power or PoCXP<br>• PC motherboard model<br><br>*In the Vision Point app, use menu option Help > Collect diagnostic info. | • Vision Point Diagnostic Info (or frame grabber being utilized)<br>• Serial Number of Camera<br>• XML dump and/or description of how the camera is being utilized<br>• Description of issue<br>• SFP+ module model<br>• CoaXPress/Fiber cable model and length<br>• External power or PoCXP | • Range Extender Model<br>• Serial Number of Range Extender<br>• SFP+ module model<br>• CoaXPress/Fiber Cable model and length<br>• PC configuration<br>• Operating System<br>• Software Revision<br>• Camera and Frame Grabber Manufacturer and Model |

KAYA VISION

Have questions about pricing, availability, documentation, or custom options?
We're always ready to assist and provide expert guidance.
Sales Inquiries: info@kaya.vision
Technical Support: support@kaya.vision
**www.kaya.vision**

KAYA Vision, Inc.
20283 State Road 7
Suite 350
Boca Raton, FL 33498
USA
**+1 561 698-2899**

2025 KAYA Vision, Inc. All rights reserved. KAYA Vision, the KAYA Vision Komodo logo, Predator, Iron, Zinc, Mercury and combinations thereof are trademarks of KAYA Vision, Inc. in the United States and/or other jurisdictions. Microsoft Windows® is a registered trademark of Microsoft Corporation. Linux® is a registered trademark of Linus Torvalds in the U.S. and other countries. JetPack® is a trademark of NVIDIA Corporation. HALCON® is a registered trademark of MVTec Software GmbH. Neither KAYA Vision, nor any of its products or services are affiliated with, endorsed by, or sponsored by National Instruments. MATLAB® is a registered trademark of The MathWorks, Inc. Cognex® is a registered trademark of Cognex Corporation. Other names are for informational purposes only and may be trademarks of their respective owners. KAYA Vision is not liable for harm or damage incurred by information contained in this document.